

Manual Técnico

AppTurnos / SWALP

Version 1.0 | 10/09/2026

Contenido

Para ver el índice: clic derecho sobre el area de abajo > Actualizar campos.

Contenido	2
Manual Técnico — AppTurnos / SWALP	6
Índice	6
1. Visión general	7
1.1 Qué es el sistema y qué problema resuelve.....	7
1.2 Stack y versiones exactas	7
1.3 Madurez del proyecto y estado actual	8
2. Puesta en marcha	9
2.1 Requisitos previos.....	9
2.2 Entorno virtual e instalación	9
2.3 Variables de entorno	9
2.4 Base de datos y migraciones	10
2.5 Datos de arranque	10
2.6 Arrancar y comprobar que funciona	11
2.7 Docker / docker-compose	11
3. Arquitectura	12
3.1 Diagrama de contexto	12
3.2 Capas y responsabilidad de cada una.....	13
3.3 Flujo de una petición de punta a punta	15
3.4 Patrones aplicados y por qué	16
3.5 Qué NO se hizo y por qué.....	19
4. Estructura del proyecto	20
4.1 Árbol de carpetas	20
4.2 Apps Django.....	21
4.3 Convenciones de nombres y de ubicación	22
4.4 Plantillas de correo	22
5. Modelos y base de datos.....	29
5.1 Diagrama entidad-relación	30
5.2 Ficha por modelo.....	31

5.3 Migraciones	41
5.4 Consultas críticas y su coste	42
6. Endpoints y URLs	44
6.1 Tabla de rutas	44
6.2 Fichas de endpoints no triviales	47
7. Servicios y lógica de dominio	57
7.1 Mapa de servicios	57
7.2 Validadores	70
7.3 Tareas programadas, outbox y comandos de gestión.....	71
7.4 La costura donde el mensaje cambia de destinatario	74
8. Reglas de negocio	75
8.1 Estados de una solicitud	75
8.2 Principios transversales	76
8.3 Reglas por área	82
9. Autenticación, autorización y seguridad	92
9.1 Modelo de usuario y roles	92
9.2 Permisos: dónde se comprueban	92
9.3 Sesiones, CSRF, CSP y cabeceras	95
9.4 Datos sensibles y qué nunca se registra	96
9.5 Páginas de error y trazabilidad de la petición	97
10. Configuración	104
Diferencias dev / test / producción	108
11. Dependencias	108
11.1 Producción (requirements.txt)	108
11.2 Desarrollo (requirements-dev.txt)	110
11.3 Frontend y externas	110
12. Recetas (how-to)	111
12.1 Añadir un tipo de solicitud	111
12.2 Añadir un campo a un modelo	111
12.3 Correr los tests	111
12.4 Depurar un fallo de validación	112
12.5 Regenerar la documentación	112

12.6 Desplegar	114
13. Despliegue	114
13.1 Topología	114
13.2 Pipeline y pasos	115
13.3 Checklist previo	116
13.4 Rollback	117
13.5 Observabilidad.....	117
13.6 Alarma obligatoria: el cierre semanal falla ABIERTO.....	120
14. Pruebas.....	122
14.1 Cómo se corren	122
14.2 Cómo está partida la suite.....	123
14.3 Base de datos de test	125
14.4 Cobertura y áreas sin cubrir	125
15. Decisiones técnicas (ADR)	126
16. Zonas frágiles y bugs conocidos	129
16.1 El catálogo de protecciones.....	129
16.2 Zonas que se rompen fácil.....	132
16.3 Bugs y riesgos abiertos	140
16.4 Resueltos que conviene recordar	142
17. Cambios técnicos relevantes	148
El aviso del formulario y el rechazo del servidor dicen la misma regla con las mismas palabras	148
El aviso de doblada permanente deja de dar por firme lo que solo está pendiente	149
«Cambio de doblada» queda deshabilitado, y el interruptor es del backend	151
El vigilante de las tareas programadas ahora avisa por su cuenta.....	152
El detalle del día de Mis Turnos dice siempre con quién es el acuerdo.....	153
Los once correos pasan a heredar una sola base	155
Una conexión SMTP por lote, no por correo.....	156
Horas a favor del explorador: la deuda también corre al revés.....	157
Levantar una auto-sanción por deuda condona la deuda del mes	158
Despliegue en AWS: se elige EC2 t4g.micro y se descarta Fargate	159
Refactorización SOLID de los cuatro puntos calientes	160

Cambios de comportamiento (no son refactor).....	162
El comentario pasa a ser obligatorio en toda decisión	164
Retirado el bloqueo temporal del día para un nuevo cambio de descanso.....	168
La cancelación de una aprobada deja de ser unilateral	169
Aviso al elegir un día de pago que ya se trabaja AM+PM	172
El día de pago de la cobertura con 2 compañeros debe estar libre.....	173
Páginas de error propias y trazabilidad de la petición	174
Seguridad de los enlaces de aprobación por correo	175
Temporada y cambio de descanso (rama fix/cambio-descanso-temporada).....	176
Rendimiento y consistencia.....	182
Vocabularios y constantes.....	182
Limpieza de código muerto	183
Calendario y calidad	183
18. Por confirmar.....	183

Manual Técnico — AppTurnos / SWALP

Documentación para el desarrollador que llega mañana y tiene que instalar el proyecto, entenderlo, tocarlo sin romperlo y desplegarlo.

Toda afirmación técnica lleva su origen entre paréntesis, en formato `ruta/archivo:línea` relativo a `AppTurnosExplora/` salvo que se indique otra raíz. Las líneas se verificaron contra el código el **2026-08-08**. Lo que no se pudo verificar está en [§ 18 Por confirmar](#).

Este documento **no reemplaza** la documentación de `docs/`: la ordena y la enlaza. Cuando un tema ya está cubierto en profundidad, aquí se resume y se remite.

La terminología de dominio (explorador, jornada, doblada, cesión, pago, temporada, alternancia, D FDS...) es la misma que en [manual_usuario.md](#) (`./manual_usuario.md`) § 1.3; ese glosario es la referencia común y no se duplica aquí.

Índice

1. [Visión general](#)
2. [Puesta en marcha](#)
3. [Arquitectura](#)
4. [Estructura del proyecto](#)
5. [Modelos y base de datos](#)
6. [Endpoints y URLs](#)
7. [Servicios y lógica de dominio](#)
8. [Reglas de negocio](#)
9. [Autenticación, autorización y seguridad](#)
10. [Configuración](#)
11. [Dependencias](#)
12. [Recetas \(how-to\)](#) (how-to)](#12-recetas-how-to)
13. [Despliegue](#)
14. [Pruebas](#)
15. [Decisiones técnicas \(ADR\)](#) (ADR)](#15-decisiones-técnicas-adr)
16. [Zonas frágiles y bugs conocidos](#)
17. [Cambios técnicos relevantes](#)
18. [Por confirmar](#)

1. Visión general

1.1 Qué es el sistema y qué problema resuelve

AppTurnos (SWALP) programa los turnos de un equipo de **exploradores** del Parque Explora y gestiona los acuerdos con los que intercambian jornadas entre ellos. Cada persona tiene una jornada base (AM o PM) y una programación de descansos que depende del calendario — festivos, mantenimiento, temporada— y de la alternancia publicada de fines de semana. Sobre esa base, el sistema permite seis tipos de acuerdo entre compañeros, cada uno con reglas propias.

El núcleo técnico no es el calendario: es el **motor de solicitudes**. Seis formularios que modifican turnos propios y *de terceros*, generan deuda de jornada, exigen doble aprobación (compañero + supervisor) y se pueden deshacer con el acuerdo de la contraparte, dentro de dos plazos de 24 horas ([ADR 009](#) (`./03-arquitectura/adr/009-cancelacion-consensuada.md`)). Casi toda la complejidad del repositorio está ahí: validar que un acuerdo es legal contra el estado *real* del día, materializarlo en filas de **Turno**, poder deshacerlo exactamente, e impedir que dos cambios concurrentes se pisen. Fuera del sistema quedan la nómina y el fichaje real: AppTurnos programa y contabiliza deuda, no asistencia.

1.2 Stack y versiones exactas

Componente	Versión / elección	Evidencia
Python	3.12 — deliberado : es la versión de Ubuntu 24.04 LTS (el SO de la EC2) y tiene soporte hasta 2028	<code>Dockerfile</code> , <code>.github/workflows/ci.yml</code>
Django	5.2.17 — sin Django REST Framework	<code>requirements.txt:4</code>
Base de datos	MySQL (8.4 en la plantilla de RDS); driver PyMySQL 1.1.2 instalado como MySQLdb si falta el nativo	<code>config/settings.py:123-124</code> (el <code>install_as_MySQLdb</code>), <code>requirements.txt:16</code>
Servidor de aplicación	Gunicorn 23.0.0, 3 workers	<code>requirements.txt:13</code>
Estáticos	Nginx desde disco en producción (143 MB); WhiteNoise 6.8.2 los sirve en el contenedor de pruebas, donde no hay Nginx	<code>config/settings.py:75</code> , § 13.1
Frontend	AdminLTE 3.2 + JavaScript vanilla, sin paso de build	<code>docs/03-arquitectura/TECNOLOGIAS_FRONTEND.md</code>

Historial	django-simple-history 3.8.0	config/settings.py:54, requirements.txt:11
Seguridad	django-axes[ipware] 7.0.1 (el extra <code>[ipware]</code> no es opcional: sin él axes ignora <code>AXES_IPWARE_*</code> y ve la IP de Nginx para todos), django-csp 4.0, django-cors-headers 4.9.0	config/settings.py:52-53,73,76,84, requirements.txt:7-9
Configuración	django-environ 0.14.0	config/settings.py:11,21-25, requirements.txt:10
Caché	LocMem en desarrollo; tabla MySQL (<code>`db://cache_appturnos`</code>) en producción, según <code>CACHE_URL</code>	config/settings.py:285-345
Exportación	openpyxl 3.1.5	requirements.txt:14
Zona horaria	America/Bogota, <code>USE_TZ=True</code> , idioma <code>es-co</code>	config/settings.py:214-217

No hay API REST pública: los endpoints JSON existen para el propio frontend.

1.3 Madurez del proyecto y estado actual

Proyecto en **pre-producción**. Lo que eso significa en la práctica:

- El esquema está maduro: 36 archivos de migración en `solicitudes/migrations/` —la última,

`0034`, añade la cancelación consensuada—, más los de `turnos`, `empleados` y `permisos` (recuento exacto de estos tres pendiente, § 18; en `permisos` la homóloga es la `0008`, y la `0009` añade `PermisoEspecial.fecha_aprobacion`).

- La suite de pruebas es amplia: 46 archivos en `solicitudes/tests/` —uno de ellos, `helpers_cancelacion.py`, es un helper compartido, no un test— y 33 repartidos entre `turnos`, `empleados`, `permisos`, `core` e `integration_tests`.

- El despliegue en AWS está documentado y hay imagen Docker, pero la infraestructura definitiva

sigue con pendientes ([ADR 005](#) (`./03-arquitectura/adr/005-pendientes-aws.md`)).

- La política CSP estricta está publicada en modo **report-only**, a la espera de confirmar que no

genera violaciones (`config/settings.py:399-418`).

- Los datos de la base de desarrollo son descartables: producción arranca limpia. No hay backfills ni migraciones de datos planificadas.

2. Puesta en marcha

Tutorial reproducible desde cero. Asume que no conoces el proyecto.

2.1 Requisitos previos

Requisito	Versión	Por qué
Python	3.12	Es la versión de la imagen (<code>Dockerfile</code>)
MySQL	8.0	Motor de <code>DATABASES</code> (<code>config/settings.py:164</code>)
Git	cualquiera	—
Microsoft Word	opcional	Solo para regenerar los PDF de documentación (usa COM)

Trampa de orientación nº 1: la raíz del proyecto Django es `AppTurnosExplora/` (ahí está `manage.py`), no la raíz del repositorio.

Trampa nº 2: el `docker-compose.yml` de la raíz del repositorio levanta **SonarQube**, no la aplicación. Los de la aplicación son `AppTurnosExplora/docker-compose.local.yml` y `AppTurnosExplora/docker-compose.hostdb.yml`.

2.2 Entorno virtual e instalación

```
cd AppTurnosExplora
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements-dev.txt # incluye requirements.txt con -r
```

`requirements-dev.txt` hereda producción con `-r requirements.txt` y añade solo herramientas de desarrollo (`requirements-dev.txt:3`).

2.3 Variables de entorno

```
copy .env.example .env          # y edita los valores
```

`.env.example` es la plantilla completa. Las variables **obligatorias** sin valor por defecto son `SECRET_KEY`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD` y `DEFAULT_FROM_EMAIL`: si falta cualquiera, `django-environ` lanza `ImproperlyConfigured` al importar `settings` y el proceso no arranca (`config/settings.py:33,170-172,261-263`). La tabla completa está en [§ 10 Configuración](#).

`ENVIRONMENT` (`development` | `production`) gobierna todo el archivo de `settings`, que es único: no hay `settings/local.py` (`config/settings.py:27`).

Deja `EMAIL_SEND_ASYNC=True` en tu `.env` —así viene en `.env.example:52`—. Con `False`, el envío del correo ocurre **dentro** del request y el formulario se queda esperando los handshakes SMTP (§ 7.3). El **default** del código sigue siendo `IS_PRODUCTION` (`config/settings.py:271`), de modo que la suite de tests conserva el envío síncrono que necesita.

2.4 Base de datos y migraciones

```
CREATE DATABASE bdturnosex CHARACTER SET utf8mb4;
```

```
python manage.py migrate
```

Si usas caché en tabla (`CACHE_URL=db://cache_appturnos`), hace falta además, una sola vez:

```
python manage.py createcachetable
```

(`config/settings.py:276-278`).

Trampa conocida en MySQL local: las tablas de zonas horarias vienen vacías y el admin de Django falla al filtrar por fecha. RDS/Aurora ya vienen pobladas; en local hay que cargarlas con `mysql_tzinfo_to_sql`.

2.5 Datos de arranque

```
python manage.py createsuperuser
python manage.py instalar_calendario_colombiano      # festivos
python manage.py actualizar_codigos_estrategia      # sincroniza codigo_estrategia
```

Además hacen falta datos que **no** se generan solos:

- Las dos filas de `Jornada` con nombre exacto `AM` y `PM`. El motor las busca por nombre literal y no se pueden borrar (`empleados/models.py:19-25,39-41`).
- Los roles `Supervisor` y `Explorador`, también por nombre exacto (`empleados/models.py:82-85`).
- Los seis `TipoSolicitudCambio` con los nombres de `core.constants.TipoSolicitud`

(`core/constants.py: CAMBIO TURNO, CAMBIO DESCANSO, CT PERMANENTE, DOBLADA, DOBLADA PERMANENTE, D FDS`).

- El calendario anual: temporada, festivos, mantenimiento, alternancia de fines de semana y descansos de semana. **Se carga a mano cada diciembre**; que un año futuro esté vacío es intencional, no un fallo. Procedimiento en [04-guias/mantenimiento-anual/](#) (./04-guias/mantenimiento-anual/).

2.6 Arrancar y comprobar que funciona

```
python manage.py runserver
```

Comprobación	Cómo	Qué debe pasar
Proceso vivo	GET /health/	{"status": "ok"}, sin tocar la base (core/health.py)
Base accesible	GET /health/ready/	Estado que sí verifica la conexión (core/health.py)
Login	GET /	Formulario de inicio de sesión (core/login/urls.py:5)
Panel	GET /dashboard/	Portada tras autenticarse (core/dashboard/urls.py:5)
Configuración	python manage.py check - -deploy	Sin errores críticos

Ninguno de los dos endpoints de salud requiere autenticación ni devuelve información del sistema: solo un estado (core/health.py, docstring del módulo).

2.7 Docker / docker-compose

Hay dos ficheros, para dos escenarios distintos:

Fichero	Qué levanta	Cuándo usarlo
docker-compose.local.yml	MySQL 8.0 efímero (puerto host 3307) + la app con la imagen de producción	Probar la imagen real sin tocar tu base local
docker-compose.hostdb.yml	Solo la app, apuntando a tu MySQL del host vía host.docker.internal	Probar la imagen contra tus datos reales

```
docker compose -f docker-compose.local.yml up --build -d # http://localhost:8000
docker compose -f docker-compose.local.yml down -v
```

Tres detalles que evitan sorpresas:

19. Ambos ficheros fijan `ENVIRONMENT=production` pero `SECURE_HTTPS=False`, para poder navegar en HTTP local sin caer en el redirect a HTTPS (`docker-compose.local.yml`, sección `environment`).
20. `docker-compose.hostdb.yml` **sobrescribe el ``command``** para arrancar solo Gunicorn, sin `migrate`: no toca el esquema de tu base real (`docker-compose.hostdb.yml`, clave `command`).
21. El puerto de MySQL en `local` se publica en **3307** para no chocar con tu MySQL local en 3306.

Guía completa: [MANUAL DOCKER LOCAL.md](#) (`./05-referencia/deployment/03-operacion/MANUAL_DOCKER_LOCAL.md`).

3. Arquitectura

3.1 Diagrama de contexto

```
flowchart TB
    EXP["Explorador<br/>(navegador)"]
    SUP["Supervisor / staff"]
    subgraph APP["AppTurnos – Django 5.2 monolito"]
        WEB["Vistas HTML<br/>AdminLTE + JS vanilla"]
        API["Endpoints JSON<br/>(consumo propio)"]
        ORQ["Motor de solicitudes<br/>orchestrator + factory + strategies"]
        TUR["Motor de turnos<br/>turnos/services"]
    end
    DB["MySQL 8.0<br/>turnos, solicitudes, deudas"]
    CACHE["Caché<br/>LocMem / Redis / tabla"]
    OUTBOX["EmailOutbox<br/>(tabla)"]
    SMTP["SMTP / SES"]
    HILO["Entrega tras commit<br/>(hilo, 1 conexión por lote)"]
    CRON["cron:<br/>procesar_email_outbox<br/>(red de seguridad)"]

    EXP --> WEB
    SUP --> WEB
    WEB --> API
    WEB --> ORQ
    ORQ --> TUR
    ORQ --> DB
    TUR --> DB
    TUR --> CACHE
    ORQ --> OUTBOX
    OUTBOX --> HILO
    CRON --> OUTBOX
    HILO --> SMTP
    CRON --> SMTP
    SMTP -->|enlace firmado de aprobación| EXP
```

Qué muestra. AppTurnos es un monolito Django único, sin servicios separados. Exploradores y supervisores usan el mismo navegador y las mismas vistas; lo que cambia es el permiso. Todo lo que modifica turnos pasa por el motor de solicitudes, que escribe en MySQL y encola correos en

la tabla `EmailOutbox` dentro de la misma transacción (`solicitudes/models.py:49-69`). La entrega ocurre **después del commit y en lote**: todos los correos de la operación salen por **una sola conexión SMTP** (`EmailOutboxService.enviar_lote`, `solicitudes/services/email_outbox_service.py:234`), en un hilo aparte cuando `EMAIL_SEND_ASYNC` está activo. El cron `procesar_email_outbox` **no es el camino normal**: es la red de seguridad que recoge lo que ese intento no logró entregar (`solicitudes/services/email_outbox_service.py:280-296`). Los correos llevan enlaces firmados que permiten aprobar sin iniciar sesión: es la única entrada al sistema que no pasa por el login. La caché guarda el estado de "Mis Turnos" y se invalida al aprobar o cancelar.

3.2 Capas y responsabilidad de cada una

Capa	Dónde vive	Responsabilidad	Qué NO hace
HTTP	<code>*/views/</code>	Parsear la petición, comprobar sesión y permisos, devolver JSON o HTML. En el alta de solicitudes la traducción a JSON está en un único punto, <code>solicitudes/views/resultado_http.py:12</code>	No contiene reglas de negocio
Orquestación	<code>solicitudes/services/solicitud_orchestrator.py</code>	Encadena los chequeos transversales y despacha al tipo	No conoce las reglas de cada tipo
Casos de uso	<code>solicitudes/use_cases/</code>	Nombra la intención: crear, aprobar, cancelar	No habla HTTP
Dominio	<code>solicitudes/domain/</code>	Máquina de estados, bloqueos, invariantes	No consulta el ORM salvo lo imprescindible
Estrategias	<code>solicitudes/services/strategies/</code>	Reglas propias de cada uno de los seis tipos	No sabe qué otros tipos existen
Validadores	<code>solicitudes/services/validators/</code>	Reglas reutilizables entre tipos. Dos familias: primitivas que lanzan <code>ValidationError</code> (<code>doblada_validator.py</code>) y	No escribe

		tramos de flujo que devuelven el mensaje (<code>doblada_flujo_validator.py</code>)	
Servicios de aplicación	<code>solicitudes/services/*_aplicacion_service.py</code>	Materializa y revierte turnos y deudas	No valida
Consultas	Managers y QuerySets del propio modelo (<code>solicitudes/models.py:136,149,256</code>)	Encapsular los <code>select_related</code> /filtros reutilizados	No decide
Modelos	<code>*/models.py</code>	Persistencia + invariantes estructurales (constraints)	—

La descripción larga está en [03-arquitectura/ARQUITECTURA.md](#) (./03-arquitectura/ARQUITECTURA.md) y la justificación de la capa de servicios en [ADR 001](#) (./03-arquitectura/adr/001-service-layer-y-orchestrator.md).

Middleware. El orden de `MIDDLEWARE` (`config/settings.py:70-85`) no es decorativo; tres posiciones están fijadas por una razón concreta:

Posición	Middleware	Por qué ahí
1.º	<code>core.errors.RequestIDMiddleware</code> (<code>config/settings.py:71</code>)	Etiqueta la petición con su identificador. Lo que quede por encima no queda etiquetado, así que va arriba del todo (§ 9.5)
2.º	<code>corsheaders.middleware.CorsMiddleware</code>	Debe responder al preflight antes que el resto
último – 1	<code>axes.middleware.AxesMiddleware</code>	La propia librería exige ir al final
último	<code>core.middleware.AperturaAnioMiddleware</code>	Bloquea al supervisor si falta planificar el año; necesita sesión y usuario ya resueltos

Fuera de producción, `debug_toolbar.middleware.DebugToolbarMiddleware` se inserta en la posición 0 (`config/settings.py:89-90`), por delante incluso del identificador.

3.3 Flujo de una petición de punta a punta

```
sequenceDiagram
    autonumber
    actor E as Explorador
    participant V as ProcesarSolicitudView
    participant U as CrearSolicitudUseCase
    participant O as SolicitudOrchestrator
    participant F as SolicitudFactory
    participant S as Strategy del tipo
    participant DB as MySQL
    participant SMTP as SMTP / SES
    actor C as Compañero
    actor SUP as Supervisor
    participant AP as Servicio de aplicación

    E->>V: POST /solicitudes/procesar-solicitud/
    V->>V: tipo_solicitud_id presente y existe
    V->>U: execute(POST, tipo, empleado)
    U->>O: procesar(...)
    O->>O: dedupe del POST idéntico (30 s)
    O->>O: sanción del solicitante
    O->>O: cierre semanal sobre las fechas objetivo
    O->>O: sanción del compañero
    O->>O: restricción médica (advierde, no bloquea)
    O->>F: validar_solicitud(tipo, datos)
    F->>S: validar_solicitud(datos)
    S-->>F: (ok, mensaje)
    F->>S: crear_solicitud(datos)
    S->>DB: SolicitudCambio + detalle (estado=pendiente)
    S->>DB: EmailOutbox (misma transacción)
    Note over S,SMTP: ya commitado: se cierra el envio_agrupado
    S->>SMTP: enviar_lote – los 3 correos por UNA conexión
    O-->>E: 201 {success, solicitud_id}

    C->>DB: aprueba como receptor
    SUP->>DB: aprueba como supervisor
    Note over F,AP: al completarse las dos aprobaciones
    F->>S: revalidar_para_aprobar(solicitud)
    F->>AP: aplicar_cambios(solicitud)
    AP->>DB: snapshot previo + turnos nuevos + deudas
    AP->>DB: estado = aprobada, fecha_resolucion = ahora

    E->>DB: pedir cancelación (≤ 24 h desde fecha_resolucion)
    Note over AP,DB: guardia LIFO + guardia de integridad (1.ª vez)
    AP->>DB: cancelacion_estado = pendiente – el estado sigue siendo aprobada
    C->>DB: responder (≤ 24 h): aprobar o rechazar
    Note over AP,DB: si rechaza o caduca → el cambio queda FIRME
    Note over AP,DB: si aprueba → guardia LIFO + integridad (2.ª vez)
    AP->>DB: restaurar snapshot, cancelar deudas, reconciliar
    AP->>DB: estado = cancelada, fecha_cancelacion = ahora
```

Qué muestra. El ciclo de vida completo de una solicitud. Los pasos 1-11 ocurren en una sola petición POST y terminan con la solicitud **pendiente**: nada se ha aplicado todavía al calendario. La aplicación real solo ocurre cuando se completan las dos aprobaciones, y va precedida de una revalidación contra el estado actual, porque entre la creación y la aprobación el mundo pudo

cambiar. La cancelación deshace exactamente lo aplicado usando el snapshot capturado antes de aplicar, y solo si las dos guardias lo permiten.

Dónde encaja el correo. Crear una solicitud dispara **tres** correos (supervisor, receptor y solicitante). Los tres se encolan dentro de la transacción y se entregan **al salir del bloque** ``envio_agrupado``, que `NotificacionService.crear_notificacion_solicitud` abre alrededor del cuerpo entero (`solicitudes/services/notificacion_service.py:40`). Ese bloque marca el límite del lote: los tres viajan por una única conexión SMTP en vez de pagar tres handshakes TLS + AUTH de ~2 s cada uno. Con `EMAIL_SEND_ASYNC` el lote sale en un hilo tras el commit, así que la respuesta al explorador no espera al SMTP (§ 7.3).

La cancelación de una solicitud aprobada son dos pasos, no uno. Cancelar deshace un acuerdo, así que quien puede deshacerlo depende de si ese acuerdo llegó a existir: una solicitud **pendiente** la retira el solicitante solo —nadie aceptó nada—, pero una **aprobada** la **pide** el solicitante y la **aprueba o rechaza el receptor**. Entre los dos pasos el **estado** sigue siendo **aprobada a propósito**: los turnos continúan aplicados y todo lo que filtra por `'aprobada'` —guardia LIFO, Mis Turnos, reconciliación— la sigue viendo vigente. Las guardias corren **dos veces**: al pedir, para no molestar al receptor con algo irreversible, y al aprobar, porque entre una cosa y otra pueden pasar 24 horas. Regla completa en § 8.2 (P4) y decisión razonada en el [ADR 009](#) (`./03-arquitectura/adr/009-cancelacion-consensuada.md`).

El paso **reconciliar** del diagrama —re-aplicar las dobladas que siguen aprobadas en esos días— es **reparación best-effort, no una validación**: se re-aplica **sin re-validar**, así que una solicitud vigente que ya no encaje con el calendario actual puede levantar `ValidationError` desde los guardias de negocio de los servicios de aplicación. Cuando pasa, esa candidata se **omite** y se registra; la cancelación en curso continúa. Política y consecuencia operativa en § 16.2.

3.4 Patrones aplicados y por qué

Strategy — un archivo por tipo de solicitud. Los seis tipos comparten la mayor parte del flujo pero difieren en reglas casi incompatibles: una doblada genera deuda, un cambio de descanso no; un CT permanente opera sobre un rango, un CT sencillo sobre un día. Resuelto con condicionales habría producido una vista de miles de líneas donde tocar un tipo rompe otro. `SolicitudStrategy` (`solicitudes/services/strategies/base_strategy.py`) define el contrato —`validar_solicitud`, `crear_solicitud`, `aplicar_cambios`, `revalidar_para_aprobar`, `get_empleados_disponibles`— y cada tipo lo implementa. El orquestador no sabe con qué tipo trabaja.

Factory con búsqueda en cuatro niveles. Los tipos viven en base de datos (`TipoSolicitudCambio`) y sus nombres los escriben personas. `SolicitudFactory.get_strategy` (`solicitudes/services/solicitud_factory.py:131`) busca en cascada: campo `codigo_estrategia` normalizado (`:143`), `codigo_estrategia` directo (`:154`), nombre directo (`:176`) y, si nada casa, estrategia por defecto con `warning` en el log (`solicitudes/services/solicitud_factory.py:190-195`).

Validadores reutilizables. Reglas como "no en día de mantenimiento" o "no contigo mismo" aplican a casi todos los tipos. `SolicitudValidator` (`solicitudes/services/validators/base_validator.py`) las concentra y los validadores específicos —`ct_validator.py`, `ct_permanente_validator.py`, `doblada_validator.py`— añaden lo suyo.

Máquina de estados propia, sin librería. Un único mapa de transiciones legales (`solicitudes/domain/estado_machine.py:11-18`) y una función que las valida (:28, lanza `EstadoTransicionError` en :47-53). El motivo de no usar `django-fsm` está en [ADR 002](#) (`./03-arquitectura/adr/002-fsm-sin-libreria-externa.md`).

Casos de uso. `use_cases/` nombra la intención por encima de la implementación: `CrearSolicitudUseCase`, `AprobarComoReceptorUseCase`, `AprobarComoSupervisorUseCase`, `AprobarAmbosRolesUseCase` (`solicitudes/use_cases/aprobar_solicitud.py:39`), `CancelarSolicitudUseCase`.

Consultas encapsuladas, sin capa de repositorios. Hubo un paquete `solicitudes/repositories/` (y otro en `empleados/`) que envolvía el ORM. Se **retiró**: de sus 14 métodos solo 2 tenían llamador, y el resto era indirección que había que leer dos veces para descubrir un `filter`. Lo que se reutiliza de verdad vive donde Django lo pone, en el manager del modelo: `SolicitudCambioQuerySet.con_relaciones()` (`solicitudes/models.py:149`), instalado como manager por defecto en `SolicitudCambio.objects` (`solicitudes/models.py:256`). El mismo camino que ya seguían `EmpleadoQuerySet` (`empleados/models.py:41`) y `TurnoActivoManager` (`turnos/models.py:41`).

Una fuente por pregunta, y el snapshot como geometría. «¿Con quién es el acuerdo de este día?» tiene dos mitades, y cada una tiene un dueño único:

Pregunta	Servicio	Qué lee
«Este día trabajo por un acuerdo : ¿cuál, con quién y en qué papel?»	<code>solicitudes/services/acuerdo_por_dia_service.py:73</code> (<code>AcuerdoPorDiaService</code>)	<code>snapshot_turnos_resultantes</code>
«Este día descanso por un acuerdo	<code>solicitudes/services/descanso_solicitud_service.py:40</code> (<code>DescansoPorSolicitudService</code>)	las solicitudes aprobadas del rango, por tipo

: ¿cuál y con quién?»		
-----------------------	--	--

El lado del trabajo **no re-deriva la geometría de cada tipo**: lee lo que el propio aplicador escribió. `snapshot_turnos_resultantes` guarda, por par (`empleado`, `fecha`), la lista de turnos que la solicitud dejó —clave "`<empleado_id>:<YYYY-MM-DD>`"—, así que una lista no vacía significa «esta solicitud dejó a esta persona trabajando ese día» y una vacía, «la dejó libre» (`acuerdo_por_dia_service.py:24-45`). Lo escribe el mismo código que aplica el cambio, de modo que no puede divergir de la realidad ni quedarse corto cuando aparece una sub-modalidad nueva.

El precio de no hacerlo así está medido: la vista de Mis Turnos tenía **ocho** consultas que reimplementaban a mano quién trabaja qué fecha en cada tipo, y esa copia nunca estuvo completa —DOBLADA PERMANENTE no tenía ninguna, CT PERMANENTE solo cubría el primer día del rango y CAMBIO DESCANSO entre semana dos de sus cuatro combinaciones—, así que los días que se caían del mapeo salían sin compañero (`turnos/api/views/turnos_mes.py:345-350`). Añadir una novena consulta arreglaba un caso y dejaba el patrón intacto para el siguiente tipo.

Dos reglas de resolución, y las dos son de dominio, no de implementación: **la última aprobada gana el día** —se recorre por `fecha_resolucion` descendente y la primera que reclama la fecha se la queda con `setdefault` (`acuerdo_por_dia_service.py:132-151`), § 8.2 P1— y la **guarda de realidad**: un snapshot solo puede hablar de un día si su `tipo_cambio` sigue coincidiendo con el del **Turno** que hay de verdad (`:115-126,145`). Sin esa guarda, una solicitud vieja cuyo día fue reescrito después seguiría nombrando a su compañero.

Outbox de correos. La fila de correo se escribe **dentro** de la misma transacción que el cambio de negocio, con clave de idempotencia única; el envío real ocurre después y se reintenta si falla (`solicitudes/models.py:49-69,81,100-101`). Sin esto, un proceso que muriera entre el COMMIT y el envío perdía el correo en silencio.

El patrón tiene **dos mitades separadas a propósito**: `encolar` es transaccional, `enviar_lote` no. Y la entrega es **siempre en lote**, incluso para un correo suelto —un lote de uno—, porque el coste real de un correo no es el mensaje sino el saludo: handshake TLS + AUTH, ~2 s medidos contra Gmail, frente a milisegundos por un canal ya abierto (`solicitudes/services/email_outbox_service.py:15-30`). El lote abre la conexión con un `open()` **explícito** antes del bucle (`:207-223`); sin ese `open()` no habría ahorro, porque `send_messages` de Django solo cierra la conexión que él mismo abrió y volvería a saludar por mensaje. Efecto secundario buscado: si el SMTP no responde, el fallo ocurre **al abrir**, antes de reclamar ninguna fila, así que una caída del servidor ya no le quema un intento a cada correo (`:219-222`).

Tres piezas deciden el **cuándo**, y las tres viven en el mismo módulo:

Pieza	Qué hace	Dónde
<code>despachar(fila_id)</code>	Punto único tras encolar: apunta al grupo abierto si lo hay; si no, delega en <code>_despachar_lote</code>	<code>email_outbox_service.py:370-382</code>
<code>envio_agrupado()</code>	Context manager reentrante que junta en un solo lote todo lo encolado dentro del bloque. Despacha en el finally	<code>:329-359</code>
<code>enviar_tras_commit(ids)</code>	Camino rápido: un <code>transaction.on_commit</code> que lanza el lote entero en un hilo y una conexión	<code>:303-325</code>

`envio_agrupado` **no** se apoya en `transaction.on_commit` para agrupar, y es deliberado: fuera de un `atomic()` —hay strategies que crean sin transacción a propósito— Django ejecuta el callback en el acto y cada correo formaría su propio grupo de uno (`:337-340`). El bloque marca el límite del lote de forma explícita, haya transacción o no.

Si una fila falla a media tanda, el lote **cierra y reabre el canal** antes de seguir: el fallo pudo ser del canal (un SMTP que corta por inactividad), y con un canal muerto fallaría todo el resto. Si la reapertura no prospera, el lote se corta ahí y las filas restantes quedan **sin reclamar**, con sus intentos intactos, para que las recoja el barrido (`:265-272`).

Todo el catálogo de protecciones —bloqueo pesimista, snapshot-once, deuda idempotente, guardias que fallan cerrado, invariantes en la base de datos— vive en ``PROTECTION_PATTERNS.md`` (raíz del repositorio, 43 patrones). Se resume en [§ 16](#).

3.5 Qué NO se hizo y por qué

Alternativa descartada	Motivo	Dónde consta
Django REST Framework	No hay consumidores externos; los endpoints JSON los usa el propio frontend	Ausente de <code>INSTALLED_APPS</code> (<code>config/settings.py:43-62</code>)
<code>django-fsm</code>	Las transiciones son pocas y fijas; una librería añadía dependencia sin cerrar el hueco real (asignar estado a mano)	ADR 002 (<code>./03-arquitectura/adr/002-fsm-sin-libreria-externa.md</code>)

Celery / broker	El único trabajo diferido son los correos. Se resolvió con tabla outbox + entrega en lote + cron de respaldo: sin broker, sin worker que vigilar y sin una segunda infraestructura que pueda caerse. Lo que costaba el envío era el handshake repetido, no la falta de paralelismo, así que compartir la conexión resolvió el problema medido sin añadir una cola	<code>solicitudes/models.py:49-69</code> , <code>solicitudes/services/email_outbox_service.py:15-30, 234-277</code>
django-redis	El backend Redis nativo de Django 5 basta; solo hace falta el cliente <code>redis</code>	<code>config/settings.py:280-281</code>
Paso de build en frontend	JS vanilla y AdminLTE servidos como estáticos; no hay bundler	<code>docs/03-arquitectura/TECNOLOGIAS_FRONTEND.md</code>
Índice único parcial en MySQL	MySQL no soporta <code>UniqueConstraint(condition=...)</code> ; se usa columna discriminante nullable	<code>turnos/models.py:55-60, 80-83</code>
Settings partidos por entorno	Un archivo único gobernado por <code>ENVIRONMENT</code>	<code>config/settings.py:23</code>

4. Estructura del proyecto

4.1 Árbol de carpetas

C:\appTurnos\	raíz del repositorio
├─ AppTurnosExplora\	← raíz del proyecto Django (manage.py)
│ ├─ config\	settings.py, urls.py, wsgi.py, db.py, paths.py
│ ├─ core\	login, dashboard, mixins, constantes, health,
│ ├─ caché	
│ ├─ empleados\	personas, roles, salas, competencias,
│ ├─ restricciones, sanciones	
│ ├─ turnos\	turnos, jornadas, días especiales, descansos,
│ ├─ alternancia	
│ ├─ solicitudes\	el dominio central: los seis tipos de solicitud
│ ├─ permisos\	PDH y permisos especiales
│ ├─ templates\	plantillas HTML (incluye emails\)
│ ├─ static\	JS, CSS, AdminLTE, plugins autohospedados
│ ├─ integration_tests\	pruebas de integración

scripts\	utilidades manuales (NO recogidas por pytest)
docs\	esta documentación
Dockerfile	
docker-compose.local.yml	app + MySQL efímero
docker-compose.hostdb.yml	app contra tu MySQL del host
PROTECTION_PATTERNS.md	43 patrones de concurrencia, idempotencia y
seguridad	
instructivos\	fuentes de negocio en .docx / .mwb
docker-compose.yml	⚠ solo SonarQube, NO la aplicación

4.2 Apps Django

App	Responsabilidad	Contiene	Depende de
core.login	Autenticación	LoginFormView, LogoutUserView (core/login/urls.py:5-6)	empleados
core.dashboard	Portada	DashboardView (core/dashboard/urls.py:5)	todas
core (no-app)	Mixins de permiso, constantes de dominio, caché, health, middleware, test runner	core/mixins.py, core/constants.py, core/health.py, core/test_runner.py	ninguna
empleados	Personas, jornadas, roles, salas, competencias, restricciones, sanciones, PDH y horas a favor (vistas)	empleados/models.py (303 líneas), empleados/views/	core
turnos	Turnos, días especiales, descansos de semana, alternancia, apertura de año, reportes	turnos/models.py (397), turnos/services/ (13 servicios), turnos/api/	empleados
solicitudes	Dominio central: seis tipos, deudas, aprobaciones, notificaciones, cierre semanal, reprogramaciones, outbox	solicitudes/models.py (847), 29 servicios, 7 estrategias, 4 validadores, 18 módulos de vistas	empleados, turnos
permisos	PDH y permisos especiales	permisos/models.py (158)	empleados, solicitudes

Dependencias en un solo sentido: `core` → `empleados` → `turnos` → `solicitudes` → `permisos`.
`core/constants.py` no importa ninguna app, precisamente para no crear ciclos
(`core/constants.py`, docstring).

4.3 Convenciones de nombres y de ubicación

Elemento	Convención	Ejemplo
Estrategia	<code>services/strategies/<tipo>_strategy.py</code> , clase <code><Tipo>Strategy</code>	<code>doblada_strategy.py</code> → <code>DobladaStrategy</code>
Validador	<code>services/validators/<tipo>_validator.py</code>	<code>doblada_validator.py</code>
Servicio de aplicación	<code>services/<tipo>_aplicacion_service.py</code> , método <code>aplicar()</code> / <code>revertir()</code>	<code>cambio_descanso_aplicacion_service.py</code>
Caso de uso	<code>use_cases/<verbo>_solicitud.py</code> , clase <code><Verbo>UseCase</code>	<code>CancelarSolicitudUseCase</code>
Vista JSON	<code>views/api_*.py</code> o <code>views/*_api.py</code> , reexportada en <code>views/__init__.py</code>	<code>views/api_fin_semana.py</code>
Respuesta JSON	Siempre <code>json_ok</code> / <code>json_error</code> de <code>core/utils/json_responses.py</code>	—
Lógica de front probable	Sin <code>document</code> ni <code>fetch</code> , en <code>static/js/utils/<tema>.js</code> , expuesta en un objeto de <code>window</code> ; su prueba en <code>tests_js/<tema>.test.cjs</code>	<code>static/js/utils/detalle-dia-mensajes.js</code> → <code>window.DetalleDiaMensajes</code>
Test	<code>test_*.py</code> , clases <code>Test*</code> , funciones <code>test_*</code>	<code>pytest.ini:3-5</code>
Literal persistente	Nunca en línea: constante en <code>core/constants.py</code>	<code>TipoSolicitud.DOBLADA</code>

Las vistas de `solicitudes` están partidas en 18 módulos y reexportadas desde `solicitudes/views/__init__.py`; el diagnóstico que lo motivó está en [01-analisis/ANALISIS_VIOLACIONES_SRP.md](#) (`./01-analisis/ANALISIS_VIOLACIONES_SRP.md`).

4.4 Plantillas de correo

Los once correos HTML del sistema heredan una sola base. Antes cada plantilla llevaba su propio `<style>` de ~200 líneas duplicado.

Hecho	Dónde vive	Por qué
Base común de correo, con los bloques <code>titulo_doc</code> , <code>estado_texto</code> , <code>estado_punto</code> , <code>antetitulo</code> , <code>titulo</code> , <code>subtitulo</code> , <code>saludo</code> , <code>contenido</code> y <code>pie</code>	<code>templates/emails/base_email.html</code> (bloques en :52,79,102-104,108,119,126)	Un único sitio donde tocar la maqueta de marca
Los once correos la heredan con <code>{% extends 'emails/base_email.html' %}</code> en su línea :1	Nueve en <code>templates/solicitudes/emails/</code> (<code>solicitud_receptor</code> , <code>solicitud_supervisor</code> , <code>solicitud_supervisor_receptor</code> , <code>confirmacion_solicitud</code> , <code>aprobacion_receptor</code> , <code>aprobacion_supervisor</code> , <code>rechazo_receptor</code> , <code>rechazo_supervisor</code> , <code>cancelacion_solicitud</code>), más <code>templates/permisos/emails/solicitud_permiso.html</code> y <code>templates/registration/aviso_password_cambiada.html</code>	Ninguno vuelve a declarar estilos propios
Cuatro parciales reutilizables: <code>_seccion.html</code> (título con barra roja), <code>_fila.html</code> (fila etiqueta/valor, con variantes <code>badge=<color></code> y <code>cita=1</code>), <code>_botones.html</code> (par rechazar/aceptar, o <code>url_unico</code> para un botón único) y <code>_item.html</code> (pastilla de icono + etiqueta/título/detalle)	<code>templates/emails/</code>	Traen el estilo en línea ya puesto: el <code>contenido</code> de cada correo solo los incluye
<code>_detalle_solicitud.html</code> conserva intacta su lógica por tipo de solicitud, pero ahora se pinta con <code>_item.html</code>	<code>templates/solicitudes/emails/_detalle_solicitud.html:21-101</code> (el <code>{% if tipo == ... %}</code> por tipo)	Lo que se solicita depende del tipo; cómo se pinta, no
Maquetación con <code><table></code> y estilos en línea: sin flexbox, grid, degradados, <code>rgba()</code> ni <code>position</code>	<code>base_email.html:24-27</code>	Varios clientes de correo eliminan el <code><style></code> del <code><head></code>
El logo va incrustado en el mensaje (<code>cid:logo-swalp</code>), no enlazado, y su <code>alt</code> va vacío	<code>base_email.html:70-71</code> ; lo adjunta <code>EmailOutboxService._incrustar_logo</code>	Una URL remota no se ve: con <code>SITE_URL=http://127.0.0.1:8000</code> el cliente resuelve

		127.0.0.1 contra la máquina de quien lee , y aun con el dominio público Gmail y Outlook bloquean las imágenes remotas hasta que el destinatario da permiso. El alt vacío es deliberado: con texto se pintaba «parque explora medellín» en rojo y partido dentro del recuadro blanco
El logo es PNG, no SVG: static/img/logo-explora-email.png	solicitudes/services/email_outbox_service.py (_RUTA_LOGO , _bytes_del_logo , _incrustar_logo), enganchado en _intentar	Ni Gmail ni Outlook renderizan SVG. Se generó rasterizando static/img/parque-explora-logo.svg —que es la versión blanca , la del sidebar oscuro— con el relleno cambiado a #e30613 . Si se regenera el logo hay que repetir ese paso. Se lee una vez por proceso (lru_cache) y, si falta, el correo sale igual sin él
PermisoNotificacionService._email_html ya no devuelve un f-string con HTML embebido: hace render_to_string('permisos/emails/solicitud_permiso.html', ...) . Su firma no cambia	permisos/services.py:145-158 (el render_to_string , en :149)	El correo de permisos usa la misma base que los de solicitudes

El logo va dentro del mensaje, no enlazado. El **** de la base apunta a **cid:logo-swalp** y quien adjunta esa parte es **EmailOutboxService._incrustar_logo**, llamado desde **_intentar**, que es el punto único por el que sale todo correo. Adjuntar convierte el mensaje en

`multipart/related` envolviendo al `multipart/alternative`; **solo** se hace si el HTML referencia el cid, para no colgarle un adjunto suelto a quien no lo pide. Se probó la estructura MIME resultante en `solicitudes/tests/test_arquitectura_correos.py`.

La trampa: `site\_url` no llega solo. Los botones «Ver en el sistema» construyen su enlace con `site_url`, y `render_to_string` se invoca **sin** ``request`` (`solicitudes/services/email_service.py:183,232,277,317,351,385,417,449,478`), así que los context processors **no** se aplican. **Todo contexto que renderice un correo tiene que pasarlo.** Por eso se añadió `'site_url': settings.SITE_URL` a los cinco contextos que no lo traían — confirmación (:320), aprobación supervisor (:356), aprobación receptor (:390), rechazo supervisor (:422) y rechazo receptor (:454)— y a `permisos/services.py:157`. Si falta, el botón apunta a un `/solicitudes/` sin host: en un correo, un enlace muerto. El fallo es **mudo** —el correo se envía igual—, y por eso lo vigila un trinquete con AST en `solicitudes/tests/test_arquitectura_correos.py`. La variable está documentada en § 10.

4.4.1 Los dos correos **PERMANENTES**: decir quién hace qué

Los correos de los dos tipos permanentes daban el dato y se callaban la consecuencia. En DOBLADA PERMANENTE decían «cede miércoles, devuelve martes», que no le cuenta a nadie quién trabaja ese día; en CT PERMANENTE no decían con qué jornada queda cada uno. Los dos se arreglan **solo en la capa de presentación del correo**: no se tocó la lógica de aplicación de ninguno de los dos tipos, solo se lee para contarlos.

Tipo	Qué dice ahora el correo	Dónde
DOBLADA PERMANENTE	Dos ítems: «Días que cede X» → <i>esos días el receptor dobla AM + PM y X descansa</i> ; «Días que devuelve X» → <i>esos días X dobla</i>	<code>templates/solicitudes/emails/_detalle_solicitud.html:88-89</code>

	AM + PM y el receptor descansa	
CT PERMANENT E	«Días del cambio» con el total de días que entran en el rango, y «Cómo queda la jornada» con el par real: <i>X pasa a PM y Y pasa a AM</i>	<code>templates/solicitudes/emails/_detalle_solicitud.html:68,70-74</code>

Para DOBLADA PERMANENTE basta con leer el detalle: los días de cesión y devolución están guardados en `DobladaPermanenteDetalle` y la plantilla los pide por `dias_cesion_legible` y `dias_devolucion_legible` (:88-89). **Para CT PERMANENTE no hay nada que leer**, y de ahí sale la pieza nueva.

En un CT PERMANENTE las jornadas no están guardadas en ninguna parte. Un CT permanente es un intercambio, y el intercambio se resuelve **día a día contra el estado real de «Mis Turnos»**: `jornadas_intercambiables_ct` devuelve el par (`js`, `jr`) con las jornadas reales solo si ese día ambos trabajan una jornada única y son contrarias, y si no devuelve `None` (`solicitudes/services/cambios_permanentes_helper.py:271-286`). No existe un campo `jornada_solicitante` que copiar al correo.

Pieza nueva	Qué hace	Dónde
<code>resumen_correo_ct_permanente(solicitud)</code>	Devuelve siempre las cinco claves <code>dias</code> , <code>total</code> , <code>primera</code> , <code>jornada_solicitante</code> , <code>jornada_receptor</code>	<code>solicitudes/services/cambios_permanentes_helper.py:969-1008</code>
<code>dias_legibles_ct_permanente(detalle)</code>	Auxiliar: los días del cambio en palabras — <code>'Martes, Jueves'</code> , las	<code>solicitudes/services/cambios_permanentes_helper.py:956-966</code>

	fechas sueltas elegidas, o 'Todos los días hábiles' cuando no se acotó ninguno— sobre <code>dias_seleccionados_desde_detalle</code>	
<code>{% resumen_ct_permanente solicitud as ct %}</code>	El <code>simple_tag</code> que le entrega ese diccionario al parcial	<code>solicitudes/templatetags/correo_solicitud.py:27-43</code>

Tres decisiones dentro de `resumen_correo_ct_permanente` que conviene no deshacer:

- **La jornada es la del PRIMER día aplicable, y el correo la presenta como tal.** Se toma `aplicables[0]` y se le pregunta el par a `jornadas_intercambiables_ct` (`cambios_permanentes_helper.py:998-1003`). El correo la enseña como lo que es —la jornada de hoy, la de partida, la que cada uno tiene *antes* del cambio—, no como una promesa: la solicitud aún no está aprobada y el resto de días se resolverá contra el estado real de cada uno cuando toque. El texto lo dice literalmente: «hoy X tiene AM y Y tiene PM» (`_detalle_solicitud.html:71`).
- **No dispara consultas por fecha.** Reutiliza `calcular_fechas_aplicables_y_excluidas_ct_permanente` (`cambios_permanentes_helper.py:930-949`), que ya precarga en lote, y envuelve la consulta del par en `precargar_ct_permanente` para el único día que necesita (`cambios_permanentes_helper.py:1000-1001`). El contexto de precarga vive en `cambios_permanentes_helper.py:132`.
- **Nunca lanza.** Si el detalle no existe, si el cálculo falla o si el par no se puede resolver, devuelve el diccionario `vacio` —todas las claves presentes, en `None` o vacías— y deja el rastro en el log (`cambios_permanentes_helper.py:984-988, 1005-1008`). La plantilla lo aprovecha con un `if`: sin par, cae a la explicación genérica «quien tiene AM pasa a PM y al revés» (`_detalle_solicitud.html:72-73`). Un correo no puede caerse ni salir a medias porque falle un adorno.

Por qué es un template tag y no una clave más del contexto. El parcial

`solicitudes/emails/_detalle_solicitud.html` lo incluyen **seis** correos, cada uno con su propio `render_to_string`: `solicitud_receptor.html:46`, `solicitud_supervisor.html:47`, `solicitud_supervisor_receptor.html:47`, `confirmacion_solicitud.html:46`, `aprobacion_receptor.html:34` y `aprobacion_supervisor.html:34` (todos bajo `templates/solicitudes/emails/`). Pasar el resumen por contexto obligaría a acordarse en los seis, y el que se olvidara **no fallaría**: mostraría un hueco en silencio. Es exactamente la trampa de `site_url` y la de los rangos en blanco. Con el tag, el parcial se lo pide solo (`{% load correo_solicitud %}` en `_detalle_solicitud.html:13`, la llamada en `:66`) y no hay nada que

recordar al añadir el séptimo correo. El módulo del tag no importa el servicio arriba: lo hace dentro de la función (`solicitudes/templatetags/correo_solicitud.py:41`), para que cargar la librería de plantillas no arrastre media capa de servicios.

La trampa: `{# ... #}` solo comenta si cabe en UNA línea. El lexer de plantillas de Django parte la fuente con `tag_re = re.compile(r"({%.*?%})|({{.*?}})|({#.??#})")` (`django/template/base.py`, Django 5.2.17) — sin `re.DOTALL`, así que el `.` no cruza el salto de línea. Un `{#` que no cierra en su misma línea no se reconoce como comentario y **sale impreso**. En un correo eso significa mandarle al explorador una nota interna de desarrollo. Pasó de verdad en cinco plantillas de correo **y en cuatro pantallas** —el consolidado de horas (tres), la lista de notificaciones y el levantamiento de sanción—; se corrigió pasándolas a `{% comment %}`, que sí es un tag de bloque (la cabecera del propio parcial es el ejemplo: `_detalle_solicitud.html:1-13`). Como el fallo no es de los correos sino de **cualquier** plantilla, el trinquete vive en `core/tests/test_plantillas_comentarios.py` y recorre línea a línea `templates/` entero.

Del mismo grupo: `add` con un objeto `date` devuelve cadena vacía. El filtro intenta `int(a)+int(b)`, luego `str + date`, se come el `TypeError` y devuelve `''` sin avisar de nada. Así, `fecha_inicio|date:"d/m/Y"|add:" a "|add:fecha_fin|date:"d/m/Y"` no daba `"17/09/2026 a 24/09/2026"` sino `""`: los rangos de vigencia de CT PERMANENTE y DOBLADA PERMANENTE salían **en blanco** y el correo se enviaba igual. Se arregla formateando las dos fechas antes con `{% with %}`, para que `add` solo vea cadenas (`_detalle_solicitud.html:53,81`).

Tests. `solicitudes/tests/test_arquitectura_correos.py` creció con tres clases, todas `SimpleTestCase` que renderizan el parcial con objetos de pega —lo que se vigila es la plantilla, no el modelo—:

Clase	Archivo:línea	Qué protege
<code>RangoDeVigenciaTest</code>	<code>solicitudes/tests/test_arquitectura_correos.py:178-241</code>	Que el rango de los dos tipos permanent es no vuelva a salir en blanco por el <code>add</code> con <code>date</code>
<code>ComentariosDePlantilla Test</code>	<code>core/tests/test_plantillas_comentarios.py</code>	Que ningún <code>{#</code> abra un comentario que no

		cierre en la misma línea, en ninguna plantilla del proyecto (no solo de correo)
DetallePermanenteTest	<code>solicitudes/tests/test_arquitectura_correos.py:267-335</code>	Que DOBLADA PERMANENTE diga quién dobla y quién descansa, que CT PERMANENTE diga con qué jornada queda cada uno y que, sin par resoluble, el correo explique igual en vez de quedarse mudo

Los dos casos de CT PERMANENTE parchean `resumen_correo_ct_permanente`, así que corren sin base de datos. `test_arquitectura_correos.py` son 11 tests y tarda 0,1 s; el de plantillas vive aparte porque su alcance ya no es el correo.

Qué no cambió. Ninguna regla de negocio, ningún endpoint, ningún modelo y ninguna migración. `resumen_correo_ct_permanente` solo **lee**: no escribe, no aplica y no altera el cálculo de fechas aplicables, que sigue siendo el mismo que usa la vista previa.

5. Modelos y base de datos

Cuatro apps con modelos: `empleados` (8 modelos), `turnos` (7), `solicitudes` (14) y `permisos` (2). Casi todos llevan `HistoricalRecords()` de `django-simple-history`, así que tienen además su

tabla `historical*` con el rastro de cambios; `Notificacion`, `EmailOutbox` y `TurnoArchivo` no lo llevan.

5.1 Diagrama entidad-relación

```
erDiagram
    User ||--|| Empleado : "OneToOne"
    Empleado ||--o{ EmpleadoRole : tiene
    Role ||--o{ EmpleadoRole : "PROTECT"
    Empleado ||--o{ CompetenciaEmpleado : "sabe salas"
    Sala ||--o{ CompetenciaEmpleado : ""
    Empleado ||--o{ RestriccionEmpleado : "restriccion medica"
    Empleado ||--o{ SancionEmpleado : "sancionado"
    Empleado ||--o{ AsignarJornadaExplorador : "jornada base"
    Jornada ||--o{ AsignarJornadaExplorador : "PROTECT"
    Empleado ||--o{ Turno : trabaja
    Jornada ||--o{ Turno : "PROTECT"
    Sala ||--o{ Turno : ""
    Jornada ||--o{ DescansoSemanaManual : descansa
    Jornada ||--o{ AsignacionEspecialManual : "trabaja el dia"

    Empleado ||--o{ SolicitudCambio : solicita
    Empleado ||--o{ SolicitudCambio : recibe
    TipoSolicitudCambio ||--o{ SolicitudCambio : tipo
    SolicitudCambio ||--o{ CambioPermanenteDetalle : "CT PERMANENTE"
    CambioPermanenteDetalle ||--o{ CambioPermanenteDia : dias
    SolicitudCambio ||--o{ DobladaDetalle : "DOBLADA / CAMBIO DESCANSO / D FDS"
    SolicitudCambio ||--o{ DobladaPermanenteDetalle : "DOBLADA PERMANENTE"
    SolicitudCambio ||--o{ DeudaExplorador : genera
    SolicitudCambio ||--o{ DeudaCorporativa : genera
    SolicitudCambio ||--o{ ReprogramacionDiaDoblada : reprograma
    SolicitudCambio ||--o{ Notificacion : notifica
    SolicitudCambio ||--o{ PDH : opcional
    Empleado ||--o{ PDH : "paga horas"
    DeudaCorporativa }o--o{ PDH : deudas_pagadas
    PermisoEspecial }o--o{ PDH : permisos_pagados
    CreditoHoras }o--o{ PDH : creditos_consumidos
    Empleado ||--o{ CreditoHoras : "se le deben horas"
    Empleado ||--o{ PermisoEspecial : pide
```

Qué muestra. El centro de gravedad es `SolicitudCambio`: toda solicitud, sea del tipo que sea, es una fila de esa tabla más **una** fila de detalle según el tipo. Hay tres tablas de detalle, no seis: `CambioPermanenteDetalle` para CT PERMANENTE, `DobladaPermanenteDetalle` para DOBLADA PERMANENTE y `DobladaDetalle` —la más cargada— para DOBLADA, CAMBIO DESCANSO y D FDS. CAMBIO TURNO no tiene detalle: le basta con `fecha_cambio_turno` y los snapshots de la propia solicitud (`solicitudes/models.py:171,238-253`). A la izquierda queda el calendario: `Turno` es el hecho materializado, mientras `Jornada`, `DescansoSemanaManual` y `AsignacionEspecialManual` son la planificación que dice quién trabaja cuándo. A la derecha, la contabilidad: `DeudaCorporativa` (30 minutos por doblada) y `PDH`, el pago que la salda.

5.2 Ficha por modelo

App empleados (empleados/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
Jornada	<code>nombre</code> (unique, choices AM/PM), <code>hora_inicio</code> , <code>hora_fin</code>	<code>unique</code> en <code>nombre</code> (:25)	Catálogo estructural. El motor la busca por nombre literal y deriva la "jornada contraria" de forma binaria; <code>NOMBRES_PROTEGIDOS = (AM, PM)</code> y la propiedad <code>es_protegida</code> impiden borrarlas (:19-23,38-41)
Empleado	<code>user</code> (OneToOne con <code>auth.User</code>), <code>nombre</code> , <code>apellido</code> , <code>cedula</code> (unique), <code>email</code> , <code>activo</code> , <code>supervisor</code> (FK a sí mismo, <code>SET_NULL</code>)	<code>empleado_activo_idx</code> , <code>empleado_super_activo_idx</code> (:58-61)	<code>supervisor</code> define a quién llega el correo de aprobación. <code>notificaciones_no_leidas_count()</code> (:66-68)
Role	<code>nombre</code> (unique)	<code>unique</code> (:87)	Catálogo estructural. <code>es_protegido</code> compara sin distinguir mayúsculas porque el permiso también lo hace: un rol "supervisor" concede acceso (:98-107). El docstring documenta la escalada de privilegios que hubo con búsqueda <code>icontains</code> (:73-81)
EmpleadoRole	<code>empleado</code> , <code>role</code>	<code>unique_together</code> (<code>empleado</code> , <code>role</code>), <code>empleado_role_com_p_idx</code> (:121-124)	<code>role</code> es <code>on_delete=PROTECT</code> a propósito: borrar "Supervisor" arrastraba en silencio todas sus asignaciones (:112-114)
Sala	<code>nombre</code> , <code>activo</code>	<code>sala_activo_idx</code> (:139-141)	—
CompetenciaEmpleado	<code>empleado</code> , <code>sala</code>	<code>unique_together</code> , <code>comp_emp_sala_idx</code> (:156-159)	Qué salas sabe atender cada persona
RestriccionEm	<code>empleado</code> ,	<code>rest_emp_fecha_id</code>	<code>clean()</code> exige <code>fecha_fin >=</code>

pleado	fecha_inicio, fecha_fin (null), recomendacion, tipo_restriccion	x, rest_tipo_idx (:179-182)	fecha_inicio (:184-187). Advierte, no bloquea: ver SolicitudOrchestrator.verificar_restriccion
SancionEmpleado	explorador, supervisor, fecha_inicio, fecha_fin, motivo, levantada_en, levantada_por (PROTECT), levantada_motivo	sanc_exp_fecha_idx, sanc_supervisor_idx (:230-233)	Invariante: una sanción no se borra, se levanta. fecha_fin es el fin planeado y no se toca; el fin real lo da levantada_en (:193-208). fecha_fin_efectiva puede quedar antes que fecha_inicio, y por eso se calcula en vez de guardarse (:249-260). levantar() es idempotente (:286-299). clean() prohíbe autosancionarse (:241-242)

App turnos (turnos/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
AsignarJornadaExplorador	explorador, jornada (PROTECT), fecha_inicio	jornada_explorador_fecha_idx (:19-21)	Sin fecha_fin: las jornadas son indefinidas (:13). La vigente es la de mayor fecha_inicio menor o igual a la fecha consultada
Turno	explorador, fecha, jornada (PROTECT), sala (nullable), tipo_cambio (null = turno normal), anulado, motivo_anulacion, activo_key (derivado, editable=False)	turno_explorador_fecha_idx; UniqueConstraint(explorador, fecha, jornada, activo_key) = turno_unico_activo_por_jornada; CheckConstraint turno_tipo_cambio_valido (:70-95)	El hecho materializado. Soft-delete auditable vía anulado. activo_key vale 1 si está activo y NULL si anulado, y se mantiene sola en save() (:98-104): existe porque MySQL no soporta índices únicos parciales. El manager por defecto

			TurnoActivoManager excluye anulados; Turno.all_objects los incluye (:27-36,63-65)
DiaEspecial	fecha, tipo (festivo/mantenimiento/temporada), recurrente, activo, año_planificacion, es_temporada, mes	dia_esp_anio_mes_tem_p_idx, dia_esp_fecha_tipo_activo_idx; UniqueConstraint(fecha, tipo) (:140-149)	mes, año_planificacion y es_temporada son derivados y se recalculan en cada save() para que no puedan contradecir a fecha/tipo (:152-164). La temporada manda sobre el mantenimiento: es_mantenimiento_efectivo devuelve False si el día cae en temporada (:186-200). ANIO_MIN=2000, ANIO_MAX=2100 como fuente única de los límites (:110-113)
TurnoArchivo	Espejo de Turno + fecha_archivado, turno_original_id; sala también nullable	turno_arch_exp_fecha_idx, turno_arch_fecha_idx (:227-230)	Sin CheckConstraint a propósito: solo recibe copias ya validadas (:218-219)
DescansoSemanalManual	fecha (lun-vie), jornada (PROTECT), motivo, activo	UniqueConstraint(fecha, jornada), descsem_fecha_activo_idx (:268-273)	En semanas de temporada o festivo el descanso deja de ser el lunes de mantenimiento y el supervisor lo fija aquí, por jornada (:240-247). clean() rechaza sábado y domingo (:275-278)
AsignacionEspecialManual	fecha, jornada_trabaja (PROTECT), tipo	UniqueConstraint(fecha), asigesp_fecha_activo	Fuente de verdad de la alternancia. Una fecha sin fila significa "sin planificar" y así se

	(finde/festivo), activo	_idx (:318-323)	reporta; nunca se inventa un grupo (:284-296). clean() valida que el tipo case con el día de la semana (:325-333)
AperturaAnioConfig	inicio_recordatorio_dia/mes, inicio_bloqueo_dia/mes, bloqueo_duro	— (singleton)	Singleton vía obtener() con get_or_create(pk=1) , para que dos peticiones concurrentes no creen dos filas (:375-382). _fecha() recorta al último día del mes y evita que un 31 configurado reviente en meses cortos (:384-388)

App solicitudes (solicitudes/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
Notificacion	destinatario, tipo, titulo, mensaje, leida, fecha_lectura, solicitud (null)	notif_dest_leida_idx, notif_tipo_idx, notif_fecha_creacion_idx (:34-38)	La campana dentro de la aplicación; independiente del correo
EmailOutbox	asunto, cuerpo_texto, cuerpo_html, remitente, reply_to, destinatarios (JSON), estado, intentos, ultimo_error, clave_idempotencia (unique, nullable), disponible_en , enviado_en	outbox_estado_disp_idx (:108-110)	Patrón outbox. MAX_INTENTOS = 5 (:81) . La fila se escribe dentro de la misma transacción que el cambio de negocio; la garantía es <i>como máximo una vez por clave más al menos una vez por reintento (:49-69)</i> . disponible_en implementa el backoff (:100-101). El modelo no sabe cómo

			se entrega: quién abre la conexión, cuándo y en qué lote lo decide <code>EmailOutboxService</code> (§ 3.4, § 7.3)
TipoSolicitudCambio	<code>nombre</code> (unique), <code>codigo_estrategia</code> (null), <code>activo</code> , <code>genera_deuda</code>	<code>tipo_sol_activo_id</code> (:129-131)	Tabla maestra de los seis tipos. <code>codigo_estrategia</code> es el primer nivel de búsqueda de la factory
SolicitudCambio	<code>explorador_solicitante</code> , <code>explorador_receptor</code> , <code>tipo_cambio</code> (FK), <code>estado</code> , <code>fecha_solicitud</code> , <code>fecha_cambio_turno</code> , <code>fecha_resolucion</code> , <code>fecha_cancelacion</code> , <code>aprobado_receptor</code> + <code>fecha</code> , <code>aprobado_supervisor</code> + <code>fecha</code> , <code>turno_origen/turno_destino</code> (SET_NULL), <code>solicitud_origen</code> , <code>reemplazada_por</code> , <code>snapshot_turnos_previos</code> , <code>snapshot_turnos_resultantes</code> , y el bloque de cancelación consensuada <code>cancelacion_estado</code> (CharField(10), <code>choices=EstadoCancelacion.CHOICES</code> , default ''), <code>cancelacion_solicitada_por</code> / <code>cancelacion_respondida_por</code> (FK Empleado, SET_NULL), <code>cancelacion_solicitada_en</code> / <code>cancelacion_respondida_en</code> (datetime, null) y <code>cancelacion_motivo</code> (texto, null) (:166-195)	5 índices: <code>receptor+fecha+estado</code> , <code>turno_origen+estado</code> , <code>turno_destino+estado</code> , - <code>fecha_resolucion+estado</code> , <code>solicitante+-fecha_solicitud</code> (:238-264)	<code>'fecha_resolucion'</code> NO se sobrescribe al cancelar : de ella dependen la ventana de 30 minutos y el orden de la guardia LIFO; para eso existe <code>fecha_cancelacion</code> (:151-165). Los dos snapshots permiten revertir y detectar que alguien más tocó el día (:217-232). Invariante del bloque de cancelación : mientras <code>cancelacion_estado == 'pendiente'</code> , <code>estado</code> sigue valiendo <code>'aprobada'</code> y los turnos siguen aplicados; la reversión solo ocurre cuando el receptor aprueba (<code>solicitudes/models.py:166-170</code>)

CambioPermanent eDetalle	solicitud (OneToOne), fecha_inicio, fecha_fin	camb_perm_solicitu d_idx, camb_perm_fecha_in icio_idx (:250-253)	clean() exige fecha_fin >= fecha_inicio (:255-258)
CambioPermanent eDia	cambio_permanente, fecha_especifica o dia_semana, tipo	3 índices + CheckConstraint camb_perman_dia_ti po_valido, que exige exactamente uno de los dos campos según tipo (:314-328)	save() rechaza sábado y domingo, tanto en dia_semana como en fecha_especifica (:330-349). Sin filas se usa el rango completo, por retrocompatibilidad (:273)
DobladaDetalle	solicitud (OneToOne), minutos_deuda (30), fecha_pago (obligatoria), tipo_cesion, jornada_cedida, es_intercambio, jornada_pago_sabado (AM/PM/AMBAS), fecha_pago_semana, jornada_cubre_en_pago, submodalidad_semana, snapshot previo y resultante, empleado_receptor	doblada_solicitud_ idx, doblada_fecha_pago _idx, doblada_receptor_f echa_idx (:470-474)	No existen dobladas abiertas: fecha_pago es obligatoria (:387- 389). es_intercambio=Tr ue es un swap de días y no genera ni altera deudas (:403-408). submodalidad_sema na distingue los sub- flujos de CAMBIO DESCANSO en temporada; las filas antiguas sin valor se tratan como intercambio_dia (:435-442). El historial excluye jornada_pago_saba do (:464)
DobladaPermanen teDetalle	solicitud (OneToOne), fecha_inicio, fecha_fin, dias_cesion/dias_devo lucion (CSV 0-6), fechas_cesion/fechas_ devolucion (CSV ISO), minutos_deuda, empleado_receptor, snapshots	dob_perm_solicitud _idx, dob_perm_fecha_idx (:537-540)	Las fechas específicas tienen prioridad sobre los días de la semana; si están vacías se expanden los weekdays (:504- 514). No se permiten domingos (:490). clean() valida el rango (:554-557)

DeudaExplorador	deudor, acreedor, solicitud_origen, fecha_pago_pactada, fecha_pago_real, estado, media_jornada, jornada_cedida	deuda_deudor_estado_idx, deuda_acreedor_estado_idx, deuda_fecha_pago_estado_idx (:627-631)	Deuda entre personas
DeudaCorporativa	explorador, solicitud_origen (SET_NULL), minutos (30), fecha_doblada, estado (activa / pagada / consumida_por_sancion / condonada / cancelada, :706-720), fecha_pago, comentario, sancion_consumidora (FK SET_NULL a SancionEmpleado, :768), fecha_consumo (:771)	deuda_corp_exp_estado_idx, deuda_corp_fecha_estado_idx, deuda_corp_fecha_dob_idx (:695-699)	Deuda con la empresa . obtener_deuda_total() suma las activa . Cinco finales , no tres , y cada uno dice algo distinto: pagada (hubo PDH), consumida_por_sancion (la extinguió el castigo cumplido), condonada (el supervisor levantó la auto-sanción de ese mes y con ella la deuda, :719; motivo de que sea estado propio en :713-718) y cancelada (corrección técnica: deuda huérfana, fin de semana deshecho). Los dos de extinción por sanción comparten los campos de trazabilidad — sancion_consumidora y fecha_consumo , la sanción responsable es una sola— y es el estado el que separa lo cumplido de lo perdonado (:765-771). Ver § 8.3 y el ADR 013 (./03-arquitectura/adr/013-condonacion-al-levantar-la-sancion.md)

ReprogramaciónDoblada	doblada_origen, explorador, fecha_original, jornada_debida, fecha_reprogramada, jornada_pago_previa, estado, motivo, registrado_por	reprog_explorador_estado_idx, reprog_doblada_idx (:773-776)	Es simétrico (sirve para solicitante o receptor) y no afecta al otro explorador . El día original se anula por soft-delete y se le resta su deuda; al programar el nuevo se le vuelve a agregar la doblada y los 30 minutos (:719-729). jornada_pago_previa se restaura al cancelar (:754-757)
CierreSolicitud esConfig	habilitado, dia_cierre, hora_cierre	— (singleton)	Si habilitado=False no hay ninguna restricción (:802). obtener() usa get_or_create(pk=1) (:815-822)
CierreSemanaOverride	semana_lunes (unique), habilitado, dia_cierre, hora_cierre	unique en semana_lunes (:830)	Ajusta el cierre de una semana concreta sobre el default global

App permisos (permisos/models.py)

Modelo	Campos clave	Constraints e índices	Notas de negocio
PDH	explorador, solicitud (opcional), fecha, horas (decimal 5,2), supervisor, tipo_registro (pago_horas), comentario, deudas_pagadas (M2M a DeudaCorporativa), permisos_pagados (M2M a PermisoEspecial), creditos_consumidos (M2M a CreditoHoras por	pdh_exp_fecha_idx, pdh_solicitud_idx, pdh_fecha_idx (:47-50)	clean() exige $0 < \text{horas} \leq 24$ (:53-58). Las horas del PDH son la suma de las deudas que salda (:22-24)

	ConsumoCreditoHoras, :37-40)		
PermisoEspecial	empleado, tipo (5 valores), es_permanente, fecha_inicio, fecha_fin, dias_semana (CSV), tiempo, especificacion, cubre (SET_NULL), motivo, estado, pagado, fecha_pago, supervisor, comentario_supervisor, jornada_trabaja, fecha_compensacion, snapshot_turnos_previos, fecha_aprobacion (DateTimeField, null=True blank=True, :94-97) y el mismo bloque de cancelación consensuada que SolicitudCambio —cancelacion_estado, cancelacion_solicitada_por/_en, cancelacion_respondida_por/_en, cancelacion_motivo— (:114-134)	perm_esp_emp_estado_idx, perm_esp_estado_idx, perm_esp_fecha_idx (:173-175)	El tipo MEDIA_JORNADA_TEMPORADA parte el día completo de temporada en dos y no genera deuda (tiempo=0, :98-107). horas_totales() multiplica por las ocurrencias si es permanente (:149-166). clean() valida el rango (:178-183). fecha_aprobacion sella el instante en que el permiso pasó a APROBADO y es el equivalente de SolicitudCambio.fecha_resolucion: de él cuelga el plazo de 24 h para pedir la cancelación. No sirve actualizado_en, que es auto_now (:137) y lo reiniciaría cualquier guardado posterior (permisos/models.py:96-104). En la cancelación consensuada la contraparte no es un receptor —el permiso no tiene—, sino el supervisor: es quien lo aprobó y quien responde por la cobertura del día (permisos/models.py:121-125)
DeudaPermiso Mes	permiso, explorador (desnormalizado a propósito, :263-265), anio, mes, minutos_generados, minutos_pagados, ocurrencias, estado (los mismos cinco valores que DeudaCorporativa , :249-258), fecha_pago,	ordering = ['anio', 'mes', 'permiso'] (permisos/models.py :290)	Un permiso × un mes: es la unidad que permite reclamar mes a mes y admitir pago parcial (ADR 011 (./03-arquitectura/adr/011-sancion-independiente-del-pago.md)). fecha_pago es la del pago que la saldó, no la del primer abono (:277-279).

	sancion_consumidora (:281), fecha_consumo (:284), historial		PermisoEspecial.pagado es un agregado derivado de estas filas: cualquier cosa que cambie sus estados tiene que rehacer el roll-up, o el permiso sigue figurando impagado en las pantallas que leen el agregado. condonada (:256) marca la deuda que perdonó el supervisor al levantar la sanción del mes
CreditoHoras	explorador, fecha_hecho (día en que trabajó de más), minutos_otorgados, minutos_consumidos (roll-up), motivo, otorgado_por, estado (activo / consumido / anulado, :373-377), historial	ordering = ['fecha_hecho', 'id'] (permisos/models.py:408), índice credito_exp_estado_idx (:410)	La dirección contraria de la deuda: lo que la corporación le debe AL explorador. Es una bolsa , no un ajuste a una deuda concreta: sobrevive al pago y el remanente queda para deudas futuras. El ordering no es cosmético — es lo que implementa el consumo FIFO , así que cambiarlo cambia qué crédito se gasta primero. minutos_pendientes (:414-416) nunca es negativo. ADR 014 (./03-arquitectura/adr/014-horas-a-favor-del-explorador.md). Ver § 7.1 y § 8.2
ConsumoCreditoHoras	pdh (CASCADE), credito (PROTECT), minutos	UniqueConstraint(pdh, credito) = consumo_credito_horas_unico (permisos/models.py:453)	Tabla intermedia con importe, copia estructural de `PagoDeudaPermisoMes` y por el mismo motivo: el consumo es parcial y puede repartirse entre varios créditos, así que sin el importe por fila borrar el PDH no sabría cuánto devolver a cada uno. PROTECT impide borrar un crédito que ya cubrió un pago

5.3 Migraciones

`solicitudes` tiene 36 archivos en `solicitudes/migrations/`. El orden de dependencias entre apps es obligatorio y no se puede invertir: `permisos/models.py:7` importa `SolicitudCambio` y `solicitudes/models.py:224,227` apunta a `'turnos.Turno'` en dos claves foráneas (por cadena, no por import), así que el orden es `empleados` → `turnos` → `solicitudes` → `permisos`.

Migraciones delicadas — las que tocan invariantes, no solo columnas:

- La que añade `Turno.activo_key` y la `UniqueConstraint` `turno_unico_activo_por_jornada` (`turnos/models.py:80-83`). Falla si la base ya tiene turnos activos duplicados; hay que limpiarlos antes.
- La que añade la `CheckConstraint` `turno_tipo_cambio_valido` (`turnos/models.py:90-94`).

Falla si existe algún `Turno.tipo_cambio` fuera de `TipoCambioTurno.TODOS`. El comando `actualizar_codigos_estrategia` normaliza el catálogo antes.

- La `UniqueConstraint(fecha, tipo)` de `DiaEspecial` (`turnos/models.py:148`): sin ella, el borrado por `año_planificacion` dejaba restos que se duplicaban al regenerar el año.
- `turnos/migrations/0016_alter_turno_sala_alter_turnoarchivo_sala.py` pone `null=True, blank=True`

en `Turno.sala` y `TurnoArchivo.sala` (:17-23). Es **relajar** una restricción, así que no puede fallar por datos existentes y no necesita limpieza previa. Motivo: la sala es **informativa** —dice en qué espacio tiene competencia el explorador, no condiciona el turno— y siendo `NOT NULL` hacía fallar la **aprobación de solicitudes** de cualquiera sin competencia cargada, porque `obtener_sala_explorador_fecha` lanzaba `ValidationError` (`turnos/models.py:56-63`).

Cuando no hay sala, la interfaz muestra **"Por asignar"** (`turnos/services/doblada_turno_service.py:39,76`). Cubierto por `turnos/tests/test_sala_opcional.py`.

- `solicitudes/migrations/0034_historicalsolicitudcambio_cancelacion_estado_and_more.py` y

`permisos/migrations/0008_historicalpermisoespecial_cancelacion_estado_and_more.py` añaden los seis campos de cancelación consensuada, **también a las tablas históricas** de `django-simple-history`. Son puramente aditivas (todos los campos admiten null o traen default ''), así que no fallan sobre datos existentes: una solicitud anterior queda con `cancelacion_estado = ''`, que es exactamente "nunca se pidió cancelar". La de `solicitudes` depende de `0033_solicitudcambio_fecha_cancelacion` (:11).

- `permisos/migrations/0009_historicalpermisoespecial_fecha_aprobacion_and_more.py` añade

`PermisoEspecial.fecha_aprobacion`, también sobre la tabla histórica, y depende de la `0008` (:9). Aditiva y con `null=True`: los permisos aprobados antes de existir el campo lo tienen vacío,

y por eso el cálculo del plazo usa el respaldo `fecha_aprobacion` or `actualizado_en` (`permisos/views.py:443`). Ya aplicada.

- `permisos/migrations/0011_alter_deudapermisomes_estado_and_more.py` y

`solicitudes/migrations/0039_alter_deudacorporativa_estado_and_more.py` añaden el estado 'condonada', y `solicitudes/migrations/0040_alter_notificacion_tipo.py` añade el tipo de

- `permisos/migrations/0012_creditohoras_consumocreditohoras_and_more.py` crea `CreditoHoras`,

`ConsumoCreditoHoras`, su histórico y el M2M PDH. `creditos_consumidos`.

`permisos/migrations/0013_alter_creditohoras_minutos_otorgados_and_more.py` mueve los topes de 0 a 24 h desde `Model.clean()` a **validadores del campo**: es **no-op en SQL** (`sqlmigrate` lo confirma), solo cambia metadatos de validación. El porqué está en § 16.2. notificación 'sancion_levantada'. **No son delicadas, y se listan aquí para que nadie las audite dos veces**: son solo `AlterField` sobre `estado`, su tabla histórica y `Notificacion.tipo`, y Django trata `choices` como un atributo que no vive en la base (`non_db_attrs`), así que no emiten `ALTER TABLE` ni pueden fallar sobre datos existentes. Sin backfill: ninguna fila anterior es 'condonada'.

En producción no hay backfills previstos: la base arranca limpia (§ 1.3).

5.4 Consultas críticas y su coste

Consulta	Dónde	Protección aplicada
Guardia LIFO al cancelar : todas las solicitudes aprobadas posteriores de las dos personas	<code>solicitudes/use_cases/cancelar_solicitud.py:423-458</code>	<code>select_related('doblada', 'doblada_permanente', 'cambio_permanente') (:447)</code> evita un N+1 al calcular los pares afectados de cada candidata
Candidatos de cobertura : jornada	<code>solicitudes/views/api_fin_semana.py: 237-241</code>	Se cargan en una consulta a <code>AsignarJornadaExplorador</code> y se indexan en el dict <code>bases</code> ; sin eso serían N consultas

base de todos los empleados activos		
"Mis Turnos" por mes	<code>turnos/api/views/turnos_mes.py</code>	Cacheado como <code>turnos_mes_<emp>_<año>_<mes></code> durante una hora (<code>CACHE_TTL_LONG = 3600</code> , <code>core/services/cache_service.py:17</code> ; escritura en <code>turnos/api/views/turnos_mes.py:213-214</code>) e invalidado con <code>CacheService.invalidar_cache_turnos_empleado</code> (<code>core/services/cache_service.py:132</code> , ver <code>config/settings.py:288-296</code>)
El «con quién» de cada día del mes	<code>AcuerdoPorDiaService.en_rango</code> (<code>solicitudes/services/acuerdo_por_dia_service.py:81</code>)	Una consulta de candidatas para todo el mes, con <code>select_related</code> de tipo, las dos personas y los dos detalles (<code>:99-117</code>), más una de <code>Turno</code> para la guarda de realidad (<code>:124-130</code>). Sustituye a las ocho consultas por tipo que hacía la vista
Barrido del outbox	<code>procesar_pendientes</code> (<code>solicitudes/services/email_outbox_service.py:280-296</code>)	Índice <code>outbox_estado_disp_idx</code> sobre (<code>estado, disponible_en</code>) (<code>solicitudes/models.py:110</code>). Selecciona <code>ids</code> con <code>values_list</code> y un <code>[:limite]</code> (50 por defecto) y los pasa a <code>enviar_lote</code> : no instancia modelos y el barrido entero va por una conexión SMTP (<code>:289-295</code>)
Matriz empleado × día de CT permanente	<code>solicitudes/services/cambios_permanentes_helper.py</code>	Precarga en lote, introducida en el commit <code>00558d8</code>

⚠ La caché es **compartida obligatoriamente** en producción: con varios workers de Gunicorn, `LocMemCache` invalidaría solo el proceso que atendió la petición y el resto seguiría sirviendo el mes viejo hasta una hora (`config/settings.py:279-296`).

6. Endpoints y URLs

Enrutado raíz en `config/urls.py`: `health/`, `health/ready/`, `admin/`, `'` (login), `dashboard/`, `empleados/`, `turnos/`, `permisos/` y `solicitudes/` con namespace (`config/urls.py:24-38`). `__debug__` solo existe con `DEBUG=True` (`config/urls.py:38-44`).

Todas las respuestas JSON se construyen con `json_ok` / `json_error` de `core/utils/json_responses.py`; `json_error` acepta `status` y un `code` textual que el frontend usa para distinguir el motivo.

6.1 Tabla de rutas

Raíz y core

Método	Ruta	Vista	Permiso	Respuesta
GET	<code>/health/</code>	<code>core.health.health</code> (<code>config/urls.py:27</code>)	ninguno	Estado, sin tocar la base
GET	<code>/health/ready/</code>	<code>core.health.readiness</code> (<code>config/urls.py:27</code>)	ninguno	Estado verificando la conexión
GET/POST	<code>/</code>	<code>LoginFormView</code> (<code>core/login/urls.py:5</code>)	ninguno	Formulario / sesión
POST	<code>/logout/</code>	<code>LogoutUserView</code> (<code>core/login/urls.py:6</code>)	sesión	Redirección
GET	<code>/dashboard/</code>	<code>DashboardView</code> (<code>core/dashboard/urls.py:5</code>)	sesión	Portada
—	<code>/admin/</code>	Django admin (<code>config/urls.py:30</code>)	<code>is_staff</code>	—

No existe ruta de recuperación de contraseña. `core/login/urls.py` declara exactamente dos rutas, `login` y `logout` (`core/login/urls.py:4-7`), y ninguna vista de reseteo de `django.contrib.auth` está incluida en `config/urls.py`. El desbloqueo lo hace un administrador. Ver § 16.

solicitudes/ (solicitudes/urls.py)

Grupo	Rutas	Permiso
Panel	<code>'</code> , <code>notificaciones-solicitudes/</code> (legacy, misma	sesión

	vista)	
Notificaciones	notificaciones/, notificaciones/<id>/marcar-leida/	sesión
Mis cosas	mis-solicitudes/, mis-favores/, solicitudes-pendientes/	sesión
Gestión	gestion-solicitudes/ y sus acciones reenviar/, cancelar/, eliminar/	supervisor
Reprogramación	reprogramaciones/, .../registrar/<id>/, .../<id>/programar/, .../<id>/cancelar/	supervisor
Cierre semanal	cierre-solicitudes/	supervisor
Aprobación web	aprobar-solicitud/<id>/, rechazar-solicitud/<id>/, aprobar-solicitud-receptor/<id>/, rechazar-solicitud-receptor/<id>/, aprobar-solicitud-ambos/<id>/, cancelar-solicitud/<id>/	sesión + rol comprobado en el caso de uso
Cancelación consensuada	cancelar-solicitud/<id>/ (el solicitante pide) y cancelar-solicitud/<id>/responder/ (el receptor decide)	sesión; el caso de uso exige ser el solicitante y el receptor respectivamente
Aprobación por correo	aprobar-email/<id>/<token>/, rechazar-email/<id>/<token>/, aprobar-receptor-email/<id>/<token>/, rechazar-receptor-email/<id>/<token>/	token HMAC, sin sesión
Creación	procesar-solicitud/ (POST único de los seis formularios), cambio-turno/, cambio-turno/solicitar/<tipo_id>/	sesión
Consulta del formulario	obtener-empleados-disponibles/, obtener-turno-	sesión

	explorador/, obtener-jornadas-rango/, obtener-cambio-aprobado/, descansos-semana-usuario/, cambio-descanso-fines/, alternancia-mes/, dfds-companeros/, dobladas-semana/, cobertura-candidatos/, obtener-detalle-solicitud/<id>/	
Previsualización	previsualizar-ct-permanente/, previsualizar-doblada-permanente/, dias-disponibles-doblada-permanente/	sesión
Doblada	obtener-exploradores-doblada/, verificar-doblada-existente/, exploradores-con-doblada/, sabado-pago-comprometido/, obtener-fechas-descanso/, verificar-coincidencia-jornadas/	sesión
Administración	tipos-solicitud/ + create/, edit/<pk>/, delete/<pk>/	supervisor

Las rutas de `permisos-detalle/` están comentadas en el archivo y **no existen** en runtime (`solicitudes/urls.py`, bloque "ADMINISTRACIÓN").

turnos/, empleados/, permisos/

Incluidas desde `config/urls.py:34-36`. Concentran los CRUD de calendario (días especiales, descansos de semana, asignación especial anual, apertura de año), de personas (salas, competencias, restricciones, sanciones, PDH, horas a favor) y de permisos especiales. La referencia de la Las rutas del crédito son `/empleados/credito-horas/ (credito_list)`, `.../create/ (credito_create)` y `.../anular/<pk>/ (credito_anular)` — `empleados/urls.py:94-97`. **No hay ruta de borrado a propósito:** un crédito es un reconocimiento de deuda hacia el explorador, y borrarlo haría desaparecer el rastro de quién lo concedió; se **anula**, igual que una sanción se levanta en vez de eliminarse. fuente de verdad del calendario está en [05-referencia/turnos/](#) (`./05-referencia/turnos/`).

6.2 Fichas de endpoints no triviales

POST /solicitudes/procesar-solicitud/

22. **Método y ruta.** `POST /solicitudes/procesar-solicitud/` (`solicitudes/urls.py`, nombre `procesar_solicitud`).
23. **Vista y archivo.** `ProcesarSolicitudView` (`solicitudes/views/procesar_solicitud.py:15`).
24. **Para qué existe.** Es la **única** puerta de creación para los seis tipos de solicitud. El tipo se elige por `tipo_solicitud_id`, no por la ruta.
25. **Permiso.** `LoginRequiredMixin` (`solicitudes/views/procesar_solicitud.py:15`). No exige rol: cualquier explorador autenticado crea solicitudes.
26. **Entrada.** `tipo_solicitud_id` (int, obligatorio) más el `POST` completo del formulario, que varía por tipo; lo parsea `solicitudes/services/solicitud_request_parser.py`.
27. **Validaciones, en orden.** (a) `tipo_solicitud_id` presente, si no 400 (:20-21); (b) el tipo existe, si no 400 (:23-26); y dentro de `SolicitudOrchestrator.procesar` (`solicitudes/services/solicitud_orchestrator.py:554`): dedupe del `POST` idéntico (:573 → :60-91), sanción del solicitante (:578 → :175-181), cierre semanal sobre las fechas objetivo (:598 → :156-172), restricción médica —advierte, no bloquea— (:611 → :312), sanción del compañero (:626 → :184-198) y por último `SolicitudFactory.validar_solicitud` y `crear_solicitud`.
28. **Respuesta 201.** El orquestador **ya no devuelve `JsonResponse`**: devuelve un `ResultadoSolicitud` (`solicitudes/services/resultado.py:32`) y la vista lo traduce con `json_desde_resultado` (`solicitudes/views/resultado_http.py:12`, invocado en `solicitudes/views/procesar_solicitud.py:30`). El **cuerpo JSON no cambió**: como `payload()` reproduce exactamente lo que emitían `json_ok/json_error` (`solicitudes/services/resultado.py:81-93`). Sigue siendo `{'success': true, 'message': ..., 'solicitud_id': ...}`, y la cobertura con dos compañeros devuelve `{'success': true, 'message': ..., 'solicitud_ids': [id, id]}` (`solicitudes/services/solicitud_orchestrator.py:283-287`).
29. **Errores.** 400 `missing_fields` (falta `tipo_solicitud_id`), 400 `invalid_type` (tipo inexistente), 400 `cierre_semanal`, 400 `creation_failed`, 403 `sancionado`, 403 `sancionado_receptor`, 409 `duplicate_request` (reenvío idéntico dentro de la ventana de 30 s), 500 `internal_error`.
30. **Efectos secundarios.** Crea `SolicitudCambio` en estado `pendiente` más su fila de detalle, y encola `EmailOutbox` en la misma transacción. **No toca `Turno`**: el calendario solo cambia al aprobar.
31. **Servicios que invoca.** `CrearSolicitudUseCase` → `SolicitudOrchestrator` → `SolicitudFactory` → la `Strategy` del tipo → los `validators`.
32. **Tests.** `solicitudes/tests/` (43 archivos), entre ellos `test_politica_temporada.py`, `test_cobertura_dos_companeros.py` y `test_cambio_doblada_candidatos.py`.
33. **Gotchas.** El dedupe usa una huella SHA-256 del `POST` completo menos el CSRF, con TTL en la constante `_DEDUPE_TTL_SEGUNDOS = 30 s` (`solicitud_orchestrator.py:34`): dos solicitudes distintas del mismo tipo no chocan, solo el reenvío idéntico (:60-90). **El TTL tiene que durar**

más que el request que protege; estuvo en 10 s y los flujos multi-compañero tardaban 13-19 s, así que el candado caducaba a mitad de la operación y un segundo POST idéntico pasaba limpio, creando el acuerdo por duplicado (:69-77). Y el **cierre semanal falla abierto**: si la comprobación revienta, la solicitud pasa y solo queda un **CRITICAL** en el log (:169-171).

POST /solicitudes/cancelar-solicitud/<solicitud_id>/

34. **Método y ruta.** `POST /solicitudes/cancelar-solicitud/<int:solicitud_id>/` (`solicitudes/urls.py:100`, nombre `cancelar_solicitud`).
35. **Vista y archivo.** `CancelarSolicitudView` (`solicitudes/views/aprobacion_views.py:68`).
36. **Para qué existe.** Acción de cancelar del **solicitante**. Hace dos cosas distintas según el estado: si la solicitud está **pendiente** la retira en el acto; si está **aprobada** **no cancela nada**, registra la petición y deja la decisión al receptor (`solicitudes/use_cases/cancelar_solicitud.py:39-97`).
37. **Permiso.** `LoginRequiredMixin` y el usuario debe tener **empleado** (:80-81); el caso de uso comprueba que sea el solicitante (`cancelar_solicitud.py:70-71`).
38. **Entrada.** `solicitud_id` en la URL; `motivo` (str, **obligatorio**, no vacío tras `strip()`) en el cuerpo `POST` (`aprobacion_views.py:82`, vía `exigir_texto_json` de `core/utils/comentarios.py:44`).
39. **Validaciones, en orden.** (a0) `motivo` con contenido, si no **400 `comentario\_requerido`** antes de tocar nada (`aprobacion_views.py:82-84`); (a) la solicitud es propia (`cancelar_solicitud.py:70`); (b) `bloquear_partes` toma el lock de las dos personas (:79); (c) si está **pendiente**, transiciona a **cancelada** y termina (:81-88); (d) si está **aprobada** entra en `_pedir_cancelacion` (:227): no hay ya una petición pendiente (:237) ni una terminal (:243), hay receptor registrado (:246), hay `fecha_resolucion` (:250), no han pasado más de **24 h** desde ella (:254, `VENTANA_PEDIR_CANCELACION_HORAS`), ninguna de las fechas está ya cumplida (:263), y pasan la **guardia LIFO** y la **de integridad** (:267).
40. **Respuesta 200.** Dos formas. Pendiente retirada: `json_ok({'message': msg, 'cancelada': True})` (`aprobacion_views.py:110`). Aprobada: `json_ok({'message': msg, 'cancelada': False, 'cancelacion_pendiente': True})` (:114). El frontend **debe** mirar **cancelada**: en el segundo caso el cambio sigue vigente.
41. **Errores.** **403 forbidden** (sin empleado o no es propia), **400 cambio_mas_reciente** (LIFO), **400 conflicto_integridad**, **400 ventana_expirada** (pasaron las 24 h desde la aprobación), **400 cancelacion_ya_pedida**, **400 cancelacion_cerrada** (rechazada o caducada: el cambio quedó firme), **400 invalid_state**, **500 internal_error** (`aprobacion_views.py:89-101,115-117`), y **400 comentario_requerido** con el texto *Escribe el motivo de la cancelación*. (`MSG_MOTIVO`, `core/utils/comentarios.py:25`) si el motivo llega vacío. El mapeo del resto se hace **inspeccionando el texto del mensaje**, no un código; `comentario_requerido` es el único que sale de una comprobación explícita.
42. **Efectos secundarios.** Si era pendiente: transición a **cancelada**, `fecha_resolucion` y `fecha_cancelacion` al mismo instante, notificación de cancelación (`cancelar_solicitud.py:81-88`, `aprobacion_views.py:107-109`). Si era aprobada: escribe **solo** los cuatro campos de la petición (`cancelacion_estado`, `..._solicitada_por`,

..._solicitada_en, ..._motivo, :271-274) y notifica al receptor. **No toca turnos ni deudas ni el `estado`.** En ambos casos invalida los contadores en caché (`aprobacion_views.py:117`, `_invalidar_contadores` en :184).

43. **Servicios.** `CancelarSolicitudUseCase.execute` → `_pedir_cancelacion`, `bloqueo_lifo` (:423), `bloqueo_integridad` (:460); después `CacheService.delete_many` y `NotificacionService.crear_notificacion_cancelacion` o `crear_notificacion_peticion_cancelacion` (`solicitudes/services/notificacion_service.py:631`).
44. **Tests.** `solicitudes/tests/test_cancelacion_lifo.py`:
`test_pedir_cancelacion_no_cancela_nada` (:150),
`test_no_se_puede_pedir_pasado_el_plazo_del_solicitante` (:263) y
`test_pedir_cancelacion_sin_motivo_se_rechaza` (:167), que hace el POST sin `motivo` y exige 400 con `code == 'comentario_requerido'` sin que se abra la petición.
45. **Gotchas.** (a) Un 200 **no** significa cancelada: hay que leer `cancelada`. (b) El mapeo de error por subcadena (`'más reciente' in msg`, `'quedó firme' in msg`) acopla la vista al texto de los mensajes: cambiar la redacción de un mensaje del caso de uso cambia el código HTTP que ve el frontend. (c) `VENTANA_CANCELACION_MINUTOS` **ya no existe en el proyecto**: era código muerto en `cancelar_solicitud.py` desde el ADR 009 y se borró junto con el bloqueo temporal del día ([ADR 010](#) (`./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md`)). Los plazos vivos son `VENTANA_PEDIR_CANCELACION_HORAS` y `VENTANA_RESPONDER_CANCELACION_HORAS` (`core/constants.py:201, 206`).

POST /solicitudes/cancelar-solicitud/<solicitud_id>/responder/

46. **Método y ruta.** `POST /solicitudes/cancelar-solicitud/<int:solicitud_id>/responder/` (`solicitudes/urls.py:102`, nombre `responder_cancelacion`).
47. **Vista y archivo.** `ResponderCancelacionView` (`solicitudes/views/aprobacion_views.py:120`).
48. **Para qué existe.** Es la **única** puerta por la que una solicitud aprobada llega a `cancelada` sin pasar por Gestión. Aprobar revierte los turnos; rechazar deja el cambio firme y cierra el asunto.
49. **Permiso.** `LoginRequiredMixin + empleado` (:131-132); el caso de uso exige ser **el receptor** de esa solicitud, comparando contra `explorador_receptor_id` (`cancelar_solicitud.py:312-313`).
50. **Entrada.** `solicitud_id` en la URL; `accion` (str, obligatorio, `aprobar` o `rechazar`, :134-137); `comentario_respuesta` (str, **obligatorio**, :139-141).
51. **Validaciones, en orden.** (a) `accion` válida (:135-137); (a2) `comentario_respuesta` con contenido, si no **400 `comentario\_requerido`** (:139-141); (b) el que responde es el receptor (`cancelar_solicitud.py:312`); (c) `cancelacion_estado == 'pendiente'` —si es terminal se explica por qué ya no se puede (:315-318)—; (d) la solicitud sigue en `aprobada` (:320); (e) **caducidad**: no han pasado más de 24 h desde `cancelacion_solicitada_en` (:331,

- `_caducar_si_vencida` en :395); (f) `bloquear_partes` toma el lock (:337); y **solo si aprueba**: (g) guardia LIFO y de integridad **otra vez** (:359), (h) ninguna fecha ya cumplida (:363).
52. **Respuesta 200.** `json_ok({'message': msg, 'aprobada': <bool>})` (`aprobacion_views.py:165`).
53. **Errores.** 403 `forbidden` (sin empleado o no es el receptor), 400 `accion_invalida`, 400 `cambio_mas_reciente`, 400 `conflicto_integridad`, 400 `comentario_requerido` (comentario vacío, texto `MSG_COMENTARIO`), 400 `plazo_vencido` (la petición caducó), 400 `invalid_state`, 500 `internal_error` (:147-156,167-168).
54. **Efectos secundarios. Rechazo:** escribe `cancelacion_estado='rechazada'` + quién y cuándo, y añade la nota al motivo; nada más (`cancelar_solicitud.py:340-350`). **Aprobación:** `_revertir_por_tipo` (:646) restaura el snapshot y cancela deudas, se refrescan desde BD los FK `turno_origen/turno_destino` que quedaron colgando tras borrar los turnos materializados (:364-372), transición a `cancelada`, `fecha_cancelacion = ahora sin tocar `fecha_resolucion``, `cancelacion_estado='aprobada'` y anotación en `comentario` (:379-389). En ambos casos: invalidación de contadores y notificación al solicitante (`aprobacion_views.py:171-177`).
55. **Servicios.** `CancelarSolicitudUseCase.responder_cancelacion` (:287) → `bloquear_partes`, `bloqueo_lifo` (:423), `bloqueo_integridad` (:460), `_revertir_por_tipo` (:646) → servicio de aplicación del tipo → reconciliación; después `CacheService.delete_many` y `NotificacionService.crear_notificacion_respuesta_cancelacion` (`notificacion_service.py:665`).
56. **Tests.** `solicitudes/tests/test_cancelacion_lifo.py`:
`test_solo_el_receptor_puede_responder` (:200),
`test_receptor_rechaza_y_el_cambio_queda_firme` (:209),
`test_receptor_aprueba_y_se_revierde` (:229),
`test_la_peticion_caduca_si_el_receptor_no_responde` (:240) y
`test_responder_cancelacion_sin_comentario_se_rechaza` (:185).
57. **Gotchas.** (a) Las guardias pueden **fallar aquí aunque pasaran al pedir**: en 24 h puede entrar otro cambio sobre los mismos días, y entonces el receptor recibe el error de algo que él no provocó. (b) La caducidad es **perezosa**: nada la marca en segundo plano, se reconoce al responder (`_caducar_si_vencida`) o se filtra al listar (`solicitudes/views/notificaciones_listas.py:173-197`). Una petición vencida no tiene efecto pendiente que aplicar, así que basta con reconocerla cuando alguien la mira. (c) Los tres estados terminales son **definitivos**: no se vuelve a `pendiente` ni se puede volver a pedir; la salida es un cambio nuevo o que un supervisor cancele desde Gestión (`cancelar_solicitud.py:415`).

POST /permisos/permisos-especiales/media-jornada/<pk>/cancelar/ y .../cancelar/responder/

58. **Método y ruta.** Los dos son `POST`; nombres `permisos_media_jornada_cancelar` (`permisos/urls.py:25-26`) y `permisos_media_jornada_cancelar_responder` (:28-30).

59. **Vista y archivo.** `PermisoMediaJornadaCancelView` (`permisos/views.py:453`) y `PermisoMediaJornadaCancelResponderView` (`:521`).
60. **Para qué existe.** El mismo acuerdo que en solicitudes, aplicado al permiso de media jornada de temporada. Como el permiso **no tiene receptor**, la contraparte es el **supervisor**: es quien aprobó el permiso y quien responde por la cobertura del día (`permisos/models.py:121-125`).
61. **Permiso.** Cancelar: dueño del permiso o supervisor (`permisos/views.py:415-417`); sobre el permiso **propio** siempre se PIDE, aunque quien pulse tenga rol de supervisor (`:440`). Responder: `_puede_responder_cancelacion(request.user, emp, permiso)` (`:538`, definida en `:599-624`), que exige rol de supervisor (`:613`), prohíbe que el dueño se responda a sí mismo (`:615`) y exige ser **el** supervisor del permiso (`:618-623`).
62. **Entrada.** `pk` en la URL; **motivo (obligatorio)** al cancelar o pedir la cancelación, validado nada más entrar al `post` y pasado como parámetro a `_pedir_cancelacion` (`:437-439, :457, :465`); al responder, `accion` —`aprobar` aprueba, cualquier otro valor rechaza (`:576`)— y `comentario (obligatorio, :571-573)`. El helper es `leer_texto(request, campo)` de `core/utils/comentarios.py:34`, compartido con solicitudes: devuelve el texto limpio o `None` y **no decide la respuesta**, que aquí la construye `_fallo` (`permisos/views.py:297`).
63. **Validaciones, en orden.** Cancelar: (a) **motivo** no vacío, si no `messages.error('Escribe el motivo de la cancelación.')` y redirección (`:420-423`); (b) el permiso no está ya `CANCELADO/RECHAZADO` (`:425`); (c) si está `PENDIENTE` se cancela en el acto (`:430-435`); (d) si es el dueño —tenga el rol que tenga— va a `_pedir_cancelacion` (`:440-441`, definida en `:449`), que comprueba petición pendiente (`:452`), terminal (`:456`), plazo de 24 h contra `fecha_aprobacion` or `actualizado_en` (`:462`) y `puede_revertir_limpio` (`:474`). Responder: autorización (`:538`), hay petición pendiente (`:543`), plazo de 24 h desde `cancelacion_solicitada_en` (`:549-550`, marca `CADUCADA` en `:551`) y **por último** el `comentario` (`:557-560`) —deliberadamente después de la caducidad, para no pedir un texto que ya no serviría de nada.
64. **Respuesta.** Ninguna de las dos devuelve JSON: son vistas de formulario que dejan un `messages` y hacen `redirect('permisos_especiales_list')`.
65. **Errores.** Todos por `messages.error` + redirección, no por código HTTP.
66. **Efectos secundarios.** Pedir: escribe los cuatro campos de la petición —`cancelacion_motivo` con el texto obligatorio— y notifica al supervisor (`:449-495`). Rechazar la cancelación: además de marcar `RECHAZADA`, **concatena el motivo a ``comentario_supervisor``** (`"Cancelación rechazada por {quien}. Motivo: ..."`, `:569-573`), cosa que antes no se hacía y dejaba al explorador sin saber por qué. Aprobar: `PermisoMediaJornadaService.revertir` (`:511`), `estado='CANCELADO'`, nota en `comentario_supervisor` con el motivo, invalidación de la caché de turnos y notificación al dueño (`:498-518, :583-587`). Aprobar el permiso —no la cancelación— exige también `comentario` (`PermisoEspecialAprobarView.post, :303-306`) y sella `fecha_aprobacion` en las dos puertas: `:314` y `PermisoEspecialResolverEmailView.get` (`:351`).
67. **Servicios.** `PermisoMediaJornadaService.puede_revertir_limpio` y `.revertir`; `_notificar_peticion_cancelacion_permiso` (`:607`) y

`_notificar_respuesta_cancelacion_permiso (:630)`, definidas a nivel de módulo en `permisos/views.py` en vez de en `NotificacionService`. Autorización y aviso resuelven la contraparte con el **mismo** helper, `_supervisor_del_permiso (:561)`, usado en `:588` y `:611`: decide y avisa la misma persona.

68. **Tests.** `permisos/tests/test_cancelacion_permiso_consenso.py`, cinco casos:
`test_pedir_cancelacion_sin_motivo_no_abre_la_peticion (:94)`,
`test_el_plazo_no_se_reinicia_al_guardar_el_permiso (:110)`,
`test_el_plazo_corre_desde_la_aprobacion (:133)`,
`test_un_supervisor_no_cancela_directo_su_propio_permiso (:143)`,
`test_el_dueno_no_puede_responderse_a_si_mismo (:158)` y
`test_solo_responde_el_supervisor_de_ese_explorador (:169)`.
69. **Gotchas.** (a) El plazo se mide contra `permiso.fecha_aprobacion` or `permiso.actualizado_en (:443)`: el respaldo es **deliberado y permanente**, no provisional — sin él, los permisos aprobados antes de la migración `0009` quedarían incancelables para siempre. (b) Si el permiso **no tiene supervisor resoluble** (`permiso.supervisor` y `empleado.supervisor` vacíos), puede responder **cualquier** supervisor (`:588-591`): es a propósito, porque si no la petición sería incontestable y caducaría siempre. (c) **Doble contrato de respuesta.** `_fallo (:297)` mira `_quiere_json (:289)`, cabeceras `X-Requested-With` y `Accept`: quien pide JSON recibe el código HTTP real y un `code` — `403 forbidden`, `400 comentario_requerido`, `409 invalid_state`, `409 plazo_vencido`—, y el navegador conserva su `messages` + redirección `302`, que es lo que espera al enviar un formulario. Antes **todo** desenlace era el mismo `302` a la lista y un cliente automatizado no distinguía éxito de rechazo. (d) El texto obligatorio lo pide el front con `static/js/permisos/comentario_obligatorio.js`, que intercepta los formularios con `data-comentario-label` y delega el diálogo en el helper común `static/js/core/comentario_obligatorio.js` (§ 17); si ese script no carga, el POST llega sin campo y la vista redirige con el mensaje de error, así que **nunca** se pierde la acción en silencio. (e) La lista necesita `mi_empleado_id` en el contexto (`:153`) para distinguir "mi permiso" del "de otro": la plantilla condiciona con `es_supervisor and p.empleado_id != mi_empleado_id`, porque el rol por sí solo ya no basta.

POST /solicitudes/aprobar-solicitud-ambos/<solicitud_id>/

70. **Método y ruta.** `POST /solicitudes/aprobar-solicitud-ambos/<int:solicitud_id>/`.
71. **Vista y archivo.** `AprobarSolicitudAmbosView`
(`solicitudes/views/aprobacion_views.py:199`).
72. **Para qué existe.** Cuando la misma persona es a la vez compañero receptor y supervisor, aprueba ambos roles en un solo paso.
73. **Permiso.** `LoginRequiredMixin` y `empleado` asociado (`:202-203`); el caso de uso valida que tenga efectivamente los dos roles sobre esa solicitud.
74. **Entrada.** `solicitud_id` en la URL; `comentario_respuesta` (str, **obligatorio**) en el POST (`:204`).

75. **Validaciones.** Primero el comentario: vacío corta con `400 `comentario_requerido` (:192-194)`. El resto, delegadas a `AprobarAmbosRolesUseCase` (`solicitudes/use_cases/aprobar_solicitud.py:39`). Al completarse ambas aprobaciones, `SolicitudAprobacionService` llama `SolicitudFactory.revalidar_para_aprobar` antes de aplicar nada (`solicitudes/services/solicitud_aprobacion_service.py:124`).
76. **Respuesta 200.** `json_ok({'message': msg}) (:213)`.
77. **Errores.** `400 comentario_requerido` (comentario vacío), `403 approval_error` si el mensaje contiene 'permisos', `400 approval_error` en el resto, `500 internal_error` (:192-204).
78. **Efectos secundarios.** Marca ambas aprobaciones con su fecha, transiciona a `aprobada`, fija `fecha_resolucion`, captura el snapshot previo, materializa los turnos, genera deudas y encola los correos.
79. **Servicios.** `AprobarAmbosRolesUseCase` → `SolicitudAprobacionService` → `SolicitudFactory.revalidar_para_aprobar` (`solicitudes/services/solicitud_factory.py:391,432`) → servicio de aplicación del tipo.
80. **Tests.** Suite de aprobación en `solicitudes/tests/`.
81. **Gotchas.** La revalidación puede fallar aunque la solicitud fuera legal al crearse: entre la creación y la aprobación el mundo cambió. Ese rechazo tardío es intencional, no un fallo. El mensaje que se devuelve **no** es el de la validación en crudo: se reencuadra en la costura (`solicitud_aprobacion_service.py:134-138`, ver § 7.4).

GET /solicitudes/cobertura-candidatos/

82. **Método y ruta.** `GET /solicitudes/cobertura-candidatos/`.
83. **Vista y archivo.** `CoberturaCandidatosView` (`solicitudes/views/api_fin_semana.py:192`).
84. **Para qué existe.** Poblar el desplegable de compañeros de la opción "Que me cubran mi día" de CAMBIO DESCANSO en temporada, con el motivo por el que cada uno puede o no.
85. **Permiso.** `LoginRequiredMixin` (:182). Un usuario sin `empleado` recibe lista vacía (:202-204).
86. **Entrada.** `fecha_trabajo` (ISO, obligatoria), `opcion` (AM|PM, obligatoria), `fecha_pago` (ISO, opcional) (:206-218).
87. **Validaciones, en orden.** (a) parámetros presentes y `opcion` en AM/PM, si no `400` (:216-218); y por cada candidato del grupo contrario (:243): (b) ya trabaja la jornada `opcion` ese día → descartado (:249-251); (c) ya tiene el día completo AM+PM → descartado (:252-254); (d) el día está comprometido en otra solicitud → descartado (:255-257); (e) no trabaja `opcion` el día de pago → descartado, porque no habría jornada que devolverle (:258-266).
88. **Respuesta 200.** `json_ok({'candidatos': [...]}),` con disponibilidad, motivo y la jornada del candidato el día que cubre y el día de pago (:192-193,204).
89. **Errores.** `400 bad_request`, con el texto `Parámetros inválidos (fecha_trabajo, opcion=AM|PM)` (:217-218).
90. **Efectos secundarios.** Ninguno: es solo lectura.
91. **Servicios.** `TurnoService.dia_comprometido_por_solicitud` y `CambioDescansoAplicacionService._jornadas_actuales` (:245-248).
92. **Tests.** `solicitudes/tests/test_cambio_doblada_candidatos.py,`
`solicitudes/tests/test_cobertura_dos_companeros.py.`

93. **Gotchas.** El filtro (e) existe porque el desplegable y la validación de envío miraban fechas distintas: el selector solo el día de cesión y el backend además el día de pago, así que ofrecía compañeros que el envío tumbaba (:258-263, commit 9c429e3). Es el patrón 37 de `PROTECTION_PATTERNS.md`. Si tocas este endpoint, toca `_validar_semana_cobertura` en el mismo commit. **Este endpoint no sabe nada del modo «2 compañeros»:** evalúa una jornada a la vez, así que la regla del día de pago libre no le corresponde y vive en `_pago_dos_debe_estar_libre` (§ 8.3). El campo `mi_jornada_pago` de la respuesta es lo que el formulario usa para adelantarla.

GET /solicitudes/dfds-companeros/

94. **Método y ruta.** `GET /solicitudes/dfds-companeros/` (`solicitudes/urls.py:114`, nombre `dfds_companeros`).
95. **Vista y archivo.** `DFDSCompanerosView` (`solicitudes/views/api_fin_semana.py:16`).
96. **Para qué existe.** Llena el selector de compañeros de los **dos** formularios de fin de semana: D FDS (cesión) y CAMBIO DESCANSO en su modalidad de finde (intercambio). Devuelve a **todos** los candidatos, disponibles o no, con el motivo: no se esconde a nadie, se explica por qué no puede (`solicitudes/views/api_fin_semana.py:18-20`).
97. **Permiso.** `LoginRequiredMixin`; sin `request.user.Empleado` responde `{'companeros': []}` (:38-40).
98. **Entrada.** `fecha` (str YYYY-MM-DD, obligatorio, el día que cedes) y `tipo_solicitud_id` (int, opcional; por defecto D FDS, :56).
99. **Validaciones, en orden.** (a) `fecha` parseable, si no `400 'bad_request'` (:42-44); (b) la fecha es sábado o domingo, si no `400 'no_finde'` (:45-46); (c) la estrategia del tipo implementa `'EstrategiaFinDeSemana'`, si no `400 'tipo_sin_selector_finde'` (:65-69).
100. **Respuesta 200.** `{'success': true, 'companeros': [{'id', 'nombre', 'etiqueta', 'dia', 'dia_fecha', 'disponible', 'motivo'}], 'otro_dia': '<sábado|domingo>'}`, ordenada por disponibles primero y luego alfabética (:89-90).
101. **Errores.** `400 bad_request`, `400 no_finde`, `400 tipo_sin_selector_finde`.
102. **Efectos secundarios.** Ninguno: solo lee.
103. **Servicios que invoca.** `SolicitudFactory.get_strategy` y, sobre la estrategia, `disponibilidad_companero` y `etiqueta_companero` (:79,85), definidos en el mixin `solicitudes/services/strategies/estrategia_fin_de_semana.py:47`.
104. **Tests.** `solicitudes/tests/test_contrato_fin_de_semana.py`:
`test_un_tipo_de_finde_contesta` (:109) y
`test_un_tipo_que_no_es_de_finde_se_rechaza_en_vez_de_inventar` (:116).
105. **Gotchas.** La regla de disponibilidad **no es la misma en los dos formularios**: el intercambio (CAMBIO DESCANSO) exige que el compañero trabaje el **otro** día del finde para poder canjearlo; la cesión (D FDS) solo exige que tenga **libre** el día que recibe (`estrategia_fin_de_semana.py:64-66`). Y `etiqueta` solo viene rellena cuando `disponible` es `true` (:85).

GET /solicitudes/aprobar-email/<solicitud_id>/<token>/ y sus tres hermanas

106. **Método y ruta.** GET, cuatro rutas: `aprobar-email/` y `rechazar-email/` (supervisor), `aprobar-receptor-email/` y `rechazar-receptor-email/` (compañero), todas con `<int:solicitud_id>/<str:token>/`.
107. **Vista y archivo.** `AprobarSolicitudEmailView` (`solicitudes/views/aprobacion_email.py:83`), `RechazarSolicitudEmailView` (`:142`), `AprobarSolicitudReceptorEmailView` (`:201`) y `RechazarSolicitudReceptorEmailView` (`:253`). Rutas en `solicitudes/urls.py:98` y siguientes.
108. **Para qué existe.** Aprobar o rechazar desde el correo con un clic, sin iniciar sesión.
109. **Permiso. Ninguno de sesión.** La autorización es el `token` de la URL, verificado por `tokens_aprobacion.verificar` (`solicitudes/views/aprobacion_email.py:140,199,251,303`; el mismo módulo respalda `EmailService._verificar_token`, `solicitudes/services/email_service.py:528`).
110. **Entrada.** `solicitud_id` (int) y `token` (str), ambos obligatorios y en la URL.
111. **Validaciones, en orden.** (a) firma y caducidad: `signing.loads(token, salt='solicitudes.aprobacion-email', max_age=...)`, que devuelve `False` ante firma inválida o token caducado (`solicitudes/services/tokens_aprobacion.py:84-91`); (b) el token debe ser de **esta** solicitud, de **este** rol y de la persona que **hoy** ocupa ese rol (`:96-107`); (c) idempotencia: `_ya_resuelto_para()` corta si la solicitud ya está resuelta o si ese rol ya respondió (`solicitudes/views/aprobacion_email.py:58-73`, llamado en `:115,174,226,278`).
112. **Respuesta 200.** Página HTML `solicitudes/aprobacion_exitosa.html` con el detalle rico de la solicitud, renderizada por `_render_resultado` (`solicitudes/views/aprobacion_email.py:49-55`). No JSON.
113. **Errores.** Token inválido o caducado → página de error. Solicitud inexistente → `get_object_or_404`. Enlace ya usado → la misma página con `ya_procesada=True` y `sin` reenviar notificaciones (`:54`).
114. **Efectos secundarios.** Los mismos que la aprobación web. En el caso ya resuelto, ninguno.
115. **Servicios.** `solicitudes/services/tokens_aprobacion.py` y los casos de uso de aprobación.
116. **Tests.** `solicitudes/tests/test_tokens_aprobacion.py` (25 tests: firma, caducidad, sal, rol y suplantación) más la suite de aprobación en `solicitudes/tests/`.
117. **Gotchas. Excepción a la regla del comentario obligatorio** (§ 8.2, P6): estas cuatro rutas son GET de un clic desde el correo y **no** piden ni motivo ni comentario; el comentario queda vacío. Es deliberado: un enlace de correo no tiene formulario donde escribirlo. Si necesitas dejar constancia, resuelve desde la aplicación. El token va firmado con `SECRET_KEY`: **todas las instancias deben compartir la misma** o el enlace fallará según a cuál encamine el ALB, y **rotar `SECRET\_KEY` invalida los enlaces ya enviados** (§ 9.2 y § 13.3). El uso único no se guarda en ninguna lista: depende del estado de la solicitud en base de datos, así que cualquier ruta que apruebe sin actualizar `estado/aprobado_*` reabre el enlace.

GET /turnos/api/mis-turnos-por-mes/

118. **Método y ruta.** GET /turnos/api/mis-turnos-por-mes/?mes=<MM>&anio=<AAAA> (turnos/api/urls.py:13, nombre turnos_api:mis_turnos_por_mes; el prefijo api/ lo pone turnos/urls.py:49).
119. **Vista y archivo.** MisTurnosPorMesView (turnos/api/views/turnos_mes.py:47), método get (:52).
120. **Para qué existe.** Es la pantalla que mira el explorador. Devuelve, día a día, qué le toca ese mes: jornada, sala, si es descanso y —lo que da nombre a esta ficha— con quién es el acuerdo que modificó el día.
121. **Permiso.** LoginRequiredMixin. No acepta un empleado por parámetro: siempre resuelve request.user.empleado (:53-56), así que nadie consulta el mes de otro por esta vía.
122. **Entrada.** mes (str/int, obligatorio, '8' o '08') y anio (str/int, obligatorio, '2026'). Se normalizan a MM/AAAA antes de formar la clave de caché (:63-72).
123. **Validaciones, en orden.** (a) el usuario tiene empleado (:53); (b) mes y anio presentes (:61); (c) mes entre 1 y 12 y ambos numéricos (:65-73); (d) acierto de caché → se devuelve tal cual y no se calcula nada (:81-83).
124. **Respuesta 200.** Un objeto plano {"YYYY-MM-DD": {...}}, una clave por día del mes —los días que la fuente de verdad decide no mostrar simplemente no aparecen (turnos/services/mis_turnos_dia.py:60-65)—. Cada día lleva jornada, sala, tipo (predeterminado \\\ cambio \\\ descanso), es_cambio, es_doblada/es_descanso, jornada_predeterminada, coincide_con_predeterminada, turno_id y, cuando aplica, es_festivo, descanso_info, permiso, restriccion y sancion. Los dos bloques del acuerdo:
• `solicitud\_info` (día TRABAJADO por un acuerdo): solicitud_id, tipo, tipo_cambio, companero_id, companero_nombre, rol ('solicitante' \\\ 'receptor'), fecha_solicitud, fecha_resolucion, fecha_relacionada (solicitudes/services/acuerdo_por_dia_service.py:47-61,236-251). Es null en un día sin acuerdo.
• `descanso\_info` (día LIBRE por un acuerdo): tipo, tipo_solicitud, origen, companero_nombre, companero_id, solicitud_id, fecha_relacionada, fecha_cesion, fecha_pago, fecha_solicitud, fecha_aprobacion, tipo_cesion, jornada_cedida (turnos/services/mis_turnos_dia.py:179-195).
125. **Errores.** 400 {'error': 'Usuario no es empleado'} (:54); 400 {'error': 'Debe enviar mes y anio'} (:61); 400 {'error': 'Mes invalido'} (:69); 400 {'error': 'anio/mes deben ser numéricos'} (:73); 500 con mensaje propio y extra: {request_id} vía json_error_inesperado (:227-228). No devuelve traza: ver § 9.5.
126. **Efectos secundarios.** Ninguna escritura de negocio. Escribe la caché turnos_mes_<emp>_<mes> con TTL de una hora (:212-214).
127. **Servicios que invoca.** TurnoService.estado_mes, DescansoPorSolicitudService, turnos/services/mis_turnos_dia.py (:192, la decisión por día) y `AcuerdoPorDiaService.en\_rango` desde _enriquecer_solicitud_info (:338, llamada en

:194-195). Además `_permisos_por_fecha` (:231), `_restricciones_por_fecha` (:282) y `_sanciones_por_fecha` (:306).

128. **Tests.** `solicitudes/tests/test_reflejo_mis_turnos.py` (12 métodos de prueba; pytest recoge 20 porque la clase de permisos hereda los ocho de la base) llama al endpoint de verdad con `self.client.get('/turnos/api/mis-turnos-por-mes/')` (:159) y comprueba el «con quién» con `_con_quien` (:163) y el «libre por» con `_libre_por` (:183); `solicitudes/tests/test_acuerdo_por_dia.py` cubre las dos reglas de resolución del servicio y la rama de PAGO REPROGRAMADO, que no sale de ningún snapshot; `core/tests/test_json_error_inesperado.py`:140,180 vigila que ningún `except` de este módulo publique `str(e)` ni traceback.
129. **Gotchas.** (a) **La caché no sabe de despliegues.** El mes se sirve del caché hasta una hora; tras desplegar un cambio que altere el JSON hay que invalidar (`CacheService.invalidar_cache_turnos_empleado`) o esperar el TTL, o se sigue entregando el JSON anterior. En los tests se hace un `cache.clear()` antes de cada consulta (`test_reflejo_mis_turnos.py`:157). (b) Con `LocMemCache` y varios workers, invalidar limpia un proceso: `CACHE_URL` es obligatorio en producción (§ 10). (c) `turno_origen/turno_destino` ya **no** es la vía principal, solo el **respaldo** para un día con cambio que ningún snapshot reclame (`_solicitud_por_turno` :369, `_respaldo_por_turno` :411); una doblada tiene dos turnos ese día y `turno_id` solo guarda el primero, por eso el respaldo recorre todos los turnos de la fecha (:418-421). (d) El frontend rotula por `tipo_cambio`, que **no** es el vocabulario de `TipoSolicitudCambio.nombre`: la traducción de los dos nombres que difieren está en `_TIPO_SOLICITUD_A_TIPO_CAMBIO` (`acuerdo_por_dia_service.py`:319-322).

7. Servicios y lógica de dominio

7.1 Mapa de servicios

`solicitudes/services/` tiene 29 módulos. Los que importan, y quién los llama:

Servicio	Qué hace	Quién lo llama
<code>solicitud_orchestrator.py</code>	Encadena los chequeos transversales —dedupe, cierre, sanciones, restricción— y despacha al tipo. Contiene además los dos flujos multi-solicitud: <code>_procesar_cobertura_dos</code> y <code>_procesar_doblada_permanente_multi</code> . El primero lleva delante	<code>CrearSolicitudUseCase</code>

	<code>_pago_dos_debe_estar_libre</code>, el único guard de pareja del archivo: mira una condición que ninguna de las dos solicitudes puede ver por separado (ver § 8.3). Los dos envuelven su bucle en <code>EmailOutboxService.envio_agrupado()</code> por fuera del <code>atomic()</code> (:278,530): los 3N correos comparten conexión y, sobre todo, nada se entrega hasta que el bloque termina, ya commitado	
<code>solicitud_factory.py</code>	Resuelve el <code>TipoSolicitudCambio</code> a su <code>Strategy</code> en cascada de cuatro niveles (:131-195) y expone <code>validar_solicitud</code>, <code>crear_solicitud</code> y <code>revalidar_para_aprobar</code> (:293)	orquestador y servicio de aprobación
<code>solicitud_aprobacion_service.py</code>	Ejecuta la aprobación: revalida (:124) y aplica	casos de uso de aprobación
<code>solicitud_request_parser.py</code>	Convierte el <code>POST</code> en el dict de datos que entienden las estrategias	orquestador
<code>cambio_descanso_aplicacion_service.py</code>	Materializa y revierte CAMBIO DESCANSO. Contiene <code>_marcar_reemplazadas</code> (:281), donde vive "la última aprobada gana por día", y <code>dia_bloqueado_para_nuevo_cambio</code> (:216), fuentes únicas de si una fecha admite un nuevo cambio de descanso: la comparten el selector de	<code>CambioDescansoStrategy</code> , <code>CambioDescansoFindesView</code> , cancelación

	findes (<code>solicitudes/views/api_fin_semana.py:369</code>) y la validación (<code>cambio_descanso_strategy.py:119</code>), así nunca se contradicen. Su regla es una sola consulta: turno con <code>tipo_cambio</code> distinto de CAMBIO DESCANSO (:243-247). El parámetro <code>excluir_id</code> sobrevive por compatibilidad de firma y hoy no hace nada (el criterio mira turnos, no solicitudes). <code>_mapa_descanso_multi</code> (:89) mete además una clave <code>`acuerdo`</code> dentro del dict del compañero — <code>solicitud_id</code>, <code>tipo_solicitud</code> y las cuatro fechas (:140- 155)—: quien pinta el día ya no tiene que volver a deducir de qué intercambio viene. Quien solo lee <code>'id'/'nombre'</code> no se entera de que está, y DescansoPorSolicitud Service la consume sin propagarla a la API (<code>descanso_solicitud_ service.py:277-291</code>)	
<code>doblada_aplicacion_ service.py</code> , <code>doblada_permanente_ aplicacion_service.py</code> , <code>d_fds_aplicacion_ service.py</code>	Lo mismo para DOBLADA, DOBLADA PERMANENTE y D FDS	sus estrategias
<code>doblada_snapshot_ service.py</code>	Captura y restaura snapshots de turnos; reconciliación de días	servicios de aplicación

	colaterales. <code>reconciliar_dobladas_aprobadas (:289)</code> re-aplica las dobladas vigentes sin re-validarlas, así que una que ya no encaje con el calendario actual puede levantar <code>ValidationError</code> desde los guardias de negocio del patrón 39. Es reparación best-effort, no validación: la llamada a <code>_reaplicar_una</code> va envuelta en <code>try/except ValidationError (:337-347)</code>, se registra con <code>logger.error</code> (id de solicitud, tipo, fechas y motivo) y se continúa con las demás, para que nunca impida una cancelación legítima. Se captura solo ValidationError; cualquier otro fallo —de BD o de programación— sigue propagándose. Consecuencia operativa en § 16.2; decisión razonada en el ADR 008 (./03-arquitectura/adr/008-reconciliacion-best-effort.md) y generalizada como patrón 40 de PROTECTION_PATTERNS.md	
<code>deuda_service.py</code>, <code>deuda_corporativa_service.py</code>, <code>doblada_deuda_service.py</code>	Generan, cancelan y saldan deudas. <code>deuda_corporativa_service.py</code> es además el dueño de la sanción automática por deuda vencida: <code>gestionar_sancion_por_deuda (:391)</code> la crea,	servicios de aplicación, PDH, <code>SancionEmpleado.levantar</code> (<code>empleados/models.py:438</code>), cron <code>revisar_sanciones_por_deuda</code>

	<code>_consumir_deudas_de_sanciones_cumplidas (:114)</code> extingue la deuda del mes cuando el castigo se cumple, y <code>condonar_deudas_por_levantamiento (:181)</code> la extingue —como 'condonada'— cuando el supervisor lo levanta. Las tres son idempotentes: solo tocan lo que sigue 'activa'. Dos ayudantes cierran el ciclo: <code>horas_condonadas_por (:258)</code>, fuente de verdad de cuántas horas se perdonaron —lee lo escrito, no lo recalcula—, y <code>notificar_condonacion (:282)</code>, que avisa al explorador por la campana con un tipo de notificación propio, 'sancion_levantada' (:330). Qué sanción es automática lo deciden dos `Q` de clase, `ES_AUTOMATICA` y `ES_MANUAL` (:68-69), por `periodo_anio` y no por el prefijo del motivo (:58-67)	
doblada_pago_service.py	Aplica el día de pago de una doblada. <code>aplicar_doblada_pago</code> es <code>@transaction.atomic (:23-24)</code> y despacha a una rama por modalidad: sábado (<code>_aplicar_pago_sabado, :99</code>), <code>jornada_cubre_en_pago</code> AMBAS/media, cesión	servicios de aplicación, PDH

	parcial, <code>jornada_cedida</code> y fallback (:63-94). Tres ramas validan antes de escribir que el acreedor trabaje el día de pago: sábado (:139-151, § 8.3), <code>jcp_media</code> (:302-309) y <code>jornada_cedida</code> en su rama parcial (:536-543, dentro del else de <code>cesion_completa</code>, no al inicio del método). Las tres usan <code>BaseValidator._explorador_trabaja</code>. Las otras tres —<code>jcp_ambas</code>, <code>cesion_parcial</code> y el fallback— no tienen guardia propia; el cierre es parcial y el motivo de por qué no puede extenderse —medido, no razonado— está en § 16.2	
<code>cierre_solicitudes_service.py</code>	Calcula la ventana de cierre semanal: <code>cutoff_para_fecha</code> (:123), <code>fecha_bloqueada</code> (:140), <code>validar_fechas</code> (:148)	<code>SolicitudOrchestrator.verificar_cierre</code>
<code>email_outbox_service.py</code>	Dueño de la entrega, no solo de la cola. Encola (<code>encolar</code>, :76), reclama por UPDATE condicional (<code>_reclamar</code>, :117) y entrega siempre en lote por una única conexión (<code>enviar_lote</code>, :234; un correo suelto es un lote de uno). <code>_intentar</code> (:151) devuelve <code>'enviado'/'fallido'/'omitido'</code> —<code>'omitido'</code> —<code>'omitido'</code> significa "no se pudo reclamar", así que no dice	servicios de aplicación, <code>notificacion_service.py</code>:40,417, <code>solicitud_orchestrator.py</code>:278,530, comando <code>procesar_email_outbox</code>

	nada del estado de la conexión, y esa distinción es la que permite al lote saber si debe renovar el canal—; <code>intentar_enviar (:202)</code> es su envoltorio booleano de siempre. El cuándo lo deciden <code>despachar (:370)</code>, <code>envio_agrupado (:329)</code> y <code>enviar_tras_commit (:303)</code>	
<code>email_service.py</code>	Plantillas, enlaces firmados y encolado. <code>_enviar_email_desde_usuario (:74)</code> es el cuello de botella único por el que pasa todo correo de la aplicación: valida, encola y delega el cuándo en <code>EmailOutboxService.d espachar(fila.id) (:140)</code>. Devuelve <code>True</code> cuando el correo quedó encolado, no cuando el SMTP lo aceptó	outbox
<code>tokens_aprobacion.py</code>	Fuente única de los tokens firmados de los enlaces de aprobación/rechazo por correo. <code>generar/verificar</code> para solicitudes de cambio (sal <code>solicitudes.aprobacion-email</code>, tipos <code>supervisor</code> y <code>receptor</code>, <code>:44-46</code>) y <code>generar_permiso/verificar_permiso</code> para permisos especiales (sal <code>permisos.aprobacion-email</code>, <code>:116</code>), de modo que un token no vale en el circuito del otro. Firma con	<code>email_service.py:146,528</code>, las cuatro vistas de <code>views/aprobacion_email.py:140,199,251,303</code> y <code>permisos/services.py:30,34</code>

	<code>django.core.signing</code> sobre <code>SECRET_KEY</code> , con caducidad (:49-52)	
<code>core/utils/error_token.py</code>	<code>render_error_token()</code> : punto único para la página de token inválido/caducado. Arma el contexto con <code>APPROVAL_LINK_MAX_AGE_DAYS</code> y con el <code>request_id</code> . <code>render_error_token_inesperado()</code> cierra un <code>except Exception</code> sin filtrar nada: deja la traza en el log y devuelve 500 con el mensaje genérico	las 15 llamadas de <code>views/aprobacion_email.py</code> y <code>permisos/views.py</code>
<code>core/utils/json_responses.py</code>	Formato único de respuesta de las APIs: <code>json_ok (:11)</code> y <code>json_error (:35, con code y extra)</code> . <code>json_error_inesperado(request, excepcion, mensaje) (:68)</code> cierra un <code>except Exception</code> sin filtrar nada: <code>logger.exception</code> con la traza (:99) y 500 con el mensaje propio del endpoint más <code>extra: {request_id} (:101-102)</code>	todas las vistas API; las 7 llamadas al helper en <code>turnos/api/views/{dias_especiales, calculo_automatizado, turnos_mes}.py</code> y <code>solicitudes/views/api_turno_jornada.py</code> . Ver § 9.5
<code>notificacion_service.py</code>	Crea filas de <code>Notificacion</code> . Para la cancelación consensuada aporta <code>crear_notificacion_peticion_cancelacion (:627, al receptor: exige acción, dice que el cambio sigue vigente y que hay plazo)</code> y	vistas y casos de uso

	<code>crear_notificacion_respuesta_cancelacion</code> (:661, al solicitante, tanto si se aprueba como si se rechaza). Las dos son solo notificación en la aplicación, sin correo: no encolan <code>EmailOutbox</code>	
<code>reprogramacion_dobla_da_service.py</code>	Anula el día no cumplido y programa el nuevo	vistas de reprogramación
<code>doblada_filtro_servi ce.py</code>	Alimenta el desplegable de compañeros de DOBLADA. <code>filtrar_empleados_si n_doblada_activa</code> (:27) une dos fuentes de «ya trabaja AM+PM ese día»: las <code>SolicitudCambio</code> de tipo DOBLADA aprobadas en la fecha, consultadas por <code>`explorador_receptor_id`</code> (:60-66), y las dobladas ya materializadas en Turno (AM y PM el mismo día, :94-98). Además resuelve la jornada a ceder (<code>obtener_jornada_a_c eder</code>, :125) y lista a quien descansa (<code>obtener_empleados_e n_descanso</code>, :190)	vistas API de doblada
<code>empleado_disponibili dad_service.py</code> , <code>solicitud_consulta_s ervice.py</code> , <code>solicitud_context_se rvice.py</code>	Alimentan desplegables y pantallas	vistas API
<code>cambios_permanentes_ helper.py</code>	Expansión y filtrado de días de CT PERMANENTE	<code>CtPermanenteStrategy</code>
<code>permiso_service.py</code>	Permisos especiales	app <code>permisos</code>
<code>permisos/credito_hor as_service.py</code>	Horas a favor del explorador.	<code>PDHCreateView</code> , <code>PDHDeleteView</code> , <code>ConsolidadoHorasService</code> , CRUD de

	<code>creditos_disponibles</code> (:37, filtra el saldo en SQL con <code>F()</code> en vez de recorrer el histórico en Python) y <code>saldo_disponible</code> (:53); <code>otorgar</code> (:66); <code>consumir</code> (:100) gasta en FIFO y comprueba el saldo antes de escribir nada, porque un consumo a medias dejaría créditos gastados contra un pago que la vista va a rechazar; <code>revertir_consumos</code> (:152) los devuelve al borrar el PDH; <code>anular</code> (:176) solo si no se consumió nada. <code>select_for_update</code> en el consumo: sin él, dos pagos simultáneos gastan el mismo crédito dos veces	crédito
<code>permisos/views.py</code> (funciones de módulo)	<code>_supervisor_del_permiso</code> (:561): fuente única de quién responde por un permiso (<code>permiso.supervisor</code> or <code>permiso.empleado.supervisor</code>); la usan la autorización (:588) y el aviso (:611), así que decide y notifica la misma persona. <code>_puede_responder_cancelacion</code> (:569): las tres reglas de quién puede responder una cancelación. <code>_notificar_peticion_cancelacion_permiso</code> (:607) y <code>_notificar_respuesta_cancelacion_permiso</code> (:630): equivalentes de las	<code>PermisoMediaJornadaCancelView</code>, <code>PermisoMediaJornadaCancelResponseView</code>

	dos anteriores para el permiso de media jornada, pero viven en la vista, no en `NotificacionService`. Duplicación consciente de la costura; anotada como deuda en § 16.3	
resultado.py	ResultadoSolicitud (:32): el resultado de dominio de crear una solicitud —éxito, status, mensaje, code, datos—. Sustituye al JsonResponse que devolvía el orquestador, sin cambiar el cuerpo JSON (:81-93)	orquestador, y resultado_http.py para traducirlo
doblada_permanente_plan.py	Parsea y valida la forma del alta multi-compañero de DOBLADA PERMANENTE antes de crear nada	SolicitudOrchestrator._procesar_doblada_permanente_multi (solicitud_orchestrator.py:366)
`acuerdo_por_dia_service.py`	Fuente única del acuerdo que puso a alguien a TRABAJAR un día. en_rango(empleado, ini, fin) -> {fecha: info} (:81) y el atajo en_fecha(empleado, fecha) (:154). No re-deriva la geometría de ningún tipo: lee los snapshots, que viven en tres sitios según el tipo — SolicitudCambio (CAMBIO TURNO, CT PERMANENTE), DobladaDetalle (DOBLADA, D FDS, CAMBIO DESCANSO) y DobladaPermanenteDetalle (DOBLADA PERMANENTE)— y los recorre por ese orden	MisTurnosPorMesView._enriquecer_solicitud_info (turnos/api/views/turnos_mes.py:355-357)

	(<code>_snapshot</code>, :163). PAGO REPROGRAMADO no pasa por ninguno de los tres y tiene rama propia sobre <code>ReprogramacionDiaDoblada</code> (<code>_agregar_pagos_reprogramados</code>, :273). Dos reglas: última aprobada gana el día (:132-151) y guarda de realidad L1-sobre-L2 (:119-130, 149). El filtro de fechas de la consulta es a propósito un superconjunto con <code>MARGEN_DIAS = 3</code> (:78,96-117): quien decide es el snapshot, fecha a fecha Dos niveles (<code>_dias_reclamados</code>, :187): el <code>snapshot_turnos_resultantes</code> dice QUÉ dejó en cada (persona, fecha) y manda cuando está; si falta, se cae a <code>snapshot_turnos_previos</code>, que solo dice QUÉ FECHAS tocó —sus valores describen el mundo anterior— y deja decidir a la guarda de realidad. El nivel 2 existe porque el resultante se empezó a capturar después que el previo: sin él, toda solicitud anterior a ese cambio deja su día sin compañero (medido en desarrollo: 51 de 61 días de un explorador, 28 tras añadirlo).	
<code>descanso_solicitud_service.py</code>	Hermano del anterior por el otro lado: el descanso	<code>TurnoService</code> , <code>mis_turnos_dia.py</code> , validadores

	atribuido a una solicitud aprobada («¿por qué descanso este día y con quién?»), para los cuatro tipos que lo generan. Devuelve, por fecha, el dict que consumen <code>TurnoService.estado_dia/estado_mes</code> y <code>MisTurnos (:13-27</code>, con <code>tipo_solicitud</code> documentado en <code>:19</code>). Desde el cambio del detalle del día incluye <code>tipo_solicitud</code> en las cuatro ramas <code>(:156,220,249,283,317)</code> y las de CAMBIO DESCANSO y DOBLADA PERMANENTE ya no devuelven <code>None</code> en <code>solicitud_id</code>, fechas de cesión/pago, <code>fecha_solicitud</code> ni <code>fecha_aprobacion</code> <code>(:277-291, :314-326)</code>. Siempre por FECHA, nunca por patrón de día de la semana	
<code>fechas_helper.py</code> , <code>solicitud_service.py</code> , <code>solicitud_validator.py</code>	Apoyo transversal	varios

`turnos/services/` (15 módulos) responde a "¿qué pasa realmente este día?":
`turno_service.py` (`estado_dia / estado_mes`, la fuente de verdad),
`descanso_semana_service.py`, `asignacion_especial_service.py`,
`alternancia_fines_semana_service.py`, `temporada_service.py`, `dia_especial_service.py`,
`festivos_rotacion_service.py`, `consolidado_horas_service.py`, `reporte_dia_service.py`,
`apertura_anio_service.py`, `jornada_service.py`, `doblada_turno_service.py`,
`turno_context_service.py`, más los dos que salieron de sus llamadores: `mis_turnos_dia.py` — la decisión por día de «Mis Turnos», antes el cuerpo del bucle de `MisTurnosPorMesView.get`; se invoca en `turnos/api/views/turnos_mes.py:192`, y su `descanso_info` incluye ahora

`tipo_solicitud` para que el día libre pueda nombrar el trámite del que viene (`turnos/services/mis_turnos_dia.py:183`)— y `reporte_dia_empleado.py` —la clasificación por empleado del reporte del supervisor, extraída de `ReporteDiaService.reporte`—. Los dos replican el orden de capas de `TurnoService.estado_dia`: reordenarlo hace que Mis Turnos y el reporte digan cosas distintas del mismo día, y `turnos/tests/test_reporte_dia_paridad.py` lo detecta.

`solicitudes/domain/` contiene lo que no depende del ORM: `estado_machine.py`, `bloqueo_partes.py`, `fechas.py`, `jornada.py`, `solicitud.py`. No hay paquete `repositories/`: las consultas reutilizadas viven en el manager del modelo (`solicitudes/models.py:136-149`). Ver § 3.4.

7.2 Validadores

Validador	Qué regla protege
<code>services/validators/base_validator.py</code>	Reglas comunes a casi todos los tipos: día de mantenimiento con la precedencia de temporada (:66), <code>es_dia_temporada</code> (:394-409) y descanso de semana manual (:419)
<code>services/validators/ct_validator.py</code>	Reglas del cambio de turno sencillo
<code>services/validators/ct_permanente_validator.py</code>	Filtra los días no aptos del rango —fines de semana, festivos, mantenimiento, temporada, descansos y días comprometidos— y rechaza si no queda ninguno (:186, 219)
<code>services/validators/doblada_validator.py</code>	Días especiales de doblada: los festivos de lunes a viernes y la temporada sí están permitidos (:170-198); el receptor que descansa el día de pago —casos 1.5/1.8 de la matriz— en <code>validar_receptor_tiene_jornada_en_fecha_pago</code> (:272-294), llamado desde <code>strategies/doblada_strategy.py:251</code> ; y el caso del deudor que descansa el día de pago (:296)
<code>services/validators/doblada_flujo_validator.py</code>	Los ocho tramos de flujo de <code>DobladaStrategy.validar_solicitud</code> , extraídos de la estrategia: jornadas y coincidencia (:115), día no comprometido (:204), cesión en festivo (:300), reglas comunes (:359), pago en sábado (:396), cobertura en el pago (:486), intercambio (:554) y <code>validar_pago_en_sabado</code> (:602). No están en <code>doblada_validator.py</code> porque devuelven el mensaje en vez de lanzarlo: <code>validar_pago_en_sabado</code> devuelve un <code>RequiereCambioTurnoPrevio</code> —subclase de <code>str</code>

	con datos colgados— y viajar dentro de un <code>ValidationError</code> lo aplanaría a texto, dejando al formulario sin el recuadro «Ir a Cambio de Turno Sencillo» (el porqué, en el docstring del módulo, :1-46)
<code>services/solicitud_validator.py</code>	Validación de nivel de solicitud, por encima del tipo. Es la fachada que reúne las primitivas de base, CT y doblada; <code>doblada_flujo_validator</code> la importa a ella, no a <code>DobladaValidator</code> (<code>services/validators/doblada_flujo_validator.py:61-64</code>)

7.3 Tareas programadas, outbox y comandos de gestión

No hay Celery ni broker: el único trabajo diferido son los correos, resuelto con la tabla `EmailOutbox`, una entrega en lote tras el commit y un cron de respaldo.

Los dos caminos de entrega

Un correo encolado tiene **un camino normal y una red de seguridad**, y confundirlos lleva a diagnósticos equivocados ("el correo no salió porque el cron no corre" suele ser falso):

Camino	Cuándo actúa	Qué hace	Dónde
Entrega tras el commit (normal)	Al cerrar el <code>envio_agrupado</code> , o al despachar un correo suelto	Un único intento del lote completo por una conexión SMTP. Con <code>EMAIL_SEND_ASYNC</code> va en un hilo y no bloquea la respuesta; sin él —desarrollo y tests— se ejecuta ahí mismo, de forma síncrona y determinista	<code>email_outbox_service.py:303-325, 362-367</code>
Barrido del cron (respaldo)	Cada ejecución de <code>procesar_email_outbox</code>	Recoge lo que el camino normal no entregó: filas pendiente o enviando con disponible_en vencido e intentos < 5 . También va en una conexión, vía	<code>email_outbox_service.py:280-296</code>

		enviar_lote	
--	--	-------------	--

El camino normal es **un intento y ya**: si falla, la fila queda lista para reintentarse pero *ningún proceso despierta a mirarla*. Por eso el cron no es opcional en producción. El backoff son [1, 5, 15, 60, 180] minutos (`email_outbox_service.py:50`) y, agotados los 5 intentos, la fila queda **fallido** y exige intervención humana.

Lo que hace barato el envío es la conexión compartida, no el hilo. Medido el 2026-09-07 contra Gmail: crear una solicitud tardaba 6,3-7,8 s en responder y **todo** ese tiempo era SMTP, porque cada uno de los tres correos abría su propia conexión (~2 s de handshake TLS + AUTH). Los flujos multi-compañero multiplicaban la cifra: cobertura con 2 compañeros son 6 correos, doblada permanente con 3 son 9. Detalle del mecanismo en § 3.4; operación en [MANUAL_OUTBOX_CORREOS.md](#) (`./05-referencia/deployment/02-correo/MANUAL_OUTBOX_CORREOS.md`).

Comandos de `solicitudes` (`solicitudes/management/commands/`):

Comando	Para qué	Frecuencia
<code>procesar_email_outbox</code>	Barrido de respaldo: reintentar las filas cuyo <code>disponible_en</code> ya venció, todas por una sola conexión SMTP. Opciones <code>--limite N</code> (50 por defecto) y <code>--resumen</code> (solo cuenta la cola, no envía) (<code>procesar_email_outbox.py:24-34</code>)	cron, continuo
<code>instalar_calendario_colombiano</code>	Carga los festivos	una vez / anual
<code>actualizar_codigos_estrategia</code>	Sincroniza <code>TipoSolicitudCambio.codigo_estrategia</code>	tras tocar el catálogo
<code>archivar_solicitudes_antiguas</code>	Mueve solicitudes viejas	periódico
<code>verificar_crons</code>	Comprueba desde fuera que los dos crons corren : el del outbox y el de sanciones. Mide la espera del correo pendiente más antiguo que ya tocaba enviar, no el tamaño de la cola —una cola larga con worker vivo se vacía sola; una fila que lleva horas parada solo se explica porque nadie la recoge (<code>verificar_crons.py:131-174</code>)—. Cada problema sale con el marcador <code>CRON_NO_EJECUTADO (:69)</code> y el comando termina con código 1 , para que sea un <i>check</i> y no un informe que alguien tiene que acordarse de leer (<code>:124-127</code>). Opciones: <code>--umbral-outbox MINUTOS</code> y <code>--json (:89-100)</code>	cron de vigilancia

alertar_crons	El mismo diagnóstico, y además avisa. Importa la clase de <code>verificar_crons</code> y ejecuta sus tres chequeos —<code>_revisar_outbox</code>, <code>_revisar_correos_fallidos</code> y <code>_revisar_sanciones</code>— sin duplicar criterio (<code>alertar_crons.py:66-73,104-108</code>); si alguno falla, manda el correo él mismo. Existe porque <code>verificar_crons</code> solo <i>diagnostica</i>: en AWS bastaba —el <code>crontab</code> de Linux avisa por correo al salir con código distinto de cero y CloudWatch enganchaba los marcadores con un <i>metric filter</i>—, pero Dokploy no tiene disparador para un Schedule que falla: sus notificaciones cubren despliegues, backups, limpieza de Docker y umbrales del servidor (<code>:4-18</code>). Envía con <code>smtplib</code> directo y no con <code>django.core.mail</code> (<code>:205-242</code>), por dos motivos: el vigilante no puede compartir maquinaria con lo que vigila —todo correo de la aplicación pasa por <code>EmailOutbox</code>, y si lo averiado es el worker del outbox la alerta se quedaría en la cola que denuncia— y el ADR 016 (<code>./03-arquitectura/adr/016-transporte-de-correo-y-fiabilidad.md</code>) restringe <code>django.core.mail</code> a <code>email_outbox_service.py</code> (<code>:20-34</code>). Reutiliza la configuración <code>EMAIL_*</code> de settings: mismo transporte, sin pasar por la cola (<code>:214-235</code>). Destinatarios: <code>--para</code> o la variable de entorno <code>ALERTAS_CRON_EMAIL</code> (separadas por coma); sin destinatarios falla ruidosamente (<code>:159-169</code>) —no se añadió un ajuste a <code>settings.py</code> a propósito: es configuración de despliegue—. Latido semanal: los lunes manda un correo de «sin novedad» aunque todo esté bien (<code>:78-79,119-121</code>), la única defensa contra que deje de correr <i>él</i>. Termina con código 1 si hay problema, incluido no haber podido enviar el aviso (<code>:142-149,154-155</code>). <code>--dry-run</code> imprime el correo sin enviarlo —y sin exigir destinatarios—, pero conserva la salida 1 si hay problemas (<code>:130-137</code>). Opciones: <code>--para</code>, <code>--dry-run</code>, <code>--umbral-outbox</code>, <code>--umbral-fallidos</code>, <code>--sin-latido</code> (<code>:86-101</code>)	Schedule diario
cancelar_deudas_fin_semana,	Saneamiento de deudas	puntual

<code>cancelar_deudas_huerfanas</code>		
<code>reaplicar_doblada (:259), corregir_doblada_cesion_total (:73)</code>	Reparación manual. ⚠ Llaman a `aplicar_doblada_pago` sin validar y sin pasar por la guardia LIFO , y se ejecutan cuando el día ya está descuadrado — el momento de más riesgo. <code>verificar_doblada.py:384</code> sugiere el de reaplicación al detectar errores. Ver § 16.2	puntual
<code>verificar_doblada, verificar_integridad_dobladas, verificar_efecto_aplicado, validar_dobladas_junio, validar_jornadas, test_factory, test_verificar_doblada_jeison, validar_fix_doblada_jeison</code>	Diagnóstico	manual

``verificar_crons`` y ``alertar_crons`` **no son duplicados — no los fusiones**. El primero *diagnostica*: imprime, admite `--json` y deja el veredicto en el código de salida, que es lo que lo hace útil como paso de un despliegue. El segundo *entrega el aviso* cuando no hay infraestructura que lo haga por él. La lógica está escrita **una sola vez**: `alertar_crons` importa la clase de `verificar_crons` y llama a sus tres `staticmethod` (`solicitudes/management/commands/alertar_crons.py:66-73,104-108`), así que un chequeo nuevo se añade en `verificar_crons` y los dos lo heredan. Lo que `alertar_crons` pone encima —destinatarios, transporte, latido y código de salida— lo cubren 16 pruebas en `solicitudes/tests/test_alertar_crons.py (:2-13)`. Los Schedules y la configuración paso a paso están en [CHECKLIST DESPLIEGUE DOKPLOY.md](#) (`./05-referencia/deployment/01-dokploy/CHECKLIST_DESPLIEGUE_DOKPLOY.md`) (FASE 9 y § 11.2).

Comandos de `turnos` (`turnos/management/commands/`): `archivar_turnos_antiguos`, `limpiar_festivos_futuros`, `materializar_alternancia`, `verificar_apertura_anio`.

⚠ Ocho de los comandos de `solicitudes` son de diagnóstico puntual, cuatro con nombres de incidencias concretas (`..._jeison`, `..._junio`). No son parte del ciclo de vida del sistema; ver § 16.

El manual de operación del outbox está en [MANUAL OUTBOX CORREOS.md](#) (`./05-referencia/deployment/02-correo/MANUAL_OUTBOX_CORREOS.md`).

7.4 La costura donde el mensaje cambia de destinatario

Los mensajes de validación nacieron en el formulario y están escritos en imperativo para **quien envía**: "Elige otra fecha de pago", "Realiza primero un cambio de turno sencillo". Los ~13 de DOBLADA viven en `solicitudes/services/validators/doblada_validator.py`. El problema aparece en la aprobación: `SolicitudAprobacionService` llama a `SolicitudFactory.revalidar_para_aprobar`

([solicitudes/services/solicitud_aprobacion_service.py:124](#)) y, si falla, quien lee el mensaje es el **aprobador**, que no puede ejecutar ninguna de esas instrucciones — la solicitud ya existe y él solo puede aprobarla o rechazarla.

Decisión: no se reescriben los mensajes; se reencuadran una sola vez en la costura. El `return` de fallo es hoy ([solicitud_aprobacion_service.py:134-138](#)):

```
return False, (
    f"No se puede aprobar: la solicitud ya no es válida con el estado actual. "
    f"Motivo: {msg_reval} "
    f"(instrucciones dirigidas a quien la envió). Recházala para que pueda rehacerse."
)
```

Antes devolvía `f"No se puede aprobar: {msg_reval}"`. El porqué de no tocar los 13 literales está comentado en el propio código ([:123-129](#)) y se sostiene en tres razones:

Razón	Dónde se comprueba
Son literales documentados en el manual de usuario	manual_usuario.md (./manual_usuario.md), tabla de errores de doblada (:976) y nota de estilo (:1431); origen del literal en doblada_validator.py:267
Hay tests que verifican subcadenas de esos mensajes	solicitudes/tests/test_matriz_dobladas.py:2121,2600,2727
CancelarSolicitudView decide el código HTTP inspeccionando el texto ('más reciente' in msg)	solicitudes/views/aprobacion_views.py:64-72 ; ver § 16.2

Es la regla 5 del patrón 39 de [PROTECTION_PATTERNS.md](#): un mensaje que atraviesa dos rutas no puede dar instrucciones de rol. Cuando el mensaje sí es propio del servicio de escritura —el guard de sábado— se aplica la regla en origen y se describe el problema en vez de ordenar nada ([solicitudes/services/doblada_pago_service.py:146-152](#)).

8. Reglas de negocio

8.1 Estados de una solicitud

```
stateDiagram-v2
    [*] --> pendiente : crear
    pendiente --> aprobada : receptor + supervisor
    pendiente --> rechazada : cualquiera rechaza
    pendiente --> cancelada : el solicitante se arrepiente
```

```

aprobada --> cancelada : el receptor aprueba la cancelación pedida
aprobada --> reemplazada : otra aprobada pisa el mismo día
aprobada --> pagada : se salda la deuda (PDH)
rechazada --> [*]
cancelada --> [*]
reemplazada --> [*]
pagada --> [*]

```

Qué muestra. Los seis estados de `SolicitudCambio` y las únicas transiciones legales. El mapa está en un solo sitio (`solicitudes/domain/estado_machine.py:11-18`) y la función `transicionar` lo hace cumplir, lanzando `EstadoTransicionError` cuando el destino no está permitido (`:28,47-53`). Cuatro estados son terminales y no tienen salida: `rechazada`, `cancelada`, `reemplazada` y `pagada` (`:14-17`). Se lee así: desde `pendiente` se puede ir a `aprobada`, `rechazada` o `cancelada`; una vez `aprobada` solo caben tres finales —cancelarla, que otra la reemplace, o que se pague su deuda—. Nada vuelve atrás.

Cuidado: hay un segundo estado, ortogonal a este. `cancelacion_estado` describe la petición de cancelación, no la solicitud, y sus valores son `'` (nunca se pidió), `pendiente`, `aprobada`, `rechazada` y `caducada` (`core/constants.py:167-195`). Los tres últimos son **terminales**: `EstadoCancelacion.TERMINALES` (`:154`) y no se vuelve nunca a `pendiente`.

```

stateDiagram-v2
    [*] --> ninguna : la solicitud se aprueba
    ninguna --> pendiente : el solicitante pide cancelar (≤ 24 h desde
    fecha_resolucion)
    pendiente --> aprobada : el receptor acepta → se revierte y estado = cancelada
    pendiente --> rechazada : el receptor se niega → el cambio queda FIRME
    pendiente --> caducada : pasan 24 h sin respuesta → el cambio queda FIRME
    aprobada --> [*]
    rechazada --> [*]
    caducada --> [*]

```

Qué muestra. El ciclo de la petición de cancelación, que corre en paralelo al de la solicitud. La clave está en la combinación `estado='aprobada' + cancelacion_estado='pendiente'`: es un estado **legítimo y frecuente**, y significa "hay una cancelación en trámite y el cambio sigue aplicado". Mientras dure, la guardia LIFO, Mis Turnos y la reconciliación siguen contando esa solicitud como vigente, que es exactamente lo que se quiere: nadie pierde su turno porque otro haya pulsado un botón que el receptor aún no ha aceptado. Solo la rama `pendiente → aprobada` toca los turnos. La caducidad no la escribe ningún proceso de fondo: se reconoce al responder (`solicitudes/use_cases/cancelar_solicitud.py:390`) y se filtra al listar (`solicitudes/views/notificaciones_listas.py:173-197`).

8.2 Principios transversales

Estas reglas atraviesan los seis formularios. Son el núcleo del dominio. P1 y P0 son hermanas: la primera dice quién manda sobre un día, la segunda quién puede deshacer lo pactado.

#	Regla	Dónde se aplica	Test que la cubre	Qué pasa si se
---	-------	-----------------	-------------------	----------------

				viola
P 1	La última aprobada gana por día. El estado efectivo de un día es lo último aprobado que lo modifica. No se encadenan cambios: la solicitud anterior sobre ese (persona, día) pasa a reemplazada , con reemplazada_por apuntando a la nueva	<code>solicitudes/services/cambio_descanso_aplicacion_service.py:281-321</code> (<code>_marcar_reemplazadas</code>), invocado en <code>:380</code> ; el estado REEMPLAZADA se define en <code>core/constants.py</code> (<code>EstadoSolicitud</code>). También manda al LEER: <code>AcuerdoPorDiaService</code> resuelve el «con quién» de un día recorriendo las aprobadas por <code>fecha_resolucion</code> descendente y quedándose con la primera que reclama la fecha (<code>solicitudes/services/acuerdo_por_dia_service.py:128-147</code>)	Suite de reemplazos en <code>solicitudes/tests/;</code> del lado de la lectura, <code>solicitudes/tests/test_acuerdo_por_dia.py:81</code> (<code>test_la_ultima_aprobada_gana_el_dia</code>)	El día queda con dos solicitudes vigentes que se contradicen y el consolidado cuenta doble
P 2	No se encadenan cambios de descanso. Solo se cede el descanso de temporada original; un intercambio nuevo cancela el anterior en vez de apilarse	<code>cambio_descanso_aplicacion_service.py:342-345</code> (documenta por qué no se marcan reemplazos cuando la solicitud no estaba aplicada) y <code>services/strategies/cambio_descanso_strategy.py:444</code>	<code>solicitudes/tests/test_politica_temporada.py</code>	Se pierde el rastro de a quién pertenece el descanso original
P 3	Los días de descanso de temporada solo se tocan desde CAMBIO DESCANSO. Son dos fechas por semana, no la temporada entera: el resto de la temporada sigue disponible para los demás formularios	Regla y motivo en <code>turnos/services/descanso_semana_service.py:44-62</code> (<code>es_dia_descanso_temporada</code>); rechazo en <code>solicitudes/services/strategies/doblada_strategy.py:194-205</code>	<code>solicitudes/tests/test_politica_temporada.py:222</code>	Un formulario ajeno cede un descanso fijado y el cómputo semanal de CAMBIO DESCANSO deja de cuadrar

P0	Cancelar deshace un acuerdo, y quién puede deshacerlo depende de si ese acuerdo llegó a existir. pendiente: nadie aceptó y nada se aplicó, el solicitante la retira solo. aprobada: el receptor ya aceptó y los turnos se movieron, así que el solicitante pide la cancelación (≤ 24 h desde fecha_resolucion) y el receptor la aprueba o la rechaza (≤ 24 h desde cancelacion_solicitada_en). Gestión (supervisor) sigue cancelando sin pasar por el receptor: es intervención administrativa, no parte del acuerdo, y execute_supervisor no cambió. Invariante: mientras cancelacion_estado == 'pendiente', estado sigue siendo	core/constants.py:167-206 (EstadoCancelacion, TERMINALES en :195, las dos ventanas de 24 h en :201 y :206), solicitudes/use_cases/cancelar_solicitud.py:39 (execute), :227 (_pedir_cancelacion), :287 (responder_cancelacion), :390 (_caducar_si_vencida), :99 (execute_supervisor, sin cambios); invariante comentado en solicitudes/models.py:166-170	solicitudes/tests/test_cancelacion_lifo.py: test_pedir_cancelacion_no_cancela_nada (:148), test_solo_el_receptor_puede_responder (:165), test_receptor_rechaza_y_el_cambio_queda_firme (:174), test_receptor_aprueba_y_se_revierde (:194), test_la_peticion_caduca_si_el_receptor_no_responde (:205), test_no_se_puede_pedir_pasado_el_plazo_del_solicitante (:228)	Una persona deshace unilateralmente un turno que la otra ya organizó. Si además se pusiera estado='cancelada' al pedir, el cambio dejaría de verse vigente mientras se decide: la guardia LIFO no lo contaría, Mis Turnos mostraría el turno viejo y la reconciliación lo desharía. Hermano de P1
------------------	---	--	---	--

	'aprobada' a propósito —los turnos siguen aplicados y todo lo que filtra por 'aprobada' la ve vigente—; la cancelación solo se materializa cuando el receptor aprueba. Rechazo y caducidad dejan el cambio firme y son definitivos			
P 4	Reversión al cancelar. La reversión restaura snapshot_turnos_previos y cancela las deudas generadas, y solo ocurre si pasan la guardia LIFO y la de integridad. Con el acuerdo (P0) esas dos guardias corren dos veces: al pedir (cancelar_solicitud.py:267), para no molestar al receptor con algo que ya es irreversible, y al aprobar (:354), porque entre una cosa y otra pueden pasar 24 horas y entrar otro cambio sobre los mismos días	solicitudes/use_cases/cancelar_solicitud.py:423 (LIFO), :460 (integridad), :580 (fechas_y_cumplidas), :646 (_revertir_por_tipo); las dos llamadas en :267 y :354. VENTANA_CANCELACION_MINUTOS ya no existe: se eliminó de este archivo (era código muerto) y del servicio de cambio de descanso (ADR 010 (<code>./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md</code>)))	Commit 40a7ed8: reversión de las cinco opciones de temporada. Ciclo de dos pasos envuelto para el resto de la suite en solicitudes/tests/helpers_cancelacion.py (cancelar_con_acuerdo)	Se pisa el cambio de otra persona: se restaura un turno que el compañero ya no tiene

P 4 b	La reconciliación posterior a una cancelación es best-effort, no todo-o-nada. Restaurado el snapshot, reconciliar_dobladas_aprobadas re-aplica sin re-validar las solicitudes aprobadas vigentes de esos días. Si una ya no encaja con el calendario actual y levanta ValidationError, se omite —no entra en el conjunto de reaplicadas, así que su snapshot_turnos_resultantes tampoco se refresca—, se registra a nivel ERROR y el bucle sigue. Ninguna solicitud ajena puede bloquear la cancelación de otra. Solo se captura ValidationError; cualquier otra excepción propaga y aborta	solicitudes/services/doblada_snapshot_service.py:286 (método), :337-347 (try/except + logger.error), :348 (reaplicadas.add fuera del try)	TestReconciliacionNoTumbalaCancelacion (solicitudes/tests/test_matriz_dobladas.py:3051, caso test_una_doblada_que_ya_no_encaja_no_impide_cancelar_otra en :3072)	Queda una solicitud aprobada cuyo efecto no está materializado en turnos. Consta en el log de ERROR (id, tipo, fechas, motivo) y lo detecta python manage.py verificar_efecto_aplicado, que además lo repara con --reparar. Lo que falta es la alerta: hoy hay que ejecutar el comando o leer el log (§ 16.2, § 18 fila 22, ADR 008 (./03-arquitectura/adr/008-reconciliacion-best-effort.md))
P 5	Re-validación al aprobar. Antes de aplicar nada se vuelve a validar contra el estado	solicitudes/services/solicitud_aprobacion_service.py:124 → solicitudes/services/solicitud_factory.py:39	Suite de aprobación en solicitudes/tests/	Se aplica un acuerdo que ya era ilegal cuando se aprobó

	actual, porque entre la creación y la aprobación el mundo pudo cambiar	1,432 → <code>services/strategies/ba se_strategy.py:92-114</code>		
P 6	Quien aprueba, rechaza o cancela algo escribe por qué. El comentario o motivo es obligatorio en toda acción que resuelve: aprobar/rechazar una solicitud (como supervisor, como receptor o en ambos roles), cancelar una solicitud, responder a una petición de cancelación, aprobar o rechazar un permiso especial, pedir o responder la cancelación de un permiso de media jornada, registrar una inasistencia de doblada y registrar un PDH. Se valida en el servidor : quitar el required del HTML no se la salta. Excepción única : la aprobación por enlace de correo (<code>solicitudes/v iews/aprobaci</code>	Fuente única: <code>`core/utls/comentarios.py`</code> . Ahí viven los textos (<code>MSG_COMENTARIO</code> , <code>MSG_MOTIVO</code> , <code>MSG_MOTIVO_INASISTENCIA</code> , <code>MSG_NOTA_PAGO</code> , :24-27), el código <code>CODE_COMENTARIO_REQUERIDO (:31)</code> y dos helpers con nombres distintos porque los contratos lo son: <code>`leer_texto(request, campo) - > str \</code>	<code>None (:34)</code> , <code>neutral</code> , <code>y</code> <code>exigir_texto_json(request, campo, mensaje) -> (texto, respuesta_400 \</code>	<code>None) (:44)</code> , <code>para vistas que hablan JSON</code> . <code>Solicitudes</code> usa el <code>segundo</code> (<code>aprobacion_views.py:31, :41, :51, :60, :83, :140, :192</code>); <code>permisos</code> usa el <code>primero</code> y <code>construye la respuesta con _fallo</code> (<code>permisos/views.py:322, :437, :571</code>). Las <code>plantillas</code> reciben los <code>mismos literales</code> por el <code>context processor</code> <code>core.context_processors.mensajes_comentario (:21, registrado en config/settings.py: 106)</code> , y de ahí <code>pasan al navegador – window.MSG_COMENTARIOS en solicitudes, data-comentario-error en permisos–</code> , de forma que <code>**cliente y</code>

on_email.py), que es un GET de un clic sin formulario			servidor avisan con las mismas palabras**. Inasistencia: solicitudes/views/ reprogramacion_v iews.py:114,121- 122. PDH: empleados/forms. py:380,380-383 (required = True + clean_comentario) y empleados/views/ pdh.py:91,133`, validado después de fecha y explorador para no tapar esos errores
--	--	--	--

8.3 Reglas por área

Regla	Dónde se aplica	Qué pasa si se viola
Un sancionado no participa en ninguna solicitud, ni como compañero: si no, bastaría con que otro la enviara en su nombre	solicitud_orchestrator.py:163-169 (solicitante), :172-186 (receptor)	La sanción es evitable
Una restricción médica advierte, no bloquea: el usuario confirma	solicitud_orchestrator.py:291 (verificar_restriccion(..., confirmar)), llamada en :579	—
El mismo POST no se procesa dos veces dentro de la ventana de _DEDUPE_TTL_SEGUNDOS (30 s)	solicitud_orchestrator.py:34,60-90	Solicitud duplicada por doble clic
No existen dobladas abiertas: fecha_pago es obligatoria	solicitudes/models.py:447	Deuda sin vencimiento
Cada doblada acumula 30 minutos de deuda corporativa	solicitudes/models.py:641,663-666	El consolidado no refleja lo trabajado

Un intercambio de dobladas (<code>es_intercambio=True</code>) no genera ni altera deudas: es un swap de días	<code>solicitudes/models.py:403-408</code>	Deuda inventada
No se paga una doblada en un sábado que el receptor descansa. Si no trabaja, no hay jornada suya que cubrir. Se valida dos veces: aguas arriba en <code>doblada_validator.py:272-294</code> (desde <code>doblada_strategy.py:251</code>) y dentro del servicio, antes de cualquier escritura, en <code>doblada_pago_service.py:139-151</code>, porque al servicio se entra desde tres rutas — creación, re-validación al aprobar y aprobación por enlace de correo— y no puede confiar en que alguien mirara. <code>aplicar_doblada_pago</code> es <code>@transaction.atomic(:23)</code>, así que el <code>ValidationError</code> no deja estado a medias. El mensaje describe el problema y no ordena una acción ("La doblada debe rehacerse con otra fecha de pago", <code>:149-150</code>): lo lee tanto el solicitante al enviar como el aprobador al aprobar, y el aprobador no puede cambiar la fecha. El motivo está comentado en el código (<code>:143-145</code>) para que no se "mejore" de vuelta a un imperativo. En la web es defensa en profundidad: creación y aprobación re-validan, y la reconciliación, que re-aplica sin validar, solo corre dentro de un <code>revertir(...)</code> protegido por la guardia LIFO. Fuera de la web no: los	El reparto de sábado, que le quita al receptor la mitad cubierta y le crea la contraria si no la tiene, le inventa un turno en un día libre	

comandos de gestión reaplicar_doblada (solicitudes/management/commands/reaplicar_doblada.py:259) y corregir_doblada_cesion_total (:73) entran al servicio sin validar y sin LIFO, así que ahí este guard es el único que hay. Por eso vive donde está la escritura. El mismo guard existe ya para dos ramas de pago entre semana (:300-307, :534-541); las otras dos siguen sin él por un motivo de diseño que se explica en § 16.2. El bug real y demostrado que apareció de camino es otro: la doble cobertura de la fila siguiente, alcanzable por aprobación normal. Cubierto por solicitudes/tests/test_matriz_dobladas.py:2635 (TestPagoSabadoReceptorD escansa: :2702 rechazo y ausencia de turno creado, :2722 contraprueba del reparto legítimo)		
Excepción: que el receptor no tenga turnos ese sábado es legítimo si la otra mitad del día ya la cubre otra doblada aprobada, la pague quien la pague. Un único helper decide, DobladaPagoService._mitad_contraria_cubierta_por_otra_doblada(solicitud, receptor, fecha_pago, jornada_sel): filtra por explorador_receptor, tipo DOBLADA/D_FDS, estado='aprobada', misma doblada__fecha_pago y doblada__jornada_pago_sabado contraria, excluyendo la	Helper: solicitudes/services/doblada_pago_service.py:99-123; consumido por el guard (:139-140) y por la rama _contraria_tambien_cubierta del reparto (:225-226). Atribución del descanso: solicitudes/services/descanso_solicitud_service.py:193-197,204-215.
No confundir con `_otra_mitad_pagada` (doblada_pago_service.py:200-206): esa consulta se parece mucho y pregunta lo contrario — el sujeto es el deudor ("¿ya trabajo yo la otra mitad de este sábado?"), no el acreedor, y por eso sí filtra por explorador_solicitante	Con el filtro por mismo deudor fallaban dos cosas: el reparto recreaba al receptor la mitad que otro deudor ya cubría (dos personas en el mismo turno) y el guard rechazaba re-aplicaciones legítimas (reconciliación, re-validación al aprobar, aprobación por enlace). Sentinelas: solicitudes/tests/test_matriz_dobladas.py:2759 y :2795

propia solicitud. No filtra por <code>`explorador_solicitante`</code>: que dos deudores distintos cubran cada mitad del sábado del acreedor es legítimo y ya está modelado en el dominio — <code>DescansoPorSolicitudService</code> atribuye el descanso del receptor acumulando las mitades de todas las solicitudes en que es receptor esa fecha, con <code>{ 'AM', 'PM' } <= e['parciales']</code>, sin mirar quién paga cada una	y no filtra por receptor: las dos mitades pueden pagarse a personas distintas. Están deliberadamente separadas y el motivo está comentado en el código (<code>:195-199</code>); unificarlas rompe una de las dos	
Tampoco se paga una doblada entre semana a un acreedor que descansa — pero solo dos de las cuatro ramas lo comprueban. Guard con <code>BaseValidator._explorador_trabaja(receptor, fecha_pago)</code> en <code>_aplicar_pago_jcp_media</code>, al inicio y antes de escribir (<code>solicitudes/services/doblada_pago_service.py:302-309</code>), y en <code>_aplicar_pago_jornada_cedida</code> dentro de su rama parcial, el <code>else</code> de <code>tipo_cesion == 'cesion_completa'</code> (<code>:536-543</code>) — deliberadamente ahí y no al inicio del método. Mismo texto en las dos: "...no trabaja el DD/MM/AAAA: ese día descansa, así que no tiene una jornada que cubrirle para pagar la doblada. La doblada debe rehacerse con otra fecha de pago." Solo estas dos porque conservan una jornada al acreedor: al re-aplicar, <code>_explorador_trabaja</code> ve un turno real y el guard no salta. <code>cesion_parcial</code>, el fallback y	Guard: <code>doblada_pago_service.py:302-309</code> y <code>:536-543</code>; motivo comentado en <code>:286-301</code> y <code>:530-535</code>	Corrección respecto a versiones anteriores de este manual: el daño de las ramas sin guardia no es "fabricarle un turno al acreedor". Auditado empíricamente con un acreedor que descansa por alternancia de fin de semana: al acreedor no se le inventa nada, se queda sin turnos, que es lo correcto. El perjudicado es el deudor, que acaba con AM+PM cubriendo jornadas que nadie iba a trabajar y, al quedar doblado, carga además los 30 minutos de deuda corporativa: trabaja de más por un turno inexistente. La rama de control <code>jcp_media</code> quedó bloqueada correctamente por su guard en esa misma medición

jornada_cedida con cesion_completa dejan al acreedor sin turnos por diseño y el guard bloquearía su propia re-aplicación ya en la primera aplicación — medido, § 16.2. Cubierto por TestGuardPagoEntreSemana (solicitudes/tests/test_matriz_dobladas.py:2948, rechazos en :2988 y :3030, regresión de re-aplicación en :3005)		
No se puede pedir doblada a quien ya trabaja AM+PM ese día — y el que trabaja AM+PM es el <code>explorador_receptor</code>, no el solicitante. En una DOBLADA los dos roles hacen lo contrario ese día: el explorador_solicitante cede su jornada y descansa, así que sigue disponible; el explorador_receptor recibe la jornada y queda doblado, así que pedirle otra sería un triple turno. DobladaFiltroService.filtrar_empleados_sin_doblada_activa construye el conjunto a excluir con values_list('explorador_receptor_id') (solicitudes/services/doblada_filtro_service.py:60-66), lo une al de las dobladas ya materializadas en Turno —AM y PM la misma fecha, :94-98— y filtra (:110-113). La regla de negocio no cambia; lo que estaba mal era el rol consultado. El porqué está en el docstring (:34-38) y en el comentario «OJO CON EL ROL» junto a la consulta (:56-59)	Consulta: doblada_filtro_service.py:60-66. Tests sentinela: solicitudes/tests/test_doblada_filtro_rol.py:51 (quien cedió sigue disponible), :58 (el receptor queda excluido) y :65 (la fuente Turno sigue vigente)	Con el filtro por explorador_solicitante_id el desplegable escondía justo a la gente libre ese día —incluida la que descansa porque te cedió a TI— y en cambio ofrecía al que estaba doblado. Doble fallo: candidatos válidos invisibles y candidatos inválidos ofrecidos, que el backend tumba al enviar (patrón 37)

CT PERMANENTE solo de lunes a viernes	solicitudes/models.py:330-349 (CambioPermanenteDia.save)	Turnos permanentes en fin de semana
DOBLADA PERMANENTE no permite domingos	solicitudes/models.py:549	Ídem
Un solo turno activo por (explorador, fecha, jornada)	turnos/models.py:80-83 , garantizado por la base de datos	La persona aparece dos veces en la misma jornada y el consolidado cuenta doble
Turno.tipo_cambio solo acepta los ocho valores de TipoCambioTurno	turnos/models.py:90-94	Un typo entra en silencio y el turno deja de contarse en los filtros de texto exacto
Un día ya usado en un cambio de descanso vuelve a estar libre de inmediato. dia_bloqueado_para_nuevo_cambio bloquea la fecha solo si hay un turno con tipo_cambio distinto de CAMBIO DESCANSO (DOBLADA, D FDS, CT...): ese bloqueo es permanente y sin ventana. Un CAMBIO DESCANSO previo no bloquea — coherente con P1, "la última aprobada gana por día". Es la fuente única que comparten el selector de fines y la validación	Regla: solicitudes/services/cambio_descanso_aplicacion_service.py:243-247 , motivo en el docstring :217-240; llamadas en solicitudes/services/strategies/cambio_descanso_strategy.py:111 y solicitudes/views/api_fin_semana.py:358 . Tests: solicitudes/tests/test_cambio_descanso.py:254 (CDReintercambioDiaTest), centinela en :312. Motivo completo: ADR 010 (./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md)	Si se reintroduce el bloqueo, un día intercambiado queda congelado para toda la plantilla mientras dure la ventana de cancelación (hoy 24 h). Si se quita el bloqueo por otro tipo de cambio, se cede un día que ya está comprometido por una doblada o un CT
La temporada manda sobre el mantenimiento	turnos/models.py:186-200 (es_mantenimiento_efectivo)	Un lunes de temporada se trata como descanso
Cobertura de día completo (2 compañeros): el día de pago tiene que estar LIBRE. Las dos solicitudes comparten una única fecha_pago , así que ese día se devuelven las dos medias jornadas —la AM a un compañero y la PM al otro— y el solicitante termina en AM+PM. Eso solo cabe partiendo de cero: con media jornada propia solo alcanza para pagarle a uno y el otro	SolicitudOrchestrator._pago_dos_debe_estar_libre , llamado al inicio de _procesar_cobertura_dos (justo tras la restricción médica, antes de construir datos_am/datos_pm). Se apoya en CambioDescansoAplicacionService._jornadas_actuales: estado real del día , no la jornada base (P-jornada-real). El formulario lo adelanta con mi_jornada_pago , que ya devuelve GET /solicitudes/cobertura-candidatos/ : recuadro rojo bloqueante	Sin el guard el rechazo llega igual, pero con el consejo equivocado: el usuario hace el Cambio de Turno, vuelve, y choca con el otro compañero. Si alguien "arregla" el síntoma relajando _validar_semana_cobertura , se crea media cobertura pagada y la otra media gratis

habría cubierto gratis. Es un guard de PAREJA, y por eso no vive en la strategy: <code>_validar_semana_cobertura</code> valida cada solicitud por separado y no puede ver el conjunto. Rechazaba el caso igual, pero por la mitad equivocada del problema (<code>tomadas & pre_sol_fp</code>, la jornada que choca) y con un mensaje que manda a hacer un Cambio de Turno sencillo — que aquí es un callejón sin salida: girar la jornada propia traslada el choque al otro compañero. Y si la <code>fecha_pago</code> fuera de mantenimiento, ese día ni siquiera se puede pedir un CT. El guard corta el par antes de validar ninguna de las dos y da el motivo real, con las dos salidas que sí existen: el día libre de la semana, o cobertura de solo AM / solo PM	<code>cob-aviso-dos-libre</code> + error al enviar, y en modo DOS se suprime el aviso de «Ir a Cambio de Turno» (<code>static/js/cambio-turno/solicitar_cambio_descanso.js</code>, <code>mostrarAvisoDosDiaLibre</code> y la salida temprana de <code>verificarPagoCobertura</code>). El desplegable de día de pago además deshabilita los días de mantenimiento: ese día no opera nadie, así que no hay jornada del compañero que cubrir. El texto del rechazo y el del aviso del formulario son un espejo sincronizado a mano (<code>solicitud_orchestrator.py:228-234</code> ↔ <code>solicitar_cambio_descanso.js:1091-1095</code>), con centinela <code>AvisoDiaPagoLibreEspejoTest</code>. Tests: <code>solicitudes/tests/test_cobertura_dos_companeros.py:29</code> (espejo), <code>:154</code> (<code>test_dia_de_pago_con_media_jornada_propia_es_rechazado</code>, que además fija que el mensaje no mande al CT) y <code>:174</code> (<code>test_dia_de_pago_donde_ya_doble_o_tampoco_sirve_para_los_dos</code>)	
Una sub-modalidad de CAMBIO DESCANSO entre semana se apaga desde el backend, no desde el formulario. Hoy está apagada <code>cambio_doblada</code>. El interruptor es la constante <code>SUBMODALIDADES_SEMANA_DESHABILITADAS</code> y se comprueba dentro de <code>validar_solicitud</code>, en la rama de ENTRE SEMANA y antes de despachar a ningún sub-validador. Como <code>revalidar_para_aprobar</code> termina llamando a	Constante: <code>solicitudes/services/strategies/cambio_descanso_strategy.py:53</code> (motivo comentado en <code>:40-52</code>); comprobación y mensaje en <code>:248-256</code>, con la etiqueta sacada de <code>DobladaDetalle.SUBMODALIDAD_SEMANA_CHOICES</code> (<code>:252</code>) para que el texto siga siendo cierto al apagar o reactivar otra. Doble entrada: <code>solicitudes/services/strategies/base_strategy.py:114</code>. Reflejo visual (solo cosmético): <code>templates/solicitudes/solicitar_cambio_descanso.html:128-133</code>.	Si el apagado viviera solo en el template, bastaría con quitar la clase desde DevTools —o tener la pestaña abierta desde antes del despliegue, con el HTML y el JS anteriores— para que el POST entrara. Y si la comprobación estuviera en la vista y no en <code>validar_solicitud</code>, una solicitud creada antes del apagado se seguiría aprobando

validar_solicitud, el apagado cubre las dos entradas: una solicitud que se hubiera colado antes del despliegue tampoco se puede aprobar. El código del sub-flujo no se borra (_validar_semana_cambio_doblada y su aplicación siguen enteras): reactivar es sacar la clave del frozenset y quitar la clase disabled de la tarjeta	Tests: solicitudes/tests/test_cambio_doblada_deshabilitado.py:74,80,93,120,131. Detalle en 05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md (./05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md)	
Una fecha sin AsignacionEspecialManual es "sin planificar" y así se reporta; nunca se infiere un grupo	turnos/models.py:305-315	El pasado se recalcula solo
Un turno anulado no cuenta como falta ni genera deuda	turnos/models.py:41-51,73	Se penaliza un día que se anuló
Una sanción no se borra: se levanta, dejando quién, cuándo y por qué	empleados/models.py:299,323-330	Se pierde el hecho disciplinario
Cumplir la auto-sanción salda la deuda del mes que la originó. Pasa a consumida_por_sancion con FK a la sanción: el castigo ES el pago	solicitudes/services/deuda_corporativa_service.py:114 (ES_AUTOMATICA y levantada_en__isnull=True en :137-139)	El explorador termina sus 15 días con el mismo saldo vencido que lo metió en ellos y vuelve a ser sancionado por lo mismo: un bucle del que no se sale
Levantar una auto-sanción por deuda CONDONA la deuda de ese mes. Pasa a 'condonada' —estado propio, ni cancelada (corrección técnica) ni consumida_por_sancion (esa la paga el castigo cumplido)— con sancion_consumidora y fecha_consumo. Solo aplica a auto-sanciones —reconocidas por el criterio único ES_AUTOMATICA, ver la regla siguiente—, solo al mes que la originó, y es idempotente. Se dispara desde	Servicio: solicitudes/services/deuda_corporativa_service.py:181 (guardas en :212-215, marca y update en :218-236, roll-up del permiso en :241-249). Disparador: empleados/models.py:409 (levantar), llamada en :438. Aviso previo al supervisor (estimación): empleados/views/sanciones.py:149, :289; el número posterior sale de horas_condonadas_por (deuda_corporativa_service.py:258), que lee lo ya escrito, y lo usan tanto el mensaje al supervisor	Es el bug que cierra: la deuda quedaba en un limbo permanente. El mes ya venció, así que no se puede pagar (Periodo.esta_vencido, solicitudes/services/sancion_deuda_calculo.py:99); esa sanción no la consumirá al llegar a su fin, porque el consumo exige levantada_en IS NULL; y no nace otra que

SancionEmpleado.levantar() y no desde la vista, dentro de su misma transacción: es el único punto de entrada al levantamiento, para que ninguna vía futura (comando, admin, proceso automático) pueda olvidarse. Lo que NO se perdona es el expediente: _periodos_ya_evaluados y _antecedente siguen viendo la sanción levantada, así que la reincidencia se mide igual. Razonado en el ADR 013 (./03-arquitectura/adr/013-condonacion-al-levantar-la-sancion.md)	(sanciones.py:316) como el aviso por campana al explorador (notificar_condonacion, :282, dentro de la misma transacción, empleados/models.py:442-443), que usa el tipo propio 'sancion_levantada' (:330, declarado en solicitudes/models.py:22 con su motivo en :18-21) para que la campana no pinte el triángulo rojo de la sanción sobre la noticia contraria (templates/solicitudes/notificaciones_list.html:65-68). Antecedente intacto: deuda_corporativa_service.py:369 (_periodos_ya_evaluados) y :340 (_antecedente). Tests: solicitudes/tests/test_consumo_deuda_por_sancion.py:145,174,192,208,226,297,318,339,362,385 y solicitudes/tests/test_sancion_automatica_deuda.py:257	lo haga, porque el mes ya cuenta como juzgado. Horas activas para siempre, ni cobrables ni saldables
Qué sanción es automática lo dice su periodo, nunca el texto del motivo. ES_AUTOMATICA = Q(periodo_anio__isnull=False) y ES_MANUAL = Q(periodo_anio__isnull=True) son el criterio único de las siete consultas que deciden algo: consumo, antecedente, meses ya juzgados, contador de pendientes, las tres de auditoría y la del barrido. El prefijo [AUTO-DEUDA] se sigue escribiendo en el motivo, pero es etiqueta de lectura: ninguna decisión depende de él	Constantes y motivo en solicitudes/services/deuda_corporativa_service.py:58-69; usos en :137 (consumo), :386 (meses ya juzgados), :574 (contador de pendientes), :731, :740 y :777 (auditoría) y :828 (barrido); el prefijo se escribe en :497	El motivo es editable desde la pantalla de sanciones. Con la detección por texto, retocar la redacción volvía la sanción invisible para las siete consultas a la vez: su deuda no se consumía al cumplirse el castigo y su mes dejaba de contar como juzgado, así que el sistema podía sancionar por segunda vez un mes ya castigado. Lo fijan test_condona_aunque_se_haya_editado_el_motivo y test_editar_el_motivo_no_reabre_el_ciclo_de_sanciones (solicitudes/tests/

		<code>test_consumo_deuda_por_sancion.py:318, :339)</code>
Una deuda ya extinguida por una sanción no se recalcula ni resucita. ' condonada ' cuenta como hecho ocurrido igual que ' pagada ' y ' consumida_por_sancion '	<code>permisos/deuda_permiso_service.py:44 (_tiene_historia, :54-55)</code> y el revert de PDH en <code>permisos/pago_horas_service.py:387</code>	Se reabre por la puerta de atrás una deuda que alguien decidió cerrar, y con ella la posibilidad de volver a sancionar por lo ya perdonado
Los roles Supervisor y Explorador se comparan exacto y están protegidos de renombrado y borrado	<code>empleados/models.py:73-85,98-107</code>	Escalada de privilegios; ya ocurrió con la búsqueda icontains
Las jornadas AM y PM no se pueden eliminar	<code>empleados/models.py:19-23,38-41</code>	El motor de turnos deja de resolver la jornada contraria
El cierre semanal bloquea las solicitudes cuyo objetivo caiga en la ventana; con habilitado=False no hay restricción	<code>solicitudes/models.py:792-803, solicitudes/services/cierre_solicitudes_service.py:140,148</code>	Se reprograma el fin de semana después de publicarlo
Un PDH pide entre 0 y 24 horas	<code>permisos/models.py:53-58</code>	Pago imposible
Un crédito de horas a favor pide entre 1 minuto y 24 h	<code>permisos/models.py:389-392</code> (validadores de campo , no <code>clean()</code>)	Crédito imposible
El crédito aplicado a un PDH no puede superar las horas que ese pago salda	<code>permisos/credito_horas_service.py:128-131</code>	Doble descuento: el consolidado restaría dos veces las mismas horas
Las horas a favor NO intervienen en la sanción. Un mes vencido sigue sancionando aunque se salde con crédito: lo sancionable es haber dejado vencer el plazo, no el importe	<code>solicitudes/services/deuda_corporativa_service.py</code> no consulta CreditoHoras ; el centinela es <code>permisos/tests/test_credito_horas.py:222</code>	Una bolsa de crédito volvería inmune a la sanción a quien nunca paga a tiempo
El permiso MEDIA_JORNADA_TEMPORADA no genera deuda (tiempo=0)	<code>permisos/models.py:113-117</code> y <code>permisos/views.py:855</code>	Deuda duplicada

El detalle formulario a formulario está en [05-referencia/solicitudes/](#) (./05-referencia/solicitudes/) y, en lenguaje de usuario, en [manual_usuario.md](#) (./manual_usuario.md).

9. Autenticación, autorización y seguridad

9.1 Modelo de usuario y roles

El usuario es el `auth.User` estándar de Django, extendido por composición: `Empleado` tiene un `OneToOneField` a `User` (`empleados/models.py:45`). No hay `AUTH_USER_MODEL` personalizado. Casi todas las vistas usan `request.user.Empleado`, y varias comprueban antes `hasattr(request.user, 'Empleado')`, porque un `User` sin `Empleado` —un superusuario creado con `createsuperuser`, por ejemplo— rompería el acceso (`solicitudes/views/aprobacion_views.py:60-61,96-97`).

Los roles son filas de `Role` unidas por `EmpleadoRole`. Dos son estructurales: `Supervisor` y `Explorador` (`empleados/models.py:82-85`). Qué puede hacer cada rol está en [manual_usuario.md](#) (`./manual_usuario.md`); aquí solo el mecanismo.

9.2 Permisos: dónde se comprueban

Capa	Mecanismo	Archivo
Sesión	<code>LoginRequiredMixin</code> de Django en cada vista	<code>solicitudes/views/*.py</code>
Rol, vistas HTML	<code>AdminRequiredMixin</code> — lanza o redirige	<code>core/mixins.py:44,57</code>
Rol, endpoints JSON	<code>SupervisorApiRequiredMixin</code> — devuelve JSON en vez de redirigir	<code>core/mixins.py:73,80</code>
Función común	<code>es_supervisor(user)</code> — la usan los dos mixins y el resto del código	<code>core/mixins.py:12-17</code>
Rol sobre una solicitud concreta	Los casos de uso: solo el solicitante cancela, solo el receptor aprueba como receptor	<code>solicitudes/use_cases/</code>
Token de correo	Token firmado con <code>django.core.signing</code> sobre <code>SECRET_KEY</code> , con caducidad y sal por circuito	<code>solicitudes/services/tokens_aprobacion.py:55-107,119-148</code>
Visibilidad del menú	Catálogo de sesiones por empleado: cada <code>Sesion</code> agrupa nombres de URL y se enciende o apaga por persona	<code>core/sesiones.py</code>

El último renglón no es un permiso más. `core/sesiones.py` es un **catálogo declarativo** que asocia cada entrada de menú con los nombres de URL que habilita (`Sesion(clave, etiqueta, grupo, [urls], defecto_explorador=...)`); apagar una sesión oculta la entrada y corta el acceso a sus rutas. Dos detalles con trampa, advertidos en el propio archivo: `sanciones_list` y `restricciones_list` quedan **fuera** de sus sesiones a propósito —son pantallas compartidas que al explorador le enseñan solo lo suyo, y bloquearlas dejaría al supervisor sin ver las propias—, y quien administra roles puede concederse cualquier permiso, así que esa sesión es la llave maestra. `is_staff` no se recorta por este mecanismo. La sesión `credito_horas` va **separada** de `pdh`: reconocer horas compromete a la corporación, mientras que un PDH solo constata lo que el explorador ya debía.

Tokens de aprobación por correo

Los enlaces de "aprobar" y "rechazar" del correo actúan **sin sesión iniciada**: el token de la URL es la única credencial. Toda la firma vive en un módulo único, `solicitudes/services/tokens_aprobacion.py`; las seis copias que antes duplicaban la verificación —una en `email_service.py`, cuatro en `views/aprobacion_email.py` y una en `permisos/services.py`— ahora delegan en él (§ 16.3, B1).

Aspecto	Cómo funciona	Evidencia
Firma	<code>signing.dumps / signing.loads</code> , HMAC-SHA256 sobre <code>SECRET_KEY</code> más marca de tiempo	<code>tokens_aprobacion.py:59-62,84</code>
Caducidad	<code>max_age = APPROVAL_LINK_MAX_AGE_DAYS × 86400</code> , por defecto 30 días	<code>tokens_aprobacion.py:51-54</code> , <code>config/settings.py:262</code>
Aislamiento por circuito	Sal <code>solicitudes.aprobacion_email</code> para solicitudes de cambio, <code>permisos.aprobacion_email</code> para permisos especiales: un token no sirve en el otro circuito aunque comparta clave	<code>tokens_aprobacion.py:44,116</code>
Vínculo con la persona	El token solo vale para quien hoy ocupa el rol; si al explorador le cambian de supervisor, el enlace del anterior deja de servir	<code>tokens_aprobacion.py:67-71,96-107</code>
Fallo cerrado	Firma inválida, token caducado, de otra solicitud o de otro rol → <code>False</code> , nunca excepción	<code>tokens_aprobacion.py:76-107</code>

Uso único	Sin lista de tokens gastados: lo impide <code>_ya_resuelto_para()</code> , que lee el estado de la solicitud en base de datos —el almacén compartido entre instancias— y también cubre el cliente de correo que pre-carga el enlace	<code>solicitudes/views/aprobacion_email.py:58-73</code>
Cobertura	25 tests dedicados	<code>solicitudes/tests/test_tokens_aprobacion.py</code>

Condición de despliegue. La verificación es *stateless*: no consulta base de datos ni caché, así que cualquier instancia detrás del balanceador valida un token emitido por otra. El precio es que **todas las instancias deben compartir la MISMA `SECRET\_KEY`** —una sola entrada en Secrets Manager o SSM, nunca un valor generado por tarea—; si no, los enlaces fallan de forma intermitente según a qué instancia encamine el ALB. Además, **rotar `SECRET\_KEY` invalida los enlaces ya enviados**: es el comportamiento correcto ante una filtración, pero hay que contarle al planificar rotaciones rutinarias (los pendientes se resuelven entrando a la aplicación). Ver § 13.3.

Esa condición ya se cumple en las dos rutas de despliegue documentadas (auditado 2026-08-09):

Evidencia	Dónde	Qué demuestra
<code>SECRET_KEY = env('SECRET_KEY'),</code> sin valor por defecto	<code>config/settings.py:32</code>	La aplicación no arranca sin la clave: <code>django-environ</code> lanza <code>ImproperlyConfigured</code> . Una tarea mal configurada muere, no se inventa una clave
Cero apariciones de <code>get_random_secret_key</code> , <code>secrets.token*</code> o <code>urandom</code> en todo el repositorio	—	El código no puede generar una clave por instancia
<code>aws secretsmanager create-secret --name swalp/SECRET_KEY</code>	<i>(era del checklist de Fargate, descartada y borrada el 2026-09-04)</i>	En el plan EC2 el secreto va en el <code>.env</code> de la instancia; ver checklist EC2+RDS (./05-referencia/deployment/99-aws/CHECKLIST_DESPLIEGUE_AWS_RDS.md) FASE 4
Instancia única con Elastic IP, sin ALB, con	checklist EC2+RDS (./05-referencia/deployment/99-	Sin reparto de tráfico no hay nada que pueda divergir

un .env en la máquina	aws/CHECKLIST_DESPLIEGUE_AWS_RDS.md)	
El Dockerfile usa SECRET_KEY=build-only como prefijo de un `RUN` , no como ENV	AppTurnosExplora/Dockerfile	La clave ficticia del build no queda en el runtime de la imagen

Queda como comprobación operativa —no del repositorio— confirmar en la consola de AWS que el secreto es efectivamente uno solo el día del despliegue. El checklist de § 13.3 lo recoge.

AUTHENTICATION_BACKENDS pone `axes.backends.AxesStandaloneBackend` **antes** del backend de Django, para que el bloqueo por intentos se evalúe primero (`config/settings.py:229-232`).

Bloqueo por intentos fallidos (`django-axes`, `config/settings.py:330-334`):

Ajuste	Valor	Efecto
AXES_FAILURE_LIMIT	5	Se bloquea al quinto intento fallido
AXES_COOLOFF_TIME	1	Una hora de espera
AXES_LOCKOUT_PARAMETERS	['username']	Bloquea por usuario, no por IP
AXES_RESET_ON_SUCCESS	True	Un acierto limpia el contador
AXES_VERBOSE	False	Menos ruido en el log

Axes se desactiva bajo tests, porque exige un `request` en `authenticate()` que `client.login()` no provee (`config/settings.py:336-341`).

No hay recuperación de contraseña: ver § 6.1 y § 16.

9.3 Sesiones, CSRF, CSP y cabeceras

Ajuste	Valor	Dónde
SESSION_COOKIE_HTTPONLY	True	<code>config/settings.py:438</code>
SESSION_COOKIE_SAMESITE / CSRF_COOKIE_SAMESITE	Lax	<code>config/settings.py:439-440</code>
SECURE_CONTENT_TYPE_NOSNIFF	True (siempre, dev y prod)	<code>config/settings.py:432</code>
X_FRAME_OPTIONS	DENY	<code>config/settings.py:433</code>
CSRF_FAILURE_VIEW	<code>core.errors.csrf_failure</code> — vista propia, no la de Django	<code>config/settings.py:445</code>
SECURE_SSL_REDIRECT +	Solo si SECURE_HTTPS	<code>config/settings.py:453-</code>

SECURE_PROXY_SSL_HEADER		457
CSRF_TRUSTED_ORIGINS	de <code>env.list</code> , vacío por defecto	<code>config/settings.py:38</code>
CORS_ALLOWED_ORIGINS	de <code>env.list</code> , localhost por defecto	<code>config/settings.py:350-353</code>
CORS_ALLOW_CREDENTIALS	True	<code>config/settings.py:335</code>
AUTH_PASSWORD_VALIDATORS	los 4 estándar de Django	<code>config/settings.py:204-209</code>

El `csrftoken` **no** es `HttpOnly` a propósito: `api-client.js` lo lee con `document.cookie` para mandarlo en la cabecera `X-CSRFToken` de las llamadas AJAX (`config/settings.py:435-438`).

CSP. Hay dos políticas. La **activa** (`CONTENT_SECURITY_POLICY`, `config/settings.py:376-401`) todavía permite `cdn.jsdelivr.net`, `cdnjs.cloudflare.com`, `fonts.googleapis.com`, `fonts.gstatic.com` y `code.ionicframework.com`. La **objetivo** (`CONTENT_SECURITY_POLICY_REPORT_ONLY`, `:409-427`) ya solo deja Google Fonts, y se publica como `Content-Security-Policy-Report-Only`: informa de las violaciones en la consola del navegador sin bloquear nada. Cuando se confirme que no aparece ninguna, basta con mover ese diccionario a `CONTENT_SECURITY_POLICY` y borrar el permisivo (`:366-375`).

Ambas mantienen `'unsafe-inline'` en `script-src` y `style-src` por los inline existentes; quitarlo exige migrar esos bloques a archivos o usar nonces (`:362-364`). Las dos declaran `frame-ancestors: 'none'` y `connect-src: 'self'`.

Regla operativa: al añadir un recurso externo nuevo hay que añadirlo a las **dos** allowlists de `config/settings.py`, o el navegador lo bloqueará. Pero el sentido de la migración es el contrario: autohospedar en `static/plugins/` en lugar de ampliar la lista (`config/settings.py:366-375`).

Las páginas de error (§ 9.5) llevan su CSS y su SVG **en línea**, así que dependen de `'unsafe-inline'` en la política activa; no añaden ningún origen externo, por lo que también pasan la política estricta en report-only. Si algún día se eliminan los inline en favor de nonces, estas cinco plantillas hay que revisarlas a mano (`templates/errors/_base_error.html:32-35`).

9.4 Datos sensibles y qué nunca se registra

Lo que el sistema guarda de personas: nombre, apellido, cédula (unique), correo, `RestriccionEmpleado.recomendacion` —texto libre de contenido médico— y `SancionEmpleado.motivo`. Ninguno de los dos últimos debe aparecer en logs ni en respuestas JSON de propósito general.

Los logs del orquestador registran **identificadores, no contenido**: `logger.info("Solicitud %d creada - tipo=%s solicitante=%s receptor=%s", ...)` (`solicitudes/services/solicitud_orchestrator.py:615`) y `logger.warning('Cobertura con`

2 compañeros revertida (solicitante=%s): %s', solicitante.id, e) (:273-274). Mantén ese criterio: identificadores sí; motivos médicos, contraseñas y tokens, nunca.

Las vistas capturan las excepciones y devuelven un mensaje genérico al cliente, dejando el traceback solo en el log con `logger.exception` (`solicitudes/views/procesar_solicitud.py:32-34`).

9.5 Páginas de error y trazabilidad de la petición

El razonamiento completo, con las alternativas descartadas, está en [ADR 007](#) (`./03-arquitectura/adr/007-paginas-de-error-propias-y-request-id.md`). Aquí va lo que hay que saber para trabajar con el sistema.

El problema. Con `DEBUG=True`, Django respondía a cualquier URL inexistente con su pantalla técnica, que imprime el **URLconf completo** —todos los endpoints, incluidos `aprobar-solicitud` y `rechazar-solicitud`—, y a un 500 con traceback, código fuente y variables locales. Es divulgación de información: CWE-215, CWE-209 y OWASP A05:2021 Security Misconfiguration. El caso concreto que lo destapó está escrito en el docstring del test de regresión (`core/tests/test_paginas_error.py:8-10`).

Las piezas. Todo vive en `core/errors.py`:

Pieza	Dónde	Qué hace
<code>RequestIDMiddleware</code>	<code>core/errors.py:51-76</code>	Genera <code>uuid4().hex[:12].upper()</code> , lo guarda en un <code>ContextVar</code> y en <code>request.request_id</code> , y lo devuelve en la cabecera <code>X-Request-ID</code>
<code>get_request_id()</code>	<code>core/errors.py:46-48</code>	Identificador de la petición en curso, o '-' fuera de una petición
<code>RequestIDFilter</code>	<code>core/errors.py:79-84</code>	<code>logging.Filter</code> que inyecta <code>record.request_id</code> para que los formatters lo impriman
<code>_render</code>	<code>core/errors.py:96-111</code>	Renderiza la plantilla <code>sin request</code> , con fallback si la propia plantilla falla
<code>bad_request / permission_denied / page_not_found</code>	<code>core/errors.py:114-131</code>	<code>handler400/403/404/500</code> , enlazados en <code>config/urls.py:56-59</code>

/ server_error		
csrf_failure	core/errors.py:153-163	CSRF_FAILURE_VIEW (config/settings.py:445)
previsualizar_error	core/errors.py:134-151	Renderiza una página a demanda para revisarla en desarrollo
Plantillas	templates/errors/_base_error.html + templates/{400,403,403_csrf,404,500}.html	Una base y cinco instancias

El identificador de petición. `RequestIDMiddleware` es el **primer** middleware (`config/settings.py:71`): lo que quede por encima no queda etiquetado. Tres decisiones que no se deben deshacer:

- **No se acepta un `X-Request-ID` entrante del cliente.** Sería un valor controlado por el atacante escrito en los logs: inyección de log y envenenamiento de trazas (`core/errors.py:58-61`). Hay test (`core/tests/test_paginas_error.py:84-91`).
- **Se usa un `ContextVar`**, no solo un atributo del `request`, porque el filtro de logging no recibe el `request` (`core/errors.py:41-43`).
- **Es aleatorio, no correlativo:** no revela cuántas peticiones ha atendido el sistema ni permite adivinar el de otro usuario (`core/errors.py:28-29`).

El usuario ve ese código en la página de error como «código de referencia», y es exactamente el mismo que viaja en la cabecera y en cada línea de log de esa petición (`core/tests/test_paginas_error.py:68-76`).

Por qué `\_render` no recibe el `request`. Usa `loader.get_template(...).render(context)` y no `render(request, ...)` (`core/errors.py:96-100`). `render()` activaría los context processors, y `core.context_processors.permisos` consulta la base de datos: en un `handler500` provocado precisamente por la BD caída, eso lanzaría una segunda excepción y Django devolvería su 500 de emergencia en texto plano (`core/errors.py:90-94`). Además Django renderiza `handler500` con contexto **vacío** —sin `request`, sin `user`, sin context processors—, así que las plantillas no pueden depender de ninguna variable obligatoria. Si aun así la plantilla falla, hay una última red que registra la excepción y devuelve un HTML mínimo (`core/errors.py:101-110`).

Por qué las plantillas son autocontenidas. No heredan de `base.html` y no cargan **nada** externo: CSS y SVG en línea (`templates/errors/_base_error.html:7-30`).

Razón	Qué evita
-------	-----------

Seguridad	No se muestra la ruta pedida, ni la excepción, ni nombres de vista. Solo el código opaco (<code>core/errors.py:124-126</code>)
Robustez	Heredar de <code>base.html</code> renderizaría el sidebar, que consulta <code>user.Empleado.notificaciones_no_leidas_count</code> y tocaría la BD posiblemente caída
Disponibilidad	La página se ve aunque WhiteNoise/S3 o la red de estáticos estén caídos

El fallo de CSRF no dice por qué falló. `csrf_failure` registra el `reason` con `logger.warning` y al usuario le muestra el mensaje genérico de sesión expirada (`core/errors.py:153-163`): decirle a un atacante si falló por «CSRF token missing» o por «Referer checking failed» le indica qué comprobación esquivar en el siguiente intento.

Cómo se revisan las páginas en desarrollo. Con `DEBUG=True` Django nunca llega a usar los handlers. Por eso se enruta `/__error__/<codigo>/` solo bajo `DEBUG` (`config/urls.py:38-48`): `/__error__/400/`, `/403/`, `/404/`, `/500/` y `/419/` —código libre para distinguir el CSRF del 403 genérico— (`core/errors.py:142-148`). Esa ruta **no existe en producción**.

El código de referencia en los envíos sin recarga. Las páginas de error solo aparecen cuando el navegador *navega*. Los seis formularios de solicitud envían por `fetch` y no recargan: ante un 500 pintaban su propio aviso y el identificador se perdía, aunque el servidor lo mandaba en la cabecera `X-Request-ID` de esa misma respuesta (`core/errors.py:51-76`). Justo el caso más grave —una solicitud que pudo quedar a medias en la base— era el que se quedaba sin código. Lo cierra `static/js/utils/codigo-referencia.js`.

Decisión	Dónde	Por qué
Se envuelve <code>`window.fetch`</code> en lugar de tocar cada envío	<code>static/js/utils/codigo-referencia.js:89-107</code>	Hay ~50 llamadas a <code>fetch</code> repartidas por los formularios, casi ninguna usa <code>ApiClient</code> y varias descartan la respuesta en su propio <code>.catch()</code> . Envolver una vez las captura todas sin tocar la lógica de envío
Solo se guarda el identificador de respuestas con error (<code>!response.ok</code>)	:94-96	Si se guardara el de cualquier respuesta, un sondeo de fondo correcto pisaría el código del fallo que el usuario acaba de ver
Vigencia de 60 s (<code>VIGENCIA_MS</code>)	:36, :40-44	Pasado ese plazo el código se considera de otro incidente y no se muestra: un código

		equivocado manda al equipo a leer la traza de otra petición, y eso es peor que no dar ninguno
Se carga antes que jQuery y el resto de scripts	<code>templates/base.html</code> (jQuery después)	Un script que se registre después no captaría las peticiones que ya hayan salido
Un stub define <code>window.CodigoReferencia</code> antes de cargar el fichero	<code>templates/base.html</code> , justo encima del <code><script src=...></code>	Sin él, si el fichero no llegara a cargarse los <code>.catch()</code> lanzarían <code>ReferenceError</code> antes de <code>rehabilitar()</code> / <code>restablecer()</code> : el usuario se quedaría sin aviso y con el formulario bloqueado. Una ayuda de diagnóstico no puede ser un punto de fallo en la propia ruta de error. El fichero real sobrescribe el stub al cargarse

Salvaguardas del shim, que **no se deben deshacer** (§ 16.1): no altera argumentos ni respuesta — devuelve la misma `Response` intacta (:101)—, **no encadena** `.catch``, así que un fallo de red sigue rechazando la promesa igual que antes (:103-105), y protege la lectura de la cabecera con `try/catch` para que una respuesta opaca no rompa la petición (:93-100).

La API tiene cuatro métodos: `ultimo()` devuelve el código vigente o `null` (:40-44); `mensaje(txt)` lo añade en **texto plano**, para los avisos que van en `text`: de `SweetAlert` (:53-58); `htmlMensaje(html)` lo añade como marcado, para los que se inyectan en `html`: — `notificar()` pasa su tercer argumento ahí, donde un `\n` no se vería— y sanea el valor con `/[^A-Za-z0-9]/g` antes de interpolarlo (:70-79); `registrar(codigo)` permite alimentarlo a mano si algún día se usa XHR (:82-86).

Ocho puntos de aviso de **envío fallido** en los seis `static/js/cambio-turno/solicitar_*.js` lo incorporan (p. ej. `solicitar_cambio_turno.js:421`, `solicitar_d_fds.js:442`, `solicitar_doblada_permanente.js:705`, `solicitar_cambio_descanso.js:1328,1362,1382`). Siete usan `htmlMensaje`, de modo que el código sale en el mismo recuadro monoespaciado en los seis formularios; `solicitar_ct_permanente.js:1748` y `solicitar_doblada.js:2862` pasaron de `text`: a `html`: justamente para no ser la excepción. El único que usa `mensaje()` es `solicitar_cambio_turno.js:421`, porque ahí el texto se compone antes de decidir en qué aviso se pinta y además se inspecciona con `includes()`. Ojo con ese punto: el código se aplica **sobre** `data.error || ...`, no solo sobre el mensaje de respaldo; envolver únicamente el respaldo hacía que el código se perdiera cada vez que el servidor mandaba texto propio. Los avisos de **carga** de listas (compañeros, dobladas) **no lo llevan a propósito**: no hay nada que reportar, se reintenta solo al cambiar de selección. El CSS del aviso está en `static/css/base_custom.css:23-36`:

`white-space: pre-line` en `.swal2-html-container`, porque los mensajes que llegan por `text:` traen saltos de línea reales, y el estilo del `<code>` del código.

La otra vía sin plantilla: los enlaces de aprobación por correo.

`templates/solicitudes/error_token.html` es una respuesta normal de una vista, no un handler, así que quedó fuera del barrido inicial. Cuatro `except Exception` de `solicitudes/views/aprobacion_email.py` hacían `render_error_token(request, f'Error al procesar la solicitud: {str(e)}')`: exactamente el CWE-209 que cierra esta sección, en una página que ve quien llega desde un correo, posiblemente sin sesión iniciada. El texto de una excepción de base de datos lleva fragmentos de SQL, nombres de tabla y de columna o el nombre de la restricción violada.

Ahora esos cuatro puntos llaman a `render_error_token_inesperado(request, e, contexto)` (`core/utils/error_token.py`), que registra la traza con `logger.exception` —el filtro le adjunta el mismo identificador que se muestra en pantalla— y devuelve el mensaje genérico `MENSAJE_INESPERADO` con un **500**. El docstring del módulo fija la regla: en `mensaje` solo va texto redactado por nosotros, nunca `str(excepcion)`.

Segundo arreglo en las mismas vistas: devolvían **200 OK** en todos los fallos, lo que los hacía invisibles para cualquier alarma que vigile códigos de error. Ahora **403** para un token inválido o expirado, **409** para un rechazo por reglas de negocio o por falta de supervisor, y **500** para lo inesperado. `render_error_token` conserva `status=200` por defecto solo por compatibilidad; hay un test que exige `status=` explícito en todas las llamadas de esas vistas.

La tercera vía: las APIs JSON. Mismo patrón, tercer frente. Nueve `except Exception` de vistas API devolvían el error crudo dentro del propio JSON:

```
except Exception as e:
    return JsonResponse({'error': f'Error al obtener festivos: {str(e)}'},
                        status=500)
```

Son respuestas normales de una vista, no handlers, así que tampoco las tocó el barrido inicial. `str(e)` de MySQL puede ser `(1054, "Unknown column 'turnos_diaespecial.descripcion' in 'field list'")` —nombres reales de tabla y columna— o `(2003, "Can't connect to MySQL server on 'swalp-prod.xxxx.rds.amazonaws.com'")`, que **expone el endpoint de RDS**: el caso más serio de los tres frentes por lo que revela, aunque no el más expuesto por quién lo alcanza.

Atenuante respecto a la fuga original del 404: las nueve **exigen sesión iniciada**. Cinco las alcanza cualquier empleado autenticado; tres solo un supervisor —las dos de `turnos/api/views/calculo_automatico.py`, con `SupervisorApiRequiredMixin (:37,83)`, y la de `solicitudes/views/gestion_solicitudes.py`, con `AdminRequiredMixin (:126)`—. Ninguna es anónima.

Punto corregido	Vista	Quién llega

<code>turnos/api/views/dias_especiales.py:186-188</code>	<code>DiasFestivosView (:11)</code>	empleado autenticado
<code>turnos/api/views/dias_especiales.py:263-265</code>	<code>DiasTemporadaView (:191)</code>	empleado autenticado
<code>turnos/api/views/dias_especiales.py:352-354</code>	<code>DiasEspecialesPorTipoView (:268)</code>	empleado autenticado
<code>turnos/api/views/calculo_automatico.py:78-80</code>	<code>CalcularMantenimientoAutomaticoView (:37)</code>	supervisor
<code>turnos/api/views/calculo_automatico.py:130-132</code>	<code>CalcularFestivosAutomaticoView (:83)</code>	supervisor
<code>turnos/api/views/turnos_mes.py:25-27</code>	turnos AM/PM de un día (<code>TurnosPorDiaView, :15</code>)	empleado autenticado
<code>solicitudes/views/api_turno_jornada.py:553-555</code>	cálculo de jornadas de un rango	empleado autenticado
<code>solicitudes/views/gestion_solicitudes.py:178-185</code>	reenvío de notificación desde Gestión	supervisor / staff

Los siete primeros llaman ahora a `json_error_inesperado(request, e, mensaje)` (`core/utils/json_responses.py:68-102`), hermano de `render_error_token_inesperado`: registra con `logger.exception (:99)` y devuelve **500** con el mensaje propio del endpoint más extra: `{request_id} (:101-102)`, el mismo código que el shim de `fetch` pinta en el aviso. El octavo no es JSON —pinta un `messages.error` en la pantalla de Gestión— y compone el código de referencia en el propio texto (`solicitudes/views/gestion_solicitudes.py:182-185`).

Decisión de diseño: mensaje específico por endpoint, no uno genérico único. Al contrario que la página del token, que usa `MENSAJE_INESPERADO`, aquí cada llamada pasa su propia frase ("No pudimos cargar los festivos. Inténtalo de nuevo.", `dias_especiales.py:188`; "No pudimos calcular los días de mantenimiento. Inténtalo de nuevo.", `calculo_automatico.py:80`). Un genérico único habría sido más barato de mantener, pero degrada la experiencia más de lo que exige la seguridad: el usuario pierde toda pista de **qué** falló. La regla que sí es dura la fija el docstring del helper (`core/utils/json_responses.py:96`): en `mensaje` solo va texto redactado por nosotros.

Hallazgo: seis de los nueve no dejaban traza. El arreglo no solo tapa la fuga, añade observabilidad que no existía. Tres no registraban absolutamente nada (`dias_especiales.py:186` y `:263`, `gestion_solicitudes.py:178`) y tres usaban

`logger.error(f"...{e}")` sin `exc_info`, así que guardaban el mensaje pero **no el traceback**. Solo dos estaban bien. Se estaba en lo peor de ambos mundos: el detalle se le enseñaba a quien no le sirve y no se le guardaba a quien lo necesita.

La novena, y la peor: un depurador que se quedó puesto. `MisTurnosPorMesView` (`turnos/api/views/turnos_mes.py`) no solo interpolaba `str(e)`: devolvía en el propio JSON `'traceback': error_trace if request.user.is_staff else None`. Es decir, **publicaba el traceback completo** —rutas de fichero, líneas de código y nombres de variables— a cualquier usuario `is_staff`, que aquí incluye a los supervisores. No era un descuido como las otras ocho: era una puerta abierta a propósito para depurar y nunca cerrada. La acompañaban dos `print()`, sin nivel y sin identificador de petición, de modo que en CloudWatch quedaban sueltos y sin poder cruzarlos.

El test de regresión tenía un agujero, y por eso se escapó. La primera versión comparaba **línea a línea**, así que un `JsonResponse` (en una línea y el `str(e)` en la siguiente) pasaban limpios. Ahora analiza el **bloque completo** del `except`, ignora los comentarios con `tokenize` —los propios comentarios que explican las fugas ya corregidas citan el patrón y se daban por reincidencias— y añade `test_no_se_publica_ningun_traceback`, que prohíbe que un `traceback` acabe en una respuesta aunque esté detrás de un `if request.user.is_staff`. La corrección se verificó ejecutando ambas versiones contra el código con la fuga: la antigua no la detecta, la nueva sí.

*Regla que deja este episodio: **una barrera de regresión que no se prueba contra el fallo que dice prevenir no es una barrera.** Antes de dar por buena una de estas comprobaciones, ejecútala contra el código defectuoso y comprueba que falla.*

Tests de regresión. `core/tests/test_paginas_error.py`, 19 tests; `core/tests/test_json_error_inesperado.py`, 16 tests sobre las APIs JSON (que ni el endpoint de RDS ni los nombres de tabla y columna lleguen al cliente, que el mensaje propio sí, que la traza quede en el log y que el código de referencia viaje en `extra`), uno de ellos parametrizado sobre los cinco módulos —lista `MODULOS` (:25)— que recorre el fichero con expresión regular y **falla si reaparece `str(e)` dentro de un `except Exception`** (:114-115); más `core/tests/test_error_token.py`, 8 tests sobre la página de los enlaces de correo (que no se filtre la excepción, que la traza sí quede en el log, los estados HTTP y que no reaparezca ninguna interpolación de `str(e)`) (`pytest core/tests/test_paginas_error.py --collect-only -q`), en tres bloques: que no se filtre nada de la lista `FUGAS` (:16-23), que el código de referencia funcione y sea del servidor (:60-91), que `base.html` defina el stub **antes** de cargar el fichero y este antes que jQuery, y que las cinco plantillas rendericen sin `request` y sin cargar `http(s)://` (:94-128).

10. Configuración

Un único `config/settings.py` gobernado por `ENVIRONMENT`; no hay `settings/local.py` (`config/settings.py:23`). Se lee con `django-environ` desde `.env` (`config/settings.py:11,21,25`); el porqué está en [ADR 004](#) (`./03-arquitectura/adr/004-variables-de-entorno-django-environ.md`).

Ningún valor real aparece en este manual. Los valores viven en `.env`, que no se versiona; la plantilla es `.env.example`.

Variable	Oblig.	Tipo	Por defecto	Qué se rompe si falta	Dónde se lee
<code>ENVIRONMENT</code>	sí	str (development\	<code>production)</code>	—	No arranca: gobierna todo el archivo
<code>SECRET_KEY</code>	sí	str	—	Improperly Configured al importar settings	<code>config/settings.py:33</code>
<code>DEBUG</code>	sí	bool	—	Ídem	<code>config/settings.py:34</code>
<code>ALLOWED_HOSTS</code>	no	lista	<code>['127.0.0.1', 'localhost']</code>	400 en cualquier otro host	<code>config/settings.py:35</code>
<code>CSRF_TRUSTED_ORIGINS</code>	no	lista	<code>[]</code>	POST rechazados tras un proxy con otro origen	<code>config/settings.py:39</code>
<code>DB_NAME</code>	sí	str	—	No conecta a la base	<code>config/settings.py:170</code>
<code>DB_USER</code>	sí	str	—	Ídem	<code>config/settings.py:171</code>
<code>DB_PASSWORD</code>	sí	str	—	Ídem	<code>config/settings.py:172</code>
<code>DB_HOST</code>	no	str	<code>localhost</code>	Apunta a la base equivocada	<code>config/settings.py:173</code>
<code>DB_PORT</code>	no	str	<code>3306</code>	Ídem	<code>config/settings.py:</code>

					174
DB_CONN_MAX_AGE	no	int	60 en prod, 0 en dev	Reconexión en cada petición (latencia)	config/settings.py: 142
DB_SSL_CA	no	str (ruta)	' '	Sin TLS contra RDS	config/settings.py: 157
TEST_DB_NAME	no	str	test_<DB_NAME>	Dos corridas de tests comparten base y se pisan	config/settings.py: 191
EMAIL_BACKEND	no	str	django.core.mail.backends.smtp.EmailBackend	Con ...console .EmailBackend el correo —enlace incluido— se imprime en la consola del runserver en vez de salir	config/settings.py: 257
EMAIL_HOST	no	str	smtp.gmail.com	Correo al servidor equivocado	config/settings.py: 258
EMAIL_PORT	no	int	587	Ídem	config/settings.py: 259
EMAIL_USE_TLS	no	bool	True	Credenciales en claro	config/settings.py: 260
EMAIL_HOST_USER	sí	str	—	Improperly Configured	config/settings.py: 261
EMAIL_HOST_PASSWORD	sí	str	—	Ídem	config/settings.py: 262
DEFAULT_FROM_EMAIL	sí	str	—	Ídem	config/settings.py: 263
EMAIL_TIMEOUT	no	int (s)	10	Un SMTP colgado congela el hilo de entrega hasta	config/settings.py: 266

				ese límite	
EMAIL_SEND_ASYNC	no	bool	IS_PRODUCTION (True en prod, False fuera)	Con False el lote de correo se entrega dentro del request : la respuesta espera al handshake SMTP. Se recomienda True también en desarrollo y así viene en <code>.env.example:52</code> ; en tests debe quedar 'False' para que <code>mail.outbox</code> se pueble dentro del propio test	<code>config/settings.py: 271</code>
SITE_URL	no	str	<code>http://127.0.0.1:8000</code>	Los enlaces de aprobación y los botones «Ver en el sistema» de los correos apuntan a localhost, es decir, a la máquina de quien lee (§ 4.4). El logo NO depende de esto: va incrustado en el mensaje	<code>config/settings.py: 273</code>
APPROVAL_LI	no	int	30	Nada al	<code>config/settings.py:</code>

NK_MAX_AGE_DAYS		(días)		arrancar: sin la variable rigen 30 días. Bajarla acorta la vida de los enlaces del correo ya enviados; subirla amplía la ventana en que un enlace filtrado sigue sirviendo	282, leída en solicitudes/service/s/tokens_aprobacion.py:53
CACHE_URL	no	str	' ' (LocMem)	Con más de un worker, la invalidación de "Mis Turnos" solo limpia un proceso	config/settings.py: 315
CORS_ALLOWED_ORIGINS	no	lista	localhost:8000	Peticiones cruzadas bloqueadas	config/settings.py: 353
SECURE_HTTPS	no	bool	True en prod	Sin redirección a HTTPS, o bucle de redirección si no hay proxy	config/settings.py: 511
LOG_DIR	no	str (ruta)	<BASE_DIR>/logs	Nada. Solo aplica en desarrollo: en producción no se escribe fichero y todo sale por stdout (§ 13.5)	config/settings.py: 557

Valores de `CACHE\_URL` (config/settings.py:315-348): vacío → LocMemCache, solo desarrollo con un proceso; redis://host:6379/1 o rediss://... → backend Redis nativo de Django 5,

opción recomendada en AWS; `db://cache_appturnos` → tabla en la propia MySQL, que exige `python manage.py createcachetable` una vez.

Diferencias dev / test / producción

Aspecto	development	test	production
ENVIRONMENT	development	el del .env	production
Caché	LocMem	LocMem	Redis o tabla, compartida
EMAIL_SEND_ASYNC	True por .env (el default sigue siendo False)	False	True
django-axes	activo	desactivado (config/settings.py:412-416)	activo
SECURE_HTTPS	False	False	True
Base de datos	DB_NAME	TEST_DB_NAME (config/settings.py:191)	DB_NAME
Test runner	—	core.test_runner.NoInputDiscoverRunner (config/settings.py:199)	—
debug_toolbar	solo con DEBUG=True (config/urls.py:38-44)	no	no

En Docker, ambos ficheros compose fijan `ENVIRONMENT=production` con `SECURE_HTTPS=False`, para poder navegar en HTTP local (§ 2.7).

11. Dependencias

11.1 Producción (requirements.txt)

Paquete	Versión	Para qué	Riesgo al actualizar
Django	5.2.16	Framework	Alto: subir de mayor exige revisar, entre otras cosas, <code>CheckConstraint(condition=...)</code>

			, que cambió de nombre respecto a check= — el proyecto usa las dos formas (turnos/models.py:91 vs solicitudes/models.py:322)
PyMySQL	1.1.2	Driver MySQL puro Python, instalado como MySQLdb si falta el nativo (config/settings.py:115-119)	Medio: install_as_MySQLdb() es sensible a la versión
django-environ	0.14.0	Lee .env y tipa las variables	Bajo
django-simple-history	3.8.0	Tabla historical* por modelo	Alto: cada subida suele traer migraciones nuevas en todos los modelos con historial, que aquí son casi todos
django-axes	7.0.1	Bloqueo por intentos fallidos de login	Medio: la rama 7.x cambió nombres de ajustes de bloqueo
django-csp	4.0	Cabeceras CSP	Medio: la 4.0 introdujo el formato de diccionario CONTENT_SECURITY_POLICY que usa este proyecto
django-cors-headers	4.9.0	CORS	Bajo
django-widget-tweaks	1.5.0	Retoques de widgets en plantillas	Bajo
whitenoise	6.8.2	Sirve los estáticos desde el propio proceso, sin Nginx en el contenedor	Bajo
gunicorn	23.0.0	Servidor WSGI de producción	Bajo
redis	5.2.1	Solo el cliente. El backend de caché es el nativo de Django 5; no hace falta django-redis (comentario en requirements.txt)	Bajo
openpyxl	3.1.5	Exportación de reportes a Excel	Bajo
cryptography , cffi , pyparser	46.0.3 / 2.0.0 / 2.23	Transitivas de TLS	Medio: cryptography exige ruedas binarias compatibles con el intérprete

python-dateutil	2.9.0.post0	Aritmética de fechas	Bajo
tzdata	2025.2	Zonas horarias en sistemas sin base tz	Bajo pero necesario : <code>USE_TZ=True</code> con <i>America/Bogota</i>
asgiref, sqlparse, six	3.11.0 / 0.5.3 / 1.17.0	Transitivas de Django	Bajo

Todas las versiones están **fijadas con** ``==``. Es deliberado: sin lockfile, el pin exacto es lo único que hace reproducible un `pip install`.

11.2 Desarrollo (requirements-dev.txt)

Hereda producción con `-r requirements.txt` y añade cinco herramientas: `django-debug-toolbar` 6.2.0, `pytest` 8.3.4, `pytest-django` 4.9.0, `pytest-xdist` 3.8.0 y `pytest-cov` 7.1.0.

`pytest-xdist` no está solo por velocidad: da a cada worker su propia base de test con sufijo `_gwN`, lo que elimina de raíz que dos corridas simultáneas se pisen la misma base `test_*` (comentario de `requirements-dev.txt` sobre `pytest-xdist`).

11.3 Frontend y externas

- **Sin paso de build.** AdminLTE 3.2 y JavaScript vanilla servidos como estáticos (`docs/03-arquitectura/TECNOLOGIAS_FRONTEND.md`).
- **Autohospedados en ``static/plugins/``:** flatpickr, chart.js, sweetalert2, fullcalendar, ionicons y Font Awesome 5.15.4 (`config/settings.py:349-352, 394-396`).
- **Lo único externo que queda:** Google Fonts (`fonts.googleapis.com`, `fonts.gstatic.com`) en `base.html` y el login. Es lo único que la política CSP objetivo sigue permitiendo (`config/settings.py:396`).
- **SMTP / SES.** Se configura por variables de entorno (§ 10); el envío pasa siempre por `EmailOutbox`.
- **AWS.** RDS o Aurora para MySQL, ElastiCache opcional para la caché; ver § 13 y [05-referencia/deployment/](#) (`./05-referencia/deployment/`).

No se encontró integración activa con Google Apps Script en el código de `AppTurnosExplora/`; ver § 18.

12. Recetas (how-to)

12.1 Añadir un tipo de solicitud

Guía completa: [04-guias/GUIA_AGREGAR_NUEVO_TIPO_SOLICITUD.md](#) (./04-guias/GUIA_AGREGAR_NUEVO_TIPO_SOLICITUD.md). El recorrido, resumido:

130. Añade el nombre a `TipoSolicitud` en `core/constants.py` y su equivalente en `TipoCambioTurno` si escribe turnos; actualiza `MAPA_SOLICITUD_A_TURNO`.
131. Amplía `TipoCambioTurno.TODOS` y migra: la `CheckConstraint turno_tipo_cambio_valido` (`turnos/models.py:90-94`) rechazará el valor nuevo hasta entonces.
132. Crea la fila de `TipoSolicitudCambio` con ese nombre y su `codigo_estrategia`.
133. Crea `services/strategies/<tipo>_strategy.py` implementando el contrato de `SolicitudStrategy` (`services/strategies/base_strategy.py`): `validar_solicitud`, `crear_solicitud`, `aplicar_cambios`, `revalidar_para_aprobar (:84)` y `get_empleados_disponibles`.
134. Regístrala en `SolicitudFactory` (`services/solicitud_factory.py:131-195`). Si cae en la estrategia por defecto verás un `warning` en el log (`:182-187`).
135. Si genera o revierte turnos, crea `services/<tipo>_aplicacion_service.py` con `snapshot` previo y resultante, y engánchalo en `_revertir_por_tipo` (`use_cases/cancelar_solicitud.py:646`).
136. Ejecuta `python manage.py actualizar_codigos_estrategia`.
137. Escribe el test **antes** de tocar el frontend.

12.2 Añadir un campo a un modelo

```
python manage.py makemigrations <app>
python manage.py migrate
```

Con `django-simple-history` la migración toca **dos** tablas: la del modelo y su `historical*`. Si el campo es obligatorio, dale `default` o hazlo `null=True`: la tabla histórica ya tiene filas y no puede quedarse sin valor. Si el campo participa en un invariante, exprésalo como `constraints` en `Meta` y no solo en `clean()`: `clean()` no corre en `save()` (patrón 26 de `PROTECTION_PATTERNS.md`, y por eso `Turno` valida `tipo_cambio` con una `CheckConstraint`, `turnos/models.py:90-94`).

12.3 Correr los tests

```
cd AppTurnosExplora
```

```

pytest                                # toda la suite
pytest -n 4                            # en paralelo – unos 3 minutos
pytest solicitudes/tests/test_politica_temporada.py -v
pytest --cov=solicitudes --cov-report=term-missing

```

`pytest.ini` fija `DJANGO_SETTINGS_MODULE=config.settings` y los `testpaths` (`pytest.ini:2,11-18`). Ver § 14.

12.4 Depurar un fallo de validación

138. Identifica el tipo: `SolicitudCambio.tipo_cambio.nombre` frente a `TipoSolicitud` de `core/constants.py`. Si estás filtrando `Turno.tipo_cambio`, el vocabulario es **otro** (`TipoCambioTurno`); confundirlos no da error, da un filtro que no casa con nada (`core/constants.py`, docstring del módulo).

139. Localiza la estrategia con `SolicitudFactory.get_strategy` (`services/solicitud_factory.py:131`). Si cayó en la estrategia por defecto hay un **warning** en el log (`:182-187`): ahí está el problema.

140. Los chequeos transversales fallan **antes** que la estrategia. Mira el `code` de la respuesta JSON: `cierre_semanal`, `sancionado`, `sancionado_receptor` y `duplicate_request` señalan al orquestador, no al tipo.

141. Si falla al **aprobar** y no al crear, es la re-validación (P5): el estado del día cambió entre medias.

142. Si el desplegable ofrece a alguien que el envío rechaza, es el patrón 37: selector y validación miran fechas distintas (`solicitudes/views/api_fin_semana.py:267-274`).

12.5 Regenerar la documentación

```

python .claude/skills/project-documentation-master/scripts/md_to_pdf.py \
  AppTurnosExplora/docs/manual_tecnico.md \
  AppTurnosExplora/docs/pdf/Manual_Tecnico.pdf \
  --titulo "Manual Tecnico" --subtitulo "AppTurnos / SWALP"

```

Requiere Microsoft Word (usa COM). El PDF **nunca** se edita a mano: se regenera desde el Markdown, que es la fuente de verdad.

El exportador fallaba en silencio. `ExportAsFixedFormat` podía no escribir nada sin lanzar excepción: el script imprimía «PDF generado», salía con código 0 y en disco seguía el PDF de la vez anterior. Solo se notaba mirando la fecha del archivo. Desde ahora `md_to_pdf.py` **aparta** el PDF anterior a `.anterior` antes de exportar y comprueba después que Word escribió uno nuevo (`apartar_destino` y `verificar_pdf`): si no lo hizo, restaura el anterior —no deja un hueco— y sale con error diciendo qué mirar, que casi siempre es un visor con el PDF abierto bloqueando el archivo, o una instancia de Word colgada. El mensaje de éxito ahora incluye el tamaño.

Si Word aborta con *"no podemos guardar el archivo porque es de solo lectura"*, casi siempre hay un **visor con el PDF abierto** bloqueando la escritura (Adobe Reader lo hace); ciérralo y repite. El atributo del archivo puede seguir siendo normal, así que el mensaje despista.

El aviso automático de documentación desfasada

Un hook **Stop** (`.claude/hooks/docs-desactualizados.sh`) avisa al terminar una sesión de trabajo cuando la documentación pudo quedarse atrás. Emite **dos** avisos distintos, que conviene no confundir:

Aviso	Cómo se decide	Qué hacer
Falta exportar	El <code>.md</code> es más nuevo que su PDF (por fecha de archivo: son dos artefactos del mismo árbol)	Regenerar el PDF con el comando de arriba
Código sin revisar	El commit actual o el trabajo sin confirmar no coinciden con el sello <code>docs/.revisado-hasta</code>	Juzgar si esos cambios afectan a los manuales, actuar en consecuencia y sellar

Sellar una revisión —tanto si hubo que escribir algo como si se concluyó que no hacía falta—:

```
bash .claude/hooks/sellar-docs.sh
```

El sello guarda dos datos: el `commit` revisado y una huella **por contenido** del código aún sin confirmar. La segunda línea es la que evita el ruido: sin ella, cualquier rama con trabajo en curso dejaría el aviso saltando en cada sesión hasta commitear.

Por qué un sello y no la fecha de los PDF (que es como funcionaba antes): el `mtime` no sobrevive a git — `git clone` escribe todos los archivos con la hora del clon, y `checkout`, `pull` o `stash pop` lo revuelven igual, así que el resultado pasaba a depender del orden de escritura. Además confundía *"el PDF está viejo"* con *"falta documentación"*: un cambio de código sin efecto en los manuales dejaba el aviso saltando para siempre, porque revisarlo y concluir "no hacía falta" no dejaba rastro. Y un aviso que salta siempre se acaba ignorando siempre.

Dos limitaciones conocidas, ambas a propósito:

143. **Los hooks no se versionan.** `.gitignore:44` ignora `.claude/` entero, así que

`docs-desactualizados.sh` y `sellar-docs.sh` viven solo en la máquina de quien los escribió; el sello sí se versiona. Sacar `hooks/` de esa regla **no basta**: el hook lo dispara el bloque **"Stop"** de `.claude/settings.json`, también ignorado, y ese archivo mezcla la configuración del proyecto con ~200 líneas de permisos locales (rutas de la máquina, scripts de depuración). Si algún día el repo lo trabaja más de una persona, la forma correcta es versionar `hooks/` y `settings.json`, moviendo antes el allowlist de permisos a `settings.local.json`, que sí es personal. Hacerlo a medias deja scripts en el árbol que nadie ejecuta.

144. **La lista de archivos "sin commitear" peca de larga.** El sello guarda una huella del conjunto,

no la lista de archivos, así que el hook sabe *que* algo cambió pero no *cuál* respecto a la última revisión: enumera todo lo modificado frente a **HEAD**. Con lo ya commiteado no ocurre (ahí compara **sello..HEAD** y es exacto). Nunca oculta un archivo que cambió; muestra de más. Se corregiría guardando **archivo:hash** en el sello, a cambio de un sello de decenas de líneas que ensucia cada diff — no compensa con el volumen actual.

Un cambio que solo toca tests **no** dispara el aviso: la especificación de rutas excluye **tests/**, **migrations/**, **docs/** y los estáticos de terceros.

12.6 Desplegar

Ver § 13 y los checklists de [05-referencia/deployment/](/05-referencia/deployment/) (`./05-referencia/deployment/`).

13. Despliegue

13.1 Topología

Decidido el 2026-09-04: EC2 `t4g.micro` + RDS `db.t4g.micro` Single-AZ. ECS Fargate quedó descartada por presupuesto (~\$64/mes con IVA frente a ~\$32) y su checklist se borró. El razonamiento completo, con precios verificados, está en

[arquitectura-aws-rds-recomendada.md](/05-referencia/deployment/99-aws/arquitectura-aws-rds-recomendada.md) (`./05-referencia/deployment/99-aws/arquitectura-aws-rds-recomendada.md`)

§5 y §8.

```
flowchart TB
    USER["Navegador<br/>explorador / supervisor"]
    DNS["swalp.parqueexplora.org<br/>registro A (lo agrega IT)"]
    subgraph AWS["AWS - us-east-1"]
        EIP["Elastic IP<br/>(IP publica fija)"]
        subgraph EC2BOX["EC2 t4g.micro (Ubuntu 24.04 ARM)"]
            NGINX["Nginx - termina TLS (Let's Encrypt)<br/>sirve /static/ desde disco"]
        end
        GUNI["Gunicorn 3 workers<br/>Django"]
        CRONJ["crontab:<br/>procesar_email_outbox (5 min)<br/>revisar_sanciones_por_deuda (diario)"]
        end
        RDS["RDS MySQL db.t4g.micro<br/>Single-AZ, TLS con DB_SSL_CA<br/>+ tabla cache_appturnos"]
        LOGS["CloudWatch Logs"]
        end
        SES["Amazon SES"]

    USER -->|HTTPS| DNS
    DNS --> EIP
    EIP --> NGINX
    NGINX -->|socket Unix + X-Forwarded-Proto/For| GUNI
```

```
GUNI --> RDS
GUNI --> LOGS
CRONJ --> RDS
CRONJ --> SES
SES -->|enlace firmado| USER
```

Qué muestra. Un solo proceso de aplicación detrás de **Nginx en la propia instancia** — sin balanceador, que a esta escala solo añadiría ~\$18/mes. Nginx termina el TLS y reenvía por HTTP con `X-Forwarded-Proto`; sin `SECURE_PROXY_SSL_HEADER`, `SECURE_SSL_REDIRECT` provocaría un bucle infinito de redirecciones (`config/settings.py:512`).

Los estáticos los sirve Nginx desde disco, no WhiteNoise: son 143 MB y pasarlos por los workers de Gunicorn desperdiciaría los 2-3 que caben en 1 GB de RAM. El middleware de WhiteNoise sigue en la lista (`config/settings.py:75`) porque es quien los sirve **en el contenedor** de pruebas locales (§ 2.7), donde no hay Nginx.

Gunicorn corre con 3 workers, y de ahí que la caché **tenga que ser compartida** y nunca `LocMemCache` (`config/settings.py:285-345`). En producción es `'CACHE_URL=db://cache_appturnos'`, la tabla dentro de la propia RDS: ElastiCache (~\$12/mes) se sale del presupuesto. Exige `python manage.py createcachetable` una vez.

El correo no viaja dentro de la petición web: con `EMAIL_SEND_ASYNC` la entrega ocurre tras el commit, en un hilo del propio proceso Gunicorn, y todo el lote sale por una única conexión SMTP. Lo que ese intento no logre entregar lo recoge un **cron** independiente que barre `EmailOutbox`. En EC2 son líneas de `crontab`, que cuestan \$0 — en Fargate habrían necesitado EventBridge y ~8.640 lanzamientos de tarea al mes.

La base es MySQL gestionada, con TLS si `DB_SSL_CA` apunta al certificado (`config/settings.py:157-160`).

Infraestructura como código: los recursos AWS del diagrama (security groups, EC2, Elastic IP, RDS, alarmas de costo) están en `infra/cloudformation/swalp-infra.yaml` (`./infra/cloudformation/swalp-infra.yaml`). La configuración *dentro* del servidor (Nginx, certbot, systemd, crontab, `.env`) es manual a propósito: ver arquitectura §12.1.

Paso a paso: [CHECKLIST DESPLIEGUE AWS RDS.md](#) (`./05-referencia/deployment/99-aws/CHECKLIST_DESPLIEGUE_AWS_RDS.md`) (FASES 0-11) y [MANUAL DESPLIEGUE EC2.md](#) (`./05-referencia/deployment/99-aws/MANUAL_DESPLIEGUE_EC2.md`). Cómo encajan la URL, la IP y las variables está en arquitectura §11.

13.2 Pipeline y pasos

CI: sí existe (`.github/workflows/ci.yml`, en la raíz del repositorio, no dentro de `AppTurnosExplora/`). Corre en cualquier rama, con dos jobs: `test` (bloqueante) levanta MySQL 8.0, carga las tablas de zona horaria, ejecuta `check`, `check --deploy` con entorno de producción simulado —así `core.E001` impide desplegar sin caché compartida— y la suite con gate de cobertura; y `lint` (informativo) con ruff, bandit y pip-audit.

CD: pendiente a propósito. Un `deploy.yml` que entre por SSH necesita que la EC2 exista para tener `SSH_HOST`, `SSH_USER` y `SSH_KEY`; se creará justo después de la FASE 6 del checklist. Hasta entonces el despliegue es manual:

145. Desplegar la pila: `aws cloudformation deploy --template-file infra/cloudformation/swalp-infra.yaml`.
146. Por SSH: `git pull` y `pip install -r requirements.txt`.
147. Aplicar migraciones: `python manage.py migrate`.
148. ``python manage.py createcachetable`` — solo la primera vez, pero **obligatorio**: sin la tabla la app arranca y falla en la primera carga de Mis Turnos.

149. `python manage.py collectstatic --noinput` → `STATIC_ROOT` (`config/settings.py:224`), de donde

los sirve Nginx.

150. `sudo systemctl restart appturnosex` (Gunicorn con 3 workers).
151. Verificar `/health/` y `/health/ready/`.

Ojo: `docker-compose.hostdb.yml` **sobrescribe el ``command``** para arrancar solo Gunicorn sin `migrate` (§ 2.7). En producción el `migrate` es un paso explícito, no implícito.

13.3 Checklist previo

- [] `python manage.py check --deploy` sin errores críticos. Conviene convertirlo en ****puerta del**

pipeline** con `--fail-level WARNING`, para que un `DEBUG=True` colado no llegue a desplegarse.

- [] `ENVIRONMENT=production` y `DEBUG=False`.
- [] `SECRET_KEY` distinta de la de desarrollo. **Pendiente real hoy:** con

`DEBUG=False SECURE_HTTPS=True python manage.py check --deploy` el único aviso que queda es `security.W009` — la clave sigue siendo la autogenerada con prefijo `django-insecure-`, y además estuvo expuesta en pantallas de debug. Con ella se firman las cookies de sesión: quien la tenga puede falsificar la sesión de cualquier usuario, incluido un superusuario. Generar la nueva con `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"` y guardarla en Secrets Manager. No se ha rotado en el repositorio a propósito: invalida todas las sesiones activas, así que es una decisión de despliegue.

- [] `SECRET_KEY` **idéntica en todas las instancias**: una sola entrada en Secrets Manager o SSM, nunca un valor generado por tarea. **El procedimiento documentado ya lo garantiza** (§ 9.2); esto se marca comprobando en la consola que `swalp/SECRET_KEY` es un único secreto y que el task definition lo inyecta en `secrets:`, no en `environment:`. Firma los tokens de aprobación por correo, y si cada instancia usa la suya los enlaces fallan de forma intermitente

según a cuál encamine el ALB (`solicitudes/services/tokens_aprobacion.py:59-62,84`). Rotarla invalida los enlaces ya enviados: avisa antes.

- [] `APPROVAL_LINK_MAX_AGE_DAYS` revisada si el plazo de respuesta real no cabe en 30 días (`config/settings.py:262`).
- [] `ALLOWED_HOSTS` y `CSRF_TRUSTED_ORIGINS` con el dominio real.
- [] `SITE_URL` con el dominio real: si no, los enlaces de aprobación del correo apuntan a `127.0.0.1` (`config/settings.py:253`).
- [] `CACHE_URL` apuntando a Redis o a tabla. **Nunca vacío con más de un worker.**
- [] Si es tabla: `python manage.py createcachetable` ejecutado una vez.
- [] `DB_SSL_CA` con el bundle de RDS.
- [] `SECURE_HTTPS=True` y **Nginx** reenviando `X-Forwarded-Proto` y `X-Forwarded-For` (el segundo lo necesita axes; ver arquitectura §11.5 y `manage.py verificar_ip_cliente`).
- [] Cron de `procesar_email_outbox` activo.
- [] **Alarma del cierre semanal creada** (§ 13.6): metric filter sobre

CIERRE SEMANAL INOPERATIVO + alarma con umbral 1 hacia SNS. Sin esto, el cierre puede quedar desactivado de hecho sin que nadie se entere: la comprobación falla **abierta** a propósito y su única señal es una línea de log.

- [] Calendario del año cargado (§ 2.5): festivos, temporada, mantenimiento, alternancia y descansos de semana.
- [] Los seis `TipoSolicitudCambio` creados con los nombres exactos de `core/constants.py`.
- [] Los roles `Supervisor` y `Explorador` y las jornadas `AM` y `PM` existentes.

Checklists completos: [CHECKLIST DESPLIEGUE AWS RDS.md](#) (`./05-referencia/deployment/99-aws/CHECKLIST_DESPLIEGUE_AWS_RDS.md`) y [CONFIGURACION PRODUCCION.md](#) (`./05-referencia/deployment/03-operacion/CONFIGURACION_PRODUCCION.md`).

13.4 Rollback

Volver a la imagen anterior es inmediato. **Las migraciones no:** varias añaden constraints que no se pueden revertir sin perder la garantía (§ 5.3). Regla práctica: si el despliegue incluyó migraciones que crean `UniqueConstraint` o `CheckConstraint`, el rollback de código es seguro pero el de esquema exige revisión manual. Antes de cualquier despliegue con migraciones, snapshot de RDS.

13.5 Observabilidad

Qué	Dónde se mira
Salud del proceso	<code>GET /health/</code> — no toca la base

Salud de la base	GET /health/ready/
Errores de aplicación	logger.exception en vistas y servicios → stdout → CloudWatch
Traza de una petición concreta	El código de referencia que el usuario ve en la página de error (§ 9.5). Filtrarlo en CloudWatch devuelve todas sus líneas
Cierre semanal caído	Alarma sobre CIERRE SEMANAL INOPERATIVO (ver § 13.6). Buscarlo a mano no basta: el fallo es silencioso
Tipo de solicitud sin estrategia	warning de la factory (solicitudes/services/solicitud_factory.py:190-195)
Los crons no corren	Dos comandos, dos trabajos. python manage.py verificar_crons (--json para consumo automático) diagnostica: comprueba el del outbox y el de sanciones, cada problema sale marcado con `CRON_NO_EJECUTADO` o `CORREOS_FALLIDOS` y termina con código 1 (solicitudes/management/commands/verificar_crons.py:69-70,124-127). python manage.py alertar_crons corre esos mismos tres chequeos y manda el correo él mismo (solicitudes/management/commands/alertar_crons.py:104-108), porque Dokploy no dispara notificación cuando un Schedule falla: destinatarios en --para o en la variable de entorno `ALERTAS_CRON_EMAIL` —sin ninguno falla ruidosamente (:159-169)— y transporte smtplib directo, al margen del outbox que vigila (:205-242). Los lunes manda un latido de «sin novedad» aunque todo esté bien (:119-121): un vigilante mudo y uno muerto se ven igual desde fuera. Probarlo con --dry-run, que imprime el correo sin enviarlo (:130-137)
Correos atascados	EmailOutbox con estado='fallido' — agotó los 5 intentos (solicitudes/models.py:73,81). Nadie las recoge ya: procesar_email_outbox --resumen las lista para que se vean
Solicitudes atascadas	Pantalla /solicitudes/gestion-solicitudes/
Auditoría de cambios	Tablas historical* de django-simple-history

Configuración de LOGGING

Definida en config/settings.py:617-700, con la elección de destinos en :582-592. Los errores **no** se le muestran al usuario (§ 9.5): salen por aquí, y solo por aquí. Dos destinos, cada uno por un motivo distinto:

Handler	Destino	Por qué
console	stdout	En EC2, systemd captura stdout en el journal y el agente de CloudWatch lo publica; el

		proceso no depende de ningún fichero propio. (Era también el motivo en contenedores, donde el log driver <code>awslogs</code> recoge stdout directamente)
<code>file</code>	<code>logs/appturnos.log</code> , sin rotación. Solo en desarrollo	Comodidad local: poder abrir el log sin depender de la consola donde corre <code>runserver</code> . En producción este handler no se usa (ver abajo)

Loggers configurados: `solicitudes` e `turnos` a `INFO`; `django.request` a `ERROR` —**aquí aterrizan los 500 con su traceback completo**, es el logger que sustituye a la pantalla de debug—; `django.security` a `WARNING` (fallos de CSRF, `Host` fuera de `ALLOWED_HOSTS`); y `core.errors`. La raíz, a `WARNING`.

El filtro ``request_id``. `core.errors.RequestIDFilter` inyecta el identificador en cada registro, y ambos formatters lo imprimen entre corchetes. Por eso buscar el código que el usuario reporta reconstruye la petición entera:

```
aws logs filter-log-events --log-group-name /swalp/app --filter-pattern
'"A3F91C2B7D01"'
```

Fuera de una petición el campo vale `-`.

En producción no se escribe fichero, y es deliberado. `LOG_A_FICHERO` queda en `False` en cuanto `IS_PRODUCTION`, así que `_DESTINOS` se reduce a `['console']`. La razón que se mantiene en pie con el plan EC2 es la tercera, y es suficiente por sí sola:

Escribir a fichero desde varios workers corrompe el log. Con `--workers 3` hay tres procesos con su propio handler sobre el mismo fichero; al rotar, los tres renombran a la vez. En Linux no da error: se pierden líneas y algún worker sigue escribiendo en un fichero ya desenlazado.

Matiz que cambió al elegir EC2.** Las otras dos razones que figuraban aquí —que el fichero no sobrevive al reinicio de la tarea y que no hay máquina donde hacer `tail -f`— eran *argumentos de contenedor****, y con una EC2 ya no aplican: el disco persiste y sí se puede entrar por SSH. La decisión **no cambia**, porque la corrupción entre workers basta y porque el journal de `systemd` ya da lo que daría el fichero, pero conviene no repetir un motivo que dejó de ser cierto.*

La aplicación **emite** eventos; dónde se guardan es decisión de la plataforma. En desarrollo, si `LOG_DIR` no se puede crear o no es escribible, la aplicación tampoco se cae: renuncia al fichero y todo sale por stdout.

Fijado por `core/tests/test_logging_destinos.py` (4 tests), que además impide que vuelva a colarse un handler con rotación.

*Retención y datos personales. Estos logs contienen nombres de empleado y detalles de solicitudes. La retención se configura **en el grupo de CloudWatch**, no en el fichero, y el acceso al grupo se restringe por IAM. Logs/ está en `.gitignore`: no se versiona nunca.*

13.6 Alarma obligatoria: el cierre semanal falla ABIERTO

`ProcesarSolicitudOrchestrator.verificar_cierre` envuelve toda la comprobación del cierre en un `try/except Exception`. Si la verificación se rompe, **la solicitud se acepta sin validar el cierre** y solo queda un `logger.critical` (`solicitudes/services/solicitud_orchestrator.py:152-160`).

La decisión de fallar abierto es deliberada y está razonada en el docstring: un *fail-closed* dejaría a **todos** los exploradores sin poder enviar ninguna solicitud, mientras que dejar pasar alguna fuera de plazo el supervisor todavía puede rechazarla a mano. Romper el formulario a todo el mundo es peor que colar una solicitud.

Pero esa decisión solo es aceptable si alguien se entera. El comentario del código dice que se registra en CRITICAL "para que el fallo sea visible en alertas"; esa frase asume una alerta que **hay que crear en el despliegue**. Sin ella, el cierre semanal queda desactivado de hecho y nadie lo nota: no hay error visible ni formulario roto, simplemente empiezan a entrar solicitudes fuera de plazo una a una. Es un fallo silencioso y progresivo, el peor tipo.

Hoy el único destino de los logs es `console` → stdout (`config/settings.py`, bloque `LOGGING`), y `ADMINS` no está definido, así que **no existe ningún envío automático**.

EC2 (el despliegue elegido)

Requiere el **agente de CloudWatch** en la instancia, publicando el journal de systemd al grupo `/swalp/app` (FASE 11 del checklist). Con eso, faltan el filtro y la alarma:

```
# 1. Tema SNS al que suscribir a quien deba enterarse
aws sns create-topic --name swalp-alertas
aws sns subscribe --topic-arn <arn-del-tema> --protocol email --notification-
endpoint <tu-correo>

# 2. Metric filter: cuenta las apariciones del texto en el log
aws logs put-metric-filter \
  --log-group-name /swalp/app \
  --filter-name swalp-cierre-inoperativo \
  --filter-pattern '"CIERRE SEMANAL INOPERATIVO"' \
  --metric-transformations \

metricName=CierreSemanalInoperativo,metricNamespace=SWALP,metricValue=1,defaultValu
e=0
```

```
# 3. Alarma: una sola aparición ya es motivo de aviso
aws cloudwatch put-metric-alarm \
  --alarm-name swalp-cierre-semanal-inoperativo \
  --namespace SWALP --metric-name CierreSemanalInoperativo \
  --statistic Sum --period 300 --evaluation-periods 1 \
  --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions <arn-del-tema>
```

El umbral es 1: una sola aparición significa que el cierre ya no está protegiendo nada. El **Sum** sobre 5 minutos además distingue un fallo puntual de una avería continua (doscientas apariciones en una hora no es lo mismo que una suelta).

Conviene una segunda alarma con el mismo patrón sobre **?ERROR ?CRITICAL** para no depender de haber previsto cada texto concreto.

EC2 (instancia única)

No hay CloudWatch por defecto: los logs de Unicorn van a **journald**. Dos opciones:

- Instalar el **CloudWatch agent** apuntando al journal de la unidad y aplicar el mismo filtro.
- Sin agente, un cron diario que avise:

```
journalctl -u appturnosex --since "24 hours ago" | grep -q "CIERRE SEMANAL
INOPERATIVO" \
  && echo "Cierre semanal inoperativo en las ultimas 24h" | mail -s "SWALP:
alerta" <tu-correo>
```

Qué hacer cuando la alarma suena (runbook)

Que suene significa **dos** cosas, y la segunda se olvida: la comprobación está rota **y**, mientras lo estuvo, entraron solicitudes sin validar el cierre. Arreglar solo lo primero deja solicitudes coladas para un fin de semana ya cerrado que nadie va a mirar, porque no dieron error.

Prioridad: no es una caída. La aplicación sigue funcionando y el supervisor conserva la última palabra sobre cada solicitud. Se atiende en horario laboral. Lo que no se puede es ignorar: cada día que pasa son más solicitudes que revisar hacia atrás.

152. **Causa.** El log lleva **exc_info=True**: la traza completa viaja en el propio mensaje de CloudWatch. No hay que reproducir nada. Causas típicas: base inaccesible o con *timeout*, dato mal cargado en **CierreSolicitudesConfig / CierreSemanaOverride**, festivos o mantenimientos sin cargar para el año, o un fallo introducido en el último despliegue.

153. **Ventana afectada.** Anotar la hora del **primer CRITICAL**: es el momento en que dejó de haber protección.

154. **Encontrar lo que se coló.** Esta consulta reutiliza el mismo servicio que hace la validación,

así que aplica exactamente el criterio que se saltó:

```
# python manage.py shell
from solicitudes.models import SolicitudCambio
from solicitudes.services.cierre_solicitudes_service import
CierreSolicitudesService as C
from django.utils.dateparse import parse_datetime

desde = parse_datetime('2026-08-15 03:12:00+00:00') # hora del primer CRITICAL
for s in SolicitudCambio.objects.filter(fecha_solicitud__gte=desde,
estado='pendiente'):
    f, msg = C.validar_fechas([s.fecha_cambio_turno])
    if f:
        print(s.id, s.explorador_solicitante, s.fecha_cambio_turno, '->', msg)
```

Lo que aparezca **no debería haber entrado**; el supervisor lo rechaza desde `/solicitudes/gestion-solicitudes/`.

⚠ Cubre las solicitudes de fecha única (`fecha_cambio_turno`). Los tipos **permanentes** guardan su rango en tablas relacionadas (`CambioPermanenteDetalle`) y hay que revisarlos aparte.

155. **Arreglar y confirmar de verdad.** Cuando la alarma vuelva a OK, crear una solicitud de prueba

para una fecha **dentro** de la ventana cerrada: debe rechazarse con "Cierre de solicitudes activo..." y código `cierre_semanal`. Que deje de haber **CRITICAL** no prueba que la validación funcione — prueba que dejó de ejecutarse la rama del `except`.

Por qué NO se resuelve con mail_admins

Django trae `AdminEmailHandler`, que enviaría el CRITICAL por correo con solo definir `ADMINS`. Se descarta a propósito: ese handler envía **de forma síncrona y dentro del hilo del request**, abriendo su propia conexión y saltándose el patrón *outbox*. Es exactamente el bloqueo por handshake SMTP que el proyecto eliminó sacando la entrega del request (`EMAIL_SEND_ASYNC`) y juntándola en un solo lote por conexión (§ 7.3). Además, un fallo de correo es una de las causas plausibles de la avería que se quiere avisar: no conviene que la alarma dependa del subsistema que puede estar roto. La alarma vive fuera de la aplicación.

14. Pruebas

14.1 Cómo se corren

```
cd AppTurnosExplora
pytest # suite completa
pytest -n 4 # en paralelo — unos 3 minutos
```

`pytest.ini` fija el módulo de settings, los patrones de descubrimiento `—test_*.py`, `Test*`, `test_*`— y `--strict-markers` (`pytest.ini:2-10`).

Dos banderas propias, para los tests que caducan. Ninguna corre por defecto:

Bandera	Qué hace	Cuándo se lanza
<code>--dias-en-el-futuro=N</code>	Corre la suite entera como si hoy fuese dentro de N días (<code>conftest.py:67-71</code>)	Antes de un despliegue, o al tocar sanciones, deudas y vencimientos
<code>--anio-planificado</code>	Finge que el checklist de apertura de año está completo, parcheando <code>AperturaAnioService.completo</code> (<code>conftest.py:104,111-113</code>)	Ensayo de diciembre, junto con <code>--dias-en-el-futuro=90</code>

Existen porque la suite tiene una clase de fallo que el CI no ve: el test que fija en el código una fecha que solo es válida "por ahora". Pasa en verde durante semanas y revienta un día cualquiera sin que nadie haya tocado nada; ya ocurrió tres veces a la vez, con tres causas distintas (`conftest.py:11-30`). **Los fallos que produce `--dias-en-el-futuro` son el hallazgo, no un error de la corrida**, y el arreglo correcto es inyectar `hoy` en el servicio, no congelar el reloj (`conftest.py:38-42`). El procedimiento completo está en [05-referencia/deployment/MANUAL TESTS QUE CADUCAN.md](#) (`./05-referencia/deployment/03-operacion/MANUAL_TESTS_QUE_CADUCAN.md`).

14.2 Cómo está partida la suite

El módulo de horas a favor añade `permisos/tests/test_credito_horas.py` (23 casos en 7 clases). Dos son **centinelas de política**, no de código:

`test_el_credito_no_evita_la_sancion_de_un_mes_vencido (:222)` fija que el crédito no exime de la sanción, y `test_el_minimo_del_campo_esta_alineado_con_el_paso (:322)` impide que vuelva el desfase de `min / step` descrito en § 16.2. El resto cubre el FIFO, el reparto entre varios créditos, la reversión al borrar el PDH y el caso que originó el módulo.

<code>testpath</code>	Archivos	Qué cubre
<code>solicitudes/tests</code>	44	El dominio central: los seis tipos, validadores, aprobación, cancelación, reversión y temporada
<code>solicitudes/tests/helpers_cancelacion.py</code>	—	No es un test, es un helper compartido. <code>cancelar_con_acuerdo(solicitud, solicitante=None, receptor=None)</code> envuelve el ciclo de dos pasos de PO —pedir y aprobar— y devuelve (<code>ok, mensaje</code>) con el mismo contrato que el caso de uso; si la

		petición falla, devuelve ese fallo sin llegar a responder. Úsalo en cualquier test que solo quiera llegar a cancelada (reversión de turnos, deudas, reconciliación) en vez de repetir los dos pasos. Para probar el acuerdo en sí —que pedir no cancela, que rechazar deja el cambio firme, la caducidad, quién puede responder— el sitio es <code>test_cancelacion_lifo.py</code>
<code>turnos/tests, empleados/tests, permisos/tests, core/tests, integration_tests</code>	33 entre todos	Calendario, personas, permisos, utilidades y flujos de punta a punta

La **entrega** del correo tiene el suyo: `solicitudes/tests/test_email_outbox.py`. Además de la entrega, la idempotencia y el uso del outbox desde `EmailService`, cubre el mecanismo de lote con dos clases: `OutboxUnaConexionPorLoteTest (:189)` —que un lote abre **una** conexión y no una por correo, que un SMTP caído no quema intentos, y la renovación del canal a media tanda— y `OutboxEnvioAgrupadoTest (:285)` —que todo lo encolado dentro de un `envio_agrupado` sale en un único `enviar_lote`—.

La seguridad de los enlaces de correo tiene archivo propio:

`solicitudes/tests/test_tokens_aprobacion.py`, 25 tests (pytest `solicitudes/tests/test_tokens_aprobacion.py --collect-only -q`), que cubren firma, caducidad, aislamiento por sal, rol equivocado y suplantación.

Los `testpaths` están declarados en `pytest.ini:11-18`. `AppTurnosExplora/scripts/` **no** está en la lista: son utilidades manuales y pytest no las recoge.

El «con quién» del detalle del día se prueba por los dos lados.

`solicitudes/tests/test_reflejo_mis_turnos.py` monta los **seis** trámites de verdad, los aprueba y consulta el endpoint real (`:128`); dos helpers concentran la comprobación: `_con_quien(empleado, fecha, companero, rol) (:132)` para el día **trabajado** —exige `solicitud_info` con compañero, papel y fecha de aprobación— y `_libre_por(empleado, fecha, companero, tipo_solicitud) (:152)` para el día **libre**. Se aplican a los dos lados de cada tipo, que es donde estaban los huecos. Las dos reglas de resolución del servicio, que ningún test de integración puede provocar a voluntad, viven aparte en `solicitudes/tests/test_acuerdo_por_dia.py` (10 casos, snapshots escritos a mano a propósito, `:1-20`): prioridad por `fecha_resolucion (:75)`, guarda de realidad (`:87`), sin turno real no hay acuerdo (`:99`), lista vacía = día libre (`:104`), estado no aprobado (`:114`) y bordes del rango (`:122`).

Pruebas de JavaScript. Sin `npm install`, sin `package.json` y sin `node_modules`: el runner que trae Node.

```
node --test tests_js/*.test.cjs
```

Los textos del detalle del día son lógica **pura** (`static/js/utils/detalle-dia-mensajes.js`, `cero document` y `cero fetch`, motivo en `:1-20`) precisamente para poder probarlos así: `tests_js/detalle-dia-mensajes.test.cjs` los cubre con **31** casos —nombre del acuerdo, fin de semana, etiqueta de jornada, «con quién», mensaje de cambio y mensaje de descanso—. El detalle está en ``tests_js/README.md`` (`./tests_js/README.md`).

14.3 Base de datos de test

Cuatro decisiones que evitan fallos fantasma:

- `TEST_DB_NAME` da un nombre explícito a la base de test, por defecto `test_<DB_NAME>` (`config/settings.py:186`). Sin él, dos corridas simultáneas compartían base y producían fallos masivos sin relación con el código.
- `pytest-xdist` da a cada worker su propia base con sufijo `_gwN` (comentario de `requirements-dev.txt`). Por eso `-n 4` no solo va más rápido: aísla.
- `TEST_RUNNER = 'core.test_runner.NoInputDiscoverRunner'` evita que una base `test_*` huérfana —de una corrida que murió a medias— deje la ejecución esperando un "yes" que nadie puede escribir (`config/settings.py:190-194`).
- `django-axes` está desactivado bajo `pytest` (`config/settings.py:336-341`).

14.4 Cobertura y áreas sin cubrir

El inventario de tests está en `05-referencia/pruebas/INVENTARIO_TESTS.md` (`./05-referencia/pruebas/INVENTARIO_TESTS.md`).

Áreas con menos cobertura relativa, medidas por número de archivos de test frente al peso del módulo: la app `permisos/` (dos modelos, PDH y permisos especiales) y las vistas de `empleados/` (salas, competencias, restricciones, indicadores). El porcentaje exacto de cobertura no se midió en esta pasada: ver § 18. La suite completa está en verde con **1 754 tests** (`pytest --collect-only -q`, 2026-09-09). El circuito de cancelación consensuada tiene ocho pruebas propias del lado de `solicitudes` (`test_cancelacion_lifo.py:150-263`) y nueve del lado de `permisos` (`permisos/tests/test_cancelacion_permiso_consenso.py`), que cubren el plazo medido desde `fecha_aprobacion`, las tres reglas de autorización, el rechazo en servidor cuando falta el comentario o el motivo (`test_cancelacion_lifo.py:167` y `:185`; `test_cancelacion_permiso_consenso.py:144`), que el literal de la plantilla sea el mismo que el del servidor (`:94`) y el doble contrato de respuesta: `400` con `code` para el cliente JSON (`:117`) y `302` con su mensaje para el navegador (`:134`).

15. Decisiones técnicas (ADR)

ADR	Título	En una línea
001 (./03-arquitectura/adr/001-service-layer-y-orchestrator.md)	Service Layer y patrón Orchestrator	La lógica sale de las vistas a servicios, con un orquestador que encadena los chequeos transversales
002 (./03-arquitectura/adr/002-fsm-sin-libreria-externa.md)	Máquina de estados sin librería externa (django-fsm)	Las transiciones son pocas y fijas; la librería añadía dependencia sin cerrar el hueco real
003 (./03-arquitectura/adr/003-on-commit-para-notificaciones.md)	<code>transaction.on_commit()</code> para desacoplar notificaciones	Un fallo de correo no revierte la operación de negocio. Superado en parte por el outbox (<code>solicitudes/models.py:49-69</code>), que además garantiza la entrega. <code>on_commit</code> sigue vivo, pero solo como temporizador del camino rápido (<code>EmailOutboxService.enviar_tras_commit, solicitudes/services/email_outbox_service.py:303-325</code>); el agrupado en lote NO se apoya en él, y el porqué está en § 3.4
004 (./03-arquitectura/adr/004-variables-de-entorno-django-environ.md)	Variables de entorno con django-environ	Configuración fuera del código, un único <code>settings.py</code>
005 (./03-arquitectura/adr/005-pendientes-aws.md)	Pendientes para despliegue en AWS	Lista de trabajo de infraestructura
006 (./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md)	<code>django.core.signing</code> para los tokens de aprobación por correo	Una credencial que actúa sin sesión se firma en un solo sitio con <code>SECRET_KEY</code> y caduca. Recoge por qué se descartaron el HMAC propio, la tabla de tokens gastados y la caché
007 (./03-arquitectura/adr/007-paginas-de-error-propias-y-	Páginas de error propias e identificador de petición	Las pantallas técnicas de Django filtraban el URLconf completo y el traceback. Se sustituyen por cinco plantillas autocontenidas y se traza cada petición con un código opaco que el usuario puede reportar. Ver §

request-id.md)		9.5
009 (./03-arquitectura/adr/009-cancelacion-consensuada.md)	Cancelar una solicitud aprobada exige el acuerdo de la contraparte	La cancelación unilateral deshacía el turno que la otra persona ya había organizado. Ahora el solicitante pide y el receptor decide , con dos ventanas de 24 h. Contrapartida asumida: mientras se decide, estado sigue siendo ' aprobada ' y las guardias corren dos veces. Ver § 8.2 (P0) y § 8.1
010 (./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md)	El día de un cambio de descanso queda libre en cuanto se aplica	El bloqueo temporal del día venía calcado de la ventana de cancelación de 30 min; con la cancelación a 24 h alinearlos habría congelado el día un día entero para toda la plantilla. Se protege la elección de día de descanso , no la reversibilidad. Contrapartida asumida: reutilizar el día mientras vive la cancelación anterior deja al LIFO impidiendo revertirla. Ver § 8.3 y § 16.2
011 (./03-arquitectura/adr/011-sancion-independiente-del-pago.md)	La sanción por deuda se cumple entera, y la deuda se mide por meses	Pagar dejó de levantar la sanción (era una fianza reembolsable), la unidad sancionable pasa a ser (explorador, año, mes) alimentada por dobladas y permisos, cumplir el castigo extingue la deuda de ese mes y el nivel de reincidencia prescribe a los 45 días. Su punto 3 queda enmendado por el 013 . Ver § 8.3
013 (./03-arquitectura/adr/013-condonacion-al-levantar-la-sancion.md)	Levantar una auto-sanción por deuda condona la deuda del mes que la originó	La deuda de una sanción levantada quedaba en un limbo permanente —ni cobrable, ni pagable, ni extingible—. Ahora pasa a ' condonada ' desde el propio levantar() . Contrapartida asumida: levantar cuesta horas reales y no se deshace; se avisa al supervisor antes de confirmar. Ver § 5.2 y § 8.3
014 (./03-arquitectura/adr/014-horas-a-favor-del-explorador.md)	Las horas a favor del explorador son una bolsa, y no tocan la sanción	La deuda también corre al revés: lo que la corporación le debe se registra como crédito, se gasta en FIFO dentro de un PDH y no exime de la sanción por un mes vencido
008 (./03-arquitectura/adr/008-reconciliacion-best-effort.md)	La reconciliación tras cancelar es una reparación best-effort	Un ValidationError de una solicitud ajena abortaba la cancelación entera y dejaba al usuario sin salida. Ahora esa pieza se omite y se registra. Contrapartida asumida: consistencia diferida, auditable con verificar_efecto_aplicado . Ver § 8.2 (P4b) y § 16.2
012 (./03-arquitectura/adr/012-contrato-parcial-para-estrategias-de-fin-	El selector de compañeros de fin de semana es un contrato aparte que solo implementan CAMBIO	La base traía la regla del INTERCAMBIO por defecto, así que D FDS tenía que contradecirla y las otras cuatro contestaban con la regla equivocada. Descarta por MEDICIÓN la división ISolicitudCicloVida/ISolicitudPantalla que

de-semana.md)	DESCANSO y D FDS, no un método de la base de las seis	pedía la auditoría: las seis estrategias implementan ambos lados, así que no separa nada. Ver § 7.2 y § 16
015 (./03-arquitectura/adr/015-entrega-de-correo-en-lote.md)	El correo se entrega en lote sobre una conexión, y el lote se declara a mano	El coste de un correo es el saludo, no el mensaje: 6,3 s medidos por solicitud, tres handshakes. Un <code>open()</code> explícito comparte el canal; el lote se marca con <code>envio_agrupado()</code> y no con <code>on_commit</code> , que fuera de <code>atomic()</code> corre en el acto. Abrirlo por fuera del <code>atomic()</code> cerró B5. Reinterpreta el 003 (./03-arquitectura/adr/003-on-commit-para-notificaciones.md) sin enmendarlo. Ver § 3.4 y § 7.3
016 (./03-arquitectura/adr/016-transporte-de-correo-y-fiabilidad.md)	El transporte del correo es de un tercero; la fiabilidad no	"¿Código propio o servicio de terceros?" mezcla cuatro capas : composición y fiabilidad se quedan en casa — ningún proveedor vende atomicidad con la transacción de negocio—, el transporte sale de Gmail hacia SES por SMTP (cero líneas de código, 0,60 USD/mes, y DKIM que alinea con el dominio propio: hoy no alinea y son 300 personas con el correo en spam), y la retroalimentación de rebotes y quejas es una capa que no existe y que es el precio de entrada a SES. Corrige la petición a IT: tres CNAME de Easy DKIM, sin tocar el SPF del ápice del que depende Workspace. Descarta SendGrid/Resend por volumen antes que por precio, y Celery/SQS porque sería una capa <i>más</i> , no una sustituta. Ver § 3.4

Los cuatro primeros están fechados en 2026-06 y marcados **Implementado**; el 005 no lleva cabecera de estado y es una lista de pendientes, no una decisión cerrada.

Regla: una decisión técnica nueva se registra como ADR en `docs/03-arquitectura/adr/`, no como párrafo suelto en este manual. Decisiones tomadas y documentadas fuera del formato ADR que deberían tener el suyo: la `UniqueConstraint` con columna discriminante `activo_key`, y `core/constants.py` como fuente única de vocabularios. Ver § 18.

Del **patrón outbox**, el ADR 015 registra el lado de la ENTREGA (lote, canal compartido, quién decide el cuándo). Lo que sigue sin ADR propio es su mitad transaccional —por qué la fila se escribe **dentro** de la transacción del cambio de negocio y qué garantía da eso—, hoy documentada en el docstring de `EmailOutbox` (`solicitudes/models.py:49-69`) y en `MANUAL_OUTBOX_CORREOS.md`.

Al mecanismo de entrega del correo se le añadieron **tres invariantes con forma de patrón** que no están registrados en `PROTECTION_PATTERNS.md` —el ADR 015 los razona, pero el catálogo de protecciones es otro sitio y otro público—, y que son exactamente la clase de cosa que alguien deshace sin querer:

Invariante	Dónde vive	Qué se rompe al tocarlo
Un TTL de deduplicación debe durar más que el request que protege. No es un número estético	<code>solicitud_orchestrator.py:34,69-77</code>	Bajarlo a 10 s reabre el bug: el candado caduca a mitad de un flujo multi-compañero y un segundo POST idéntico crea el acuerdo por duplicado
El grupo de envío se abre POR FUERA del <code>`atomic()`</code> . Meterlo dentro devuelve el SMTP al interior de la transacción y reabre el correo irretirable (B5)	<code>solicitud_orchestrator.py:269-274,515-523</code>	Un correo sale por una solicitud que el rollback después borra
El fallo de conexión ocurre ANTES de reclamar filas. Si <code>_abrir_conexion</code> se moviera dentro del bucle, o si un fallo al abrir se contara como intento, una caída del SMTP volvería a quemarle intentos a correos que no tienen culpa	<code>email_outbox_service.py:207-223,254-256</code>	Cinco caídas seguidas dejan correos en fallido para siempre

16. Zonas frágiles y bugs conocidos

16.1 El catálogo de protecciones

`PROTECTION_PATTERNS.md` (raíz del repositorio) recoge **43 patrones** aplicados, cada uno con su motivo y el caso real que lo provocó. Es lectura obligatoria antes de tocar cualquier flujo de escritura. Los que más restringen lo que puedes hacer:

#	Patrón	Qué te obliga a hacer
2	Select-for-update + transacción	Bloqueo pesimista sobre las filas que vas a modificar
3	Idempotent guards / snapshot-once	El snapshot se captura una sola vez , antes de aplicar; re-aplicar no lo reescribe
21	Deuda idempotente por (explorador, fecha)	Reintentar no duplica deuda

22	Reconciliación completa	Cubrir todos los modelos de detalle, no solo el del tipo que tocaste
25	Guardias que fallan cerrado	Ante la duda, bloquear
26	Invariantes estructurales en la base de datos	Un clean() no basta: no corre en save()
27	Una sola noción de "hoy": timezone.localtime()	Nunca date.today()
28	El turno real (L1) manda sobre la atribución de descanso (L2)	Validar contra el estado real, no contra la jornada base
29	Un hueco en L2 no da error: da un dato falso	Comprobar la ausencia explícitamente
30	Un snapshot solo vale si nadie tocó el día	De ahí snapshot_turnos_resultantes y la guardia de integridad
31	Un catálogo que gobierna permisos se compara EXACTO y se protege	Role y Jornada
32	Un hecho que se registra no se borra, y su fin previsto no es su fin real	SancionEmpleado
33	Al re-aplicar, el conjunto de días afectados se cierra antes de escribir	Incluir los días colaterales
35	La regla se valida sobre el campo que manda al aplicar	No sobre el que la declara
37	Un desplegable de candidatos filtra por las mismas fechas que valida el envío	Selector y validación se cambian en el mismo commit
38	Una credencial no se genera al vuelo: se inyecta y se verifica al arrancar	Ver § 9.5 y el ADR 006 (./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md)
39	La guardia vive donde está la ESCRITURA, no solo en la estrategia (PROTECTION_PATTERNS.md:1651)	Todo *_aplicacion_service / *_pago_service valida sus precondiciones antes de la primera escritura, aunque ya estén validadas aguas arriba; lanza, no arregla ; y su mensaje no da instrucciones de rol. Ampliado con la auditoría de esta rama: la puerta abierta son los comandos de gestión , el cierre de aplicar_doblada_pago es parcial y con su motivo, el principio "una guardia

		que consulta el estado posterior no sirve en un método cuyo efecto es cambiar ese estado", y el aviso de no mover la guardia de <code>jornada_cedida</code> al inicio del método
40	Una reparación no aborta por una pieza que no encaja (<code>PROTECTION_PATTERNS.md:1732</code>)	Un proceso de <i>reparación</i> captura el fallo de negocio de cada pieza, lo registra y sigue; uno de <i>validación</i> falla cerrado. Ver P4b y <code>reconciliar_dobladas_aprobadas</code>
41	Un nombre privado consumido desde fuera es un contrato falso (<code>PROTECTION_PATTERNS.md:1787</code>)	Si otro módulo importa tu <code>_nombre</code> , o lo renombra a público o dejas de exportarlo. No hay tercera opción
42	Trinquete: la deuda conocida se congela y solo puede menguar (<code>PROTECTION_PATTERNS.md:1846</code>)	La base conocida se declara en un test, no en un documento: así no puede crecer sin que alguien lo vea
43	Quien decide sobre una operación no puede ser candidato de esa operación (<code>PROTECTION_PATTERNS.md:1896</code>)	El supervisor aprueba los cambios de turno; queda fuera de los desplegables de compañero de los seis formularios

La regla más generalizable del patrón 39 es la 4: un ``create()`` dentro de un ``if not ...exists()`` es la señal de alarma. Pregúntate si ese registro falta porque *no debía existir*, en vez de tratarlo como un hueco que rellenar. Ese fue exactamente el bug del reparto de sábado: al acreedor que descansaba se le "completaba" la jornada ausente, inventándole un turno.

⚠ Las cinco ramas restantes de ``aplicar_doblada_pago`` están auditadas y el cierre es **PARCIAL**. `jcp_ambas` está limpia en este eje. `jcp_media` y `jornada_cedida` repetían la forma del bug de sábado y ya llevan el guard (`doblada_pago_service.py:302-309` y `:536-543`). `cesion_parcial` y el fallback **siguen expuestas**: no fabrican turno, pero validan con la jornada base en vez de con "¿trabaja?".

Qué daño hacen exactamente (auditado empíricamente, con un acreedor que descansa por alternancia de fin de semana; corrige lo que decían versiones anteriores de este manual): al **acreedor no se le inventa ningún turno** — se queda sin turnos, que es lo correcto. El perjudicado es el **deudor**: termina con **AM+PM**, cubriendo jornadas que **nadie** iba a trabajar, y al quedar doblado se le generan además los **30 minutos de deuda corporativa**. Trabaja de más por un turno inexistente. En la misma medición, la rama de control `jcp_media` quedó **bloqueada** por su guard, como se esperaba.

El LIFO **no las protege del todo**. Tapa la reconciliación, pero no los comandos de gestión `reaplicar_doblada` (`solicitudes/management/commands/reaplicar_doblada.py:259`) y

`corregir_doblada_cesion_total (:73)`, que llaman a `aplicar_doblada_pago` sin validar y sin pasar por el LIFO — y se ejecutan justo cuando un día ya está descuadrado.

Por qué solo dos ramas y no cuatro: `cesion_parcial`, el fallback y `jornada_cedida` en `cesion_completa` dejan al acreedor **sin turnos por diseño**. Como `TurnoService.estado_dia` cuenta el descanso por solicitud aprobada y no admite excluir la solicitud en curso, el guard bloquearía su **propia re-aplicación**. Esto está **medido, no razonado**: se instrumentó `BaseValidator._explorador_trabaja(acreedor, fecha_pago)` antes de aplicar en esas ramas y ya devuelve `False`, porque en cuanto la solicitud pasa a `aprobada` `DescansoPorSolicitudService` le atribuye el descanso a **esa misma solicitud**. Es decir, el guard no bloquearía solo la re-aplicación: se bloquearía a sí mismo en la **primera** aplicación. No es "arriesgado": es que no funciona. Detalle, invariante y la solución pendiente (`snapshot_turnos_previos`) en § 16.2; la limitación viva, en § 18, filas 19 y 20.

El archivo tiene además una matriz de patrones por flujo (`PROTECTION_PATTERNS.md:1151`), un checklist para flujos nuevos (`:1541`) y una sección de **deuda conocida pendiente de decisión** (`:1731`) que no se debe aplicar todavía.

⚠ `PROTECTION_PATTERNS.md` vive **fuera** de `docs/`, lo que rompe el "una sola fuente de verdad" del árbol documental. Ver § 18.

16.2 Zonas que se rompen fácil

Zona	Por qué es frágil	Qué hacer
Los tres campos llamados <code>tipo_cambio</code>	<code>SolicitudCambio.tipo_cambio</code> (FK), <code>Turno.tipo_cambio</code> y <code>TurnoArchivo.tipo_cambio</code> (texto) no comparten vocabulario . Mezclarlos no da error: da un filtro que no casa con ninguna fila y una validación que deja de aplicarse en silencio. Ya pasó, con <code>tipo_cambio__nombre='CT'</code> comparando un <code>codigo_estrategia</code> contra un <code>nombre</code> (<code>core/constants.py</code> , docstring)	Usar siempre <code>core/constants.py</code> y <code>MAPA_SOLICITUD_A_TURNO</code>
Un <code><input type="number"></code> con <code>min</code> y <code>step</code> que no casan	En HTML5 los pasos se cuentan desde el `min`, no desde cero . <code>CreditoHorasForm.horas</code> se declaró con <code>min_value=0.01</code> pensando solo en "que no sea cero" y <code>step=0.25</code> : las flechas del campo ofrecían 0.01, 0.26, 0.51 ... valores que el propio <code>clean_horas</code> rechazaba después por no ser múltiplos de 0.25, y encima con un segundo error contradictorio (ver la	El <code>min_value</code> tiene que ser el mismo cuarto de hora que el <code>step</code> (<code>empleados/forms.py:409-417</code>). Centinela: <code>permisos/tests/test_credito_horas.py:322</code>

	fila siguiente)	
Validar en Model.clean() un campo que el formulario no expone	Model.clean() corre siempre , aunque el formulario ya haya fallado en otro campo. Con el tope de horas ahí, un importe inválido dejaba minutos_otorgados en None y añadía «deben ser mayores a cero» encima del error real, diciéndole al usuario que había escrito cero cuando había escrito 0,51	Los topes van como validadores del campo (permisos/models.py:389-392): un ModelForm que no lo expone lo excluye, y el full_clean() del servicio los sigue aplicando. Centinela: permisos/tests/test_credito_horas.py:333
El bloqueo del día para un nuevo cambio de descanso	dia_bloqueado_para_nuevo_cambio (solicitudes/services/cambio_descanso_aplicacion_service.py:216) tuvo un bloqueo temporal de 30 min tras un CAMBIO DESCANSO previo, para que reutilizar el día no rompiera el revert de una cancelación aún posible. Se retiró : con la cancelación consensuada a 24 h, alinearlas habría congelado el día un día entero para toda la plantilla. Decisión de producto: se protege la elección de día de descanso , no la reversibilidad. Contrapartida asumida: si alguien reutiliza el día mientras la cancelación anterior sigue viva, la guardia LIFO impide revertirla. Al leer el código, la regla parece "incompleta": no lo es	No reintroduzcas el bloqueo sin revisar el ADR 010 (./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md). El motivo está en el docstring (:217-240) y el centinela es test_dia_bloqueado_para_nuevo_cambio_directo (solicitudes/tests/test_cambio_descanso.py:312), que comprueba los dos lados del intercambio justo después de aplicar. Tampoco excluir_id : es compatibilidad de firma con las llamadas de re-validación
La guarda de realidad de AcuerdoPorDiaService	Un snapshot_turnos_resultantes describe el mundo en el momento en que se aplicó esa solicitud, no el de hoy. Si otra solicitud reescribe el día después, el snapshot viejo sigue reclamándolo y el detalle del día nombraría al compañero equivocado. Por eso el snapshot solo vale si su tipo_cambio coincide con el del Turno real de ese día (solicitudes/services/acuerdo_por_dia_service.py:119-130,149) — mismo patrón que ya usa DescansoPorSolicitudService (L1 manda sobre L2)	No quites el cruce contra Turno "porque el snapshot ya lo dice": el snapshot dice lo que hubo , no lo que hay . Centinela: test_un_snapshot_que_ya_no_describe_el_turno_no_reclama_el_dia (solicitudes/tests/test_acuerdo_por_dia.py:93). Tampoco recortes MARGEN_DIAS = 3 (:78): el finde de un CAMBIO DESCANSO mueve el día al sábado/domingo contiguo y la ventana de la consulta tiene que abarcarlo

La caché del mes de Mis Turnos frente a un despliegue	<code>MisTurnosPorMesView</code> guarda el JSON del mes por empleado durante una hora (<code>turnos/api/views/turnos_mes.py:78, 212-214</code>). La caché no sabe que el código cambió: tras desplegar algo que altere la forma o el contenido de la respuesta —como el bloque <code>solicitud_info</code> —, quien ya tenía su mes cacheado sigue recibiendo el JSON viejo hasta que caduque	Invalidar tras el despliegue (<code>CacheService.invalidar_cache_turnos_empleado, core/services/cache_service.py:132</code>) o reiniciar el backend de caché; si no, esperar el TTL. En los tests, <code>cache.clear()</code> antes de cada lectura (<code>solicitudes/tests/test_reflejo_mis_turnos.py:157</code>). Un cambio en la respuesta que "no se ve" en producción es casi siempre esto
El orden de carga de los scripts de Mis Turnos	<code>static/js/mis_turnos.js</code> toma los textos de <code>window.DetalleDiaMensajes</code> en una constante de módulo (<code>static/js/mis_turnos.js:551</code>). Si el <code><script></code> del helper se carga después , la constante queda <code>undefined</code> y el detalle del día revienta al abrirlo	El helper va antes que <code>mis_turnos.js</code> en la plantilla (<code>templates/turnos/mis_turnos.html:90-95</code> : el helper en <code>:91</code> , <code>mis_turnos.js</code> en <code>:95</code>). No reordenes los <code><script></code> ni muevas el helper a un <code>defer</code> distinto del de su consumidor
Reversión de una cancelación	Restaurar el snapshot pisa cambios ajenos si alguien tocó el día. Las dos guardias no ven permisos especiales, reprogramaciones ni ediciones del admin (<code>use_cases/cancelar_solicitud.py:460</code>)	No añadir rutas de escritura de turnos que salten los servicios de aplicación
La pareja <code>estado='aprobada' + cancelacion_estado='pendiente'</code>	Es un estado legítimo , no un residuo: significa "cancelación en trámite, cambio todavía aplicado" (§ 8.1, § 8.2 P0). Toda consulta que filtre por <code>estado='aprobada'</code> la sigue viendo vigente, y así debe ser	No "arregles" el <code>estado</code> al pedir la cancelación, ni añadas <code>.exclude(cancelacion_estado='pendiente')</code> a las consultas de guardia, Mis Turnos o reconciliación: dejarías el turno sin dueño mientras la otra persona decide. Si necesitas listar las peticiones, hazlo en una consulta aparte , como <code>SolicitudesPendientesListView._cancelaciones_pendientes</code> (<code>solicitudes/views/notificaciones_listas.py:173</code>)
Quién decide sobre un permiso ya aprobado	Dos preguntas que es fácil confundir: de quién es el permiso y qué rol tiene quien pulsa. Mezclarlas dejó que un explorador con rol de supervisor se	Invariantes: (a) sobre el permiso propio siempre se PIDE, sea cual sea el rol (<code>permisos/views.py:424</code>), y el dueño no se responde a sí mismo

	autoaprobara la cancelación de su propio permiso (B9-B11, § 16.4). Además, la contraparte se resolvía en dos sitios: la autorización miraba el rol y el aviso miraba <code>permiso.supervisor</code> or <code>permiso.empleado.supervisor</code>	(:585); (b) la contraparte sale de una sola función, <code>_supervisor_del_permiso (:561)</code>, que usan la autorización (:588) y la notificación (:611) — si añades otro punto que decida o avise, llámala también; (c) el caso "sin supervisor resoluble → vale cualquier supervisor" (:588-591) no es un agujero: sin él la petición sería incontestable; (d) el plazo se mide con <code>fecha_aprobacion</code> or <code>actualizado_en (:443)</code> y el respaldo se queda. Lo fija <code>permisos/tests/test_cancelacion_permiso_consenso.py</code> (5 casos)
La caducidad de una petición de cancelación	Es perezosa: nada la marca en segundo plano. La escribe quien intenta responder (<code>_caducar_si_vencida, use_cases/cancelar_solicitud.py:390</code>) y la lista la excluye por fecha en vez de marcarla (<code>notificaciones_listas.py:186-197</code>)	No añadas un cron que "limpie" las caducadas creyendo que falta: una petición vencida no tiene efecto pendiente que aplicar, el cambio simplemente sigue vigente. Y si listas una vencida, le das al receptor un botón que va a fallar
Role y Jornada	Se buscan por nombre literal. Renombrar "Supervisor" deja la operación sin supervisores; borrar AM/PM rompe el motor	Los NOMBRES_PROTEGIDOS lo impiden; no los ablandes (<code>empleados/models.py:73-85,19-23</code>)
Turnos duplicados	El invariante se sostenía por la disciplina de delete-then-create repetida en unos quince sitios	Ahora lo garantiza la base de datos (<code>turnos/models.py:73-83</code>). No desactives esa constraint
El reparto de sábado de una doblada	<code>_aplicar_pago_sabado</code> da por sentado que el receptor trabaja: le quita la mitad cubierta y le crea la contraria si no la tiene (<code>solicitudes/services/doblada_pago_service.py:126-137</code>). Con un receptor que descansa, eso no falla: escribe un turno falso. Y al servicio se entra desde tres rutas distintas, así que una validación solo en la estrategia no lo protege	Invariante: el guard de :139-151 va antes de cualquier escritura. Su excepción y la rama <code>_contraria_tambien_cubierta</code> del reparto (:225-226) comparten ya un único helper, <code>_mitad_contraria_cubierta_por_otra_doblada (:99-123)</code>: no vuelvas a duplicar el criterio. Y no le añadas un filtro por <code>`explorador_solicitante`</code>: reintroducirlo revive la doble cobertura cuando dos deudores distintos cubren cada mitad (test sentinela)

		<code>solicitudes/tests/test_matriz_dobladas.py:2759</code>). Registrado como patrón 39 de <code>PROTECTION_PATTERNS.md (:1585)</code>
Pago de doblada entre semana: dos ramas guardadas, dos todavía expuestas (cierre parcial)	Cuatro ramas escribían el día de pago sin comprobar que el acreedor trabaje. Ya guardadas: <code>_aplicar_pago_jcp_media</code> (guard al inicio, <code>solicitudes/services/doblada_pago_service.py:302-309</code>) y <code>_aplicar_pago_jornada_cedida</code> en su rama parcial (guard dentro del <code>else</code> de <code>cesion_completa</code>, :536-543). Ambas usan <code>BaseValidator._explorador_trabaja</code> y lanzan <code>ValidationError</code> con el mismo texto que el de sábado ("...no trabaja el DD/MM/AAAA: ese día descansa... La doblada debe rehacerse con otra fecha de pago"). Siguen expuestas: <code>_aplicar_pago_cesion_parcial</code> (:353-445) y <code>_aplicar_pago_fallback</code> (:567-642), que no le fabrican ningún turno al acreedor —se queda sin turnos, que es lo correcto— pero validan con <code>JornadaService.get_jornada_explorador_fecha</code> — devuelve la jornada BASE ("esta persona es AM"), no si trabaja ese día; recibe una fecha, así que parece responder lo segundo y responde lo primero, y contesta 'AM' igual un día de descanso. En <code>cesion_parcial</code> se entra a ese camino justo cuando el acreedor no tiene turnos, el aspecto de un día libre; el <code>raise</code> existente (:391-395) solo salta si nunca se le asignó jornada base, que es otra situación. La puerta abierta son los comandos de gestión, no la reconciliación: <code>reaplicar_doblada</code> (<code>solicitudes/management/commands/reaplicar_doblada.py:259</code>) y	Invariante 1 — por qué solo dos ramas. <code>cesion_parcial</code>, el fallback y <code>jornada_cedida</code> con <code>cesion_completa</code> dejan al acreedor sin turnos por diseño (en <code>cesion_parcial</code> está escrito "El acreedor descansa", <code>solicitudes/services/doblada_pago_service.py:425-426</code>; <code>eliminar_turnos_explorador(receptor, ...)</code> en :521 y :634). Como <code>TurnoService.estado_dia</code> incluye el descanso por solicitud aprobada y no admite excluir la solicitud en curso, al RE-aplicar la misma solicitud —que es exactamente lo que hace <code>reaplicar_doblada</code>— el acreedor consta descansando por culpa de ella y el guard bloquearía su propia re-aplicación. Una comprobación sobre el estado POSTERIOR no distingue "descansa porque yo lo dejé así" de "descansa porque no le tocaba". Y no es una sospecha: se midió <code>BaseValidator._explorador_trabaja(acreedor, fecha_pago)</code> antes de aplicar en esas ramas y ya devuelve <code>False</code>, porque con la solicitud en <code>aprobada</code> <code>DescansoPorSolicitudService</code> atribuye el descanso a esa misma solicitud (<code>solicitudes/services/descanso_solicitud_service.py:193-197, 204-215</code>). El guard se bloquearía a sí mismo ya en la primera aplicación, no solo al repetirla. No es "arriesgado": no funciona. La solución correcta sería comparar contra <code>snapshot_turnos_previos</code> (el mundo anterior a aplicar): no se ha

	<code>corregir_doblada_cesion_total (:73)</code> llaman a <code>aplicar_doblada_pago</code> sin validar y sin pasar por el LIFO; <code>verificar_doblada.py:384</code> imprime el de re-aplicación como sugerencia al detectar errores, y lo indican dos guías de operación (<code>docs/05-referencia/solicitudes/dobladas/CHECKLIST_SOLUCION_DOBLADA.md</code>, <code>GUIA_RAPIDA_REAPLICAR_DOBLADA.md</code>), que ya llevan un aviso destacado: <code>reaplicar_doblada</code> no valida nada, hay que comprobar en Mis Turnos que el acreedor trabaje la fecha de pago —la jornada asignada no sirve: dice "es AM" aunque ese día descansa— y si descansa no reaplicar. Es el momento de más riesgo: se ejecutan cuando el día ya está descuadrado. El LIFO solo tapa la vía de la reconciliación (<code>doblada_snapshot_service.py:463</code> dentro de un <code>revertir(...)</code>; <code>use_cases/cancelar_solicitud.py:182,223,254,420,444</code>).
El daño, medido (corrige lo que decía este manual): no es "le fabrican un turno al acreedor". Al acreedor no se le inventa nada: se queda sin turnos. El perjudicado es el deudor, que acaba con AM+PM cubriendo jornadas que nadie iba a trabajar y carga los 30 minutos de deuda corporativa por quedar doblado. Trabaja de más por un turno inexistente	hecho, es un cambio de más calado y está sin decidir (§ 18, fila 20). Invariante 2: las dos guardadas lo son porque conservan una jornada al acreedor; al re-aplicar, <code>_explorador_trabaja</code> ve un turno real y el guard no salta. No muevas el guard de <code>jornada_cedida</code> al inicio del método: caería sobre <code>cesion_completa</code> y rompería su re-aplicación. Invariante 3: si se debilita el LIFO, las dos ramas expuestas quedan además sin la cobertura de la reconciliación. Lo fijan <code>TestLifoProtegeLaReconciliacion</code> (<code>solicitudes/tests/test_matriz_dobladas.py:2864</code>) y <code>TestGuardPagoEntreSemana</code> (<code>:2948</code>: rechazo en <code>:2988</code> y <code>:3030</code>, y la regresión <code>test_jcp_media_reaplicar_no_se_autoengana</code> en <code>:3005</code>, que demuestra que re-aplicar no se bloquea a sí mismo). Verificado que los dos tests de rechazo tienen dientes: neutralizando los guards, fallan. Sigue siendo señal del patrón 39
La reconciliación tras cancelar (<code>reconciliar_dobladas_aprobadas</code>)	Re-aplica las dobladas vigentes de esos días sin re-validarlas (<code>solicitudes/services/doblada_snapshot_service.py:286</code>). Desde que los servicios de aplicación llevan guardias de negocio (patrón 39), una solicitud vigente que ya no encaja con el calendario actual levanta <code>ValidationError</code> al re-aplicarse —	Riesgo cerrado. <code>_reaplicar_una</code> va dentro de un <code>try/except ValidationError</code> (<code>doblada_snapshot_service.py:337-347</code>): se registra con <code>logger.error</code> —id de solicitud, tipo, fechas y motivo— y se continúa con las demás. Razón: la reconciliación es reparación best-effort, no una

	estado demostrado empíricamente: una doblada aprobada cuyo pago cae en un sábado donde el acreedor ya no trabaja. Antes ese error subía por el bucle y abortaba la transacción de cancelación: el usuario no podía cancelar por culpa de una solicitud ajena que él no puede arreglar	validación; si una pieza no se puede recolocar se anota y se sigue, pero nunca debe impedir una cancelación legítima. Invariante: capturar solo <code>ValidationError</code> (precondición de negocio); cualquier otro fallo —BD, programación— debe seguir propagándose, porque ahí sí conviene abortar. No amplíes ese except. Consecuencia operativa: cuando ocurre, la solicitud sigue aprobada pero su efecto queda sin materializar, y no salta ningún aviso. El auditor sí existe: <code>python manage.py verificar_efecto_aplicado</code> lista las solicitudes aprobadas cuyo efecto ya no está en los turnos y <code>--reparar</code> las re-materializa; funciona precisamente porque el resultante de la omitida no se refrescó. La deuda que queda es la alerta automática sobre ese log de ERROR (§ 18). Razonado en el ADR 008 (<code>./03-arquitectura/adr/008-reconciliacion-best-effort.md</code>), junto con las alternativas descartadas (re-validar antes de re-aplicar, cancelar en cascada, marcar la solicitud como inválida). Test: <code>TestReconciliacionNoTumbaLaCancelacion</code> (<code>solicitudes/tests/test_matriz_dobladas.py:3051</code>, caso en <code>:3072</code>), verificado con dientes: cambiando el tipo de excepción capturada, falla
Las dos consultas gemelas del sábado en <code>doblada_pago_service.py</code>	<code>_mitad_contraria_cubierta_por_otra_doblada (:99-123)</code> y <code>_otra_mitad_pagada (:200-206)</code> tienen casi el mismo <code>filter()</code> y preguntan cosas opuestas: la primera por el acreedor (¿alguien cubre su otra mitad?), la segunda por el deudor (¿ya trabajo yo la otra mitad?). Por eso una filtra por <code>explorador_receptor</code> y la otra por <code>explorador_solicitante</code>.	Invariante: se quedan separadas. El motivo está comentado en el código (<code>:195-199</code>) y en el aviso ⚠ del patrón 39. Hay además una tercera copia del criterio en <code>services/validators/doblada_flujo_validator.py:253-269</code> (<code>_es_complemento_sabado</code>), asimétrica a propósito y hoy verificada como inofensiva (§ 18, fila 17)

	Unificarlas "porque están duplicadas" rompe una de las dos	
Los dos selectores de pago de doblada	El de sábado y el de día de semana (<code>sincronizarSelectorPagoSabado :398</code> y <code>sincronizarOpcionesCubrePagoReceptorDoblada :444</code> , en <code>static/js/cambio-turno/solicitar_doblada.js</code>) resuelven la misma pregunta con datos que llegan en dos respuestas asíncronas independientes y en orden no garantizado (el receptor puede resolverse después de la fecha). Por eso la decisión de sábado se guarda en <code>sabadoDatosDeudor (:81)</code> y se reevalúa desde ambos flujos (<code>:1439</code> , <code>:1540</code>)	No devolver la lógica a un bloque inline dentro de un <code>.then</code> . Al cambiar la fecha de pago hay que resetear <code>sabadoDatosDeudor</code> junto a <code>estadoReceptorPago (:1064)</code> . Patrón 37: si cambias el criterio del selector, cambias en el mismo commit el del backend
Selectores de compañero	Un desplegable que filtra por criterios distintos a los que valida el envío ofrece candidatos que el backend tumba. Variante propia de DOBLADA: confundir el rol . El filtro de <code>doblada_filtro_service.py</code> mira <code>explorador_receptor_id</code> porque el receptor es quien queda AM+PM; el solicitante cede y descansa , y sigue disponible. Consultar el rol equivocado no da error —devuelve una lista plausible pero invertida—, así que solo se detecta cuando alguien echa en falta a un compañero	Patrón 37; ocurrió en <code>9c429e3</code> y en <code>4dd6cd6</code> . Invariante: en cualquier consulta sobre «quién está doblado» el sujeto es <code>explorador_receptor (doblada_filtro_service.py:60-66)</code> ; no lo cambies a <code>explorador_solicitante</code> . Lo fija <code>solicitudes/tests/test_doblada_filtro_rol.py</code> (3 casos). Candidato a patrón nuevo en <code>PROTECTION_PATTERNS.md</code> : «en una relación asimétrica, nombra el rol en el nombre de la variable»
Caché de "Mis Turnos"	Con <code>LocMemCache</code> y varios workers, la invalidación limpia un solo proceso	<code>CACHE_URL</code> obligatorio en producción (<code>config/settings.py:279-296</code>)
El shim de <code>window.fetch</code>	Está en el camino de todas las peticiones del frontend. Un <code>.catch</code> encadenado convertiría un fallo de red en promesa resuelta y los formularios dejarían de mostrar su aviso; devolver algo distinto de la <code>Response</code> original rompería a todos sus consumidores	Invariante: no encadenar <code>.catch</code> , no alterar argumentos ni respuesta, y mantener la lectura de la cabecera dentro del <code>try/catch (static/js/utils/codigo-referencia.js:89-107)</code> . Candidato a patrón nuevo en <code>PROTECTION_PATTERNS.md</code>
Un <code>except</code>	Es la vía más fácil de reabrir CWE-209:	Invariante: usar

Exception nuevo en una vista API	<code>JsonResponse({'error': str(e)})</code> parece inofensivo y puede acabar publicando el endpoint de RDS. No pasa por ningún handler de Django	<code>json_error_inesperado(request, e, mensaje)</code> (<code>core/utils/json_responses.py:68</code>) con texto propio. El test parametrizado de <code>core/tests/test_json_error_inesperado.py:114-115</code> vigila los cinco módulos ya saneados, pero no los que se añadan después : al crear un módulo API nuevo, súmalo a MODULOS (:25) . Ver § 9.5
Mapeo de errores por subcadena	<code>CancelarSolicitudView</code> decide el código HTTP inspeccionando el texto del mensaje (<code>'más reciente' in msg</code>)	Cambiar la redacción de un mensaje cambia el HTTP que ve el frontend (<code>solicitudes/views/aprobacion_views.py:64-72</code>)

16.3 Bugs y riesgos abiertos

*Revisado contra el código el 2026-09-04. Dos entradas que figuraban aquí ya estaban resueltas y se movieron a § 16.4 (B3, recuperación de contraseña; B8, CSP report-only). Además, **B6 y B7 estaban duplicados**: el mismo identificador nombraba un bug abierto y otro cerrado, con contenidos distintos. Los abiertos se renumeraron a **B15** y **B16**; los cerrados conservan su número original para no romper referencias externas.*

#	Síntoma	Causa	Evidencia
B4	El cierre semanal falla abierto. Si la comprobación revienta, la solicitud se acepta y solo queda un CRITICAL en el log	Decisión deliberada y documentada: romper el formulario a todos los exploradores es peor que colar una solicitud fuera de plazo, que el	<code>solicitudes/services/solicitud_orchestrator.py:144-161</code> , § 13.6

		supervisor aún puede rechazar. El riesgo era que nadie vigilara ese log. Mitigado con procedimiento: la alarma sobre CIERRE SEMANAL INOPERATIVO es ahora un ítem obligatorio del despliegue (§ 13.6, § 13.3 y el checklist EC2+RDS FASE 11). El código no cambia: fallar abierto sigue siendo lo correcto; lo que faltaba era el aviso. Queda abierto hasta que la alarma exista de verdad en AWS — créala y	
--	--	--	--

		pruébala	
B15	Comandos de diagnóstico con nombre de incidencia en el árbol de producción: <code>test_verificar_doblada_jeison,</code> <code>validar_fix_doblada_jeison,</code> <code>validar_dobladas_junio</code> <code>, test_factory</code>	Scripts puntuales que nunca se retiraron. Confunden a quien llega nuevo y ensucian <code>manage.p</code> y <code>help</code>	<code>solicitudes/management/commands/</code> Ojo al retirarlos (hallazgo de B20): <code>scripts/maintenance/test_alternancia_31_enero.py</code> es el ÚNICO llamador de <code>AlternanciaFinesSemanaService.jornada_descansa_sabado</code> y <code>jornada_descansa_domingo</code> . Se conservaron en el barrido de código muerto justamente por eso; si algún día se borra ese script, esas dos funciones quedan muertas y hay que quitarlas con él.
B18	La estrategia de alertas está escrita para Fargate y el despliegue es EC2. Sin agente de CloudWatch, stdout va a journald: no existe el grupo de logs `/swalp/app` que el propio comentario manda consultar, y los <i>metric filters</i> sobre CRON_NO_EJECUTADO no tienen sobre qué montarse. Es lo que decide si te enteras de que un correo falló, así que invalida a B17 y a cualquier alerta construida encima	El comentario se escribió cuando el destino era ECS/Fargate; el commit <code>dd4228c</code> eligió EC2 y descartó Fargate, y la justificación no se revisó. Dos de sus tres argumentos ya no aplican: sí hay máquina a la que entrar, y el disco no es efímero	<code>config/settings.py:536-540</code> (la justificación), <code>:547</code> (el comando que apunta a un grupo inexistente), <code>:589-592</code> (<code>LOG_A_FICHERO=False</code> en producción). Hay que elegir: agente de CloudWatch en el <code>UserData</code> de <code>infra/cloudformation/swalp-infra.yaml</code> (~0,50 USD/mes) o aceptar journald y reescribir esos comentarios. Ver ADR 016 (<code>./03-arquitectura/adr/016-transporte-de-correo-y-fiabilidad.md</code>), pendiente 3

16.4 Resueltos que conviene recordar

#	Qué pasaba	Cómo se cerró
B16	Rutas comentadas para un modelo	No era temporal: la migración

(cerra do)	que ya no existe. Cinco líneas # COMENTADO TEMPORALMENTE en solicitudes/urls.py (el import y cuatro path) más cuatro plantillas permisodetalle_*.html	0004_remove_historicalpermisodetalle_histo ry_user_and_more ejecuta migrations.DeleteModel(name='PermisoDetalle'). La tabla no está en la base de datos, así que esas vistas no podrían volver aunque se descomentaran. El código muerto incluso arrastraba un bug que nunca llegó a correr: la ruta delete apuntaba a PermisoDetalleUpdateView
B20 (cerra do)	Código muerto en la capa de servicios: 30 funciones, 6 plantillas y 5 rutas comentadas. Dos casos eran algo peor que inertes — prometían trabajar: EmpleadoService.validar_rest ricciones_turno devolvía True siempre, con las validaciones reales en comentarios (# Aquí irían...); e ItemChecklist.bloqueado_por_dependencia era una @property que devolvía False fijo, con un comentario que decía "lo resuelve el servicio al construir la lista" — imposible, porque una property de solo lectura no admite asignación (quien lo resuelve es la vista, con otro atributo). Un tercero, DeudaService.pagar_deuda, sugería un flujo de pago que el dominio no tiene: la deuda entre exploradores nace ya en 'pagada' porque la fecha se pacta en la propia solicitud (views/favores.py, docstring)	Sedimento de refactorizaciones: delegados de una línea que quedaron al extraer servicios, versiones genéricas sustituidas por las de rol (crear_notificacion_aprobacion/_rechazo → _supervisor/_receptor), una cuarta copia de la validación de año que ya hacen turnos/forms.py, las vistas y la API, y consultas de deuda que el código resuelve atacando el modelo directamente
B17 (cerra do)	Un correo que agotaba los cinco reintentos no avisaba a nadie. verificar_crons los contaba, pero su ok dependía solo de la espera del pendiente más antiguo: mil correos agotados daban check verde y salida 0. Detectar una solicitud que el supervisor nunca vio dependía de que alguien abriera el admin por su cuenta	Check propio, _revisar_correos_fallidos, con marcador distinto (CORREOS_FALLIDOS): reutilizar CRON_NO_EJECUTADO habría hecho sonar la alarma de "nadie programó la tarea" por algo que no es lo suyo, y una alarma con dos causas deja de decir qué hacer. La semántica del check del cron no se tocó: un fallido sigue sin acusarlo. Mira solo las últimas 24 h para que la alarma se pueda apagar —reintentando desde el admin, o sola— mientras el total sigue informándose sin dar alarma. solicitudes/management/commands/verificar_crons.py; 7 pruebas nuevas en

		<code>test_verificar_crons.py</code> (21 en total). ADR 016 (<code>./03-arquitectura/adr/016-transporte-de-correo-y-fiabilidad.md</code>)
B19 (cerrodo)	El único código que ataba la aplicación a SMTP estaba muerto. <code>EmailService._configurar_email_backend</code> instanciaba a mano el backend SMTP y no lo llamaba nadie	Borrados la función y su import. Ahora <code>email_outbox_service.py</code> es el único fichero de la aplicación que importa <code>django.core.mail</code> , así que "cambiar de proveedor es configuración, no código" pasó de afirmación a hecho comprobable : <code>grep -rn "django\.core\.mail" --include=*.py</code> . fuera de tests solo debe devolver ese fichero y la línea de <code>EMAIL_BACKEND</code> . 26 pruebas de correo en verde sin tocar una línea
B5 (cerrodo)	Correo irretirable en los flujos multi-solicitud. En cobertura con dos compañeros y en doblada permanente con N, si la segunda creación fallaba el <code>atomic()</code> borraba las filas de negocio, pero el correo de la primera ya había salido y no se puede desenviar. El propio comentario del código lo documentaba como limitación asumida	El <code>envio_agrupado()</code> se abre por fuera del <code>atomic()</code> y solo despacha al salir del bloque, con la transacción ya commitada (<code>solicitudes/services/solicitud_orchestrator.py:269-286,509-531</code>). Si la transacción revierte, el rollback se lleva por delante las filas del outbox y el lote se queda vacío por sí solo: la atomicidad la sigue dando la base de datos (<code>solicitudes/services/email_outbox_service.py:354-359</code>). Ya no hay ninguna ventana en la que un correo salga por una solicitud que después desaparece
B1 (cerrodo)	Los enlaces de aprobación por correo se firmaban con una clave escrita en el código , <code>b'secret_key_change_this'</code> , con el comentario "Cambiar en producción". Quien la conociera — está en el repositorio — podía fabricar un enlace válido para cualquier solicitud y aprobarla sin sesión. La verificación estaba duplicada seis veces : una en <code>email_service.py</code> , cuatro en <code>views/aprobacion_email.py</code> y una en <code>permisos/services.py</code> (esta última, de otra app, ni siquiera estaba documentada). Cualquier arreglo había que hacerlo seis veces, y bastaba olvidar una para reabrir el agujero	Se centralizó todo en <code>solicitudes/services/tokens_aprobacion.py</code> , que firma con <code>django.core.signing</code> sobre <code>SECRET_KEY</code> . Los seis puntos delegan en él (<code>email_service.py:146,528</code> ; <code>views/aprobacion_email.py:140,199,251,303</code> ; <code>permisos/services.py:30,34</code>). Por eso el módulo está centralizado: no lo vuelvas a duplicar. El literal solo sobrevive dentro de <code>solicitudes/tests/test_tokens_aprobacion.py:68</code> , donde se usa a propósito para comprobar que un token firmado con la clave vieja ya no se acepta
B2 (cerrodo)	Los tokens no caducaban ni se invalidaban tras usarse: eran	La firma incluye marca de tiempo y se valida con <code>max_age = APPROVAL_LINK_MAX_AGE_DAYS</code>

do)	deterministas sobre (solicitud_id, empleado_id, tipo)	(tokens_aprobacion.py:51-52,84). El uso único lo garantiza <code>_ya_resuelto_para()</code> contra el estado en base de datos (views/aprobacion_email.py:58-73), no una lista en memoria. Ver § 9.2
B6 (cerrodo)	Seis mensajes que ve el usuario tenían los acentos rotos por doble codificación UTF-8: en pantalla se leía literalmente <code>Token inválido o expirado</code> y <code>No se encontró supervisor para esta solicitud</code> . Eran 31 secuencias en 5 archivos de <code>solicitudes/views/</code>	Corregidas todas. La comprobación de que no vuelven: <code>`grep -rn "Ãi\`</code>
B7 (cerrodo)	La página de error del token no decía el plazo ni qué hacer, y un mismo mensaje cubre cuatro causas distintas (caducado, manipulado, ajeno, rol equivocado)	<code>templates/solicitudes/error_token.html</code> nombra ahora los <code>{{ dias_validez }}</code> días y remite a resolver desde la aplicación. El contexto se arma en <code>core/utills/error_token.py</code> , punto único por el que pasan las 15 llamadas de <code>solicitudes</code> y <code>permisos</code> : así ninguna vista puede olvidarse de pasar el dato y dejar la frase coja
B9 (cerrodo)	En permisos, el plazo de 24 h para pedir la cancelación se medía contra <code>actualizado_en</code> , que es <code>auto_now</code> : cualquier guardado posterior del permiso —un comentario del supervisor, la propia petición— reiniciaba el reloj. El modelo no tenía el equivalente de <code>SolicitudCambio.fecha_resolucion</code>	Campo propio <code>PermisoEspecial.fecha_aprobacion</code> (permisos/models.py:101-104; migración <code>permisos/0009</code> , aditiva y también sobre la tabla histórica, depende de la <code>0008</code> , ya aplicada), sellado al aprobar en las dos puertas: <code>PermisoEspecialAprobarView.post</code> (permisos/views.py:522) y <code>PermisoEspecialResolverEmailView.get</code> (:340). El cálculo usa <code>permiso.fecha_aprobacion</code> or <code>permiso.actualizado_en</code> (:443). El respaldo es un invariante, no algo provisional : sin él, los permisos aprobados antes de la migración quedarían incancelables para siempre. No lo quites. Tests :110 y :133 de <code>permisos/tests/test_cancelacion_permiso_consensio.py</code>
B10 (cerrodo)	<code>PermisoMediaJornadaCancelarView.post</code> bifurcaba con <code>if es_dueno and not es_sup</code> , así que un explorador que además fuera supervisor caía en la rama de cancelación directa y se autoaprobaba la cancelación de su	La condición es ahora <code>if es_dueno</code> (permisos/views.py:424): decide de quién es el permiso , no qué rol tiene quien pulsa. Sobre lo propio siempre se PIDE. La otra mitad de la regla vive en <code>_puede_responder_cancelacion</code> (ver B11). Test <code>test_un_supervisor_no_cancela_directo_su_propio_permiso</code> (:143)

	propio permiso	
B11 (cerrodo)	PermisoMediaJornadaCancelResponderView solo comprobaba _es_supervisor(request.user) : cualquier supervisor decidía sobre el permiso de cualquier explorador, aunque el aviso se mandara solo al suyo. Control por rol en vez de por relación	Dos helpers de módulo en permisos/views.py : _supervisor_del_permiso (:561) , fuente única de quién responde por el permiso — permiso.supervisor or permiso.empleado.supervisor —, usada por la autorización (:588) y por el aviso (:611) para que decida y avise la misma persona; y _puede_responder_cancelacion (:569) , con tres reglas: rol de supervisor (:583), el dueño no se responde a sí mismo (:585) y hay que ser el supervisor concreto (:588-591), salvo que el permiso no tenga ninguno resoluble , en cuyo caso vale cualquiera —matiz deliberado: si no, la petición sería incontestable y caducaría siempre—. La vista carga con select_related('empleado', 'empleado_supervisor', 'supervisor') (:534). Tests :158 y :169
B13 (cerrodo)	`VENTANA_CANCELACION_MINUTOS = 30` existía dos veces con significados distintos : legado sin uso en solicitudes/use_cases/cancelar_solicitud.py y regla viva en cambio_descanso_aplicacion_service.py , donde sostenía el bloqueo temporal del día. Quien leyera una creería haber entendido la otra	Cerrado por eliminación, no por el renombrado que se anticipaba . Se retiró el bloqueo temporal —la premisa que lo sostenía murió con la cancelación consensuada, ver ADR 010 (./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md)— y con él la constante en los dos archivos: la del use_case era código muerto sin ningún uso. VENTANA_CANCELACION_MINUTOS ya no existe en el proyecto ; los plazos vivos son VENTANA_PEDIR_CANCELACION_HORAS y VENTANA_RESPONDER_CANCELACION_HORAS (core/constants.py:201,206). Suite en verde: 959 tests
B14 (cerrodo)	El modal de aprobación/rechazo de solicitudes enviaba el texto en el campo comentario , pero todas las vistas de aprobacion_views.py leen comentario_respuesta . El comentario del supervisor nunca se guardaba : se escribía, se enviaba y se perdía en el camino, sin error visible	El front envía ya comentario_respuesta (static/js/solicitudes/solicitudes_pendientes_list.js , con el motivo comentado en el propio código para que no se “corrija” de vuelta). Al hacerse obligatorio el comentario (P6) el fallo habría salido igualmente: el backend rechazaría con comentario_requerido un formulario aparentemente relleno
B3 (cerrodo)	No había recuperación de contraseña . Tras 5 intentos fallidos el usuario quedaba bloqueado una hora y su única salida era un administrador	Implementada y enrutada: core/login/urls.py declara las seis rutas del ciclo completo — password_change, password_change_done, password_reset, password_reset_done, password_reset_confirm y

		<code>password_reset_complete</code> (<code>core/login/urls.py:19-35</code>). Se declaran una a una en vez de con <code>include('django.contrib.auth.urls')</code> porque ese incluye trae su propio <code>login/logout</code>, que chocarían con las vistas propias; los nombres sí son los estándar de Django, así que <code>{% url 'password_reset' %}</code> y los enlaces que genera <code>PasswordResetForm</code> funcionan sin configuración extra. El correo de reseteo no pasa por <code>EmailOutbox</code> a propósito (<code>core/login/views.py:135-137</code>). Los límites de axes siguen vigentes (<code>config/settings.py:362-363</code>). Manual completo: MANUAL_RECUPERAR_CONTRASENA.md (<code>./05-referencia/deployment/02-correo/MANUAL_RECUPERAR_CONTRASENA.md</code>)
B8 (cerrodo)	CSP estricta en modo report-only indefinido. La política activa permitía cuatro CDN que ninguna plantilla usaba ya	La migración a estáticos locales terminó. <code>jsDelivr</code> (<code>flatpickr</code>, <code>chart.js</code>, <code>sweetalert2</code>, <code>fullcalendar</code>), <code>cdnjs</code> (<code>Font Awesome</code>) e <code>ionicons</code> se autohospedan en <code>static/plugins/</code> y ya no los referencia ninguna plantilla. Barridas las 116 plantillas, el único origen externo que queda es Google Fonts; confirmado eso, la segunda cabecera <code>REPORT-ONLY</code> se borró y sus valores son ahora los que se aplican de verdad (<code>config/settings.py:424-433</code>). Sigue abierto por decisión —no por olvido— el <code>'unsafe-inline'</code> en <code>script-src</code>, razonado en el propio <code>settings.py</code>
B12 (cerrodo)	Los dos mensajes de bloqueo de <code>cambio_descanso_strategy.py</code> aconsejaban "cancela ese cambio dentro de los 30 min de aprobado": desde el acuerdo de cancelación consensuada (ADR 009) ni el plazo son 30 minutos ni la cancelación depende de quien lee el mensaje. Un consejo inejecutable	Reescritos (<code>solicitudes/services/strategies/cambio_descanso_strategy.py:120-128</code>): ahora dicen esperar a que pase la ventana o pedirselo al supervisor. En esa pasada solo cambió el consejo. Después, el bloqueo temporal en sí se retiró (ADR 010) y el segundo mensaje, el de "cambio de descanso reciente", quedó inalcanzable y se eliminó: hoy queda uno solo, el de otro tipo de cambio aplicado (<code>solicitudes/services/strategies/cambio_descanso_strategy.py:117-120</code>). Ver B13

Los dos primeros están generalizados como **patrón 38** de `PROTECTION_PATTERNS.md` (una credencial que actúa sin sesión se firma en un solo sitio, caduca y falla cerrada) y razonados en el [ADR 006](#) (`./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md`).

17. Cambios técnicos relevantes

Changelog para desarrolladores, del más reciente al más antiguo. Cada línea corresponde a un commit real del repositorio. Las filas que antes figuraban como *(sin commit aún)* ya llevan su hash corto: el trabajo de la rama `fix/cambio-descanso-temporada` está confirmado y fusionado en `main`. Varias filas comparten hash porque un mismo commit cierra código, tests y notas de un mismo hallazgo; las filas puramente documentales apuntan al commit de documentación que las introdujo (`9dc4af1`).

El aviso del formulario y el rechazo del servidor dicen la misma regla con las mismas palabras

(Trabajo en curso: `solicitudes/services/solicitud_orchestrator.py`, `static/js/cambio-turno/solicitar_cambio_descanso.js` y su suite están en el árbol, sin confirmar.)

La regla del día de pago libre en la cobertura con dos compañeros (§ 8.3) tenía **dos redacciones**: una en el aviso bloqueante del formulario y otra en el rechazo del POST. Es la misma regla, y el usuario puede toparse con ella por cualquiera de los dos caminos.

Cambio	Impacto
El servidor adopta la redacción del formulario (<code>solicitud_orchestrator.py:228-234</code>)	Un solo texto para una sola regla, se llegue por donde se llegue
Había un tercer sitio con la misma regla en otra redacción —el bullet de la lista «Faltan datos» al pulsar Enviar—, que el centinela no veía porque vive en el mismo archivo JS que el recuadro rojo. Se unificó por construcción: tres constantes en <code>solicitar_cambio_descanso.js:1064-1067</code> (<code>DOS_LIBRE_TITULAR</code> , <code>DOS_LIBRE_MOTIVO</code> , <code>DOS_LIBRE_SALIDA</code>) que consumen el recuadro (<code>:1091-1095</code>) y el bullet (<code>:1291-1296</code>)	Dentro del front el literal existe una sola vez , así que los dos avisos no pueden separarse sin tocar la constante: garantía más fuerte que un test, y por eso ahí no hay test . La sincronización manual y el centinela solo hacen falta en el cruce front ↔ servidor , que son lenguajes distintos y no pueden compartir el literal
La única diferencia que queda es deliberada: el formulario nombra el día libre concreto (<code>solicitar_cambio_descanso.js:1075-1083</code>), el servidor no	El servidor no consulta el desplegable, así que no puede sugerir un día; y el formulario solo lo nombra si sigue siendo una opción viva
En el JS las negritas envuelven frases completas (<code>:1091-1095</code>)	Si partieran una frase por dentro, el <code></code> quedaría en medio y la frase núcleo dejaría de ser

	localizable para el centinela
Los dos fuentes llevan un comentario que apunta al otro (<code>solicitud_orchestrator.py:222-227</code> , <code>solicitar_cambio_descanso.js:1085-1090</code>)	La sincronización es manual : quien edite un lado tiene el otro señalado en la misma pantalla
AvisoDiaPagoLibreEspejoTest (<code>solicitudes/tests/test_cobertura_dos_companeros.py:29</code> , caso :60)	Lee los dos fuentes y exige las tres frases núcleo literales (:44-49). Las sigue encontrando porque ahora viven en las constantes, y la costura de <code>_COSTURA</code> (:54) une los dos trozos de <code>DOS_LIBRE_MOTIVO</code> . Si falla, se ajusta el lado que quedó atrás; no se relaja la comparación

Detalle y textos literales en [05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md](#) (./05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md) (§ 6, cobertura con DOS compañeros).

El aviso de doblada permanente deja de dar por firme lo que solo está pendiente

(Trabajo en curso: los cambios de

``solicitudes/services/strategies/doblada_permanente_strategy.py`` y de su suite están en el árbol, todavía sin confirmar.)

La **regla no cambia**: una doblada permanente del compañero, pendiente o aprobada, que comparta alguna fecha sigue bloqueando el nuevo acuerdo. Lo que cambia es el aviso, que era uno solo y decía «ya tiene una doblada permanente en esas fechas» incluso cuando el otro acuerdo estaba solo **pendiente de aprobación**: daba por firme lo que aún podía cancelarse.

Cambio	Impacto
Tres mensajes en lugar de uno, ramificando por <code>det.solicitud.estado</code> y —si es pendiente — por <code>det.solicitud.explorador_solicitante_id</code> (<code>doblada_permanente_strategy.py:278-297</code>)	El usuario sabe si lo que le bloquea es un acuerdo cerrado o uno que todavía se puede caer, y actúa en

	consecuencia
La variante «la pendiente es tuya»: "Ya enviaste una doblada permanente a {compañero}..." (: 283-288)	Es el caso frecuente. Mandarle a «elegir otro compañero» cuando la pendiente es suya lo despista: lo que tiene que hacer es esperar su propia solicitud
El listado de fechas del choque (detalle , :267-269) se mantiene en las tres variantes	El aviso sigue señalando la fecha exacta que bloquea
La rama compara <code>str(det.solicitud.estado or '').lower() == 'pendiente' (:278)</code> , no == 'pendiente' a secas	El filtro que trae otros va por ORM y MySQL no distingue mayúsculas : una fila guardada como 'PENDIENTE' llegaba hasta aquí y, comparada en Python tal cual, se anunciaba como aprobada — el error exacto que los tres mensajes vienen a

	corregir (motivo en :274-277)
<code>solicitudes/tests/test_doblada_permanente.py: :DobladaPermanenteOtro AcuerdoTest: test_el_aviso_distingue_pendiente_de_aprobada (:355) y test_si_la_pendiente_es_propia_el_aviso_lo_dice (:371)</code>	El primero fija que el aviso de una pendiente no contiene "aprobada"; el segundo, que el de la propia empieza por "Ya enviaste"

Mensajes literales y motivo en [05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md](#)
(./05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md) (§ 5, regla 9).

«Cambio de doblada» queda deshabilitado, y el interruptor es del backend

(Trabajo en curso: `solicitudes/services/strategies/cambio\_descanso\_strategy.py`, la tarjeta del template y `solicitudes/tests/test\_cambio\_doblada\_deshabilitado.py` están en el árbol, sin confirmar.)

Cambio	Impacto
<code>SUBMODALIDADES_SEMANA_DESHABILITADAS = frozenset({'cambio_doblada'}) (cambio_descanso_strategy.py:53)</code>	Un conjunto, no un if con un literal: apagar o reactivar otra sub-modalidad es editar una línea, y el test <code>test_el_interruptor_es_el_conjunto_y_n o_un_literal_suelto (:131</code> del archivo de tests) lo vigila
La comprobación va en validar_solicitud , en la rama de ENTRE SEMANA y antes de despachar (:248-256)	Cubre creación y aprobación: <code>revalidar_para_aprobar</code> pasa por ahí (<code>base_strategy.py:114</code>), así que una solicitud colada antes del apagado tampoco se aprueba
El mensaje nombra la opción apagada con la etiqueta de SUBMODALIDAD_SEMANA_CHOICES (:252) y ofrece salida: «Elige otra de las opciones de esa semana»	El texto sigue siendo verdad si mañana se apaga otra, y el usuario no queda atascado sin saber qué hacer
La clase disabled de la tarjeta (<code>templates/solicitudes/solicitar_cambi</code>	Se quita desde DevTools, y una pestaña abierta desde antes del despliegue conserva el HTML y el

o_descanso.html:128-133) es solo el reflejo visual	JS viejos: el POST llega igual al servidor. Por eso la defensa está en el backend
El código del sub-flujo no se elimina	_validar_semana_cambio_doblada y su aplicación siguen enteras. Reactivar = sacar la clave del frozenset + quitar la clase disabled

Regla en § 8.3; sub-modalidades y mecanismo completo en [05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md](#) (./05-referencia/solicitudes/REGLAS_NEGOCIO_SOLICITUDES.md) (§ 6, modalidad ENTRE SEMANA).

El vigilante de las tareas programadas ahora avisa por su cuenta

(Trabajo en curso: `solicitudes/management/commands/alertar_crons.py` y su suite están en el árbol, todavía sin confirmar. El marcador `CORREOS_FALLIDOS` que este comando reenvía sí está commitado, en `0b8d896`.)

Cambio	Impacto
Nuevo <code>python manage.py alertar_crons</code> . Ejecuta los tres chequeos de verificar_crons reutilizando su clase (alertar_crons.py:66-73,104-108) y, si alguno falla, envía el correo de alerta	verificar_crons solo diagnóstica. En AWS bastaba: el crontab de Linux avisa por correo al salir con código distinto de cero y CloudWatch enganchaba los marcadores con un <i>metric filter</i> . En Dokploy no existe ninguna de las dos cosas — sus notificaciones cubren despliegues, backups, limpieza de Docker y umbrales del servidor, pero no hay disparador para un Schedule que falla (:4-18)—
Transporte <code>smtplib</code> directo, no <code>django.core.mail</code> (:205-242)	Dos motivos que apuntan al mismo sitio: el vigilante no puede compartir maquinaria con lo que vigila —si lo averiado es el worker del outbox, una alerta encolada se quedaría esperando junto a los correos que denuncia— y el ADR 016 (./03-arquitectura/adr/016-transporte-de-correo-y-fiabilidad.md) fija que fuera de los tests django.core.mail solo puede aparecer en email_outbox_service.py , así que un import aquí rompería ese invariante en silencio (:20-34). Se reutiliza la configuración EMAIL_* de settings: mismo transporte, sin cola (:214-235)
Destinatarios por <code>--para</code> o <code>ALERTAS_CRON_EMAIL</code> , sin ajuste nuevo en settings.py (:159-169)	Es configuración de despliegue y en Dokploy vive junto al resto de variables de la aplicación. Sin destinatarios el comando falla ruidosamente : un vigilante mal configurado que calla es peor que

	ninguno
Latido semanal los lunes (:78-79,119-121)	Única defensa contra que deje de correr este comando: un vigilante mudo y uno muerto se ven igual desde fuera. Semanal y no diario a propósito —un aviso diario de que no pasa nada se deja de leer en dos semanas, y entonces el latido no vale nada—
Código de salida `1` también cuando el aviso no se pudo enviar (:142-149)	El Schedule queda en rojo además de haber avisado. No se relanza la excepción: se informa por stderr para que el traceback no tape el diagnóstico, que es lo que alguien va a leer
El cuerpo del correo lleva un pie con qué hacer ante cada marcador (:191-202)	CRON_NO_EJECUTADO → revisar los Schedules de Dokploy (swalp-outbox , swalp-sanciones); CORREOS_FALLIDOS → /admin/solicitudes/emailoutbox/ , filtrar por «fallido» y decidir
Pruebas: solicitudes/tests/test_alertar_crons.py, 16 casos	Prueban solo lo que este comando añade encima de verificar_crons —silencio cuando todo va bien, aviso, destinatarios, no dejar fila en el outbox, fallo de envío, latido y --dry-run —, para que los criterios de los chequeos no queden duplicados en dos suites (:2-13)

División de trabajo entre los dos comandos en § 7.3; operación en § 13.5.

El detalle del día de Mis Turnos dice siempre con quién es el acuerdo

(*sin commit aún*). Cambio de **lectura**: ninguna regla de negocio, ningún modelo, ninguna migración y ninguna ruta cambian. Lo que cambia es de dónde sale el dato que se enseña y qué palabras se usan para enseñarlo.

El problema. **MisTurnosPorMesView._enriquecer_solicitud_info** tenía ~240 líneas con **ocho** consultas que reimplementaban a mano la geometría de cada tipo —quién trabaja qué fecha—, y esa copia nunca estuvo completa: DOBLADA PERMANENTE no tenía ninguna consulta, CT PERMANENTE solo cubría el primer día del rango (**turno_origen/turno_destino** apuntan al primer turno creado, **solicitudes/services/strategies/ct_permanente_strategy.py**) y CAMBIO DESCANSO entre semana solo dos de sus cuatro combinaciones —las sub-modalidades de temporada tienen otra geometría—. Los días que se caían del mapeo salían **sin compañero**: «ahora trabajas DOBLADA», sin decir para quién.

Qué cambió	Detalle	Riesgo / contrapartida
Servicio nuevo	Fuente única del acuerdo que puso a alguien a	El snapshot es ahora dato

<code>`solicitudes/services/acuerdo_por_dia_service.py` (AcuerdoPorDiaService)</code>	trabajar un día: <code>en_rango (:81)</code> y <code>en_fecha (:154)</code> . Hermano de <code>DescansoPorSolicitudService</code> , que resuelve el lado del descanso. Lee los snapshots en los tres sitios donde viven (<code>:163</code>) y trata PAGO REPROGRAMADO en rama propia (<code>:273</code>). Ver § 3.4	de lectura , no solo de reversión: un aplicador que deje de capturarlo deja el día sin compañero. La guarda de realidad (<code>:119-130,149</code>) impide lo contrario, que un snapshot viejo reclame un día ya reescrito
La vista delega	<code>_enriquecer_solicitud_info (turnos/api/views/turnos_mes.py:338)</code> pasa de ~240 líneas a una llamada; el fichero baja de 577 a 422 líneas. La firma recibe ahora <code>fecha_inicio</code> , <code>fecha_fin</code> . <code>turno_origen/turno_destino</code> queda como respaldo en <code>_solicitud_por_turno (:369)</code> y <code>_respaldo_por_turno (:411)</code>	El respaldo solo entra si el día es cambio o doblada y ningún snapshot lo reclama (<code>:363-365</code>)
El día LIBRE también nombra su trámite	<code>DescansoPorSolicitudService</code> añade <code>tipo_solicitud</code> a las cuatro ramas y deja de devolver <code>None</code> en <code>solicitud_id</code> y fechas para CAMBIO DESCANSO (<code>solicitudes/services/descanso_solicitud_service.py:277-291</code>) y DOBLADA PERMANENTE (<code>:314-326</code>). El dato viaja dentro del compañero, como clave acuerdo , desde donde ya se sabía: <code>_mapa_descanso_multi (solicitudes/services/cambio_descanso_aplicacion_service.py:140-155)</code> . <code>mis_turnos_dia.py</code> lo publica en <code>descanso_info (:183)</code>	acuerdo es una clave interna: <code>DescansoPorSolicitudService</code> la consume y no la propaga a la API. Quien solo lee <code>'id'/'nombre'</code> del compañero no se entera de que está
Textos del detalle del día a un módulo puro	Nuevo <code>static/js/utils/detalle-dia-mensajes.js (window.DetalleDiaMensajes):</code> <code>nombreAcuerdo (:49)</code> , <code>esFinDeSemana (:57)</code> , <code>loQueTrabaja (:74)</code> , <code>etiquetaJornada (:80)</code> , <code>conQuien (:92)</code> , <code>mensajeCambio (:116)</code> y <code>mensajeDescanso (:160)</code> . <code>static/js/mis_turnos.js</code> se queda con el HTML (<code>:551</code>)	Se carga antes que <code>mis_turnos.js (templates/turnos/mis_turnos.html:91)</code> , consumidor en <code>:95</code>). Ver § 16.2
Vocabulario	El detalle rotulaba "Cambio de turno" para todos los tipos. Ahora traduce por <code>Turno.tipo_cambio (NOMBRE_ACUERDO,</code>	Son dos vocabularios distintos y siguen siéndolo (§ 16.2, primera

	detalle-dia-mensajes.js:28-38), y el backend salva la distancia entre TipoSolicitudCambio.nombre y Turno.tipo_cambio en _TIPO_SOLICITUD_A_TIPO_CAMBIO (acuerdo_por_dia_service.py:319-322)	fila). El mapa traduce solo los dos nombres que difieren
Pruebas	Nuevas: solicitudes/tests/test_acuerdo_por_dia.py (10 casos) y tests_js/detalle-dia-mensajes.test.cjs (31 casos). Ampliada: test_reflejo_mis_turnos.py con _con_quien (:163) y _libre_por (:183), aplicados a los seis tipos por ambos lados, y con la guardia _exige_snapshot_resultante (:130) dentro de _crear_y_aplicar (:113): aplicar CUALQUIER tipo tiene que dejar snapshot_turnos_resultantes, o el detalle de ese tipo se queda mudo sin que ningún test de turnos se entere. Suite completa en verde: 1 754	§ 14.2

Al desplegar: MisTurnosPorMesView cachea el mes por empleado durante una hora. Hay que invalidar (CacheService.invalidar_cache_turnos_empleado) o esperar el TTL, o se sigue sirviendo el JSON antiguo y el cambio "no se ve" (§ 16.2).

Los once correos pasan a heredar una sola base

(sin commit aún). Cambio de **presentación**: ninguna regla de negocio, ningún endpoint, ningún modelo y ninguna migración cambian. EmailService._enviar_email_desde_usuario sigue siendo el cuello de botella único y el outbox funciona igual. El manual de usuario no se toca: el explorador no ve ni un flujo ni un mensaje distinto.

Cada plantilla de correo llevaba su propio <style> de ~200 líneas, repetido. Ahora las once extienden templates/emails/base_email.html y componen su cuerpo con cuatro parciales (_seccion, _fila, _botones, _item). El detalle, en § 4.4. Lo que le importa al próximo que añada un correo:

- ``site_url`` es obligatorio en el contexto. Los botones «Ver en el sistema» lo usan y `render_to_string` corre sin `request`, así que los context processors no llegan. Se añadió `'site_url': settings.SITE_URL` a cinco contextos de `solicitudes/services/email_service.py` (:320,356,390,422,454) y a

`permisos/services.py:157`. Es la trampa del cambio, y la vigila `solicitudes/tests/test_arquitectura_correos.py`.

- **Logo nuevo e incrustado:** `static/img/logo-explora-email.png`, rasterizado desde `static/img/parque-explora-logo.svg` (la versión blanca del sidebar) con el relleno cambiado a `#e30613`, porque en correo el SVG no se renderiza. Viaja **dentro** del mensaje como `cid:logo-swalp` (lo adjunta `EmailOutboxService._incrustar_logo`): enlazado no se vería, ni con `SITE_URL` en localhost ni con el dominio público, porque los clientes bloquean las remotas.
- **Correo de permisos con plantilla propia:**
`templates/permisos/emails/solicitud_permiso.html`.

`PermisoNotificacionService._email_html` deja de construir HTML con un f-string y llama a `render_to_string` (`permisos/services.py:149`); su firma no cambia.

Una conexión SMTP por lote, no por correo

(*sin commit aún*). Medido en logs el **2026-09-07**: crear una solicitud tardaba **6,3-7,8 s** en responder y todo ese tiempo era SMTP. Cada solicitud manda tres correos —supervisor, receptor y solicitante— y cada uno abría **su propia** conexión: ~2 s de handshake TLS + AUTH contra Gmail. Los flujos multi-compañero multiplicaban la cifra (cobertura con 2 compañeros = 6 correos; doblada permanente con 3 = 9).

Ningún modelo, ningún endpoint y ninguna regla de negocio cambian. Lo que cambia es **cómo se entrega**:

- `EmailOutboxService.enviar_lote(ids)` es ahora el **único camino de envío** (`solicitudes/services/email_outbox_service.py:234`). Abre la conexión con un `open()` **explícito** antes del bucle y la reutiliza; sin ese `open()` no habría ahorro, porque `send_messages` de Django solo cierra la conexión que él mismo abrió. Un correo suelto es un lote de uno.
- `_intentar (:151)` devuelve `'enviado' | 'fallido' | 'omitido'`. `'omitido'` significa que `**no`

se pudo reclamar la fila** y por tanto no dice nada del estado de la conexión: esa distinción es la que permite al lote saber si debe renovar el canal. `intentar_enviar (:202)` sobrevive con su contrato booleano de siempre, como envoltorio.

- `_abrir_conexion (:207)` devuelve `None` si el SMTP no responde. Efecto que importa: el fallo ocurre **antes** de reclamar ninguna fila, así que una caída del servidor ya no le quema un intento a cada correo. Antes, cinco caídas seguidas dejaban un correo en `fallido` para siempre sin que su destinatario tuviera nada que ver.
- `envio_agrupado() (:329)`: context manager **reentrante** que junta en un solo lote todo lo

encolado dentro del bloque, y despacha en el `finally`. **No** se apoya en `transaction.on_commit`, a propósito: fuera de un `atomic()` Django ejecutaría el callback en el acto y cada correo formaría su propio grupo de uno.

- `despachar(fila_id) (:370)` queda como punto único que decide **cuándo** se entrega;

`enviar_tras_commit (:303)` acepta un id o una lista y manda todo el lote en **un** hilo; y

`procesar_pendientes (:280)` —el barrido del cron— también va por una sola conexión.

Alrededor: `EmailService._enviar_email_desde_usuario` ya no decide el modo de envío, delega en `despachar (solicitudes/services/email_service.py:140)`; `NotificacionService` envuelve sus dos puntos multi-correo en `envio_agrupado` y extrae el cuerpo a `_notificar_solicitud_creada` y `_notificar_aprobacion_receptor` para no reindentar 120 líneas (`:40,417`).

Dos cambios en el orquestador, y **el segundo no es rendimiento**:

156. `_procesar_cobertura_dos (:201)` y `_procesar_doblada_permanente_multi (:387)` envuelven su

bucle en `envio_agrupado()` **por fuera** del `atomic()` (`:278,530`). Además de compartir conexión, nada se entrega hasta que el bloque termina, **ya commitado**: eso cierra el bug B5 —el correo de la primera solicitud salía aunque el rollback borrara las dos— porque ahora el rollback se lleva también las filas del outbox. Ver § 16.4.

157. El TTL del candado anti doble-submit sube de 10 s a **30 s**, en la constante

`_DEDUPE_TTL_SEGUNDOS (:34)`. El TTL debe sobrevivir al request que protege, y los flujos multi-compañero tardaban 13-19 s: el candado caducaba a mitad de la operación y dejaba pasar un segundo POST idéntico.

Configuración: `EMAIL_SEND_ASYNC=True` se añade al `.env` de desarrollo y a `.env.example:52`. El **default de `config/settings.py:271` no cambia**: sigue siendo `IS_PRODUCTION`, de modo que los tests siguen enviando síncrono y `mail.outbox` se puebla dentro del propio test.

15 tests nuevos en `solicitudes/tests/test_email_outbox.py`

(`OutboxUnaConexionPorLoteTest`, `OutboxEnvioAgrupadoTest`); suite completa en verde, 1.729

pasados. Operación en [MANUAL_OUTBOX_CORREOS.md](#) (`./05-referencia/deployment/02-`

`correo/MANUAL_OUTBOX_CORREOS.md`).

Horas a favor del explorador: la deuda también corre al revés

980fe17. Hasta aquí el sistema de horas era **unidireccional**: solo modelaba lo que el explorador debe a la corporación, y `DeudaPermisoMes.minutos_pendientes` lo decía literalmente —*"Nunca negativo: un exceso no genera crédito"* (`permisos/models.py:303-305`)—. Pero el caso contrario ocurre: si su jornada es de 08:00 a 14:00 y el supervisor le pide entrar a las 07:00, esa hora se la deben, y sin sitio donde anotarla se perdía en cuanto se saldaba la deuda menor que tuviera encima.

Modelos nuevos `CreditoHoras` y `ConsumoCreditoHoras` (`permisos/models.py:355,435`), servicio `CreditoHorasService` (`permisos/credito_horas_service.py`) y CRUD propio (`empleados/views/credito.py`, `empleados/urls.py:94-97`, sesión `credito_horas` en `core/sesiones.py`). Migraciones `permisos/0012_*` y `0013_*`.

Las decisiones y las alternativas descartadas están en el [ADR 014](#) (`./03-arquitectura/adr/014-horas-a-favor-del-explorador.md`). En resumen:

- **Es una bolsa, no un ajuste a una deuda concreta.** El crédito sobrevive al pago y el remanente queda para deudas futuras. De ahí el consumo **FIFO** por `fecha_hecho`, que vive en el `ordering` del modelo (`:408`) y no en el servicio.

- **No es un tipo de transacción nuevo: es un medio de pago dentro del PDH existente.** Eso evita

duplicar la vista, el servicio de pago y la lógica de reversión. `PDHCreateView` lo consume tras crear el pago y deshace el PDH entero si no se puede aplicar (`empleados/views/pdh.py:169-189`); `PDHDeleteView` lo devuelve a la bolsa (`:246`). La estructura es una **copia deliberada** de `DeudaPermisoMes` + `PagoDeudaPermisoMes`.

- **No toca la sanción.** Un mes vencido sigue sancionando aunque se salde con crédito: lo sancionable es haber dejado vencer el plazo, no el importe. `sancion_deuda_calculo.py` y `deuda_corporativa_service.py` quedaron **sin tocar**, y el centinela `permisos/tests/test_credito_horas.py:222` lo fija.

En el consolidado, `total_horas` se mantiene intacta —la leen la vista de morosos y las plantillas— y el neto va aparte en `saldo_netto`, que **puede ser negativo** cuando la corporación le debe a él (`turnos/services/consolidado_horas_service.py:286-303,330-333`).

Dos correcciones posteriores, del mismo día y del mismo error de base —validar en el sitio equivocado—, están descritas en § 16.2: el desfase `min / step` del campo de horas y el segundo mensaje de error contradictorio que producía `Model.clean()`.

Levantar una auto-sanción por deuda condona la deuda del mes

Estado nuevo `'condonada'` en `DeudaCorporativa` (`solicitudes/models.py:719`) y `DeudaPermisoMes` (`permisos/models.py:256`), y servicio `DeudaCorporativaService.condonar_deudas_por_levantamiento` (`solicitudes/services/deuda_corporativa_service.py:169`) disparado desde `SancionEmpleado.levantar()` (`empleados/models.py:438`), dentro de su misma transacción. Cierra un estado sin salida: la deuda de una sanción levantada no se podía pagar (mes vencido), no la consumía esa sanción (`levantada_en IS NULL`) ni ninguna nueva (el mes ya cuenta como juzgado). Se perdona la deuda, no el antecedente. Efectos colaterales alineados: `_tiene_historia` (`permisos/deuda_permiso_service.py:44`) y el revert de PDH (`permisos/pago_horas_service.py:387`) tratan `'condonada'` como hecho ocurrido, y el consolidado de horas la muestra aparte, separada de lo cumplido

(turnos/services/consolidado_horas_service.py:75,137). Migraciones permisos/0011_* y solicitudes/0039_*: solo AlterField de choices, sin ALTER TABLE. Razonado en el [ADR 013](#) (./03-arquitectura/adr/013-condonacion-al-levantar-la-sancion.md), que enmienda el punto 3 del [ADR 011](#) (./03-arquitectura/adr/011-sancion-independiente-del-pago.md).

En la misma tanda: **el criterio de "esta sanción es automática" pasa a ser el periodo en las siete consultas que deciden algo** —dos Q de clase, ES_AUTOMATICA y ES_MANUAL (solicitudes/services/deuda_corporativa_service.py:58-69)—, con lo que editar el motivo deja de poder reabrir el ciclo de sanciones sobre un mes ya juzgado; el explorador **recibe el aviso por la campana** (notificar_condonacion, :282, disparado desde levantar()) dentro de la misma transacción, empleados/models.py:442-443), con tipo de notificación propio 'sancion_levantada' (solicitudes/models.py:22, migración solicitudes/0040_alter_notificacion_tipo.py); las horas que se comunican tras el hecho salen de horas_condonadas_por (:258), que **lee lo escrito** en vez de recalcular la estimación previa (empleados/views/sanciones.py:149); y el consolidado renombra total_saldado_sancion → total_extinguido_sancion, saldadas_sancion → extinguidas_sancion y _saldadas_por_sancion → _extinguidas_por_sancion (turnos/services/consolidado_horas_service.py:75,302-303), alineando el código con lo que ya decía la pantalla: **extinguido**, porque nadie pagó. Tests en solicitudes/tests/test_consumo_deuda_por_sancion.py:339,362,385 y turnos/tests/test_consolidado_extinguido_sancion.py.

Despliegue en AWS: se elige EC2 t4g.micro y se descarta Fargate

Commit [dd4228c](#). Cierra la decisión de infraestructura con un techo de **150.000 COP/mes** (~\$37 USD) y la escala real de la primera producción: **300 usuarios / 30 solicitudes-día**, un tercio de lo que asumía la documentación anterior.

Qué	Resultado
Cómputo	EC2 t4g.micro + RDS db.t4g.micro Single-AZ ≈ \$27/mes + IVA (~130.000 COP)
Fargate	Descartada . Contando los crons por EventBridge —8.640 lanzamientos/mes que la tabla de costes original no incluía— sale en ~\$64/mes con IVA. Su checklist se borró; sigue en el historial en e0e4388
Caché	CACHE_URL=db://cache_appturnos . ElastiCache (~\$12/mes) se sale del presupuesto, y Redis en la propia EC2 se comería 200-400 MB del único GB de RAM
IaC	infra/cloudformation/swalp-infra.yaml , 11 recursos. Ubuntu 24.04 LTS y

	MySQL 8.4
Python	Se mantiene 3.12 en Dockerfile y CI: es la versión de Ubuntu 24.04 LTS, tiene soporte hasta 2028, y subir solo el contenedor crearía divergencia con producción

Errores de documentación corregidos en esa pasada (los cuatro estaban en documentos que se daban por buenos):

158. La arquitectura afirmaba que el burst de CPU estaba *"cubierto por los créditos"*. **Es falso:** RDS fuerza modo *Unlimited* y **no se puede desactivar**, así que factura el excedente. En EC2 sí se puede, y la plantilla lo fija con `CreditSpecification: CPUCredits: standard`.
159. El presupuesto omitía el **disco EBS de la EC2** (\$1,60/mes) y el **IVA colombiano del 19 %**, que es lo que decide si `t4g.small` cabe (no cabe en on-demand).
160. **FASE 0 del checklist estaba obsoleta:** sus tres puntos ya estaban en el código, y el 0.1 pedía

instalar `django-axes` sin el extra `[ipware]` — seguirlo habría roto en silencio la detección de IP y bloqueado a los 300 usuarios al quinto fallo de login de cualquiera.

161. **FASE 5 nunca ejecutaba `createcachetable`**, pese a que el `.env` ya fijaba `CACHE_URL=db://`.

La app arrancaba, `check --deploy` pasaba, y petaba en la primera carga de Mis Turnos.

Secciones nuevas en [arquitectura-aws-rds-recomendada.md](#) (./05-referencia/deployment/99-aws/arquitectura-aws-rds-recomendada.md): **§11** (la URL, la IP y las variables paso a paso, con el orden que impone certbot) y **§12** (qué va en YAML y qué no, más el orden completo de despliegue).

Refactorización SOLID de los cuatro puntos calientes

Reparto de responsabilidades, sin cambio de comportamiento salvo donde se indica. Los cuatro sitios que la auditoría señalaba como "cada corrección futura tendrá que entrar a ciegas".

Comit	Qué cambió	Por qué
7ebea0f	El orquestador de altas deja de hablar HTTP. <code>SolicitudOrchestrator.procesar</code> devuelve <code>ResultadoSolicitud</code> (<code>solicitudes/services/resultado.py:32</code>) en vez de <code>JsonResponse</code> ; la traducción vive en un único punto, <code>json_desde_resultado</code> (<code>solicitudes/views/resultado_http.py:12</code>), llamado desde	Crear una solicitud desde un test, un comando o una tarea obligaba a fabricar un POST y leer el cuerpo con <code>json.loads</code> . El cuerpo JSON no cambia: <code>como_payload()</code> reproduce lo que emitían <code>json_ok/json_error</code>

	<code>solicitudes/views/procesar_solicitud.py:30</code>	<code>(resultado.py:81-93)</code>
708e07a	Los ocho tramos de validación salen de <code>`DobladaStrategy`</code> a <code>solicitudes/services/validators/doblada_flujo_validator.py</code> . <code>doblada_strategy.py</code> baja de 1.630 a 928 líneas	Los ocho métodos recibían <code>self</code> y ninguno lo usaba. No van a <code>doblada_validator.py</code> porque devuelven el mensaje en vez de lanzarlo, y <code>validar_pago_en_sabado</code> devuelve un <code>RequiereCambioTurnoPrevio</code> que un <code>ValidationError</code> aplanaría a texto (§ 7.2)
5857373	El selector de fin de semana pertenece a dos estrategias, no a las seis. <code>disponibilidad_companero</code> y <code>etiqueta_companero</code> salen de <code>SolicitudStrategy</code> al mixin <code>EstrategiaFinDeSemana</code> (<code>solicitudes/services/strategies/estrategia_fin_de_semana.py:47</code>)	La base tomaba partido por un subtipo: traía la regla del INTERCAMBIO por defecto, y D FDS —cuya regla es la CESIÓN— tenía que contradecir a su propia base. Cambio de comportamiento , ver la fila de <code>/solicitudes/dfds-companeros/</code> más abajo
bb7c04d	Leer el formulario se separa de orquestar el alta en DOBLADA PERMANENTE: <code>solicitudes/services/doblada_permanente_plan.py</code> parsea y valida la forma del alta multi-compañero; <code>_procesar_doblada_permanente_multi</code> baja de 236 a 150 líneas (<code>solicitud_orchestrator.py:366</code>)	—
1dc0113	La decisión por día sale de la vista de Mis Turnos a <code>turnos/services/mis_turnos_dia.py</code> ; <code>MisTurnosPorMesView.get</code> baja de 345 a 176 líneas	184 de esas 345 líneas eran el cuerpo de un solo bucle, en la pantalla que más gente mira. El contexto viaja en <code>ContextoMes</code> en vez de nueve parámetros sueltos
fffe d9e	La clasificación por empleado sale de <code>`ReporteDiaService`</code> a <code>turnos/services/reporte_dia_empleado.py</code> ; <code>reporte()</code> baja de 340 a 225 líneas	El servicio se queda con lo suyo: reunir en LOTE los datos del día para evitar las N+1. El orden de las capas es el contrato —replica el de <code>TurnoService.estado_dia</code> — y <code>turnos/tests/test_reporte_dia_paridad.py</code> detecta cualquier reordenamiento
f2af451	Se retira la capa de repositorios. Desaparecen <code>solicitudes/repositories/</code> y <code>empleados/repositories/</code> ; sobrevive <code>SolicitudCambioQuerySet.con_relaciones()</code> (<code>solicitudes/models.py:149</code>) instalada como	De sus 14 métodos solo 2 tenían llamador. Se consolida en el camino nativo de Django, el mismo que ya seguían <code>EmpleadoQuerySet</code> (<code>empleados/models.py:41</code>) y

	manager por defecto (:256)	TurnoActivoManager (turnos/models.py:41)
--	----------------------------	---

Cambios de comportamiento (no son refactor)

Commit	Qué cambió	Efecto visible
5857373	`GET /solicitudes/dfds-companeros/` devuelve 400 `tipo_sin_selector_finde` si el tipo de solicitud no es de fin de semana (solicitudes/views/api_fin_semana.py:65-69)	Antes contestaba 200 con una lista calculada con la regla equivocada: pedirle los compañeros de un sábado con el id de DOBLADA aplicaba la regla del intercambio y devolvía una lista verosímil y sin sentido, sin fallar y sin avisar. Ficha completa en § 6.2
7cdb192	El mensaje de rechazo por año operativo cambia de texto. Ya no promete que la apertura de año desbloquee la fecha; dice que se podrá a partir del 1 de enero (core/utils/anio_operativo.py:63-67)	La regla no cambió . El texto anterior mandaba al supervisor a completar el checklist esperando que le desbloqueara la fecha: la apertura <i>prepara</i> el año siguiente, no adelanta el momento de pisarlo. Quien seguía esa pista completaba los cinco ítems y se encontraba el mismo rechazo (:57-62)
d5c1f7	`PagoHorasService.deudas_pendientes` devuelve tres claves nuevas por	Aviso previo al

f	periodo: <code>fin_de_plazo</code> , <code>dias_restantes</code> y <code>urgente</code> (<code>permisos/pago_horas_service.py:184-186</code>)	vencimiento, no solo el aviso de que ya cerró. <code>dias_restantes</code> vale 0 el último día hábil del plazo —hoy todavía se puede pagar— y es negativo si el mes ya venció; <code>urgente</code> marca la última semana (:90-91)
311d281	<code>`Turno.sala` y `TurnoArchivo.sala` admiten NULL</code> (migración <code>turnos/migrations/0016_alter_turno_sala_alter_turno_archivo_sala.py</code>)	La interfaz muestra "Por asignar" cuando falta. Antes, un explorador sin competencia cargada hacía fallar la aprobación de la solicitud por un dato que no condiciona la operación. Detalle en § 5.3
48f7ae2	Nuevo <code>`python manage.py verificar_crons`</code>	Comprueba desde fuera que los dos crons corren; marcador <code>CRON_NO_EJECUTADO</code> y salida distinta de cero. Ver § 7.3 y § 13.6
9627d7c, 02e420e	Nuevas banderas <code>`pytest --dias-en-el-futuro=N`</code> y <code>`--anio-planificado`</code>	Detectan los tests que caducan solos. Ver § 14.1

El comentario pasa a ser obligatorio en toda decisión

Aprobar, rechazar y cancelar movían turnos reales sin dejar una sola línea escrita: el campo existía en los modelos y estaba en blanco casi siempre. Ahora se exige, y se exige **en el servidor** — borrar el `required` del HTML no sirve de nada. Suite completa en verde: **965 tests**.

Una segunda pasada consolidó lo que la primera dejó repartido: los helpers, los textos y la convención de etiqueta. Las tablas de abajo describen ya el estado final.

Qué cambió	Dónde	Notas
Módulo nuevo <code>core/utils/comentarios.py</code> : textos, código de error y dos helpers	<code>MSG_* (:24-27)</code> , <code>CODE_COMENTARIO_REQUERIDO (:31)</code> , <code>leer_texto (:34)</code> , <code>exigir_texto_json (:44)</code>	Nombres distintos porque los contratos lo son: <code>leer_texto</code> devuelve <code>`str`</code>
Aplicado a las siete vistas de decisión, vía <code>exigir_texto_json</code>	<code>AprobarSolicitudView (:31)</code> , <code>AprobarSolicitudReceptorView (:41)</code> , <code>RechazarSolicitudView (:51)</code> , <code>RechazarSolicitudReceptorView (:60)</code> y <code>AprobarSolicitudAmbosView (:192)</code> con <code>comentario_respuesta</code> ; <code>CancelarSolicitudView</code> con <code>motivo (:83)</code> ; <code>ResponderCancelacionView</code> con <code>comentario_respuesta (:140)</code>	Fichas en § 6.2
Mismo criterio en permisos, con <code>leer_texto + _fallo</code>	<code>permisos/views.py:322</code> (aprobar/rechazar), <code>:437</code> (cancelar o pedir la cancelación; el motivo se valida al entrar al <code>post</code> y <code>_pedir_cancelacion</code> lo recibe por parámetro, <code>:457,465</code>), <code>:571</code> (responder)	Rechazar una cancelación concatena ahora el motivo a <code>`comentario_supervisor`</code> , que antes se descartaba
Códigos HTTP en permisos	<code>_quiere_json</code> (<code>permisos/views.py:289</code>) y <code>_fallo (:297)</code>	Quien pide JSON (<code>X-Requested-With, Accept</code>) recibe <code>403 forbidden</code> , <code>400 comentario_requerido</code> , <code>409 invalid_state</code> o <code>409 plazo_vencido</code> ; el navegador conserva <code>messages + 302</code> . Antes todo

		desenlace era el mismo 302 y un cliente no distinguía éxito de rechazo
Los textos llegan a las plantillas desde el servidor	Context processor <code>core.context_processors.mensajes_comentario (:21)</code> , registrado en <code>config/settings.py:106</code>	Expone MSG_COMENTARIO y MSG_MOTIVO a todas las plantillas, así nadie tiene que acordarse al añadir una pantalla. Los literales estuvieron en tres sitios a la vez (constantes, vistas de permisos y atributos del HTML) y se desincronizaban
motivo obligatorio al registrar una inasistencia de doblada	<code>solicitudes/views/reprogramacion_views.py:114,121-122</code>	Desaparece el motivo or None : la notificación al explorador siempre lleva el motivo entre paréntesis (:140)
Nota del PDH obligatoria	<code>empleados/forms.py:380,382-385</code> (<code>required = True + clean_comentario</code>), <code>empleados/views/pdh.py:91,155</code>	En la vista se valida después de fecha y explorador, a propósito, para no tapar esos mensajes
Helper de front común <code>static/js/core/comentario_obligatorio.js</code>	<code>window.ComentarioObligatorio.swal ({...})</code> ; lo usan los cuatro Swal de solicitudes (<code>mis_solicitudes_list.js:514, :608</code> ; <code>solicitudes_pendientes_list.js:153, :200</code>) y el interceptor de permisos (:36)	Opciones propias etiqueta, error, marcador ; el resto pasa a SweetAlert. En <code>didOpen</code> pinta el asterisco rojo sobre <code>.swal2-input-label</code> acotado a <code>Swal.getPopup()</code> . Los textos vienen de <code>window.MSG_COMENTARIOS</code> , que sirve

		la plantilla
Front: required en las plantillas	templates/solicitudes/solicitudes_pendientes_list.html:253, templates/empleados/pdh_create.html:79, templates/solicitudes/reprogramacion_registrar.html:49	Comodidad, no defensa: la que cuenta es la del servidor
Archivo nuevo static/js/permisos/comentario_obligatorio.js	Cargado en templates/permisos/permisos_especiales_list.html:177, después del helper común (:176)	Sustituye a los <code>onsubmit="return confirm(...)"</code> de la lista de permisos. Ya no construye su propio Swal: delega en <code>ComentarioObligatorio.swal</code>
Una sola convención de etiqueta	Plantillas y Swal	El asterisco rojo <code>*</code> es la marca de campo obligatorio de la aplicación (~40 usos). Los Swal decían <code>(obligatorio)</code> porque <code>inputLabel</code> se inserta como texto plano; ahora el helper lo pinta a mano y <code>data-comentario-label</code> ya no lo lleva

Contrato del interceptor de permisos. `comentario_obligatorio.js` engancha el `submit` de todo `<form data-comentario-label>`, pide el texto con `Swal` (con *fallback* a `window.prompt` si `Swal` no está), lo inyecta como `<input type="hidden">` y reenvía el formulario marcándolo con `data-comentario-listo="1"` para no volver a preguntar. Los cinco atributos:

Atributo	Obligatorio	Para qué	Por defecto

data-comentario-label	Sí	Activa el interceptor y titula el campo de texto	—
data-comentario-campo	No	Nombre del input oculto que se envía; debe coincidir con el que lee la vista (comentario o motivo)	comentario
data-comentario-titulo	No	Título del diálogo	¿Confirmas la acción?
data-comentario-texto	No	Cuerpo HTML del diálogo: qué va a pasar si confirma	vacío
data-comentario-error	No	Texto del error cuando el campo llega vacío. Lo pone la plantilla desde el contexto (<code>{{ MSG_COMENTARIO }}</code> / <code>{{ MSG_MOTIVO }}</code>), así repite literalmente el del servidor: dos redacciones para la misma falta hacen dudar de si el problema es otro (<code>static/js/permisos/comentario_obligatorio.js:21</code>)	ComentarioObligatorio. RESPALDO , genérico a propósito para no recrear una segunda copia del texto bueno

Ejemplos de uso en `templates/permisos/permisos_especiales_list.html:102-104` (aprobar), `:109-111` (rechazar), `:124-133` (responder a una cancelación) y `:146-148` (cancelar o pedir la cancelación, con `data-comentario-campo="motivo"`).

Bug corregido de paso. El modal de aprobación enviaba el campo como **comentario** y el backend lee `comentario_respuesta`: el comentario del supervisor nunca llegaba a guardarse. Ver § 16.4, fila B14.

Tests que sujetan la regla. `solicitudes/tests/test_cancelacion_lifo.py:167` y `:185` (pedir y responder sin texto → 400 **comentario_requerido**, sin que se abra ni se resuelva nada); `permisos/tests/test_cancelacion_permiso_consenso.py:94` (**el literal renderizado en la plantilla es exactamente el de `core.utils.comentarios`** — la garantía de que las copias no vuelvan a divergir), `:117` (cliente JSON → 400 con **code**) y `:134` (navegador → 302 con su mensaje).

Excepción conocida. La aprobación por enlace de correo (`solicitudes/views/aprobacion_email.py`) es un GET de un clic y no exige comentario: no hay formulario donde escribirlo.

Tests. Cinco archivos existentes se actualizaron para incluir el campo en sus POST (`solicitudes/tests/test_cancelacion_integridad.py`, `test_cambio_descanso.py`, `test_cambio_descanso_revert_semana.py`, `test_cambio_turno_revert.py`,

permisos/tests/test_media_jornada_revert.py) y se añadieron tres que fijan el rechazo en servidor: solicitudes/tests/test_cancelacion_lifo.py:167 y :185, y permisos/tests/test_cancelacion_permiso_consenso.py:94.

Retirado el bloqueo temporal del día para un nuevo cambio de descanso

Cierre del hilo abierto por la cancelación consensuada. Implementado y probado: **959 tests** en verde. Razonado en el [ADR 010](#) (./03-arquitectura/adr/010-dia-de-descanso-libre-tras-el-intercambio.md).

Qué cambió	Detalle	Riesgo / contrapartida
Regla. dia_bloqueado_para_nuevo_cambio se reduce a una: turno ese día con tipo_cambio distinto de CAMBIO DESCANSO → bloqueado, siempre. Cualquier otra cosa → libre	Desaparecen las dos ramas que miraban la ventana (la del lado que trabaja, recorriendo solicitudes; y la del lado que descansa, vía <code>_mapa_descanso(dentro_ventana=True)</code>). La función pasa de ~50 líneas a un único <code>Turno.objects...exists()</code> (<code>solicitudes/services/cambio_descanso_aplicacion_service.py:243-247</code>)	Reutilizar el día mientras la cancelación anterior sigue viva deja al LIFO impidiendo revertirla. . Asumido y escrito en el docstring (<code>:201-224</code>). Zona frágil: § 16.2
Eliminada `VENTANA_CANCELACION_MINUTOS` `de los dos sitios	Del servicio de cambio de descanso (sostenía el bloqueo) y de <code>use_cases/cancelar_solicitud.py</code> (código muerto, sin ningún uso). La constante ya no existe en el proyecto: B13 cerrado por eliminación (§ 16.4)	—
Parámetro `dentro_ventana` retirado de _mapa_descanso y _mapa_descanso_multi (firmas y filtro)	Solo existía para ese cálculo (<code>:76, :90</code>)	—

<code>`excluir_id`</code> se conserva en <code>dia_bloqueado_para_nuevo_cambio</code> por compatibilidad con las llamadas existentes	Con la regla actual no hace nada : el criterio mira turnos, no solicitudes. Documentado así en el docstring	Parece un filtro activo y no lo es; no lo uses como si excluir a algo
Rastro actualizado. Un solo mensaje de bloqueo en <code>cambio_descanso_strategy.py:119-128</code> (el segundo era inalcanzable) y docstring de <code>_trabaja_dia</code> corregido (<code>:91</code>); comentario del selector de fines en <code>solicitudes/views/api_fin_semana.py:326-330</code>	Texto literal del mensaje que queda: "Ese día ya tiene un cambio aplicado (TIPOS). Para usarlo en un intercambio, primero hay que deshacer ese cambio: pídeselo a tu compañero si aún está en plazo, o a tu supervisor."	El frontend no debe mapear este texto a códigos
Tests. <code>CDVentanaCancelacionTest</code> → <code>CDReintercambioDiaTest</code> (<code>solicitudes/tests/test_cambio_descanso.py:254</code>) y ahora afirma lo contrario: el día queda libre desde el primer minuto	El test directo (<code>:312</code>) comprueba los dos lados del intercambio justo después de aplicar, para que falle si alguien reintroduce el bloqueo	Es el centinela de la decisión: no lo relajes

La cancelación de una aprobada deja de ser unilateral

Cambio de **regla de negocio**, no de implementación: es el más profundo desde "la última aprobada gana por día". Implementado, migrado y con la suite en verde (**959 tests**).

Qué cambió	Detalle	Riesgo / contrapartida
Regla. Cancelar una solicitud aprobada ya no la cancela: la pide el solicitante y el receptor la aprueba o la rechaza	<code>CancelarSolicitudUseCase.execute</code> bifurca por estado (<code>solicitudes/use_cases/cancelar_solicitud.py:87,96</code>): <code>pendiente</code> → retirada inmediata; <code>aprobada</code> → <code>_pedir_cancelacion (:232)</code> , que no cancela nada . La reversión vive ahora solo en <code>responder_cancelacion (:292)</code> . <code>execute_supervisor</code>	Un flujo de un paso pasa a ser de dos, con dos actores y dos plazos. El frontend debe leer cancelada en la respuesta: un 200 ya no significa cancelada

	(:105) no cambió: Gestión sigue cancelando directo	
Vocabulario. EstadoCancelacion en core/constants.py:167 , con NINGUNA=' ' , PENDIENTE , APROBADA , RECHAZADA , CADUCADA , sus CHOICES y TERMINALES (:154)	Los tres terminales son definitivos: no se vuelve a pendiente . Ventanas: VENTANA_PEDIR_CANCELACION_HORAS = 24 desde fecha_resolucion (:160) y VENTANA_RESPONDER_CANCELACION_HORAS = 24 desde cancelacion_solicitada_en (:165)	Antes eran 30 minutos y la cancelación era inmediata; con una respuesta de por medio, 30 minutos no alcanzan para que nadie conteste
Modelos. Seis campos nuevos, idénticos en SolicitudCambio (solicitudes/models.py:166-195) y PermisoEspecial (permisos/models.py:134-149)	cancelacion_estado , cancelacion_solicitada_por/_en , cancelacion_respondida_por/_en , cancelacion_motivo . Migraciones solicitudes/0034 y permisos/0008 , ya aplicadas, aditivas y también sobre las tablas históricas. Después, permisos/0009 añadió PermisoEspecial.fecha_aprobacion (misma forma: aditiva, histórica, depende de la 0008)	Dos modelos con el mismo bloque de campos y sin abstracción común: cualquier cambio hay que hacerlo dos veces
Invariante. Mientras la petición está pendiente, estado sigue siendo ' aprobada ' a propósito	Los turnos siguen aplicados y todas las consultas que filtran por ' aprobada ' —guardia LIFO, Mis Turnos, reconciliación— la ven vigente (solicitudes/models.py:166-170)	Es el punto que más fácil se "arregla" por error. Ver § 16.2
Las guardias corren dos veces. LIFO, integridad y fechas_ya_cumplidas no cambiaron; cambió cuándo se ejecutan	Al pedir (:272), para no molestar al receptor con algo ya irreversible; y al aprobar (:359), porque pueden haber pasado 24 h	El receptor puede recibir un error causado por un tercero que tocó esos días mientras él decidía
Caducidad perezosa. _caducar_si_vencida (:395) marca CADUCADA al leer o	Una petición vencida no tiene efecto pendiente que aplicar: el cambio sigue vigente, basta con	Sin cron ni tarea: si nadie la mira, se queda en pendiente en base de datos aunque a efectos de

responder, no un proceso de fondo	reconocerla cuando alguien la mira. _mensaje_cancelacion_cerrada (:415) explica por qué no se puede reintentar	negocio esté vencida
Endpoints. Nuevo POST /solicitudes/cancelar-solicitud/<id>/responder/ (ResponderCancelacionView, solicitudes/views/aprobacion_views.py:120, solicitudes/urls.py:102)	El de cancelar acepta ahora motivo y devuelve {'cancelada': bool, 'cancelacion_pendiente': bool}; códigos nuevos cancelacion_ya_pedida y cancelacion_cerrada, y ventana_expirada pasa a dispararse por el plazo de 24 h. El nuevo usa forbidden, cambio_mas_reciente, conflicto_integridad, plazo_vencido, invalid_state. Fichas completas en § 6.2	El mapeo de código de error sigue haciéndose por subcadena del mensaje
Permisos. Nuevo POST /permisos/permisos-especiales/media-jornada/<pk>/cancelar/responder/ (PermisoMediaJornadaCancelResponderView, permisos/views.py:582) y la vista de cancelar bifurca dueño / supervisor (:424)	La contraparte es el supervisor: el permiso no tiene receptor, y es quien lo aprobó y responde por la cobertura del día	Los tres defectos que quedaron abiertos en esta ruta (B9, B10, B11) ya están cerrados: § 16.4
Permisos, segunda pasada (cierre de B9-B12). Campo PermisoEspecial.fecha_aprobacion + migración permisos/0009; if es_dueno en vez de if es_dueno and not es_sup (permisos/views.py:424); helpers _supervisor_del_permiso (:561) y _puede_responder_cancelacion (:569); select_related en la vista de responder (:515);	El plazo cuelga de una fecha propia, con el respaldo permanente fecha_aprobacion or actualizado_en (:443); sobre el permiso propio siempre se PIDE, tenga quien pulse el rol que tenga; responde el supervisor del permiso, salvo que no haya ninguno resoluble. La plantilla distingue "mi permiso" del "de otro" por empleado, no por rol	Cinco tests nuevos en permisos/tests/test_cancelacion_permiso_consenso.py. Suite en verde: 959 tests. En esa tanda el bloqueo de 30 min de dia_bloqueado_para_nuevo_cambio no cambió; se retiró después (ver la entrada siguiente y el ADR 010)

<code>mi_empleado_id</code> en el contexto de <code>PermisoEspecialListView (:153)</code> ; y los dos mensajes de <code>cambio_descanso_strategy.py:124-128</code> reescritos		
Notificaciones. <code>crear_notificacion_peticion_cancelacion</code> (<code>notificacion_service.py:631</code>) y <code>crear_notificacion_respuesta_cancelacion</code> (:661)	Sin correo: solo notificación en la aplicación. En permisos hay dos equivalentes a nivel de módulo dentro de <code>permisos/views.py</code> (:607, :630)	La lógica de aviso vive en dos sitios distintos según la app
Listas. <code>SolicitudesPendientesListView.get_context_data</code> añade <code>cancelaciones_pendientes</code> (<code>solicitudes/views/notificaciones_listas.py:170,173</code>)	Consulta aparte , porque esas solicitudes están aprobada y no pendiente . Excluye las vencidas por fecha en vez de marcarlas	Sin esta lista la petición no tendría dónde verse y caducaría siempre
Tests. Helper compartido <code>solicitudes/tests/helpers_cancelacion.py</code> (<code>cancelar_con_acuerdo</code>), seis pruebas nuevas del acuerdo en <code>test_cancelacion_lifo.py</code> (:148-:243) y cinco de permisos en <code>permisos/tests/test_cancelacion_permiso_consenso.py</code>	El helper evita que cada test que solo quiere llegar a cancelada se acople a la forma del ciclo de dos pasos	Suite en verde: 959 tests

Aviso al elegir un día de pago que ya se trabaja AM+PM

Commit [e93ce3b](#) (*fix(cobertura): avisar cuando el día de pago ya se trabaja AM+PM*).

Cambio	Impacto
El aviso de "ese día ya doblas, no te queda jornada con la que pagar" se adelanta a la elección del día	<code>_validar_semana_cobertura</code> ya lo rechazaba, pero el usuario se enteraba al enviar, con el formulario entero relleno
La sincronización del select compartido de compañero se limita a	En cobertura el compañero se

los sub-tipos que lo usan (intercambio_dia y jornadas_partidas)	elige en cob-select-1 y el compartido está oculto y vacío: sincronizar contra él borraba la selección buena, y el formulario pedía un compañero que ya estaba elegido
Tests en solicitudes/tests/test_cambio_doblada_candidatos.py	Regresión de ambas mitades

El día de pago de la cobertura con 2 compañeros debe estar libre

Continuación directa del cambio anterior, sobre el mismo formulario. Regla completa en § 8.3.

El caso que lo destapó: cobertura de **día completo (2 compañeros)** con el día de pago puesto en un día donde el solicitante ya trabajaba la AM. El backend lo rechazaba —bien— pero por la jornada que chocaba, y el mensaje mandaba a hacer un **Cambio de Turno sencillo**. Ese consejo no lleva a ninguna parte: las dos solicitudes comparten **fecha_pago**, así que ese día se pagan las **dos** medias jornadas, y girar la jornada propia solo traslada el choque al otro compañero. Si además el día fuera de mantenimiento, ahí no se puede pedir un CT siquiera.

Cambio	Impacto
SolicitudOrchestrator._pago_dos_debe_estar_libre , llamado al inicio de _procesar_cobertura_dos	Guard de pareja: mira una condición que ninguna de las dos solicitudes ve por separado, así que no cabía en _validar_semana_cobertura . Corta antes de validar ninguna y da el motivo real más las dos salidas que existen (día libre, o solo AM / solo PM)
Recuadro rojo bloqueante cob-aviso-dos-libre + error al enviar, sobre mi_jornada_pago (dato que el endpoint de candidatos ya devolvía)	El usuario lo ve al elegir el día, no tras rellenar el formulario entero. Mismo criterio que el commit e93ce3b de arriba
En modo DOS se suprime el aviso «Ir a Cambio de Turno» (salida temprana de verificarPagoCobertura)	Sigue vivo en <i>Solo AM / Solo PM</i> , donde sí resuelve. Retirarlo del modo DOS es la mitad del arreglo: el consejo equivocado era el daño
El desplegable de día de pago deshabilita los días de mantenimiento, con el motivo a la vista	Ese día no opera nadie: no hay jornada del compañero que cubrir. Antes se podían elegir y el envío moría más adelante sin explicar por qué
solicitudes/tests/test_cobertura_dos_companeros.py : dos casos nuevos y uno reencuadrado	Uno de ellos fija que el mensaje no mande al Cambio de Turno. test_si_falla_una_jornada_no_se_crea_ninguna se reescribió para seguir probando

	la atomicidad por jornada: su escenario antiguo lo intercepta ahora el guard nuevo
--	--

Páginas de error propias y trazabilidad de la petición

Commit [65c08ac](#) (*feat(errores): paginas de error propias y codigo de referencia por peticion*), rama [fix/cambio-descanso-temporada](#). Detalle completo en § 9.5 y [ADR 007](#) (./03-arquitectura/adr/007-paginas-de-error-propias-y-request-id.md).

Cambio	Impacto
Nuevo <code>core/errors.py</code>: <code>RequestIDMiddleware</code> , <code>RequestIDFilter</code> , los cuatro handlers, <code>csrf_failure</code> y <code>previsualizar_error</code>	Con <code>DEBUG=True</code> cualquier URL inexistente devolvía el URLconf completo, con todos los endpoints de aprobación y rechazo (CWE-215)
Nuevas plantillas <code>templates/errors/_base_error.html</code> y <code>templates/{400,403,403_csrf,404,500}.html</code>	Autocontenidas: no heredan de <code>base.html</code> ni cargan recursos externos, así que funcionan con la base o los estáticos caídos
<code>handler400/403/404/500</code> en <code>config/urls.py</code> ; ruta de previsualización <code>/__error__/<codigo>/</code> solo bajo <code>DEBUG</code>	Las páginas se pueden revisar en desarrollo, donde Django nunca llega a usar los handlers
<code>core.errors.RequestIDMiddleware</code> como primer middleware y <code>CSRF_FAILURE_VIEW</code> propio	Cada petición lleva código opaco en <code>X-Request-ID</code> ; el motivo exacto del fallo CSRF deja de mostrarse al cliente
LOGGING reescrito: filtro <code>request_id</code> en ambos formatters, loggers <code>django.security</code> y <code>core.errors</code> , y solo stdout en producción (el fichero quedó como comodidad de desarrollo)	El detalle del error va al equipo, no al usuario. Ver § 13.5
Se cierra la misma fuga en la página de los enlaces de correo: los cuatro <code>except Exception</code> de <code>views/aprobacion_email.py</code> dejan de imprimir <code>str(e)</code> , y esas vistas pasan de responder siempre 200 a 403/409/500	Era CWE-209 en una página visible sin sesión iniciada, y el 200 la hacía invisible para las alarmas. Nuevo <code>core/tests/test_error_token.py</code> (8 tests)
Se cierra el tercer frente, las APIs JSON: nuevo helper <code>json_error_inesperado</code> en <code>core/utils/json_responses.py:68</code> y nueve <code>except Exception</code> corregidos en <code>dias_especiales.py</code> , <code>calculo_automatico.py</code> , <code>turnos_mes.py</code> , <code>api_turno_jornada.py</code> y <code>gestion_solicitudes.py</code>	<code>str(e)</code> de MySQL llegaba en el JSON: nombres de tabla y columna, y en el peor caso el endpoint de RDS. Seis de los ocho no dejaban traza (tres no registraban nada y tres usaban <code>logger.error</code> sin <code>exc_info</code>), así que el cambio además añade observabilidad. Se conserva un mensaje especifico por endpoint en vez de un genérico único. Nuevo <code>core/tests/test_json_error_inesperado</code>

	.py (16 tests). La novena era la peor: <code>MisTurnosPorMesView</code> publicaba el traceback entero a cualquier <code>is_staff</code> , y la primera versión del test —que comparaba línea a línea— no la detectaba. Ver § 9.5
Nuevo <code>core/tests/test_paginas_error.py</code> (19 tests) y <code>logs/</code> en <code>.gitignore</code>	Regresión sobre las fugas, el código de referencia y la autonomía de las plantillas. Los dos últimos se añadieron después, al descubrir que los <code>.catch()</code> de los formularios lanzarían <code>ReferenceError</code> si el stub de <code>CodigoReferencia</code> no estuviera
Nuevo <code>static/js/utils/codigo-referencia.js</code> , cargado en <code>templates/base.html:378</code> antes que jQuery, más el CSS de <code>static/css/base_custom.css:23-36</code> y ocho avisos de envío fallido en los seis <code>static/js/cambio-turno/solicitar_*.js</code>	Los formularios envían por <code>fetch</code> y no recargan: ante un 500 no pasaban por ninguna plantilla de error y el código se perdía pese a venir en <code>X-Request-ID</code> . Un shim sobre <code>window.fetch</code> lo recoge solo en respuestas con error y lo mantiene 60 s. Ver § 9.5

Seguridad de los enlaces de aprobación por correo

Commit [425ab8e](#) (*feat(seguridad): firmar los enlaces de aprobacion por correo*), rama `fix/cambio-descanso-temporada`.

Cambio	Impacto
Nuevo <code>solicitudes/services/tokens_aprobacion.py</code> : fuente única de los tokens firmados, con <code>django.core.signing</code> sobre <code>SECRET_KEY</code> en lugar de una clave literal	Cierra B1 y B2 (§ 16.4). Sal propia por circuito: <code>solicitudes.aprobacion-email (:44)</code> y <code>permisos.aprobacion-email (:116)</code>
Se eliminan las seis copias duplicadas de la verificación: <code>services/email_service.py:528</code> , las cuatro de <code>views/aprobacion_email.py:140,199,251,303</code> y <code>permisos/services.py:34</code>	Un solo sitio que auditar. <code>permisos/</code> queda cubierto por primera vez
Nueva variable <code>APPROVAL_LINK_MAX_AGE_DAYS</code> (default 30) en <code>config/settings.py:262</code> y <code>.env.example:43</code>	Los enlaces caducan. Ver § 10
Nuevo <code>solicitudes/tests/test_tokens_aprobacion.py</code> (25 tests)	Firma, caducidad, sal, rol y suplantación
Condición operativa nueva: misma <code>'SECRET_KEY'</code> en todas las instancias	Añadida al checklist de despliegue (§ 13.3)

Generalizado como patrón 38 de PROTECTION_PATTERNS.md y razonado en el ADR 006 (./03-arquitectura/adr/006-tokens-firmados-para-aprobacion-por-correo.md)	El ADR recoge las alternativas descartadas (HMAC propio, tabla de tokens gastados, caché) y por qué la solución <i>stateless</i> es la que encaja con varias instancias tras un ALB
Corregidas 31 secuencias de mojibake en 5 archivos de solicitudes/views/	Cierra B6 (§ 16.4): el usuario ya no lee "Token inválido" en pantalla
Nuevo core/utils/error_token.py y plantilla error_token.html con el plazo y qué hacer	Cierra B7 (§ 16.4). Las 15 llamadas pasan por un punto único
Los seis print(...) de services/email_service.py pasan a logger.exception	Un print no lleva nivel ni traza: ese texto no llegaba útil a CloudWatch. Ahora sí, con la traza de la excepción

Temporada y cambio de descanso (rama fix/cambio-descanso-temporada)

Com mit	Cambio	Impacto
4c9 0e4 7	El filtro de compañeros de DOBLADA consulta el rol correcto. DobladaFiltroService.filtrar_empleados_sin_doblada_activa excluía por explorador_solicitante_id ; ahora excluye por explorador_receptor_id , con la variable renombrada a empleados_con_doblada_receptor (solicitudes/services/doblada_filtro_service.py:60-66). La regla no cambia —«no se puede pedir doblada a quien ya trabaja AM+PM ese día»—, cambia el rol que se pregunta: quien se dobla es el receptor ; el solicitante cede su jornada y descansa . Docstring reescrito (:34-38) y comentario «OJO CON EL ROL» junto a la consulta (:56-59); las dos fuentes de exclusión —solicitudes aprobadas y AM+PM reales en Turno (:94-98)— se siguen uniendo igual (:103-105)	El desplegable escondía justo a la gente libre ese día, incluida la que descansaba porque te había cedido a TI, y en cambio ofrecía a quien ya estaba doblado: el backend la tumbaba al enviar (patrón 37). Sentinelas nuevo solicitudes/tests/test_doblada_filtro_rol.py con 3 casos: el cedente sigue disponible (:51), el receptor queda excluido (:58) y la fuente Turno sigue vigente (:65). Ver § 7.1, § 8.3 y § 16.2
2a8 e90 2	El pago en sábado exige que el receptor trabaje ese día. Guard nuevo al principio de _aplicar_pago_sabado , antes de escribir (solicitudes/services/doblada_pago_service.py:139-151), con la excepción de que la mitad contraria ya esté cubierta por otra doblada aprobada (:99-123)	Cierra la vía por la que la aprobación por enlace de correo y la re-validación al aprobar creaban un turno en un día de descanso del receptor. Nueva TestPagoSabadoReceptorDescansa (solicitudes/tests/test_matriz_dobladas.py:2635); suite en verde, 936

		tests
2a8 e90 2	La mitad contraria del sábado la puede cubrir cualquier deudor, no solo el mismo. Las dos consultas que expresaban el criterio por separado (excepción del guard y rama <code>_contraria_tambien_cubierta</code>) se unifican en <code>DobladaPagoService._mitad_contraria_cubierta_por_otra_doblada</code> (<code>solicitudes/services/doblada_pago_service.py:99-123</code>), que deja de filtrar por <code>`explorador_solicitante`</code> . Se reformula el <code>ValidationError</code> del guard: describe el problema en vez de ordenar "elige otra fecha", porque también lo lee el aprobador (<code>:146-152</code>)	Corrige un bug preexistente de doble cobertura —el reparto recreaba al acreedor la mitad que otro deudor ya cubría— y otro del guard —rechazaba re-aplicaciones legítimas (<code>reconciliar_dobladas_aprobadas</code> , re-validación al aprobar, enlace de correo) cuando el acreedor ya se había quedado sin turnos—. Nuevos tests en <code>TestPagoSabadoReceptorDescansa:solicitudes/tests/test_matriz_dobladas.py:2759</code> y <code>:2795</code> ; suite en verde, 938 tests
93f fbb 0	El mensaje de la re-validación al aprobar se reencuadra en la costura. <code>SolicitudAprobacionService</code> ya no devuelve <code>f"No se puede aprobar: {msg_reval}"</code> : cita el motivo como tal, advierte de que las instrucciones son para quien envió y dice al aprobador qué puede hacer (<code>solicitudes/services/solicitud_aprobacion_service.py:130-134</code>)	Los ~13 mensajes de DOBLADA están en imperativo para quien envía y al aprobar los lee el aprobador, que no puede ejecutarlos. No se reescribieron : son literales documentados, hay tests que verifican subcadenas y <code>CancelarSolicitudView</code> decide el HTTP inspeccionando texto. Ver § 7.4
2a8 e90 2	<code>`otra_mitad_pagada`</code> queda marcada como deliberadamente NO unificada con <code>_mitad_contraria_cubierta_por_otra_doblada</code> , con el porqué en el código (<code>doblada_pago_service.py:195-199</code>)	Se parecen y preguntan lo contrario: deudor vs. acreedor. Sin la nota, el próximo refactor las funde y rompe una de las dos. Ver § 16.2
5e0 531 2	Patrón 39 registrado en <code>PROTECTION_PATTERNS.md:1651</code> ("la guardia vive donde está la ESCRITURA, no solo en la estrategia"), con cinco reglas y cuatro filas de registro de cambios	Cierra la fila 15 de § 18. Su regla 4 — <code>create()</code> dentro de <code>if not ...exists()</code> es señal de alarma— generaliza el bug del reparto de sábado. La limitación que registraba —"solo se auditó la rama de sábado"— quedó cerrada por la auditoría posterior de esta misma rama (§ 16.2)
2a8 e90 2	Auditadas las otras cinco ramas de <code>`aplicar_doblada_pago`</code> (cierra la fila 16 de § 18). Resultado rama por rama: <code>_aplicar_pago_jcp_ambas</code> limpia en este eje (borra los turnos del acreedor y solo crea para el deudor, <code>doblada_pago_service.py:257-270</code>); <code>_aplicar_pago_jcp_media</code> (<code>:321-329</code>) y	Auditoría. La conclusión de entonces — "ninguna tiene guardia propia y son inalcanzables gracias al LIFO, así que no se añaden guards" — quedó revisada en la entrada siguiente : el LIFO tapa la reconciliación, no los comandos de gestión

	<code>_aplicar_pago_jornada_cedida (:509-519)</code> tienen la misma forma que el bug de sábado; <code>_aplicar_pago_cesion_parcial (:364)</code> y <code>_aplicar_pago_fallback (:540-554)</code> no fabrican turno pero validan con la jornada base en vez de con "¿trabaja?". Sin cambio de comportamiento en esa tanda: solo notas en el código	
2a8 e90 2	Nueva <code>`TestLifoProtegeLaReconciliacion`</code> (<code>solicitudes/tests/test_matriz_doblada s.py:2864</code>): <code>test_no_se_puede_cancelar_por_debajo_de una_doblada_que_paga_ese_dia (:2902)</code> y su contraprueba sin solape de día (<code>:2931</code>)	Fija por test la barrera del LIFO. Ya no es la única: dos de esas cuatro ramas llevan guard propio desde la entrada de abajo, y el LIFO nunca cubrió los comandos de gestión. Suite en verde entonces, 941 tests
9dc 4af 1	Matizado el encuadre del guard de sábado (§ 8.3): en la web es defensa en profundidad —creación y aprobación re-validan, y la reconciliación la tapa el LIFO—, pero fuera de la web es el único guard: los comandos de gestión entran sin validar	El bug real y demostrado de esa tanda es el otro: la doble cobertura cuando dos deudores cubren cada mitad del sábado, alcanzable por aprobación normal sin reconciliación
2a8 e90 2	Guard del patrón 39 en DOS ramas de pago entre semana — las únicas donde es demostrablemente seguro. <code>_aplicar_pago_jcp_media</code>: guard al inicio, antes de escribir (<code>solicitudes/services/doblada_pago_service.py:302-309</code>). <code>_aplicar_pago_jornada_cedida</code>: guard dentro de su rama parcial (el <code>else</code> de <code>tipo_cesion == 'cesion_completa', :536-543</code>), no al inicio del método. Ambos con <code>BaseValidator._explorador_trabaja(receptor, fecha_pago)</code> y <code>ValidationError</code> con el texto "...no trabaja el DD/MM/AAAA: ese día descansa, así que no tiene una jornada que cubrirle para pagar la doblada. La doblada debe rehacerse con otra fecha de pago."	Cambio de comportamiento real, no comentarios. Motivo: los comandos <code>reaplicar_doblada</code> (<code>solicitudes/management/commands /reaplicar_doblada.py:259</code>) y <code>corregir_doblada_cesion_total (:73)</code> llaman a <code>aplicar_doblada_pago sin validar y sin LIFO</code>, y no son scripts olvidados: <code>verificar_doblada.py:384</code> sugiere el de reaplicación al detectar errores y dos guías de operación lo indican. Es el momento de más riesgo: el día ya está descuadrado. Corrige la afirmación anterior: el LIFO tapa la reconciliación, NO los comandos. Cierre PARCIAL: de las cuatro ramas expuestas, dos cubiertas y dos siguen expuestas
9dc 4af 1	Por qué solo dos ramas y no cuatro (hallazgo de diseño, documentado en § 16.1 y § 16.2). <code>_aplicar_pago_cesion_parcial</code>, <code>_aplicar_pago_fallback</code> y <code>_aplicar_pago_jornada_cedida</code> en <code>cesion_completa</code> dejan al acreedor sin turnos por diseño (<code>doblada_pago_service.py:425-426, :521, :634</code>). Como	Una comprobación sobre el estado POSTERIOR no distingue "descansa porque yo lo dejé así" de "descansa porque no le tocaba". Las dos ramas guardadas lo son porque conservan una jornada al acreedor. Queda en § 18, fila 20

	<code>TurnoService.estado_dia</code> incluye el descanso por solicitud aprobada y no admite excluir la solicitud en curso, al re-aplicar la misma solicitud el guard bloquearía su propia re-aplicación. La solución correcta —comparar contra <code>snapshot_turnos_previos</code>— no se ha hecho: es de más calado y está sin decidir	
2a8 e90 2	Nueva <code>TestGuardPagoEntreSemana</code> (<code>solicitudes/tests/test_matriz_doblada.py:2948</code>): rechazo en <code>jcp_media (:2988)</code> y en <code>jornada_cedida</code> parcial (<code>:3030</code>), más la regresión <code>test_jcp_media_reaplicar_no_se_autoengana (:3005)</code>	La regresión es la clave: demuestra que re-aplicar no se bloquea a sí mismo. Se verificó que los dos de rechazo tienen dientes (neutralizando los guards, fallan). Suite en verde, 944 tests
9dc 4af 1	Hallazgo menor registrado: la rama guardada de <code>_aplicar_pago_jornada_cedida</code> solo se alcanza con <code>tipo_cesion vacío</code> — el despachador manda <code>cesion_parcial_am/pm</code> a <code>_aplicar_pago_cesion_parcial</code> (<code>doblada_pago_service.py:83-84</code>) y <code>cesion_completa</code> va por la rama que borra los turnos (<code>:520-521</code>). <code>TIPO_CESION_CHOICES</code> (<code>solicitudes/models.py:360-364</code>) solo tiene esos tres valores	Con producción arrancando limpia, esa combinación probablemente no se dé: el guard es barato pero protege poco. Se documenta para que nadie lo lea como cobertura real
2a8 e90 2	Verificada la asimetría de <code>_es_complemento_sabado</code> (<code>services/validators/doblada_flujo_validator.py:253-269</code>, extraído de <code>strategies/doblada_strategy.py</code>) con test propio: <code>test_media_cubierta_no_compromete_el_sabado_del_acreedor</code> (<code>solicitudes/tests/test_matriz_doblada.py:2804</code>)	Deja de ser duda abierta (fila 17 de § 18). Con una sola mitad cubierta el acreedor no está comprometido, así que la excepción ni se consulta. Suite en verde, 939 tests
fc9 721 8	Frontend de doblada: la decisión del selector de pago en sábado sale del <code>.then</code> inline y pasa a <code>sincronizarSelectorPagoSabado()</code> (<code>static/js/cambio-turno/solicitar_doblada.js:398</code>), con estado propio <code>sabadoDatosDeudor (:81)</code> para sobrevivir al orden de llegada de las dos respuestas asíncronas. Nuevo criterio de ocultado <code>estadoReceptorPago === 'descansando'</code>	Alinea la rama de sábado con <code>sincronizarOpcionesCubrePagoReceptorDoblada (:444)</code>, que ya miraba al receptor

	(:416)	
fc9 721 8	El recuadro rojo "No se puede enviar la solicitud" se pinta para cualquier caso con esRechazado , no solo el del emisor sin jornada que ceder (<code>solicitar_doblada.js:2587</code>)	Los casos 1.2 / 1.5-1.8 / 1.6 de la matriz calculaban el mensaje solo para el envío mientras la pantalla mostraba el resumen en azul: el usuario descubría el rechazo al pulsar Enviar
91c fcb 2	La reconciliación ya no puede tumbar una cancelación. <code>reconciliar_dobladas_aprobadas</code> envuelve <code>_reaplicar_una</code> en <code>try/except</code> <code>ValidationError</code> (<code>solicitudes/services/doblada_snapshot_service.py:337-347</code>): registra con <code>logger.error</code> (id, tipo, fechas, motivo) y continúa con las demás. Se captura solo <code>ValidationError</code> ; cualquier otro fallo sigue propagándose	Cambio de comportamiento y riesgo CERRADO. Desde que los servicios de aplicación tienen guardias de negocio (patrón 39), una solicitud vigente que ya no encaja con el calendario levanta <code>ValidationError</code> al re-aplicarse — estado demostrado : doblada aprobada cuyo pago cae en un sábado donde el acreedor ya no trabaja —, y ese error abortaba la transacción entera: el usuario no podía cancelar por culpa de una solicitud ajena. La reconciliación es reparación best-effort, no validación . Contrapartida: la solicitud omitida sigue aprobada con su efecto sin materializar y solo consta en los logs (§ 16.2, § 18). Test <code>TestReconciliacionNoTumbaLaCancelacion</code> (<code>solicitudes/tests/test_matriz_dobladas.py:3051</code>), verificado con dientes. Suite en verde, 945 tests
9dc 4af 1	Corregida la caracterización del daño de las ramas sin guardia (§ 8.3, § 16.1, § 16.2). Auditado empíricamente con un acreedor que descansa por alternancia de fin de semana	Lo que decía este manual —"le fabrican un turno al ACREEDOR"— es falso para <code>_aplicar_pago_cesion_parcial</code> , <code>_aplicar_pago_fallback</code> y <code>_aplicar_pago_jornada_cedida/cesion_completa</code> . Al acreedor no se le inventa nada: se queda sin turnos, que es lo correcto. El perjudicado es el deudor , que acaba con AM+PM cubriendo jornadas que nadie iba a trabajar y carga los 30 minutos de deuda corporativa . La rama de control <code>jcp_media</code> quedó bloqueada por su guard
9dc 4af 1	El "no se puede poner el guard ahí" pasa de razonado a MEDIDO . Se instrumentó <code>BaseValidator._explorador_trabaja(acreedor, fecha_pago)</code> antes de aplicar en esas	Ya devuelve False : con la solicitud en aprobada , <code>DescansoPorSolicitudService</code> le atribuye el descanso a esa misma solicitud .

	ramas	El guard no bloquearía solo la re-aplicación: se bloquearía a sí mismo en la primera aplicación. No es "arriesgado", es que no funciona . Refuerza § 16.2 y la fila 20 de § 18
9dc 4af 1	Verificaciones que descartan riesgos (medidas, no supuestas): el guard de <code>jcp_media</code> no estorba en el caso normal —con un acreedor con doblada real se aplica sin problema—; el ciclo completo funciona —dos personas con jornadas simples (AM y PM) → doblada aprobada (uno queda AM+PM, el otro descansa, en cesión y en pago) → cancelación → ambos vuelven a sus jornadas simples en los dos días , pasando por la reconciliación y sin error—; y ningún formulario queda bloqueado por los cambios de UI	Los cambios de UI solo ocultan un panel que no aplicaba y adelantan un aviso de rechazo que ya existía
5e0 531 2	Avisos añadidos a las guías de operación: <code>docs/05-referencia/solicitudes/dobladas/GUIA_RAPIDA_REAPLICAR_DOBLADA.md</code> y <code>CHECKLIST_SOLUCION_DOBLADA.md</code>	reaplicar_doblada no valida nada: antes de usarlo hay que comprobar en Mis Turnos que el acreedor trabaje la fecha de pago (la jornada asignada no sirve: dice "es AM" aunque ese día descansa); si descansa, no reaplicar . Es la mitigación por procedimiento de las tres ramas que siguen sin guard (§ 18, fila 20)
5e0 531 2	Patrón 39 ampliado en <code>PROTECTION_PATTERNS.md</code>	Añade la puerta de los comandos de gestión, el cierre parcial con su motivo, el principio "una guardia que consulta el estado posterior no sirve en un método cuyo efecto es cambiar ese estado" y el aviso de no mover la guardia de <code>jornada_cedida</code> al inicio del método
40a 7ed 8	Tests de la reversión de las 5 opciones de temporada	Cubre P4 sobre CAMBIO DESCANSO
5bd 991 c	El select de compañero manda sobre la variable en memoria	Frontend: <code>static/js/cambio-turno/solicitar_cambio_descanso.js</code>
4bd dab 7	Los días de descanso fijados de temporada solo se cambian desde CAMBIO DESCANSO	Regla P3. Añade <code>DescansoSemanaService.es_dia_descanso_temporada</code> (<code>turnos/services/descanso_semana_service.py:44</code>) y el rechazo en

		<code>doblada_strategy.py:201-297</code>
9c4 29e 3	El desplegable y la validación miran la misma fecha	Patrón 37; origen del filtro por día de pago en <code>CoberturaCandidatosView</code>
a76 07d 7	Doblada permanente: medir el choque con el compañero por FECHA	Corrige falsos negativos de disponibilidad
56e b79 3	Ceder un festivo entero cubre el día completo (AM+PM)	—

Rendimiento y consistencia

Commit	Cambio
00558d8	CT permanente: precarga en lote de la matriz empleado × día
1fe8d6e y 1a5111e	Reconciliación: cerrar los días colaterales antes de re-aplicar (patrón 33)
1967790	Integridad: refrescar solo las fechas reconciliadas
0c3ccc8	Mis Turnos y D FDS: un día de fin de semana es DÍA COMPLETO, no una DOBLADA
a437c41	Registrar la hora real de la cancelación → campo <code>fecha_cancelacion</code> (<code>solicitudes/models.py:161-165</code>)
db728c9	Cambio descanso: ejercitar de verdad la regla del día en curso

Vocabularios y constantes

Commit	Cambio
6680164	Se crea <code>core/constants.py</code> como fuente única de los tres vocabularios
2442bc3	La base de datos valida <code>Turno.tipo_cambio:</code> choices + <code>CheckConstraint</code> (<code>turnos/models.py:90-94</code>)
52bec93	Los servicios que escriben turnos usan <code>TipoCambioTurno</code>

80b435e	Se extraen a constantes los literales repetidos de bajo riesgo
cdd8a2c	Se corrige la validación CT+doblada, que nunca disparaba
15056c2	Revert: se quita el bloqueo CT+doblada porque contradice la regla 18

Limpieza de código muerto

4f0798d (DiaEspecialCreateView), 4599f55 (alias /dashboard/ y API alternancia-finde), 99f0ac1 (PDHVisualizarListView), 1a18284 (tres vistas placeholder nunca implementadas), 19ca11e (CRUD manual de turnos, muerto desde 2025) y 02b1330 (.pyc versionados en core/).

Calendario y calidad

9d87547 introduce bloqueo optimista al guardar el año y elimina la ruta de baja; 4dd6cd6 hace que el sistema diga **por qué** un compañero no cubre una fecha; 339cdee añade la configuración de SonarQube, la cobertura y las correcciones del triaje.

18. Por confirmar

#	Afirmación pendiente	Dónde se buscó	Por qué no se pudo verificar
1~ ~	~~Alcance real de B5 (correo irretirable en cobertura con dos compañeros)~~	solicitudes/services/solicitud_orchestrator.py:269-286, solicitudes/services/email_outbox_service.py:354-359	RESUELTO 2026-09-08 — la duda ya no aplica: el despacho se movió al cierre del <code>envio_agrupado</code> , que se abre por fuera del <code>atomic()</code> , así que nada se entrega antes del commit. Si la transacción revierte, el rollback se lleva las filas del outbox y el lote queda vacío. Ver B5 en § 16.4
2~ ~	~~Lo mismo para <code>_procesar_doblada_permanente_multi</code> ~~	solicitud_orchestrator.py:509-531	RESUELTO 2026-09-08 — mismo mecanismo y misma conclusión
33	Que el ahorro del lote se mantenga contra SES en producción . Las cifras del manual (~2 s de handshake,	Logs de desarrollo	No hay entorno de producción con SES contra el que medir. Repetir la medición tras el despliegue y ajustar el número

	6,3-7,8 s por solicitud) están medidas contra Gmail desde desarrollo el 2026-09-07 ; SES tiene otra latencia y otros límites de mensajes por conexión, y ese límite es justo lo que dispara la renovación de canal a media tanda (<code>solicitudes/services/email_outbox_service.py:265-272</code>)		si difiere
~~ 34 ~~	~~Que la rama de PAGO REPROGRAMADO de AcuerdoPorDiaService nombre al compañero correcto~~	<code>solicitudes/tests/test_acuerdo_por_dia.py:156-198</code>	RESUELTO 2026-09-09 — cuatro casos cubren la rama: que nombre al compañero de la doblada ORIGINAL y devuelva <code>fecha_relacionada</code> = el día incumplido, que una reprogramación cancelada no reclame el día, que pase por la misma guarda de realidad que las demás, y que un acuerdo con snapshot le gane el día (la rama se añade al final, solo sobre fechas que nadie reclamó)
35	Si el despliegue invalida la caché del mes de Mis Turnos	<code>docs/05-referencia/deployment/99-aws/MANUAL_DESPLIEGUE_EC2.md</code> y § 13.2	No aparece ningún paso de invalidación ni de reinicio del backend de caché. Con el TTL de una hora (<code>core/services/cache_service.py:17</code>), un cambio en la respuesta puede tardar hasta 60 minutos en verse; conviene añadirlo al checklist de § 13.3 y decidir si se hace por comando o reiniciando la caché
36	Si algún consumidor de <code>solicitud_info</code> fuera de <code>static/js/mis_turnos.js</code> asume el formato anterior	<code>grep</code> sobre <code>static/</code>	Hoy el único consumidor es <code>mis_turnos.js</code> . El bloque conserva las mismas claves que antes, pero ahora se rellena en días que antes llegaban null (doblada permanente, CT

			permanente a partir del segundo día, cambio de descanso entre semana): un consumidor que trate <code>solicitud_info != null</code> como "esto es un CAMBIO TURNO" se equivocaría
3	Porcentaje de cobertura de tests actual	No se ejecutó <code>pytest --cov</code>	Requiere una base MySQL levantada
4	Número exacto de migraciones de <code>turnos</code> , <code>empleados</code> y <code>permisos</code>	<code>*/migrations/</code>	Solo se contaron las de <code>solicitudes</code> (35 archivos). La § 1.3 cita 33/15/8/8 de una pasada anterior; el dato de <code>solicitudes</code> ya no coincide y conviene recomtar los cuatro
5	Contenido de <code>instructivos/*.docx</code> y <code>*.mwb</code> (reglas dictadas por el negocio)	<code>instructivos/</code> , raíz del repositorio	Formato binario, no contrastable con el código
6	Contenido de <code>04-guias/manuales/*.docx</code> ("requisito doblada", "Proceso completo doblada", "estructura sql") y de <code>PLAN_CORREO_TRANSACCIONAL_Y_LATENCIA.docx</code>	<code>docs/04-guias/manuales/</code> , <code>docs/05-referencia/deployment/</code>	Ídem
7	Si existe integración con Google Apps Script	Búsqueda en <code>AppTurnosExplora/</code>	No se encontró referencia en el código; puede vivir fuera del repositorio
8	Qué vistas concretas de <code>turnos/</code> , <code>empleados/</code> y <code>permisos/</code> exigen rol supervisor	<code>core/mixins.py</code> , <code>*/urls.py</code>	Se verificó el mecanismo (<code>AdminRequiredMixin</code> , <code>SupervisorApiRequiredMixin</code>), no la lista vista por vista
11	Si las horas a favor deben caducar (p. ej. a los 6 meses)	Decisión de negocio, no de código	Hoy <code>CreditoHoras</code> no caduca nunca . El sitio natural para implementarlo sería un campo <code>vence_el</code> más un filtro en <code>creditos_disponibles</code> (<code>permisos/credito_horas_service.py:37</code>), pero activarlo requiere que el negocio lo decida

12	Si un crédito debería poder aplicarse sin un PDH asociado	<code>permisos/credito_horas_service.py:128-131</code>	Por diseño el crédito es un medio de pago dentro de un PDH y no puede superar lo que ese pago salda. No se ha planteado el caso de un explorador con crédito y sin ninguna deuda que saldar: hoy el saldo simplemente se acumula
13	Cobertura real del CRUD de crédito en navegador	Solo pruebas de Django (Client)	Los 23 casos de <code>permisos/tests/test_credito_horas.py</code> cubren servicio, formulario y vistas por HTTP, pero no el comportamiento del <code><input type=number></code> en un navegador real, que es justo donde apareció el fallo del <code>min/step</code> (§ 16.2)
~~ 9~ ~	~~Si la CSP report-only ha generado violaciones desde su publicación~~	YA NO APLICA (2026-09-04)	La cabecera report-only se borró : barridas las 116 plantillas, el único origen externo que queda es Google Fonts, y sus valores son ahora los que se aplican de verdad (<code>config/settings.py:424-433</code>). Ver B8 en § 16.4
~~ 10 ~~	~~Contenido exacto del <code>Dockerfile</code> ~~	RESUELTO (2026-09-04)	Leído completo: <code>python:3.12-slim</code> , usuario <code>appuser</code> sin privilegios, <code>HEALTHCHECK</code> que acepta cualquier respuesta <code>< 500</code> sobre <code>/health/</code> —porque en producción <code>ALLOWED_HOSTS</code> hace que <code>127.0.0.1</code> responda 400 y exigir 200 mataría el contenedor en bucle—, y <code>CMD</code> con <code>migrate + Gunicorn a GUNICORN_WORKERS</code> (3 por defecto; no se calcula con <code>nproc</code> porque dentro de un contenedor devuelve los núcleos del anfitrión)

11	Que la plantilla <code>infra/cloudformation/swalp-infra.yaml</code> despliega sin errores en AWS	La plantilla	Solo se validó la sintaxis YAML y los tipos de recurso; no se ha ejecutado contra una cuenta real. Probarla primero con <code>--stack-name swalp-test</code> y borrarla después, comprobando que <code>delete-stack</code> no deja recursos huérfanos cobrando
12	Las fechas de fin de soporte estándar de MySQL 8.0 y 8.4 en RDS	Documentación de AWS vía Context7	La documentación describe el mecanismo (Extended Support se cobra por vCPU-hora) pero sus fechas concretas dependen de la región y del momento. Verificar antes de desplegar con <code>aws rds describe-db-major-engine-versions --engine mysql</code>
13	Los precios unitarios de AWS citados en la arquitectura (\$5)	Páginas de precios de terceros (Vantage, Holori) y de AWS	Contrastados entre varias fuentes y coherentes entre sí, pero no verificados contra la factura real ni contra la Pricing Calculator con la cuenta del proyecto . El IVA colombiano del 19 % está confirmado como tarifa general, no como trato fiscal concreto de esta entidad
11	Frecuencia real del cron de <code>procesar_email_outbox</code> en producción	MANUAL_OUTBOX_CORREOS.md (./05-referencia/deployment/02-correo/MANUAL_OUTBOX_CORREOS.md)	No está en el código: es configuración de infraestructura
~~ 23 ~~	~~Inconsistencia entre las dos vías de extinción de deuda por sanción: la condonación reconocía la auto-sanción por periodo y el consumo por el prefijo del motivo~~	<code>solicitudes/services/deuda_corporativa_service.py:58-69</code>	RESUELTO 2026-09-05 — el criterio por periodo se extendió a las siete consultas que deciden algo, con dos Q de clase (<code>ES_AUTOMATICA</code> , <code>ES_MANUAL</code>). El prefijo queda como etiqueta de lectura. Cierra además la vía de doble sanción sobre un mes ya juzgado. Test: <code>test_editar_el_motivo_n</code>

			o_reabre_el_ciclo_de_sanciones (solicitudes/tests/test_consumo_deuda_por_sancion.py:339)
12	Si la sección "deuda conocida pendiente de decisión" de <code>PROTECTION_PATTERNS.md</code> :2180 sigue vigente	<code>PROTECTION_PATTERNS.md</code>	Se leyó el índice, no el contenido completo de esa sección
13	Si <code>CierreSolicitudesConfig.habilitado</code> está activo en el entorno real	<code>solicitudes/models.py:802</code>	El valor por defecto es <code>False</code> ; el estado real es un dato de producción
24	Si la plantilla <code>templates/solicitudes/solicitudes_pendientes_list.html</code> presenta las <code>cancelaciones_pendientes</code> con los dos botones y con el plazo restante visible	La plantilla es el único archivo que referencia <code>cancelaciones_pendientes</code> fuera de la vista	Se comprobó la referencia, no el marcado ni la experiencia resultante. Sin plazo a la vista, el receptor no sabe que su silencio decide
25	Si algún consumidor del frontend de <code>POST /solicitudes/cancelar-solicitud/<id>/</code> sigue asumiendo que un 200 significa "cancelada"	JS de las pantallas de solicitudes	No se auditó el JavaScript en esta pasada. El contrato cambió: ahora hay que leer <code>cancelada</code>
26	Si la cancelación consensuada merece entrar en <code>PROTECTION_PATTERNS.md</code> como patrón propio ("un efecto ya aplicado y pactado no se deshace por una sola de las partes; mientras se decide, el efecto sigue vigente")	<code>PROTECTION_PATTERNS.md</code>	No está registrado. El invariante <code>estado='aprobada' + cancelacion_estado='pendiente'</code> es exactamente la clase de cosa que alguien "arregla" sin querer
20	(Sucesora parcial de la 19.) <code>_aplicar_pago_cesion_parcial</code> , <code>_aplicar_pago_fallback</code> y <code>_aplicar_pago_jornada_cedida</code> en <code>cesion_completa</code> siguen	<code>solicitudes/services/doblada_pago_service.py:353-445</code> (<code>cesion_parcial</code> , nota en :366-388), :520-521 (<code>cesion_completa</code>) y :567-642 (<code>fallback</code> , nota en :581-585); <code>snapshot_turnos_previos</code>	Es un cambio de calado (cambia el criterio de qué estado se consulta al aplicar) y afecta a la reconciliación y a los comandos de reparación. No se aborda en esta tanda

	sin guard y no pueden llevar el del patrón 39 tal cual: dejan al acreedor sin turnos por diseño, y TurnoService.estado_dia no sabe excluir la solicitud en curso, así que el guard bloquearía su propia re-aplicación (reaplicar_doblada). La solución correcta —comparar contra snapshot_turnos_previos, el mundo ANTES de aplicar— está sin decidir y sin implementar. SIGUE ABIERTA. Precisiones de esta pasada: (a) el guard no es "arriesgado", es que no funciona — medido: _explorador_trabaja ya devuelve False antes de la primera aplicación, porque la propia solicitud aprobada le atribuye el descanso; (b) el daño real no es inventarle un turno al acreedor, sino dejar al deudor con AM+PM y 30 minutos de deuda corporativa por jornadas que nadie iba a trabajar; (c) solo se llega por consola (comandos de gestión), y esas dos guías ya llevan aviso destacado — mitigación por procedimiento, no cierre	en solicitudes/models.py	
21	Si la rama parcial de _aplicar_pago_jornada_cedida —la que acaba de recibir guard— es alcanzable en la práctica. Solo se entra con tipo_cesion vacío: el despachador manda cesion_parcial_am/pm a otra rama	doblada_pago_service.py: 62-94, :520-542; solicitudes/models.py:360-364	Habría que auditar los datos de producción, que arranca limpia. El guard es barato, pero protege poco: no se ha demostrado que la combinación se dé

	(<code>doblada_pago_service.py:83-84</code>) y <code>cesion_completa</code> va por la que borra los turnos (<code>:520-521</code>), y <code>TIPO_CESION_CHOICES</code> (<code>solicitudes/models.py:360-364</code>) solo tiene esos tres valores		
22	Deuda operativa, parcialmente cubierta: cuando la reconciliación omite una solicitud que no pudo re-aplicar (<code>doblada_snapshot_service.py:337-347</code>), esa solicitud sigue aprobada con su efecto sin materializar. El comando de verificación ya existe — <code>verificar_efecto_aplicado</code> compara <code>snapshot_turnos_resultantes</code> contra los turnos reales y repara con <code>--reparar</code>—, y detecta estos casos porque el resultante de la omitida no se refresca (<code>:348</code>). Lo que sigue abierto es la alerta: hay que acordarse de ejecutarlo o leer el log	<code>solicitudes/services/dobladada_snapshot_service.py:286,332-342,343;</code> <code>solicitudes/management/commands/verificar_efecto_aplicado.py</code>	Falta una alarma de CloudWatch sobre el patrón Reconciliación : del log de ERROR (el grupo y el filtrado por <code>request_id</code> ya existen, ADR 007), o <code>verificar_efecto_aplicado</code> en tarea programada. Decisión y contrapartidas en el ADR 008 (<code>./03-arquitectura/adr/008-reconciliacion-best-effort.md</code>)
27	~~Si EstrategiaFinDeSemana merece un ADR propio~~ — resuelto: escrito como ADR 012 (<code>./03-arquitectura/adr/012-contrato-parcial-para-estrategias-de-fin-de-semana.md</code>) el 2026-09-04, con las tres alternativas descartadas	<code>docs/03-arquitectura/adr/</code>	Se propuso y se aprobó; no se escribió por iniciativa propia
28	Si la retirada de la capa de	<code>docs/03-</code>	No se auditaron los ADR uno a

	repositorios (f2af451) merece corregir o retirar algún ADR existente que la diera por vigente	arquitectura/adr/, docs/03-arquitectura/ARQUITECTURA.md	uno en esta pasada; el manual ya no la menciona como capa viva (§ 3.2, § 3.4, § 4)
29	Si docs/03-arquitectura/ARQUITECTURA.md y docs/05-referencia/ siguen describiendo solicitudes/repositorios/, DobladaStrategy con los ocho tramos dentro, o MisTurnosPorMesView con la decisión por día —o con las ocho consultas de enriquecimiento— dentro de la vista	Los propios documentos	Esta pasada verificó las citas de manual_tecnico.md ; los documentos enlazados no se revisaron cita a cita
30	Si el frontend de los dos formularios de fin de semana maneja el nuevo 400 `tipo_sin_selector_finde` o lo trata como error genérico	static/js/cambio-turno/solicitar_d_fds.js, solicitar_cambio_descanso.js	El código de error es nuevo (5857373); no se revisó el JS consumidor
31	Si existe ya una alarma real en AWS sobre el marcador CRON_NO_EJECUTADO , o solo existe el comando que lo emite	Consola de AWS	Mismo patrón que la fila B4 del cierre semanal: la herramienta existe, la alarma es operativa y no se puede comprobar desde el repositorio
~~ 32 ~~	~~Si las citas de las secciones de despliegue, seguridad y permisos siguen vigentes~~	RESUELTO 2026-09-04 — se verificaron las 336 citas restantes abriendo cada archivo en la línea citada. 37 estaban rotas y quedaron corregidas: las 12 de config/settings.py (ENVIRONMENT, DB_NAME, DB_USER, zona horaria, validadores, CORS, caché, debug toolbar, django-environ) y 25 más repartidas en permisos/, empleados/, turnos/models.py, config/urls.py, tokens_aprobacion.py , plantillas y tests. Una no era un desplazamiento sino otro	

		modelo: «una sanción no se borra, se levanta» apuntaba a <code>empleados/models.py:193</code> (el modelo <code>Role</code>) en vez de a <code>SancionEmpleado</code> (:299, 323-330)	
~~ 19 ~~	~~No se ha barrido toda forma de llegar a <code>_aplicar_pago_cesion_parcial</code> y <code>_aplicar_pago_fallback</code> con un acreedor que descansa: solo se descartó la vía de la reconciliación (la tapa el LIFO)~~	PARCIALMENTE RESUELTA — apareció la vía que faltaba: los comandos de gestión <code>reaplicar_doblada</code> (<code>solicitudes/management/commands/reaplicar_doblada.py:259</code>) y <code>corregir_doblada_cesion_total</code> (:73) llaman a <code>aplicar_doblada_pago</code> sin validar y sin pasar por el LIFO , y son operación documentada (<code>verificar_doblada.py:384</code> los sugiere). Dos ramas se cerraron con <code>guard</code> (<code>jcp_media</code> , <code>jornada_cedida</code> parcial); estas dos no. Sucesora: fila 20	Ya no es una duda de alcanzabilidad: la vía existe y está documentada como procedimiento de reparación
~~ 16 ~~	~~Si las otras ramas de <code>aplicar_doblada_pago</code> (<code>jcp_ambas</code> , <code>jcp_media</code> , <code>cesion_parcial</code> , <code>jornada_cedida</code> , <code>fallback</code>) tienen el mismo punto ciego que tenía la de sábado~~	RESUELTA — auditadas las cinco. <code>jcp_ambas</code> limpia en este eje; <code>jcp_media</code> y <code>jornada_cedida</code> repetían la forma del bug de sábado y ya llevan guard (<code>doblada_pago_service.py:302-309</code> , :536-543); <code>cesion_parcial</code> y <code>fallback</code> no fabrican turno pero validan con la jornada base y siguen expuestas . Detalle e invariantes en § 16.2	La conclusión de entonces ("inalcanzables por el LIFO, no se añaden guards") era falsa : el LIFO tapa la reconciliación, no los comandos de gestión. Cierre parcial : 2 de 4. Limitaciones vivas: filas 20 y 21
~~ 15 ~~	~~Si el guard de sábado merece entrar en <code>PROTECTION_PATTERNS.md</code> como patrón propio~~	RESUELTO — registrado como patrón 39 en <code>PROTECTION_PATTERNS.md:1651</code> , con cinco reglas y cuatro filas en el registro de cambios	Se leyó el archivo: no lo cubría el patrón 25 (fallar cerrado habla del sentido de la decisión, no de dónde vive la guardia)
~~ 17 ~~	~~Tercera copia del criterio en <code>_es_complemento_sabado</code> : falta decidir si autorizar	RESUELTO — la asimetría es inofensiva y queda fijada con test: <code>test_media_cubierta_no_c</code>	<code>_es_complemento_sabado</code> (<code>strategies/doblada_strategy.py:288-304</code>) solo se consulta cuando el acreedor ya

	también en la estrategia el caso de dos deudores distintos~~	ompromete_el_sabado_del_acreedor (solicitudes/tests/test_matriz_dobladas.py:2804)	está comprometido , y para estarlo hacen falta las dos mitades del sábado cubiertas. Con una sola mitad cubierta el acreedor sigue trabajando la otra: no hay bloqueo que esquivar y el segundo deudor —aunque sea otra persona— pasa la validación sin necesitar la excepción. El test comprueba las dos direcciones (media cubierta → NO comprometido; ambas → Sí) y se caerá avisando si la atribución de descanso pasara a marcar el día con media jornada cubierta. Siguen existiendo tres copias del criterio: no las unifiques (§ 16.2)
~~ 14 ~~	~~Si la SECRET_KEY procede de una sola entrada de Secrets Manager/SSM o de un valor por tarea~~	RESUELTO — sí procede de una sola entrada; ver § 9.2	La auditoría lo resolvió con fuentes del propio repositorio: el checklist de Fargate crea swalp/SECRET_KEY como secreto único y lo inyecta vía secrets: del task definition, y la ruta EC2 es de instancia única. Además la aplicación no puede generar una clave propia

Fin del manual técnico. Fuente de verdad: `AppTurnosExplora/docs/manual\_tecnico.md`. El PDF de `docs/pdf/Manual\_Tecnico.pdf` se genera desde este archivo y nunca se edita a mano.